

Generate the data of the different materials of Vehicle Composition based on different studies and previous data

Matthias Roesslein - Empa

11 April 2025

Abstract

Calculate for given data the standard composition of a vehicle and their different car segments (a, b, c etc.). It is first done for aluminum alloys (Al), iron/steel alloys (Fe)

Copyright notice: Copyright © 2023 - 2025 Empa, Matthias Roesslein

This is an R Markdown Notebook. When you execute code within the notebook, the results appear beneath the code. Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Cmd+Shift+Enter*.

Data Analysis

Prepare for the analysis

Loading of the libraries

We are using tidyverse framework to rearrange and analysis the data.

```
# load libraries
library(tidyverse)
library(dplyr)
library(broom)
library(data.table)
```

```
library(stringr)
library(ggplot2)
library(scales)
library(forcats)

library("readxl") # read excel files
library("openxlsx") #The openxlsx package is highly customizable and allows you to: Write multiple data frames to different sheets of the same Excel file. Customize formatting, such as column
#library(WriteXLS) # uses Perl and not Java - Java has a licence issue and should not be used at Empa

library(rmarkdown)
```

```
#tidyverse_conflicts()
```

Define the files names of the input excel file

```
filename0_input <- "Data/0_CarSegments.xlsx"
filename1_JRCData <- "Data/1_JRCData_1980_2010.xlsx"
filename2_input <- "Data/2_VehicleCompositionNewData.xlsx"
filename3_input <- "Data/3_VehicleCompositionPredictions.xlsx"
```

Define the file name of the output excel file

```
filename21_output <- "Data/21_CompositionData.xlsx"
```

Load all the sheets of the excel file BasicData

This are the data from the different report references.

```
# Get the actual car weight per segment
carWeight <- read_excel(filename0_input, sheet = "CarWeight")

productKeys <- read_excel(filename2_input, sheet = "ProductKeys")

aluminum_content_1 <- read_excel(filename2_input, sheet = "aluminum_content_1")
aluminum_content_2 <- read_excel(filename2_input, sheet = "aluminum_content_2")
aluminum_content_3 <- read_excel(filename2_input, sheet = "aluminum_content_3")
aluminum_content_4 <- read_excel(filename2_input, sheet = "aluminum_content_4")
```

```

aluminum_content_5 <- read_excel(filename2_input, sheet = "aluminum_content_5")
aluminum_content_6 <- read_excel(filename2_input, sheet = "aluminum_content_6")
aluminum_content_7 <- read_excel(filename2_input, sheet = "aluminum_content_7")

Car_Avg_MaterialContent_1 <- read_excel(filename2_input, sheet = "Car_Avg_MaterialContent_1")
Car_Avg_MaterialContent_2 <- read_excel(filename2_input, sheet = "Car_Avg_MaterialContent_2")

steel_content_1 <- read_excel(filename2_input, sheet = "steel_content_1")
steel_content_2 <- read_excel(filename2_input, sheet = "steel_content_2")
steel_content_3 <- read_excel(filename2_input, sheet = "steel_content_3")
steel_content_4 <- read_excel(filename2_input, sheet = "steel_content_4")

magnesium_content <- read_excel(filename2_input, sheet = "magnesium_content")

derivedSUVComposition <- read_excel(filename2_input, sheet = "SUVEffect")

catalyst_content_1 <- read_excel(filename2_input, sheet = "catalyst_China_1")
catalyst_content_2_1 <- read_excel(filename2_input, sheet = "catalyst_converter_1")
catalyst_content_2_2 <- read_excel(filename2_input, sheet = "catalyst_converter_2")
catalyst_content_2_3 <- read_excel(filename2_input, sheet = "catalyst_converter_3")

EED_weight <- read_excel(filename2_input, sheet = "EED_weight")
EED_composition <- read_excel(filename2_input, sheet = "EED_composition")

EEC_ICEV_content_1 <- read_excel(filename2_input, sheet = "EEC_ICEV")
EEC_BEV_content_1 <- read_excel(filename2_input, sheet = "EEC_BEV")
EEC_HEV_content_1 <- read_excel(filename2_input, sheet = "EEC_HEV")
EEC_PHEV_content_1 <- read_excel(filename2_input, sheet = "EEC_PHEV")

BEV_content_1 <- read_excel(filename2_input, sheet = "BEV")

rest <- read_excel(filename2_input, sheet = "Rest")

derivedFeComposition <- read_excel(filename3_input, sheet = "derivedFeComposition")
derivedPGEComposition <- read_excel(filename3_input, sheet = "derivedPGEComposition")
derivedEECComposition <- read_excel(filename3_input, sheet = "derivedEECComposition")

developmentEEC <- read_excel(filename3_input, sheet = "developmentEEC")

```

Load JRC data

```

JRC_Data <- read_excel(filename1_JRCData, sheet = "JRC-Data")

```

Define essential parameters

Set the year, where the data should begin with 1980. Currently the latest report go until 2030. We assume afterwards a constant composition.

```
startDate <- 1980
```

Developement of the car weight

Show the developemt and anlyse it

```
# Get the long format to use ggplot

carWeight_long <- carWeight %>% select(-SegmentAMass) %>% pivot_longer(cols = -Year, names_to = 'Segment', values_to = 'values')

# Display the different car weights per segment over time

ggplot() +
  geom_point(data = carWeight_long, mapping = aes(x = Year, y = values, colour = Segment)) +
  geom_smooth(data = carWeight_long, mapping = aes(x = Year, y = values, colour = Segment), method = "loess", se = FALSE) +
  ggtitle("Car weight based on common types") +
  scale_x_continuous(name = "year") +
  scale_y_continuous(name = "mass per vehicle (kg)")

# Display standard cars

carWeight_long %>% filter(Segment == "StandardMass") %>%
  ggplot(mapping = aes(x = Year, y = values)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE) +
  ggtitle("Standard car weight based on common types") +
  scale_x_continuous(name = "year")
  scale_y_continuous(name = "mass per vehicle (kg)")

## <ScaleContinuousPosition>
## Range:
## Limits: 0 -- 1

# Display standard cars from 1980 t0 2010

carWeight_long %>% filter((Segment == "StandardMass") & (Year >= 1980) & (Year <= 2010)) %>%
```

```
ggplot(mapping = aes(x = Year, y = values)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  ggtitle("Standard car weight based on common types") +
  scale_x_continuous(name = "year")
  scale_y_continuous(name = "mass per vehicle (kg)")
```

```
## <ScaleContinuousPosition>
## Range:
## Limits: 0 -- 1
```

```
# Model the car weight
```

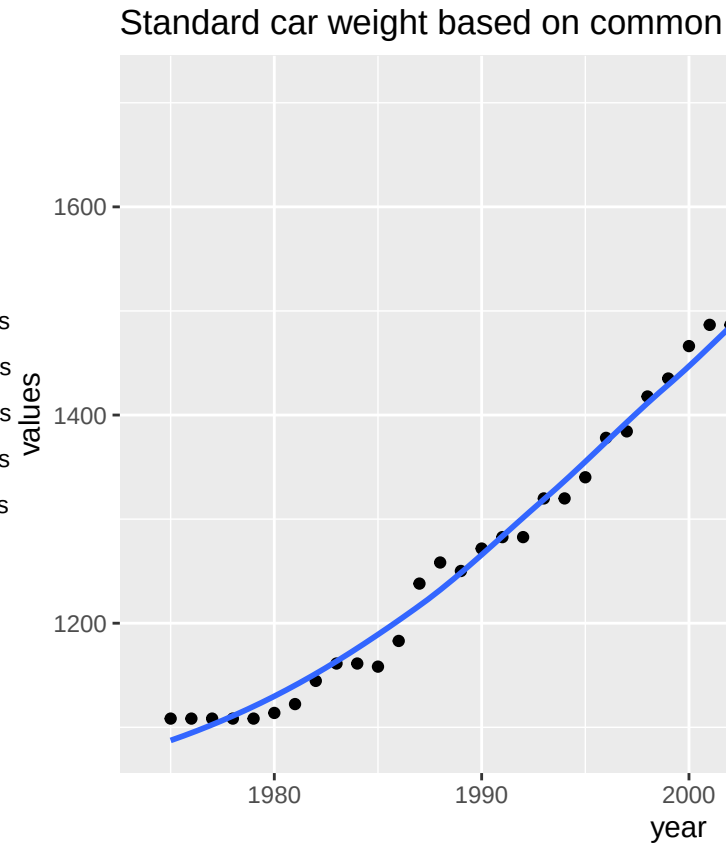
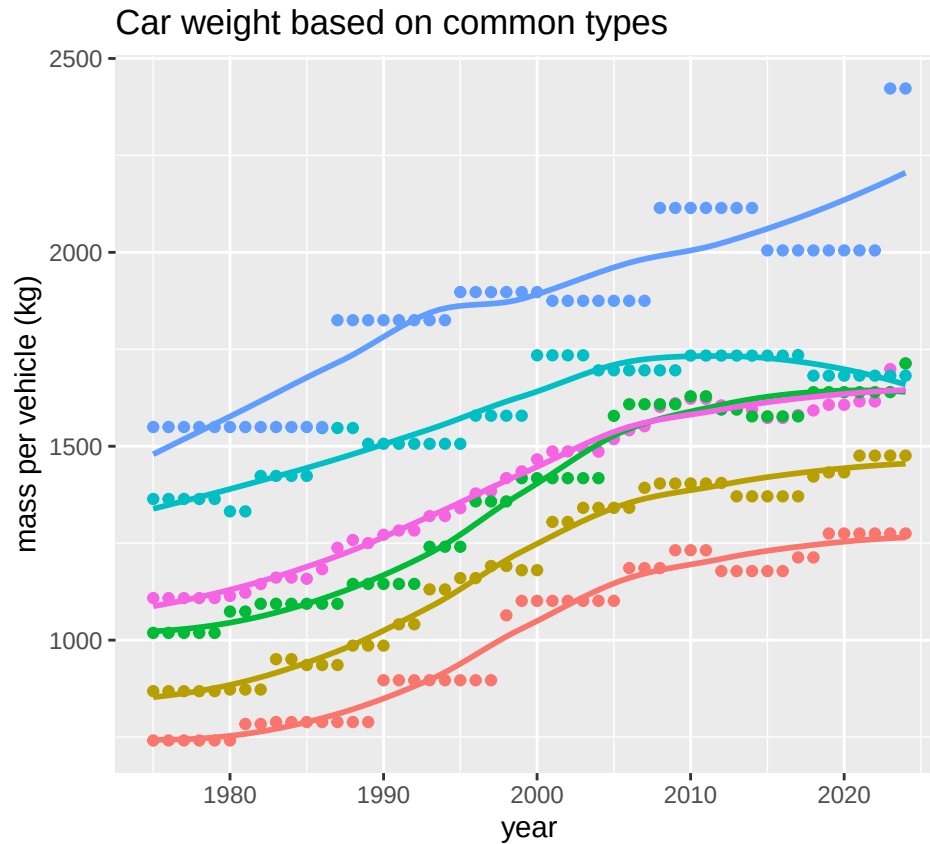
```
carWeightStandard <- carWeight_long %>% filter((Segment == "StandardMass") & (Year >= 1980) & (Year <= 2010))

modelcarWeightStandard <- lm(values ~ Year, data = carWeightStandard)
summary(modelcarWeightStandard)
```

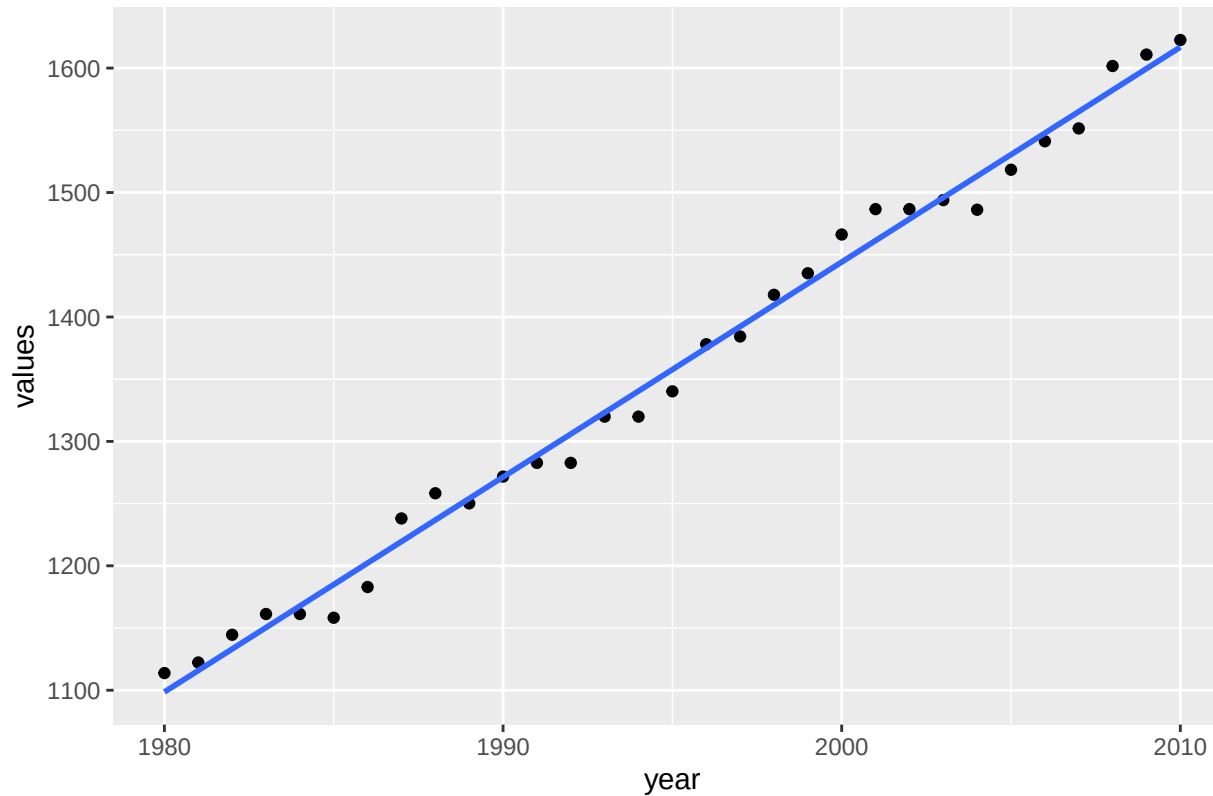
```
##
## Call:
## lm(formula = values ~ Year, data = carWeightStandard)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -27.0310 -10.0625  0.3775  11.0739  25.2792
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -3.311e+04  6.147e+02  -53.85  <2e-16 ***
## Year         1.728e+01  3.081e-01   56.06  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.35 on 29 degrees of freedom
## Multiple R-squared:  0.9909, Adjusted R-squared:  0.9905
## F-statistic: 3143 on 1 and 29 DF, p-value: < 2.2e-16
```

```
# Create a new data frame with the year you want to interpolate
start_value <- data.frame(Year = 1980)
end_value <- data.frame(Year = 2010)
```

```
# Use the predict function to get the interpolated value
startWeightCar <- predict(modelcarWeightStandard, newdata = start_value)
endWeightCar <- predict(modelcarWeightStandard, newdata = end_value)
```



Standard car weight based on common types



Alumnium content

aluminum table 1

The file filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx" with the sheet "aluminum_content_1" is based on the following report and data. 'Aluminum Content in Passenger Vehicles (Europe) Assessment 2022 and Outlook 2026, 2030 - Public Summary - April, 2023 (2023_04_Aluminum-Content_Ducker-Study_EA-Public-Summary_190423.pdf) page 26. It has been derived from the numbers in the figure and the comments.

aluminum table 2

The file `filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx"` with the sheet "aluminum_content_2" is based on the following report and data: 'Aluminum Content in Passenger Vehicles (Europe) Assessment 2022 and Outlook 2026, 2030 - Public Summary - April, 2023 (2023_04_Aluminum-Content_Ducker-Study_EA-Public-Summary_190423.pdf) page 13. It has been derived from the numbers in the figure and the comments.

```
# add the total weight

aluminum_content_1 <- aluminum_content_1 %>% group_by(year) %>% mutate(total = SheetAluminum + ExtrusionAluminum + CastingAluminum + ForgingAluminum)

# generate a long format

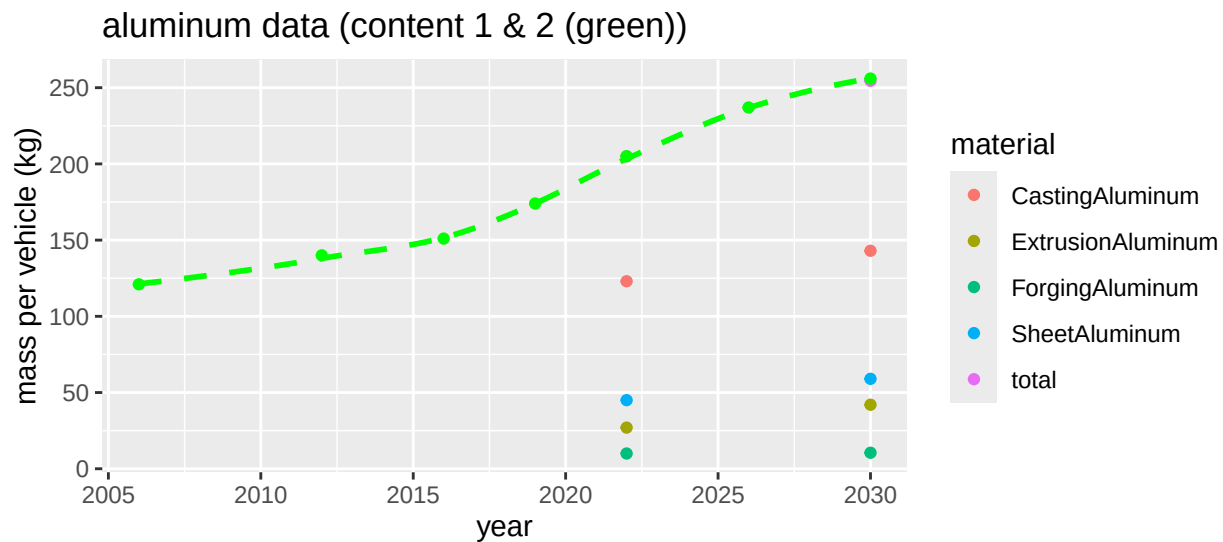
aluminum_content_1 <- aluminum_content_1 %>% pivot_longer(!year, names_to = "material", values_to = "mass")

aluminum_content_2 <- aluminum_content_2 %>% filter(scenario == "base")

## display as a figure

g <- ggplot() +
  geom_point(data = aluminum_content_1, mapping = aes(x = year, y = mass, colour = material)) +
  geom_point(data = aluminum_content_2, mapping = aes(x = year, y = totalMass, colour = "green")) +
  geom_smooth(data = aluminum_content_2, mapping = aes(x = year, y = totalMass), method = "loess", se = FALSE, linetype = "dashed", colour = "green") +
  ggtitle("aluminum data (content 1 & 2 (green))") +
  scale_y_continuous(name = "mass per vehicle (kg)") +
  coord_fixed(ratio = 0.05) # Adjust the ratio as needed

plot(g)
```



aluminum content 4

2020 data

The file filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx" with the sheet "aluminum_content_4" the 2020 are based on the "2020 North America light vehicle aluminum content and outlook - Final report summary (July 2020)" (DuckerFrontier-Aluminum-Association-2020-Content-Study-Summary-Report-FINAL.pdf) page 12.

2023 data

The second set of data comes from "Light vehicle aluminum content and outlook study - prepared for the aluminum association by Ducker April 2023 (Ducker-ATG-2023-Summary-Report-April-2023.pdf) page 13.

Conversion of all data from lb to kg

New rows are added to the data.frame with the kg values. The conversion factor is: $x \text{ lb} * 0.45359237 = y \text{ kg}$

aluminum content 5

2020 data

The file filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx" with the sheet "aluminum_content_5" the 2020 are based on the "2020 North America light vehicle aluminum content and outlook - Final report summary (July 2020)" (DuckerFrontier-Aluminum-Association-2020-Content-Study-Summary-Report-FINAL.pdf) page 13.

2023 data

The second set of data comes from "Light vehicle aluminum content and outlook study - prepared for the aluminum association by Ducker April 2023 (Ducker-ATG-2023-Summary-Report-April-2023.pdf) page 16.

Conversion of all data from lb to kg

New rows are added to the data.frame with the kg values. The conversion factor is: $x \text{ lb} * 0.45359237 = y \text{ kg}$

```
lbTokg <- 0.45359237
```

```
# aluminum_content_4
```

```
aluminum_content_4$Content2023 <- aluminum_content_4$`AluminumContent (lb) - 2023` * lbTokg
```

```
aluminum_content_4$Content2020 <- aluminum_content_4$`AluminumContent (lb) - 2020` * lbTokg
```

```
#aluminum_content_4
```

```
# aluminum_content_5
```

```
aluminum_content_5$SheetAluminum <- aluminum_content_5$`SheetAluminum (lb)` * lbTokg
```

```
aluminum_content_5$ExtrusionAluminum <- aluminum_content_5$`ExtrusionAluminum (lb)` * lbTokg
```

```
aluminum_content_5$CastingAluminum <- aluminum_content_5$`CastingAluminum (lb)` * lbTokg
```

```
aluminum_content_5$ForgingAluminum <- aluminum_content_5$`ForgingAluminum (lb)` * lbTokg
```

```
aluminum_content_5$Total <- aluminum_content_5$`Total (lb)` * lbTokg
```

```
#aluminum_content_5
```

```
# transform aluminum_content_5 to long format with kg values
```

```
aluminum_content_5_long <- aluminum_content_5 %>% pivot_longer(cols = c(SheetAluminum, ExtrusionAluminum, CastingAluminum, ForgingAluminum, Total), names_to = "materials",
```

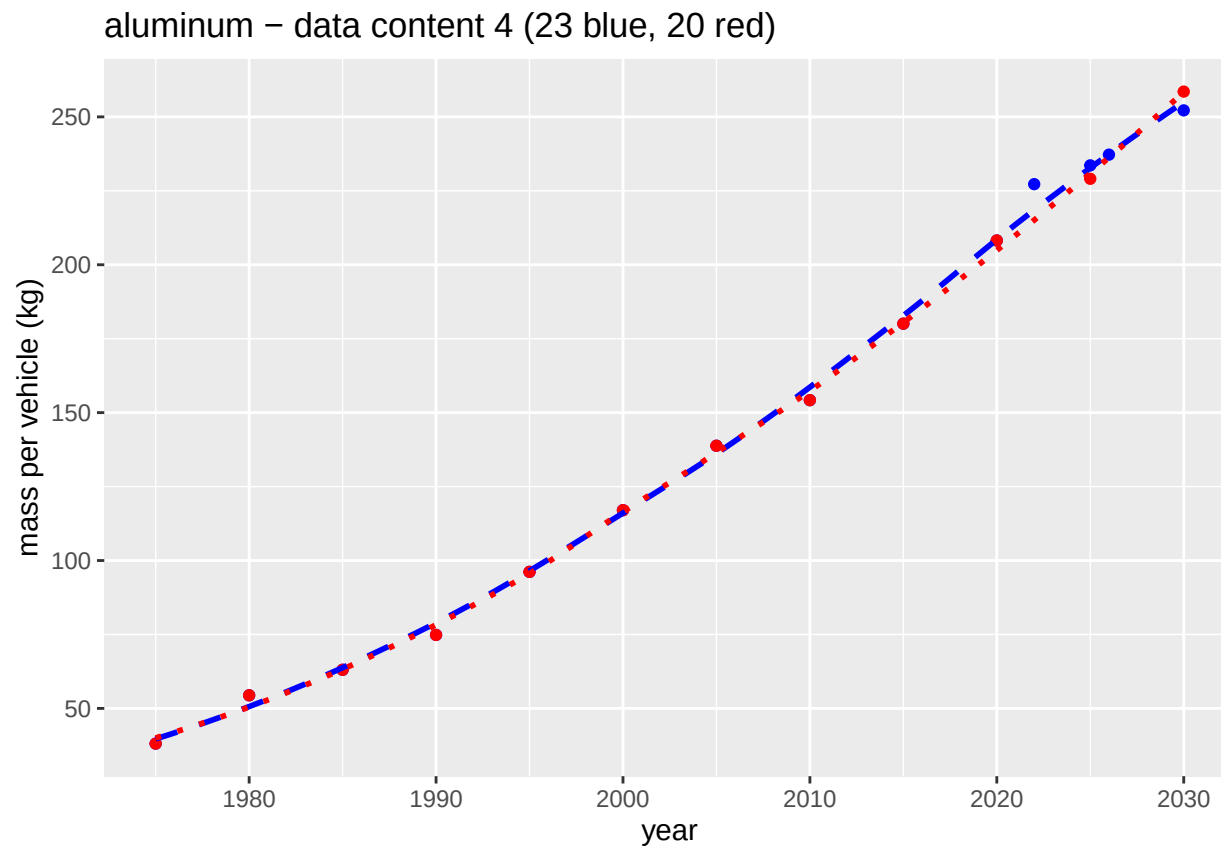
```
#aluminum_content_5_long
```

aluminum table 4 as a figure

The content of the aluminum_content_4 is shown in a figure. The 2020 numbers are given in red and the 2023 numbers are shown in blue.

```
g <- ggplot() +
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), color = "blue") +
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2020), color = "red") +
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), method = "loess", se = FALSE, linetype = "dashed", colour = "blue") +
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2020), method = "loess", se = FALSE, linetype = "dotted", colour = "red") +
  ggtitle("aluminum - data content 4 (23 blue, 20 red)") +
  scale_y_continuous(name = "mass per vehicle (kg)")

plot(g)
```

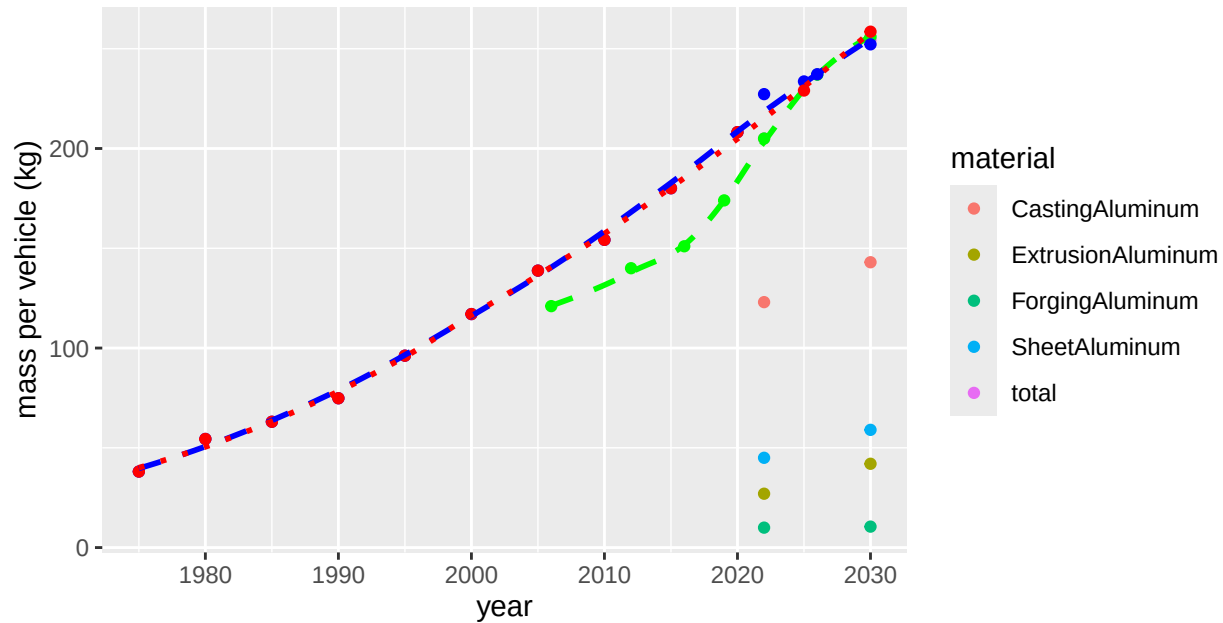


Combination of the different data set in a figure

The tables 1, 2 and 4 are combined to one figure. table one shows the different type of aluminum productions and their total in 2022 and 2030. Table 2 is shown in green with the total amount of aluminum from 2006 to 2030 with a smooth (loess) line to display the changes over time. The tidiness of the fit could be adjusted. Currently it is close to all the observed data points. The 2020 values from 1075 to 2030 are shown in blue and the 2023 in red. Only the numbers for the prediction differ slightly. The overall smooth line is a very good approximation of the overall amount of aluminum in a car. But there are some differences to the numbers provided with table 2. But they are anyway in the margins of uncertainties.

```
g <- ggplot() +  
  geom_point(data = aluminum_content_1, mapping = aes(x = year, y = mass, colour = material)) +  
  geom_point(data = aluminum_content_2, mapping = aes(x = year, y = totalMass), colour = "green") +  
  geom_smooth(data = aluminum_content_2, mapping = aes(x = year, y = totalMass), method = "loess", se = FALSE, linetype = "dashed", colour = "green") +  
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), color = "blue") +  
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2020), color = "red") +  
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), method = "loess", se = FALSE, linetype = "dashed", colour = "blue") +  
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2020), method = "loess", se = FALSE, linetype = "dotted", colour = "red") +  
  ggtitle("aluminum - data content 1, 2 (green), 4 (23 blue, 20 red)") +  
  scale_y_continuous(name = "mass per vehicle (kg)") +  
  coord_fixed(ratio = 0.15) # Adjust the ratio as needed  
  
plot(g)
```

aluminum – data content 1, 2 (green), 4 (23 blue, 20 red)



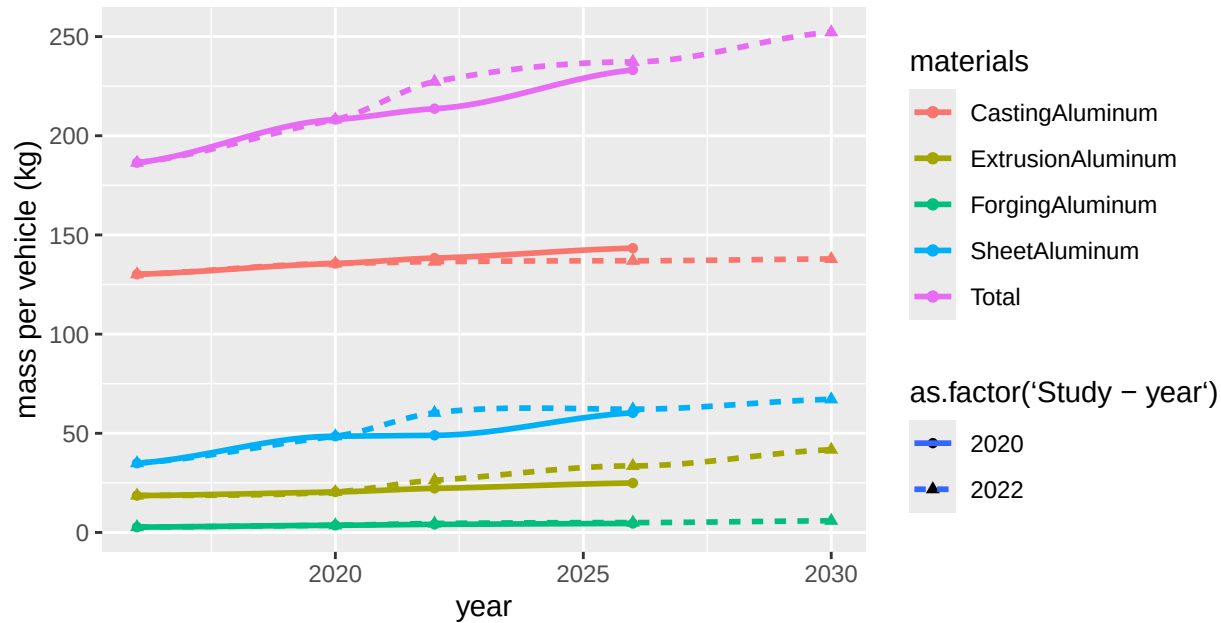
aluminum of table 5 as figure

The data of table 5 show the different types of aluminum with cast, extrusion, sheet, and forging. It is also combined to a total. The straight line and the points show the 2020 data, whereas the 2022 data are shown as dashed line and the actual data points as triangles.

```
g <- ggplot() +
  geom_point(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, shape = as.factor(`Study - year`))) +
  geom_smooth(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, linetype = as.factor(`Study - year`)),
    method = "loess", se = FALSE) +
  ggtitle("aluminum - data content 5") +
  scale_y_continuous(name = "mass per vehicle (kg)") +
  coord_fixed(ratio = 0.04) # Adjust the ratio as needed

plot(g)
```

aluminum – data content 5



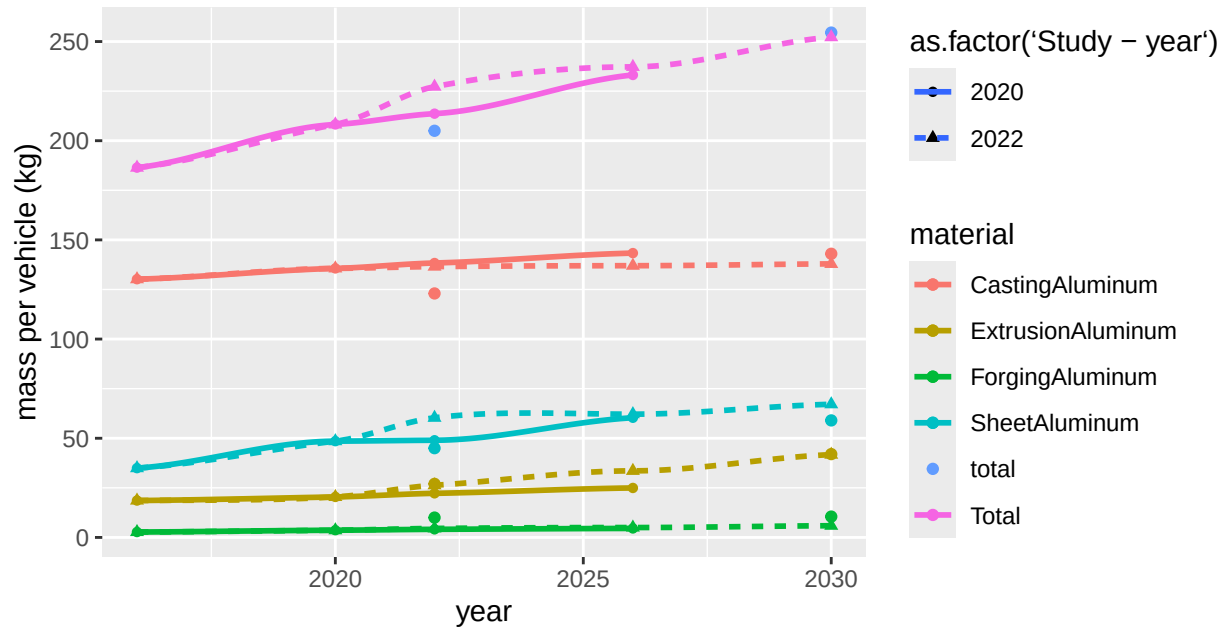
Combination of table 1 & 5 shown as a figure

The data of the different types of aluminum of table 1 and table 5 for the different years have been combined. It is shown in the following figure.

```
g <- ggplot() +
  geom_point(data = aluminum_content_1, mapping = aes(x = year, y = mass, colour = material)) +
  geom_point(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, shape = as.factor(`Study - year`))) +
  geom_smooth(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, linetype = as.factor(`Study - year`)),
    method = "loess", se = FALSE) +
  ggtitle("aluminum - data content 1 & 5") +
  scale_y_continuous(name = "mass per vehicle (kg)") +
  coord_fixed(ratio = 0.04) # Adjust the ratio as needed

plot(g)
```

aluminum – data content 1 & 5

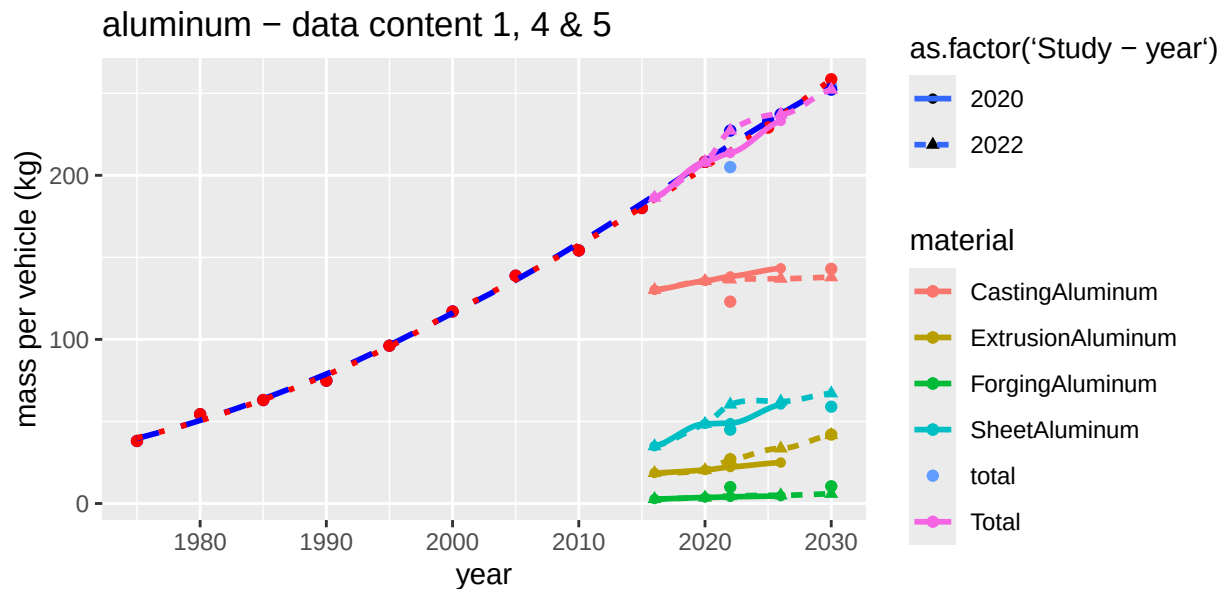


Combination of table 1, 4 & 5 shown as a figure

This figure shows the longterm behavior of the total amount of aluminum in a car from 1975 to 2030. It further includes the values for the different type of aluminum and their total from 2015 until 2030. It shows, that all the total amount of aluminum from the different studies over the years combine well. Furthermore the trends of the different aluminum types differ for some cases, whereas for other cases they agree well. It means that there are differences for the cast, sheet and extrusion aluminum between the estimates of 2020 and 2022.

```
g <- ggplot() +
  geom_point(data = aluminum_content_1, mapping = aes(x = year, y = mass, colour = material)) +
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2023, color = "blue")) +
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2020, color = "red")) +
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), method = "loess", se = FALSE, linetype = "dashed", colour = "blue") +
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2020), method = "loess", se = FALSE, linetype = "dotted", colour = "red") +
  geom_point(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, shape = as.factor('Study - year'))) +
  geom_smooth(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, linetype = as.factor('Study - year')),
    method = "loess", se = FALSE) +
  ggtitle("aluminum - data content 1, 4 & 5") +
  scale_y_continuous(name = "mass per vehicle (kg)") +
```

```
coord_fixed(ratio = 0.13) # Adjust the ratio as needed
plot(g)
```



Combine old data with all the available new data for standard car

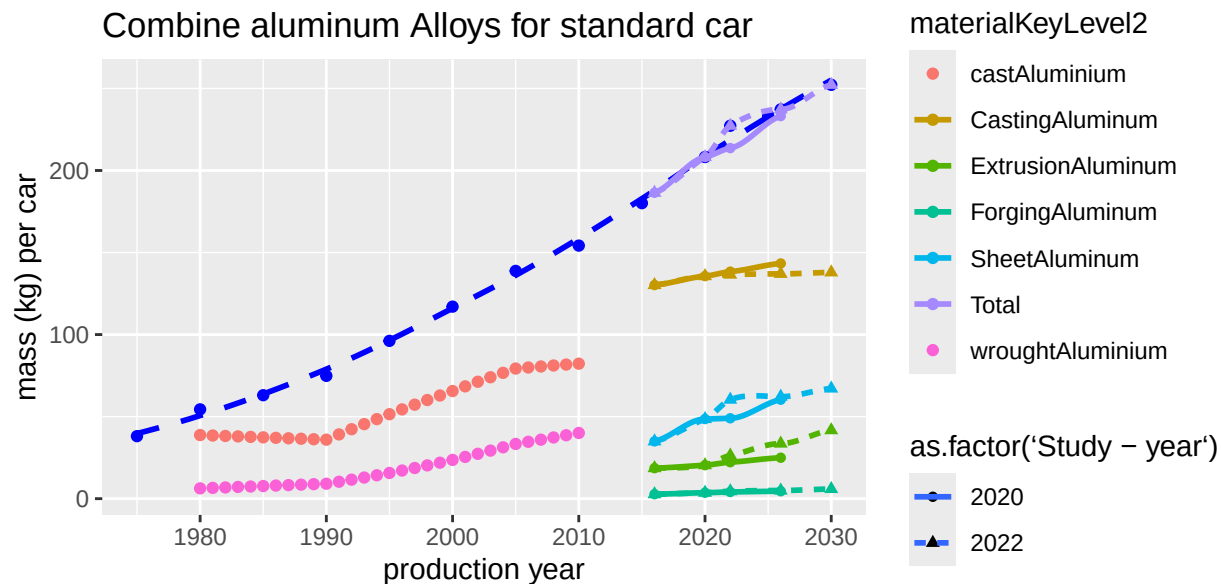
At the end all of the new data are combined with the data of the JRC study. There are some gaps between the JRC data, which last until 2010 and the new data, which begin in 2016. If the steep slope of the JRC cast aluminum is continued, then there seems to be some connection, which then flattens with the new 2022 numbers. The JRC wrought aluminum amounts seems to agree with the combined amount of the extrusion, forging and sheet aluminum combined amounts. It is difficult to judge, who to best connect both data sets.

```
temp <- JRC_Data %>% filter(materialKeyLevel1 == "AlAndAlAlloys" & is.na(element) & productKeyLevel2 == "V0101010000")

g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = as.numeric(productionYear), y = as.numeric(value), color = materialKeyLevel2)) +
  geom_point(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), color = "blue") +
  geom_smooth(data = aluminum_content_4, mapping = aes(x = year, y = Content2023), method = "loess", se = FALSE, linetype = "dashed", colour = "blue") +
  geom_point(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, shape = as.factor(`Study - year`))) +
  geom_smooth(data = aluminum_content_5_long, mapping = aes(x = year, y = mass, colour = materials, linetype = as.factor(`Study - year`)),
    method = "loess", se = FALSE) +
```

```
ggtitle("Combine aluminum Alloys for standard car") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.13) # Adjust the ratio as needed
```

```
plot(g)
```



Interpolate the values from 1980 to 2030 for totalaluminum for a standard car (V0101010000)

In a first step the total amount of aluminum used in a standard car are interpolated for each year using all the available data. For this purpose a loess fitting procedure in R is used. This allows to obtain the values for each year in the time span from 1980 to 2030.

```
# get the old data and generate long table for the interpolation
aluminum_content_4_long <- aluminum_content_4 %>% pivot_longer(cols = c("Content2023", "Content2020"), names_to = "study",
  values_to = "values", values_drop_na = TRUE)

#aluminum_content_4_long

# model the data
yearCalculate <- c(1975:2030)
yearCalculate <- as.integer(yearCalculate)
```

```

mAlttotal <- loess(values ~ year, data = aluminum_content_4_long, span = 0.85)
valueCalculate <- predict(mAlttotal, yearCalculate)
temp <- valueCalculate # this is necessary, but I have no idea why

totalAlCaluclate.standard <- data.frame(cbind(year = yearCalculate, value = temp))
totalAlCaluclate.standard <- totalAlCaluclate.standard %>% add_column(segment = "standard")
#totalAlCaluclate.standard

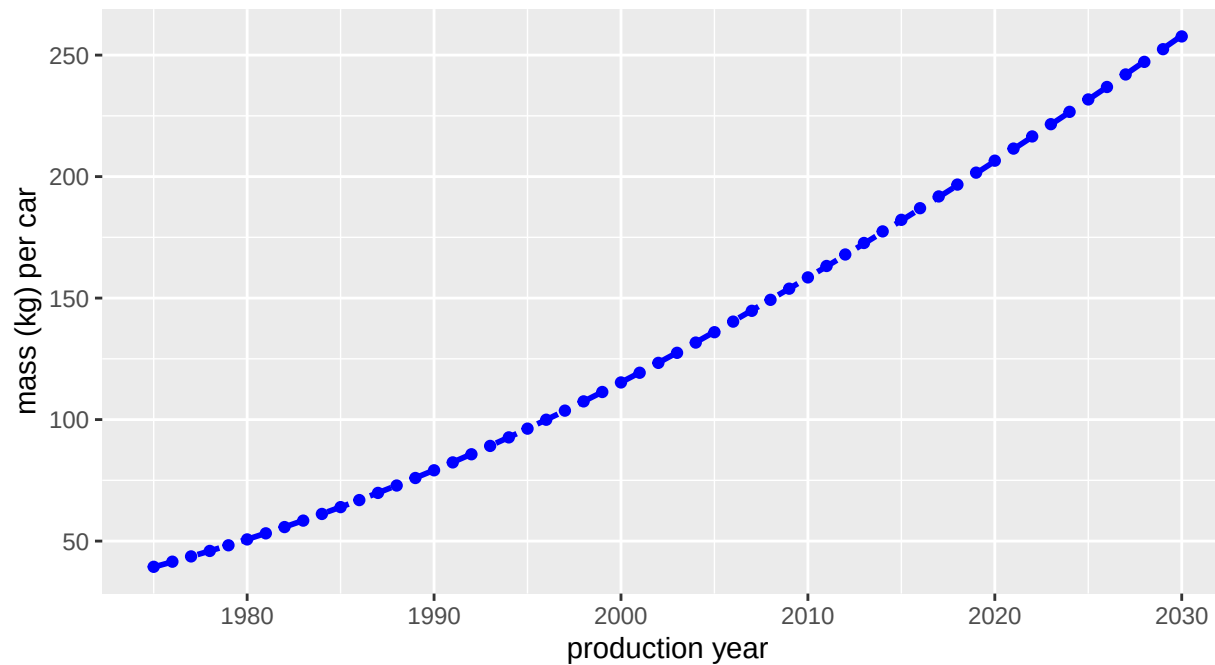
# the data are shown in a figure with the interpolated vaules as a point and the loess function as a dashed line

g <- ggplot() +
  geom_point(data = totalAlCaluclate.standard, mapping = aes(x = year, y = value), color = "blue") +
  geom_smooth(data = totalAlCaluclate.standard, mapping = aes(x = year, y = value), method = "loess", se = FALSE,
    linetype = "dashed", colour = "blue") +
  ggtitle("Total of all aluminum alloys in a standard car (interpolated)") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.13) # Adjust the ratio as needed

plot(g)

```

Total of all aluminum alloys in a standard car (interpolated)



```
# Start with 1980
totalAlCaluclate.standard <- totalAlCaluclate.standard %>% filter(year >= startDate)
#totalAlCaluclate.standard
```

Interpolate the values from 1980 to 2030 for extrusion, forging and sheet aluminum alloys (wrought aluminum) for a standard car (V0101010000)

The different type of wrought Aluminum productions are looked for their amounts in standard cars over the investigated time period. The issue is, that there are hardly any numbers before the year 2015 for the different types. Only the overall wrought aluminum is listed in older studies.

Extrusion aluminum The available data points are best modeled with a linear regression. Therefore it seems appropriated to extend the linear model approach for the years before 2016, where the first data are available. Using this linear approach the extrusion values become 0 before the year 2005, which seems reasonable.

```

yearCalculate <- c(startDate:2030)

# model the values

mAlExtrusion <- lm(ExtrusionAluminum ~ year, data = aluminum_content_5)
#summary(mAlExtrusion)

valueCalculate <- yearCalculate * mAlExtrusion$coefficients[2] + mAlExtrusion$coefficients[1]
extrusionAlCalculate.standard <- data.frame(cbind(year = yearCalculate, value = valueCalculate))
extrusionAlCalculate.standard <- extrusionAlCalculate.standard %>% add_column(segment = "standard")

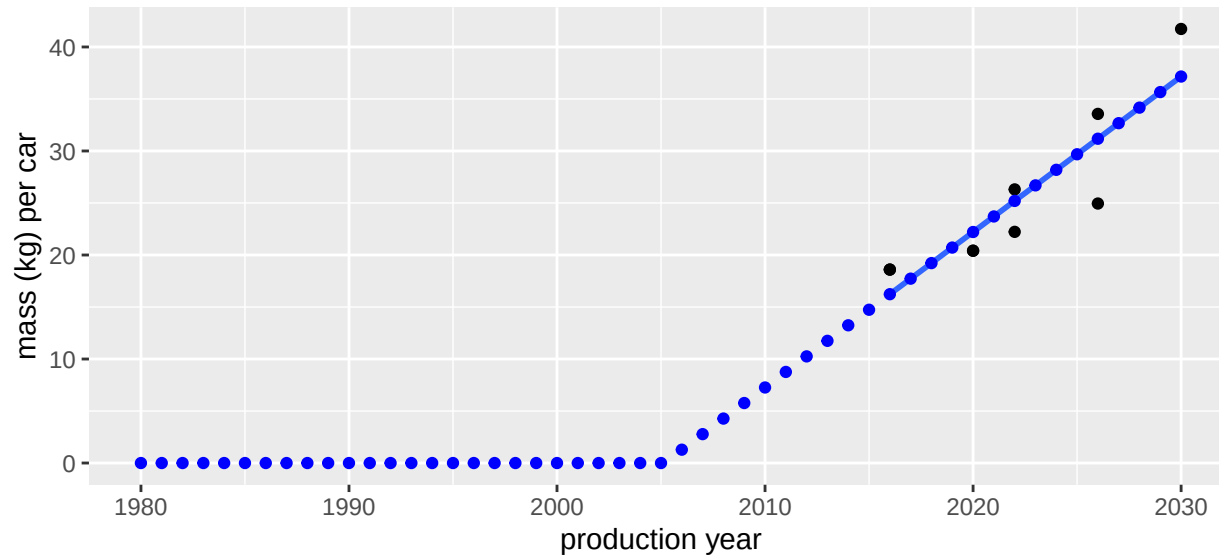
# get ride of all values < 0
extrusionAlCalculate.standard <- extrusionAlCalculate.standard %>% mutate(value = if_else(value < 0, 0, value))
#extrusionAlCalculate.standard

g <- ggplot() +
  geom_point(data = aluminum_content_5, mapping = aes(x = year, y = ExtrusionAluminum)) +
  geom_smooth(data = aluminum_content_5, mapping = aes(x = year, y = ExtrusionAluminum), method='lm', se = FALSE) +
  geom_point(data = extrusionAlCalculate.standard, mapping = aes(x = year, y = value), colour = "blue") +
  ggtitle("Extrusion aluminum alloys for standard car (linear model)") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.5) # Adjust the ratio as needed

plot(g)

```

Extrusion aluminum alloys for standard car (linear model)



Forgingaluminum The available data points are best modeled with a linear regression. Therefore it seems appropriated to extend the linear model approach for the years before 2016, where the first data are available. Using this linear approach the extrusion values become 0 before the year 2003, which seems reasonable.

```
yearCalculate <- c(startDate:2030)

# model the values

mAlForging <- lm(ForgingAluminum ~ year, data = aluminum_content_5)
#summary(mAlForging)

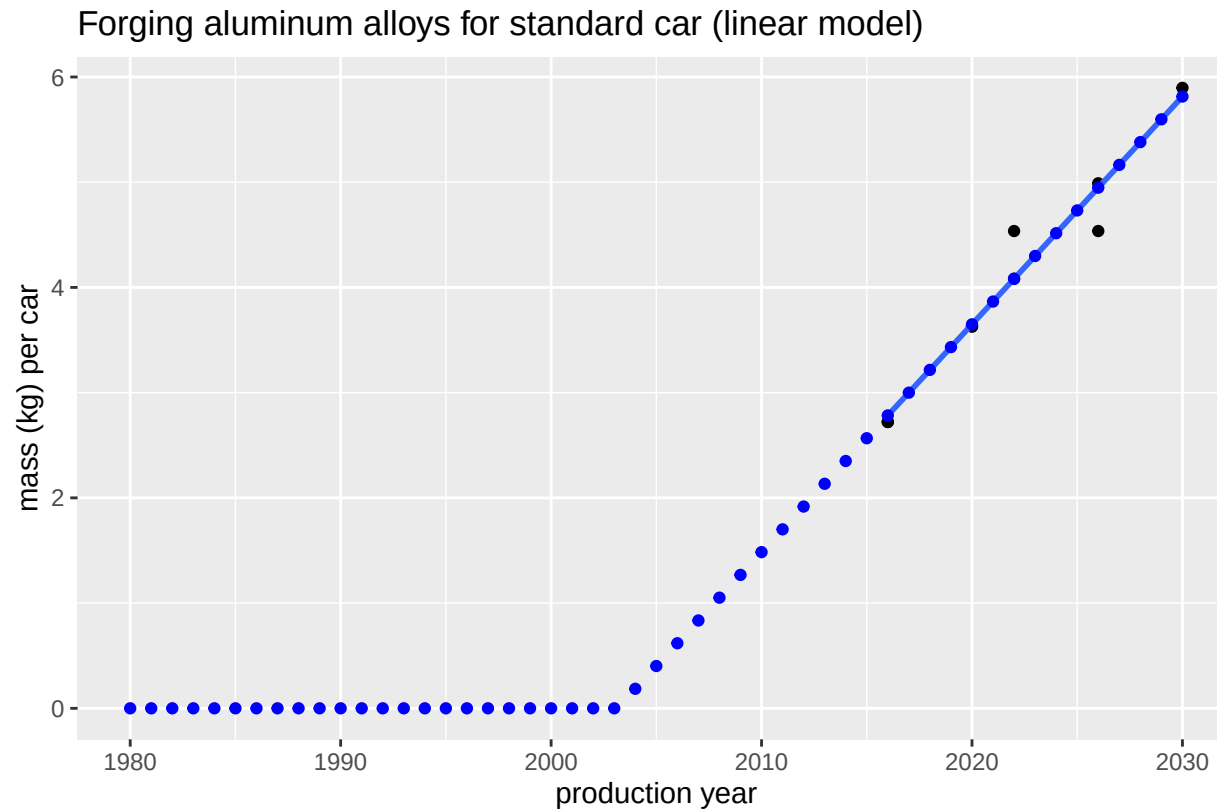
valueCalculate <- yearCalculate * mAlForging$coefficients[2] + mAlForging$coefficients[1]
forgingAlCaluclate.standard <- data.frame(cbind(year = yearCalculate, value = valueCalculate))
forgingAlCaluclate.standard <- forgingAlCaluclate.standard %>% add_column(segment = "standard")

# get ride of all values < 0
forgingAlCaluclate.standard <- forgingAlCaluclate.standard %>% mutate(value = if_else(value < 0, 0, value))
#forgingAlCaluclate.standard

g <- ggplot() +
  geom_point(data = aluminum_content_5, mapping = aes(x = year, y = ForgingAluminum)) +
  geom_smooth(data = aluminum_content_5, mapping = aes(x = year, y = ForgingAluminum), method='lm', se = FALSE) +
```

```
geom_point(data = forgingAlCaluclate.standard, mapping = aes(x = year, y = value), colour = "blue") +
ggtitle("Forging aluminum alloys for standard car (linear model)") +
xlab("production year") +
ylab("mass (kg) per car") +
coord_fixed(ratio = 5.0) # Adjust the ratio as needed
```

plot(g)



Sheetaluminum The available data points are best modeled with a linear regression. Therefore it seems appropriated to extend the linear model approach for the years before 2016, where the first data are available. Using this linear approach the extrusion values become 0 before the year 2000, which seems reasonable.

```

yearCalculate <- c(startDate:2030)

# model the values

mAlSheet <- lm(SheetAluminum ~ year, data = aluminum_content_5)
#summary(mAlSheet)

valueCalculate <- yearCalculate * mAlSheet$coefficients[2] + mAlSheet$coefficients[1]
sheetAlCaluclate.standard <- data.frame(cbind(year = yearCalculate, value = valueCalculate))
sheetAlCaluclate.standard <- sheetAlCaluclate.standard %>% add_column(segment = "standard")

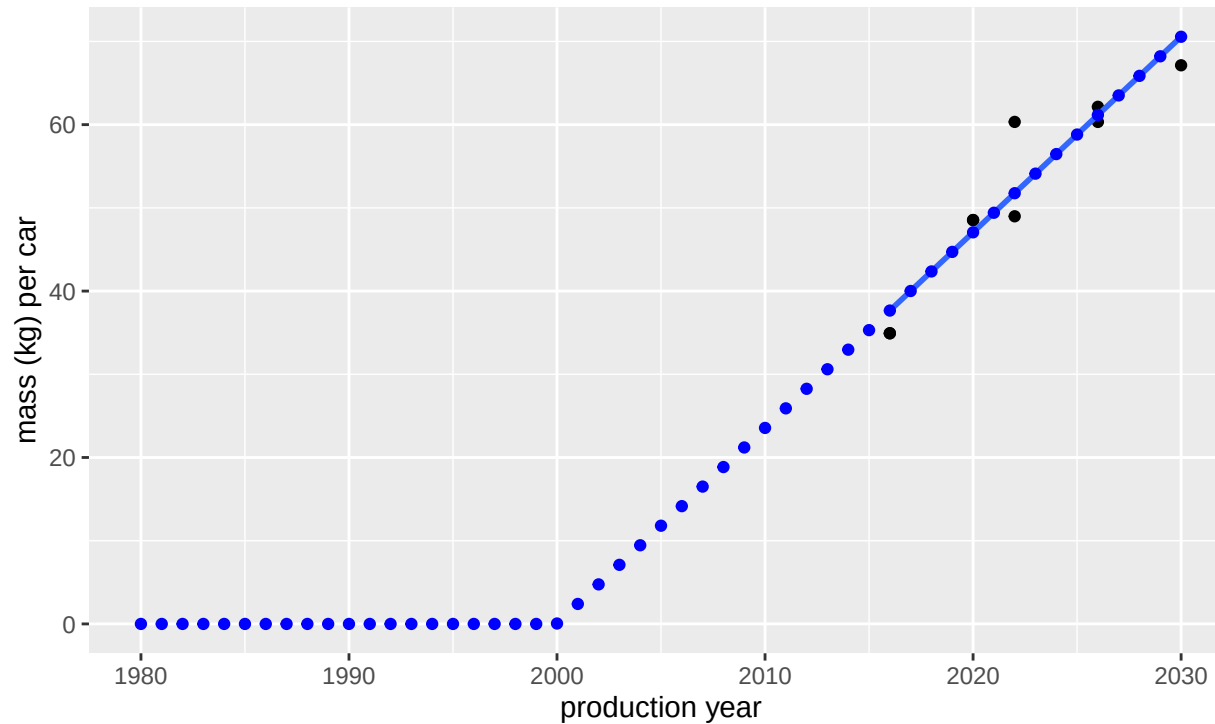
# get ride of all values < 0
sheetAlCaluclate.standard <- sheetAlCaluclate.standard %>% mutate(value = if_else(value < 0, 0, value))
#sheetAlCaluclate.standard

g <- ggplot() +
  geom_point(data = aluminum_content_5, mapping = aes(x = year, y = SheetAluminum)) +
  geom_smooth(data = aluminum_content_5, mapping = aes(x = year, y = SheetAluminum), method='lm', se = FALSE) +
  geom_point(data = sheetAlCaluclate.standard, mapping = aes(x = year, y = value), colour = "blue") +
  ggtitle("Sheet aluminum alloys for standard car (linear model)") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.4) # Adjust the ratio as needed

plot(g)

```

Sheet aluminum alloys for standard car (linear model)



The JRC data show some small amounts of wrought aluminum in cars. The amount is below 20 kg. According to the article “Automotive Aluminum Challenges Steel - As aluminum manufacturers keep pushing for increased market share in automotive design and content, their efforts may create some changes on the recycling end.” (<https://www.recyclingtoday.com/article/automotive-aluminum-challenges-steel/>), which was published in August 2001, aluminum has grown slowly to about 12% of the car weight. This is mainly cast aluminum, whereas wrought aluminum application were just introduced for the E and F segment type of cars.

castaluminum for a standard car (“V0101010000”) based on the total amount of aluminum and wrought (forging, extrusion & sheet) aluminum from 1980 - 2030

There is a extended data set for the whole amount of aluminum in cars and also the amount of the wrought aluminum are know and modeled. The amount of cast aluminum is the difference between the two values. This can be verified with independent data for cast aluminum.

```

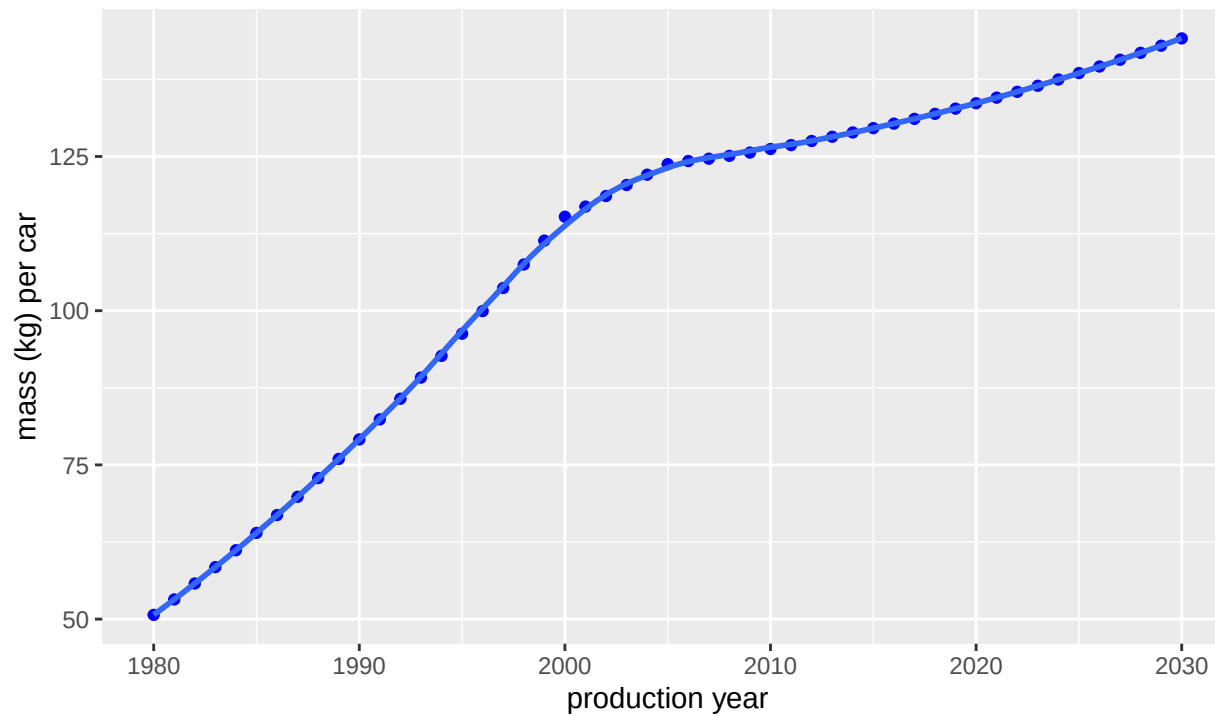
# get the total data
castingAlCalculate.standard <- totalAlCaluclate.standard %>% mutate(value = value - extrusionAlCalculate.standard$value - forgingAlCaluclate.standard$value - sheetAlCaluclate.standard$value)
#castingAlCalculate.standard

# plot the results
g <- ggplot() +
  geom_point(data = castingAlCalculate.standard, mapping = aes(x = year, y = value), colour = "blue") +
  geom_smooth(data = castingAlCalculate.standard, mapping = aes(x = year, y = value), method = "loess", se = FALSE, span = 0.4) +
  ggtitle("Casting aluminum alloys for standard car (smooth loess model)") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.3) # Adjust the ratio as needed

plot(g)

```

Casting aluminum alloys for standard car (smooth loess model)



```
# Check casting aluminum to total weight in 1980 and 2010
castingAlCalculate.standard_1980 <- castingAlCalculate.standard %>% filter(year == 1980)
castingAlCalculate.standard_2010 <- castingAlCalculate.standard %>% filter(year == 2010)

# Calculate the ratio
ratiocastingAl_1980 <- castingAlCalculate.standard_1980$value / startWeightCar
ratiocastingAl_2010 <- castingAlCalculate.standard_2010$value / startWeightCar

#ratiocastingAl_1980
#ratiocastingAl_2010
```

The smooth line does not represent the individual values for each year. It is just an approximation of the individual values shown as points and which are used.

Testing the increase in ratio of casting Al from 1980 from 4.6% to 11.5% in 2010 is in line with the overall weight increase. Therefore it is justified to not use the old JRC data, as they do not reflect the actual behavior over this timeperiod. Furthermore, they also ignores that a standard car weight increase between 1980 and 2010 by 17 kg per year.

Combination of all aluminum Alloys into one file and figure (1980 - 2030)

Combined data set of a standard car (1980 - 2030)

All the data of the different aluminum alloys are combined into one single data set. Currently there are only the data of the standard car, which is an average car between an C and D segment car.

```
# combining the data

AlCalculate.standard <- left_join(castingAlCalculate.standard, extrusionAlCalculate.standard, by = c("year", "segment"))
AlCalculate.standard <- left_join(AlCalculate.standard, forgingAlCalculate.standard, by = c("year", "segment"))
AlCalculate.standard <- left_join(AlCalculate.standard, sheetAlCalculate.standard, by = c("year", "segment"))

# renaming some of the columns

AlCalculate.standard <- AlCalculate.standard %>% rename("castingValue" = "value.x", "extrusionValue" = "value.y",
  "forgingValue" = "value.x.x", "sheetValue" = "value.y.y")

# calculate the total amount of aluminum in the data set

AlCalculate.standard <- AlCalculate.standard %>% mutate(combinedValue = castingValue + extrusionValue + forgingValue + sheetValue)

# do some rearrangement of the data set structure

AlCalculate.standard <- AlCalculate.standard %>% relocate(segment, .after = last_col())
```

```

#AlCalculate.V0101010000

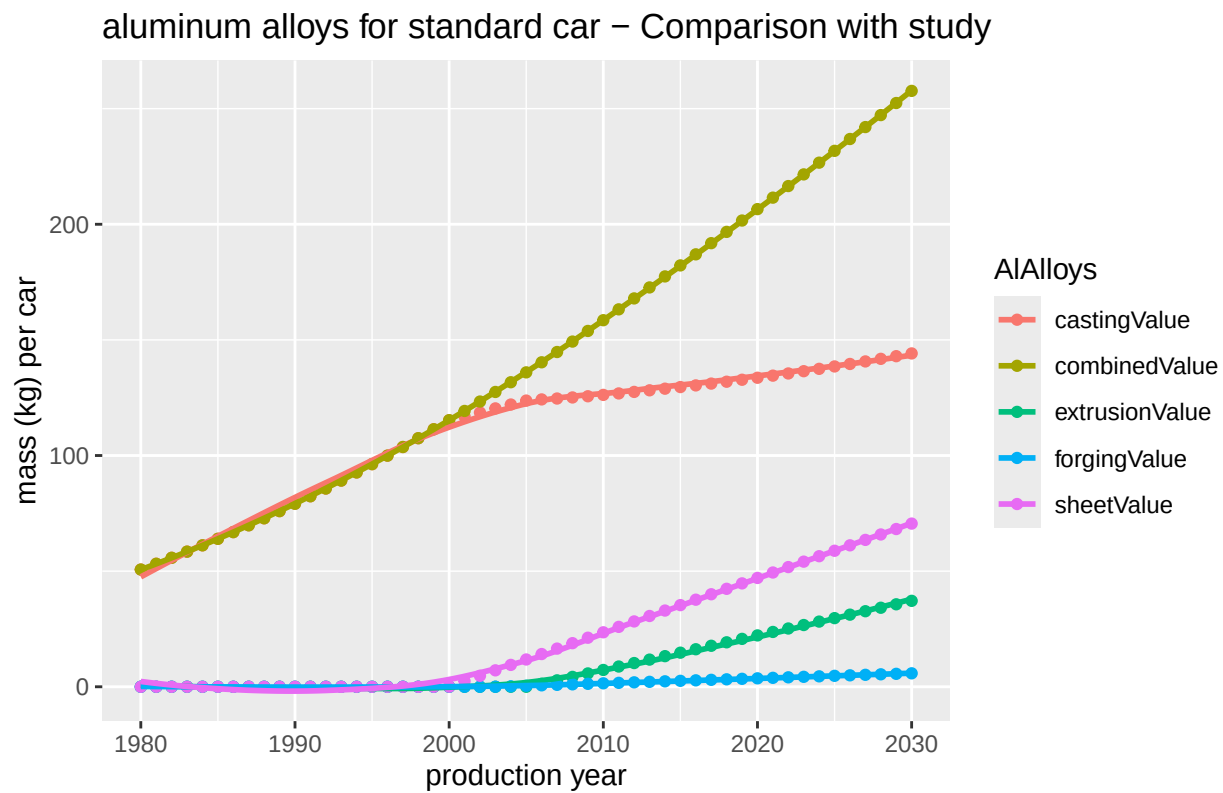
# generate long table, so that all the data in the data set can be shown in a figure

AlCalculate.standard_long <- AlCalculate.standard %>% pivot_longer(!c(year, segment), names_to = "AlAlloys", values_to = "values")
#AlCalculate.V0101010000_long

g <- ggplot() +
  geom_point(data = AlCalculate.standard_long, mapping = aes(x = year, y = values, color = AlAlloys)) +
  geom_smooth(data = AlCalculate.standard_long, mapping = aes(x = year, y = values, color = AlAlloys), method = "loess", se = FALSE) +
  ggtitle("aluminum alloys for standard car - Comparison with study") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.15) # Adjust the ratio as needed

plot(g)

```



There is a steady increase of the combined or overall amounts of aluminum in a standard car. The increase of cast aluminum is decreasing considerable with the increase of the wrought aluminum contribution with a nearly equal contribution. The figure further implies that there is a continues increase of aluminum share in cars. A proper modeling of this behavior after 2030 because necessary, as this increase can not continue in a similar way into the further future.

Calculate the ration between the different alloys and the combined amounts

In a first approximation it is assumed that the ratio of the different aluminum alloys stays the same across all the different car segments.

```
# prepare a new data frame
ratioAl.standard <- AlCalculate.standard

# calculate the ratios

ratioAl.standard <- ratioAl.standard %>% group_by(year) %>% mutate(ratioCasting = castingValue/combinedValue)
ratioAl.standard <- ratioAl.standard %>% group_by(year) %>% mutate(ratioExtrusion = extrusionValue/combinedValue)
ratioAl.standard <- ratioAl.standard %>% group_by(year) %>% mutate(ratioForging = forgingValue/combinedValue)
ratioAl.standard <- ratioAl.standard %>% group_by(year) %>% mutate(ratioSheet = sheetValue/combinedValue)

# Calculate the ratio of the combined values as a sum of all the ratios of the individual ratios. It has to be 1 and test, if there is any problem concerning the individual ratios.

ratioAl.standard <- ratioAl.standard %>% group_by(year) %>% mutate(ratioCombined = ratioCasting + ratioExtrusion + ratioForging + ratioSheet)

# rearrange the data.frame, so that it is consitent and better to understand

ratioAl.standard <- ratioAl.standard %>% relocate(segment, .after = last_col())
#ratioAl.standard
```

Display the total amount of aluminum Alloy according to the car segment

Aluminum table 3

The file filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx" with the sheet "aluminum_content_3" is based on the following report and data.'Aluminum Content in Passenger Vehicles (Europe) Assessment 2022 and Outlook 2026, 2030 - Public Summary - April, 2023 (2023_04_Aluminum-Content_Ducker-Study_EA-Public-Summary_190423.pdf) page 24 left. It has been derived from the numbers in the figure and the comments.The length information is shown on page 19 right side top.

Aluminum table 6

The file filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx" with the sheet "aluminum_content_6" is based on the following report and data.'Light Vehicle Aluminum Content and Outlook Study, Prepared for the Aluminum Association by Ducker April 2023 (Ducker-ATG-2023-Summary-

Report-April-2023.pdf) page 23 right side.

Aluminum table 7

The file filename1_inputs <- "Data/1_VehicleCompositionNewData.xlsx" with the sheet "aluminum_content_7" is based on the following report and data.'2015 North American Light Vehicle Aluminum Content Study April - Executive Summary June 2014 (2015-Ducker-Study.pdf) page 9.

```
# Ducker list the values in kg
# aluminum_content_3

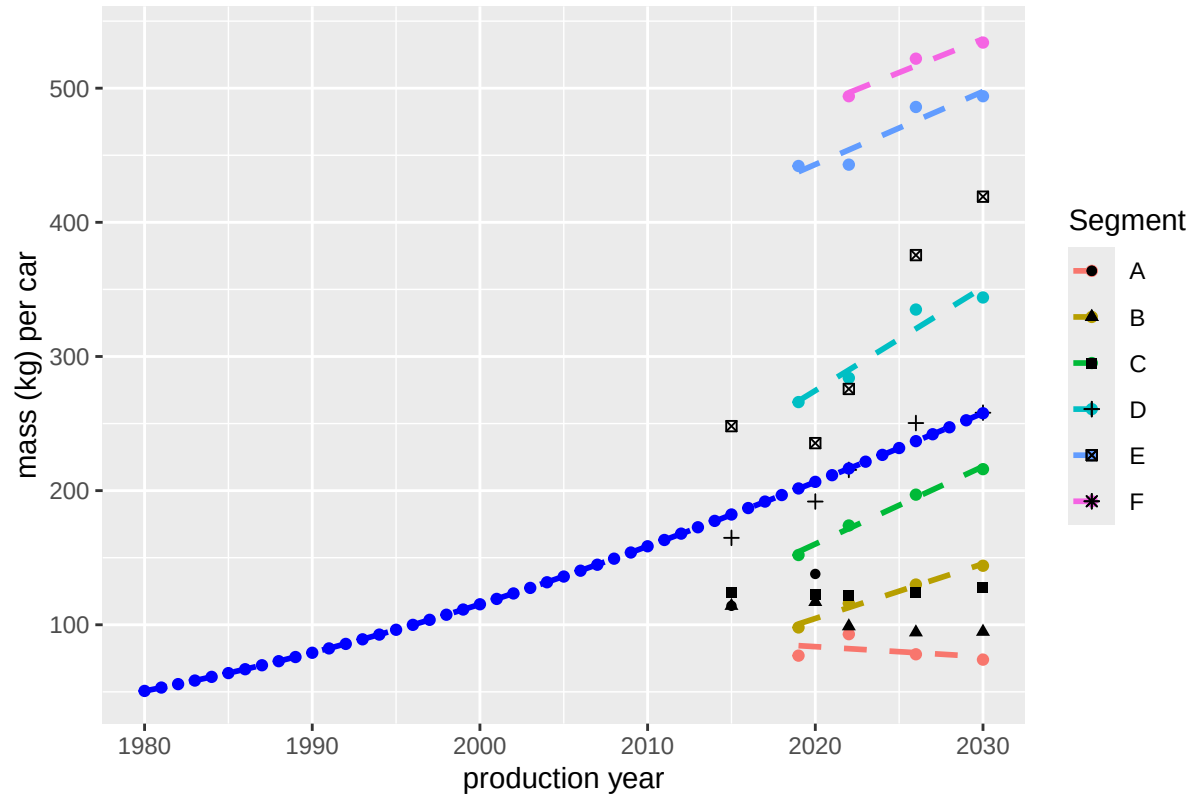
# Ducker list is in lbs and therefore it has to be transfered to kg
aluminum_content_6$totalMass <- aluminum_content_6$totalMass (in lbs)` * lbTokg
#aluminum_content_6

# Ducker list is in lbs and therefore it has to be transfered to kg
aluminum_content_7$totalMass <- aluminum_content_7$totalMass (in lbs)` * lbTokg
#aluminum_content_7

# show them in a figure
g <- ggplot() +
  geom_point(data = aluminum_content_3, mapping = aes(x = year, y = totalMass, color = Segment)) +
  geom_smooth(data = aluminum_content_3, mapping = aes(x = year, y = totalMass, color = Segment), method = "lm", se = FALSE, linetype = "dashed") +
  geom_point(data = aluminum_content_6, mapping = aes(x = year, y = totalMass, shape = Segment)) +
  geom_point(data = aluminum_content_7, mapping = aes(x = year, y = totalMass, shape = Segment)) +
  geom_point(data = ratioAl.standard, mapping = aes(x = year, y = combinedValue), color = "blue") +
  geom_smooth(data = ratioAl.standard, mapping = aes(x = year, y = combinedValue), method = "loess", se = FALSE, linetype = "dashed", color = "blue") +
  ggtitle("Total aluminum alloys for different car segments") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.08) # Adjust the ratio as needed

plot(g)
```

Total aluminum alloys for different car segments



The figure shows, that the US data are not in line with the European data, even both recent studies were made by Ducker nearly at the same time. Therefore the segment based information will use only the data of aluminum_content_3.

Interpolate the values from 1980 to 2030 for different car type (length) of total aluminum

Segment A and JA car (V0101010101 & V0101010201)

The amount of aluminum in a segment A car is shown to be constant from 2020 to 2030. But the trend for the standard car from 1980 to 2030 show a steady increase. Therefore we have to assume that the amount of aluminum in a segment A car flows this trend, but seems to level of for this segment. Hence we will use a trend which follows the overall trend.

```

#
## car segment A (V0101010101)

yearCalculate <- c(startDate:2030)

# get all the values of car segment A
temp <- aluminum_content_3 %>% filter(Segment == "A")

# get the value of 2025 to adjust everything and bring it in line
totalAl2025 <- AlCalculate.standard %>% filter(year == 2025)

# model the values (we assume it is a straight line)
ratioSegmentA <- median(temp$totalMass) / totalAl2025$combinedValue

AlCalculate.A <- AlCalculate.standard %>%
  mutate(totalAlCalculate = combinedValue * ratioSegmentA, castingAlCalculate = castingValue * ratioSegmentA,
         extrusionAlCalculate = extrusionValue * ratioSegmentA, forgingAlCalculate = forgingValue * ratioSegmentA,
         sheetAlCalculate = sheetValue * ratioSegmentA, segment = "A") %>%
  select(-castingValue, -extrusionValue, -forgingValue, -sheetValue, -combinedValue)

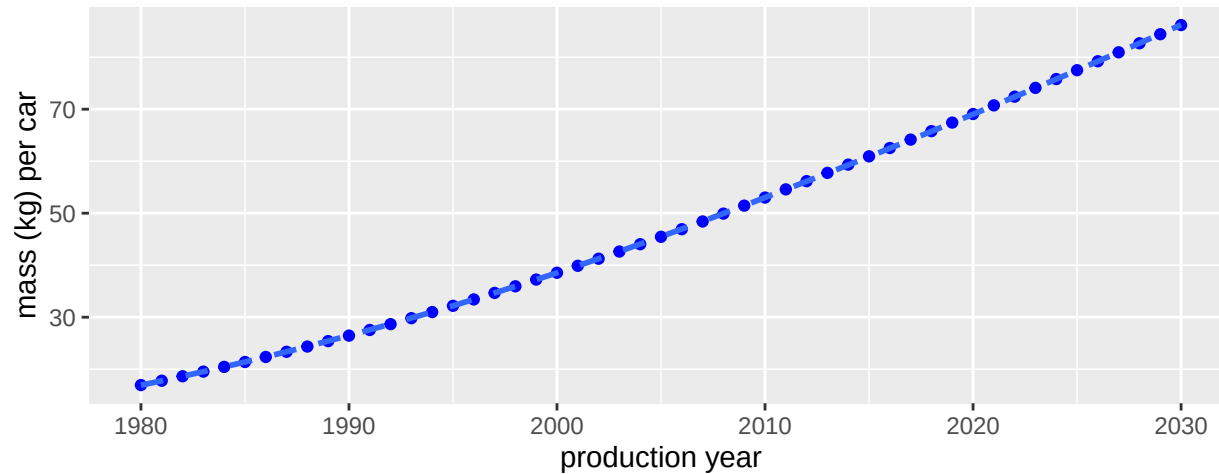
#AlCalculate.A

g <- ggplot() +
  geom_point(data = AlCalculate.A, mapping = aes(x = year, y = totalAlCalculate), color = "blue") +
  geom_smooth(data = AlCalculate.A, mapping = aes(x = year, y = totalAlCalculate), method = "loess", se = FALSE, linetype = "dashed") +
  ggtitle("Total aluminum alloys for segment A car") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.25) # Adjust the ratio as needed

plot(g)

```

Total aluminum alloys for segment A car



```
# Calculate the SUV composition
factor <- derivedSUVComposition %>% filter(segment == "A")

AlCalculate.JA <- AlCalculate.A %>%
  select(year, totalAlCalculate, castingAlCalculate, extrusionAlCalculate, forgingAlCalculate,
         sheetAlCalculate, segment) %>%
  mutate(year = year, totalAlCalculate = totalAlCalculate * factor$scaleFactor,
         castingAlCalculate = castingAlCalculate * factor$scaleFactor,
         extrusionAlCalculate = extrusionAlCalculate * factor$scaleFactor,
         forgingAlCalculate = forgingAlCalculate * factor$scaleFactor,
         sheetAlCalculate = sheetAlCalculate * factor$scaleFactor,
         segment = factor$segment_SUV
  )

#AlCalculate.JA
```

Segment B abd JB car (V0101010102 & V0101010202)

The implementation of the segment B cars follows the same tracks as for the A segment

```
#
## car segment B (V0101010102)

yearCalculate <- c(startDate:2030)
```

```

# get all the values of car segment B
temp <- aluminum_content_3 %>% filter(Segment == "B")

# get the value of 2025 to adjust everything and bring it in line
totalAl2025 <- AlCalculate.standard %>% filter(year == 2025)

# model the values (we assume it is a straight line)
ratioSegmentB <- median(temp$totalMass) / totalAl2025$combinedValue

AlCalculate.B <- AlCalculate.standard %>%
  mutate(totalAlCalculate = combinedValue * ratioSegmentB, castingAlCalculate = castingValue * ratioSegmentB,
         extrusionAlCalculate = extrusionValue * ratioSegmentB, forgingAlCalculate = forgingValue * ratioSegmentB,
         sheetAlCalculate = sheetValue * ratioSegmentB, segment = "B") %>%
  select(-castingValue, -extrusionValue, -forgingValue, -sheetValue, -combinedValue)

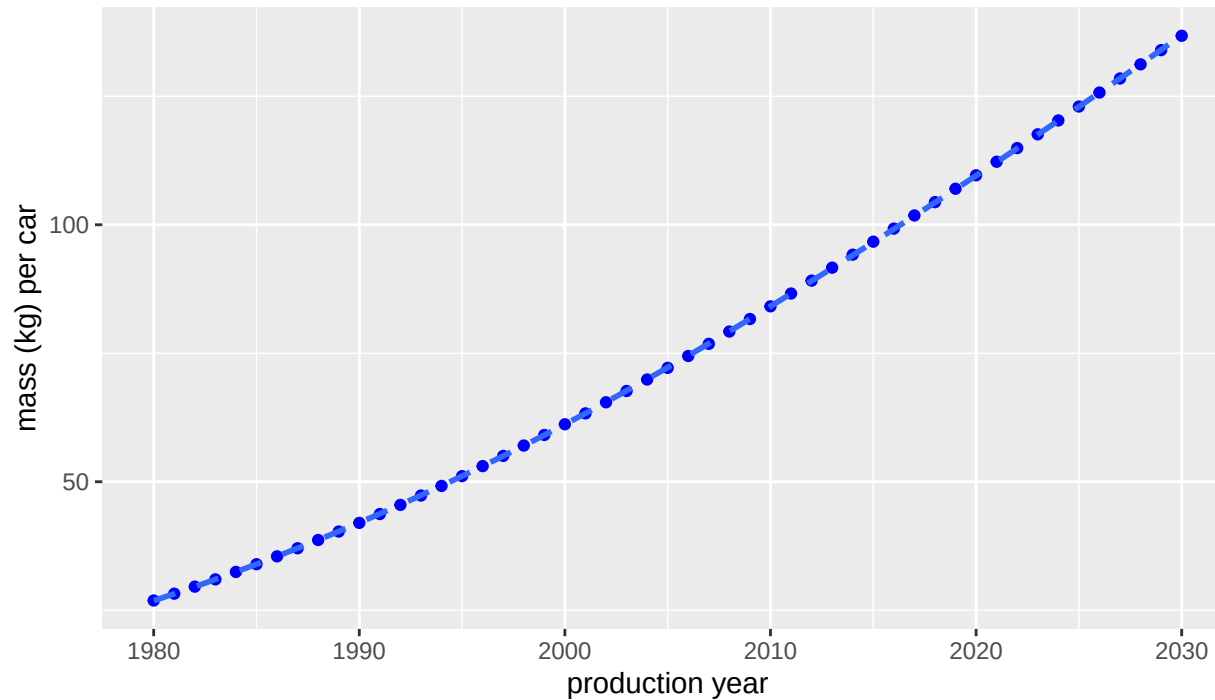
#AlCalculate.B

g <- ggplot() +
  geom_point(data = AlCalculate.B, mapping = aes(x = year, y = totalAlCalculate), color = "blue") +
  geom_smooth(data = AlCalculate.B, mapping = aes(x = year, y = totalAlCalculate), method = "loess", se = FALSE, linetype = "dashed") +
  ggtitle("Total aluminum alloys for segment B car") +
  xlab("production year") +
  ylab("mass (kg) per car") +
  coord_fixed(ratio = 0.25) # Adjust the ratio as needed

plot(g)

```

Total aluminum alloys for segment B car



```
# Calcualte the SUV composition
factor <- derivedSUVComposition %>% filter(segment == "B")

AlCalculate.JB <- AlCalculate.B %>%
  select(year, totalAlCalculate, castingAlCalculate, extrusionAlCalculate, forgingAlCalculate,
         sheetAlCalculate, segment) %>%
  mutate(year = year, totalAlCalculate = totalAlCalculate * factor$scaleFactor,
         castingAlCalculate = castingAlCalculate * factor$scaleFactor,
         extrusionAlCalculate = extrusionAlCalculate * factor$scaleFactor,
         forgingAlCalculate = forgingAlCalculate * factor$scaleFactor,
         sheetAlCalculate = sheetAlCalculate * factor$scaleFactor,
         segment = factor$segment_SUV
  )

#AlCalculate.JB
```

Segment C and JC car (V0101010103 & V0101010203)

The implementation of the segment C cars follows the same tracks as for all the segments

```
#  
## car segment C (V0101010103)  
  
yearCalculate <- c(startDate:2030)  
  
# get all the values of car segment C  
temp <- aluminum_content_3 %>% filter(Segment == "C")  
  
# get the value of 2025 to adjust everything and bring it in line  
totalAl2025 <- AlCalculate.standard %>% filter(year == 2025)  
  
# model the values (we assume it is a straight line)  
ratioSegmentC <- median(temp$totalMass) / totalAl2025$combinedValue  
  
AlCalculate.C <- AlCalculate.standard %>%  
  mutate(totalAlCalculate = combinedValue * ratioSegmentC, castingAlCalculate = castingValue * ratioSegmentC,  
         extrusionAlCalculate = extrusionValue * ratioSegmentC, forgingAlCalculate = forgingValue * ratioSegmentC,  
         sheetAlCalculate = sheetValue * ratioSegmentC, segment = "C") %>%  
  select(-castingValue, -extrusionValue, -forgingValue, -sheetValue, -combinedValue)  
  
#AlCalculate.C  
  
g <- ggplot() +  
  geom_point(data = AlCalculate.C, mapping = aes(x = year, y = totalAlCalculate), color = "blue") +  
  geom_smooth(data = AlCalculate.C, mapping = aes(x = year, y = totalAlCalculate), method = "loess", se = FALSE, linetype = "dashed") +  
  ggtitle("Total aluminum alloys for segment C car") +  
  xlab("production year") +  
  ylab("mass (kg) per car") +  
  coord_fixed(ratio = 0.25) # Adjust the ratio as needed  
  
plot(g)
```

Total aluminum alloys for segment C car



```
# Calcuale the SUV composition
factor <- derivedSUVComposition %>% filter(segment == "C")

AlCalculate.JC <- AlCalculate.C %>%
  select(year, totalAlCalculate, castingAlCalculate, extrusionAlCalculate, forgingAlCalculate,
    sheetAlCalculate, segment) %>%
  mutate(year = year, totalAlCalculate = totalAlCalculate * factor$scaleFactor,
    castingAlCalculate = castingAlCalculate * factor$scaleFactor,
    extrusionAlCalculate = extrusionAlCalculate * factor$scaleFactor,
    forgingAlCalculate = forgingAlCalculate * factor$scaleFactor,
    sheetAlCalculate = sheetAlCalculate * factor$scaleFactor,
    segment = factor$segment_SUV
  )

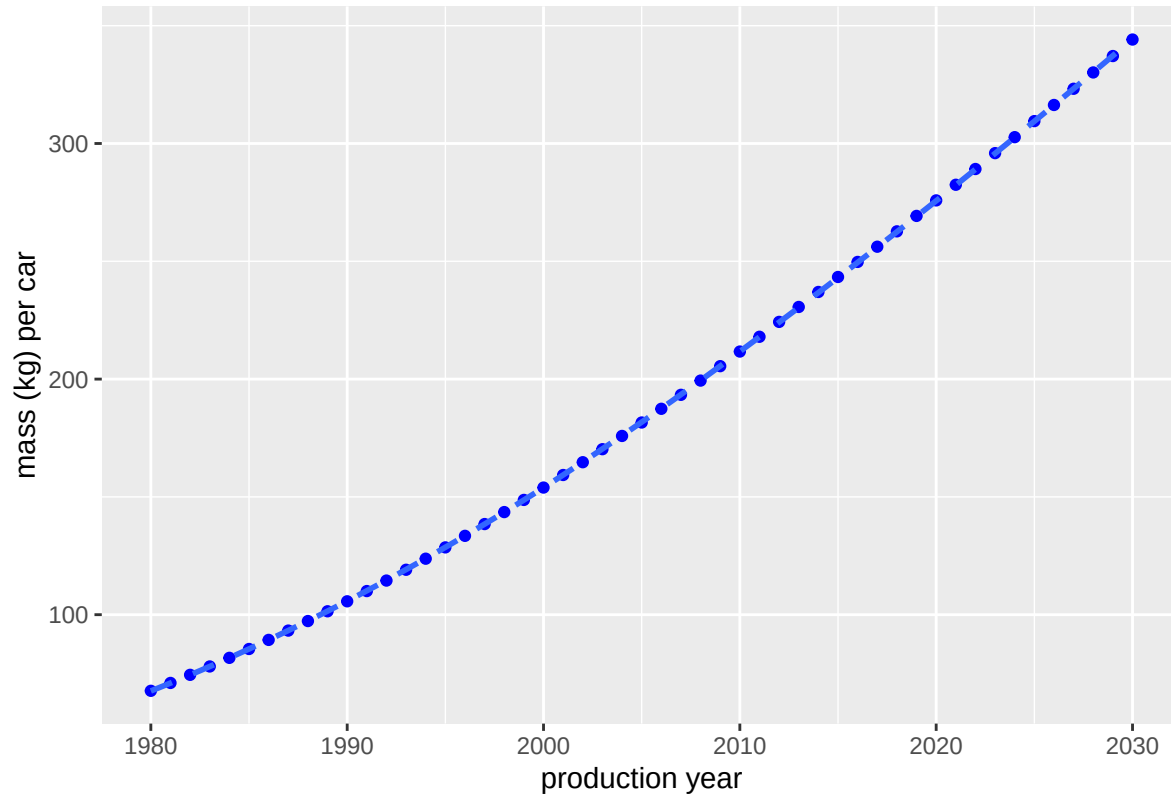
#AlCalculate.JC
```

Segment D and JD car (V0101010104 & V0101010204)

The implementation of the segment D cars follows the same tracks as for all the segments

```
#  
## car segment D (V01010104)  
  
yearCalculate <- c(startDate:2030)  
  
# get all the values of car segment D  
temp <- aluminum_content_3 %>% filter(Segment == "D")  
  
# get the value of 2025 to adjust everything and bring it in line  
totalAl2025 <- AlCalculate.standard %>% filter(year == 2025)  
  
# model the values (we assume it is a straight line)  
ratioSegmentD <- median(temp$totalMass) / totalAl2025$combinedValue  
  
AlCalculate.D <- AlCalculate.standard %>%  
  mutate(totalAlCalculate = combinedValue * ratioSegmentD, castingAlCalculate = castingValue * ratioSegmentD,  
    extrusionAlCalculate = extrusionValue * ratioSegmentD, forgingAlCalculate = forgingValue * ratioSegmentD,  
    sheetAlCalculate = sheetValue * ratioSegmentD, segment = "D") %>%  
  select(-castingValue, -extrusionValue, -forgingValue, -sheetValue, -combinedValue)  
  
#AlCalculate.D  
  
g <- ggplot() +  
  geom_point(data = AlCalculate.D, mapping = aes(x = year, y = totalAlCalculate), color = "blue") +  
  geom_smooth(data = AlCalculate.D, mapping = aes(x = year, y = totalAlCalculate), method = "loess", se = FALSE, linetype = "dashed") +  
  ggtitle("Total aluminum alloys for segment D car") +  
  xlab("production year") +  
  ylab("mass (kg) per car") +  
  coord_fixed(ratio = 0.12) # Adjust the ratio as needed  
  
plot(g)
```

Total aluminum alloys for segment D car



```
# Calcualte the SUV composition
factor <- derivedSUVComposition %>% filter(segment == "D")

AlCalculate.JD <- AlCalculate.D %>%
  select(year, totalAlCalculate, castingAlCalculate, extrusionAlCalculate, forgingAlCalculate,
    sheetAlCalculate, segment) %>%
  mutate(year = year, totalAlCalculate = totalAlCalculate * factor$scaleFactor,
    castingAlCalculate = castingAlCalculate * factor$scaleFactor,
    extrusionAlCalculate = extrusionAlCalculate * factor$scaleFactor,
    forgingAlCalculate = forgingAlCalculate * factor$scaleFactor,
    sheetAlCalculate = sheetAlCalculate * factor$scaleFactor,
    segment = factor$segment_SUV
  )

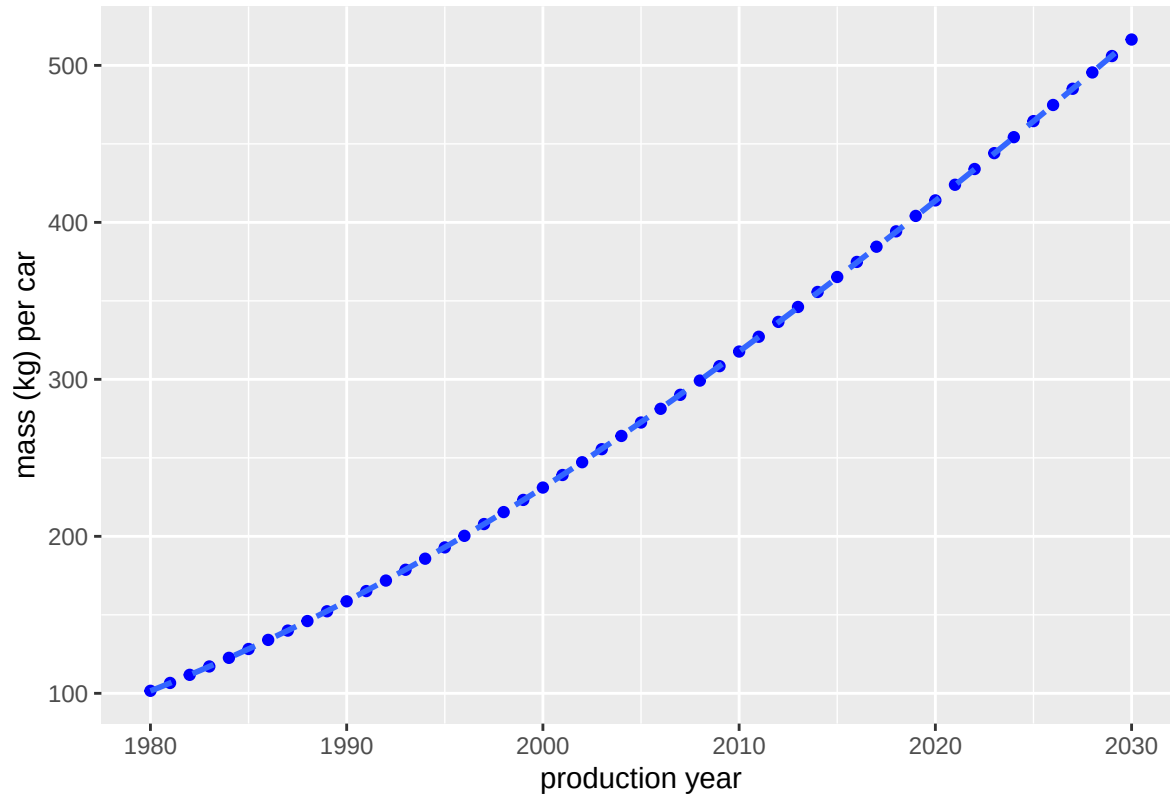
#AlCalculate.JD
```

Segment E and JE car (V0101010105 & V0101010205)

The implementation of the segment E cars follows the same tracks as for all the segments

```
#  
## car segment E (V0101010105)  
  
yearCalculate <- c(startDate:2030)  
  
# get all the values of car segment E  
temp <- aluminum_content_3 %>% filter(Segment == "E")  
  
# get the value of 2025 to adjust everything and bring it in line  
totalAl2025 <- AlCalculate.standard %>% filter(year == 2025)  
  
# model the values (we assume it is a straight line)  
ratioSegmentE <- median(temp$totalMass) / totalAl2025$combinedValue  
  
AlCalculate.E <- AlCalculate.standard %>%  
  mutate(totalAlCalculate = combinedValue * ratioSegmentE, castingAlCalculate = castingValue * ratioSegmentE,  
         extrusionAlCalculate = extrusionValue * ratioSegmentE, forgingAlCalculate = forgingValue * ratioSegmentE,  
         sheetAlCalculate = sheetValue * ratioSegmentE, segment = "E") %>%  
  select(-castingValue, -extrusionValue, -forgingValue, -sheetValue, -combinedValue)  
  
#AlCalculate.E  
  
g <- ggplot() +  
  geom_point(data = AlCalculate.E, mapping = aes(x = year, y = totalAlCalculate), color = "blue") +  
  geom_smooth(data = AlCalculate.E, mapping = aes(x = year, y = totalAlCalculate), method = "loess", se = FALSE, linetype = "dashed") +  
  ggtitle("Total aluminum alloys for segment E car") +  
  xlab("production year") +  
  ylab("mass (kg) per car") +  
  coord_fixed(ratio = 0.08) # Adjust the ratio as needed  
  
plot(g)
```

Total aluminum alloys for segment E car



```
# Calculate the SUV composition
factor <- derivedSUVComposition %>% filter(segment == "E")

AlCalculate.JE <- AlCalculate.E %>%
  select(year, totalAlCalculate, castingAlCalculate, extrusionAlCalculate, forgingAlCalculate,
    sheetAlCalculate, segment) %>%
  mutate(year = year, totalAlCalculate = totalAlCalculate * factor$scaleFactor,
    castingAlCalculate = castingAlCalculate * factor$scaleFactor,
    extrusionAlCalculate = extrusionAlCalculate * factor$scaleFactor,
    forgingAlCalculate = forgingAlCalculate * factor$scaleFactor,
    sheetAlCalculate = sheetAlCalculate * factor$scaleFactor,
    segment = factor$segment_SUV
  )

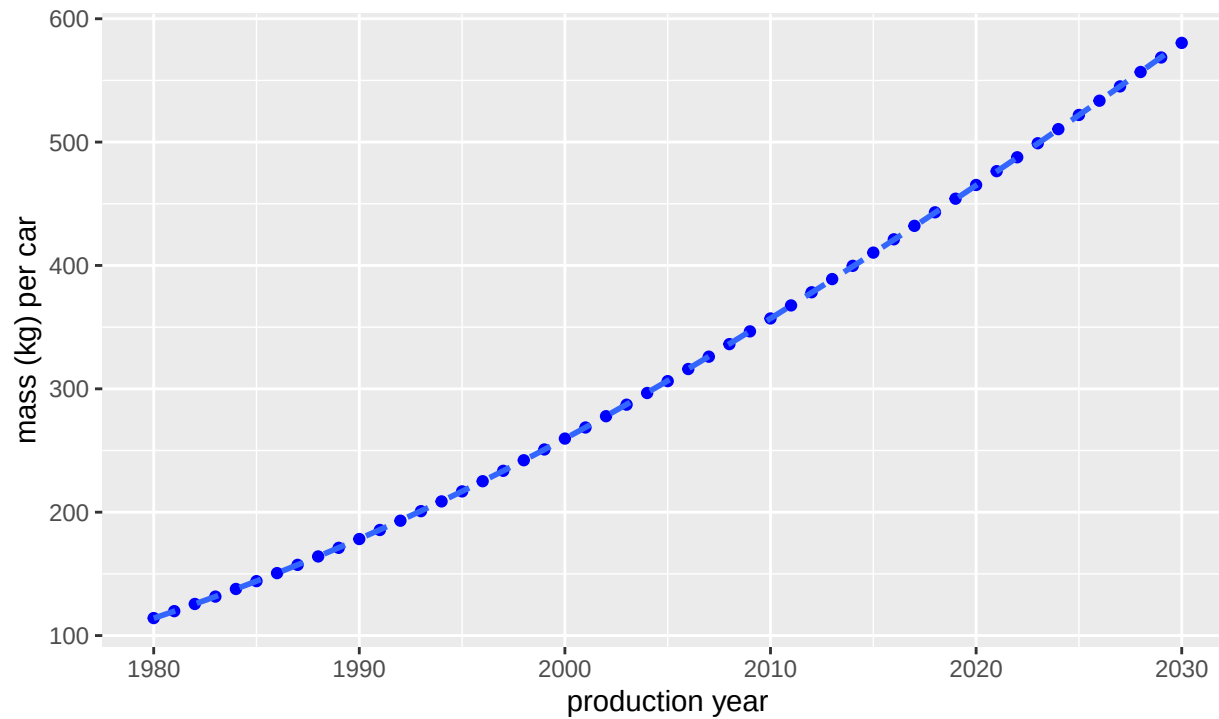
#AlCalculate.JE
```

Segment F and JF car (V0101010106 & V0101010206)

The implementation of the segment F cars follows the same tracks as for all the segments

```
#  
## car segment F (V0101010106)  
  
yearCalculate <- c(startDate:2030)  
  
# get all the values of car segment B  
temp <- aluminum_content_3 %>% filter(Segment == "F")  
  
# get the value of 2025 to adjust everything and bring it in line  
totalAl2025 <- AlCalculate.standard %>% filter(year == 2025)  
  
# model the values (we assume it is a straight line)  
ratioSegmentF <- median(temp$totalMass, na.rm = T) / totalAl2025$combinedValue  
  
AlCalculate.F <- AlCalculate.standard %>%  
  mutate(totalAlCalculate = combinedValue * ratioSegmentF, castingAlCalculate = castingValue * ratioSegmentF,  
         extrusionAlCalculate = extrusionValue * ratioSegmentF, forgingAlCalculate = forgingValue * ratioSegmentF,  
         sheetAlCalculate = sheetValue * ratioSegmentF, segment = "F") %>%  
  select(-castingValue, -extrusionValue, -forgingValue, -sheetValue, -combinedValue)  
  
#AlCalculate.F  
  
g <- ggplot() +  
  geom_point(data = AlCalculate.F, mapping = aes(x = year, y = totalAlCalculate), color = "blue") +  
  geom_smooth(data = AlCalculate.F, mapping = aes(x = year, y = totalAlCalculate), method = "loess", se = FALSE, linetype = "dashed") +  
  ggtitle("Total aluminum alloys for segment F car") +  
  xlab("production year") +  
  ylab("mass (kg) per car") +  
  coord_fixed(ratio = 0.06) # Adjust the ratio as needed  
  
plot(g)
```

Total aluminum alloys for segment F car



```
# Calculate the SUV composition
factor <- derivedSUVComposition %>% filter(segment == "F")

AlCalculate.JF <- AlCalculate.F %>%
  select(year, totalAlCalculate, castingAlCalculate, extrusionAlCalculate, forgingAlCalculate,
    sheetAlCalculate, segment) %>%
  mutate(year = year, totalAlCalculate = totalAlCalculate * factor$scaleFactor,
    castingAlCalculate = castingAlCalculate * factor$scaleFactor,
    extrusionAlCalculate = extrusionAlCalculate * factor$scaleFactor,
    forgingAlCalculate = forgingAlCalculate * factor$scaleFactor,
    sheetAlCalculate = sheetAlCalculate * factor$scaleFactor,
    segment = factor$segment_SUV
  )

#AlCalculate.JF
```

Put everything into one table

To be useful everything has to go into one table. Furthermore some rearrangements are made, so that the table is better readable and understandable.

```
# Bring the AlCalculate.V01010100 dataset into the same format as all the other ones, so that they can be combined

AlCalculate.standard <- AlCalculate.standard %>% relocate(segment, .after = last_col()) %>%
  relocate(combinedValue, .after = year) %>%
  rename(totalAlCalculate = combinedValue,
         castingAlCalculate = castingValue,
         extrusionAlCalculate = extrusionValue,
         forgingAlCalculate = forgingValue,
         sheetAlCalculate = sheetValue)

AlCalculate <- bind_rows(AlCalculate.standard, AlCalculate.A, AlCalculate.B, AlCalculate.C, AlCalculate.D, AlCalculate.E, AlCalculate.F,
                        AlCalculate.JA, AlCalculate.JB, AlCalculate.JC, AlCalculate.JD, AlCalculate.JE, AlCalculate.JF)

AlCalculate <- AlCalculate %>% mutate(productKeyLevel1 = "elvPetrol")

additionalproductKeyLevel1 <- c("elvDiesel", "elvBEV", "elvHEV", "elvPHEV", "elvOther")

# Duplicate the dataset 5 times, each time with a new description
AlCalculate <- bind_rows(AlCalculate, # Keep the original dataset
  map_dfr(additionalproductKeyLevel1, ~ AlCalculate %>%
    mutate(productKeyLevel1 = .x))
)

AlCalculate <- AlCalculate %>% left_join(productKeys, by = c("productKeyLevel1", "segment"))
#AlCalculate
```

Iron and steel composition of a car

Look at AHSS & UHSS

Table 1

Table 2 Development of AHSS & UHSS over time based on an other study

Conversion of all data.frames lb to kg (steel_content_1 & steel_content_2) New rows are added to the data.frame with the kg values

Conversion factor: $x \text{ lb} * 0.45359237 = y \text{ kg}$

```
lbTokg <- 0.45359237

# steel_content_1
steel_content_1$AHSS <- steel_content_1$`AHSS/lb` * lbTokg
steel_content_1$UHSS <- steel_content_1$`UHSS/lb` * lbTokg

# steel_content_2
steel_content_2$AHSS <- steel_content_2$`AHSS/lb` * lbTokg
steel_content_2$UHSS <- steel_content_2$`UHSS/lb` * lbTokg

# steel_content_3
steel_content_3$total <- steel_content_3$`total/lb` * lbTokg
```

Calculate the total mass, where required (steel_content_1 & steel_content_2)

```
# steel_content_1
steel_content_1$total <- steel_content_1$AHSS + steel_content_1$UHSS

# steel_content_2
steel_content_2$total <- steel_content_2$AHSS + steel_content_2$UHSS
```

Do some statistics and modeling to get the content forecast until 2030

Iron and steel with figures of current data

steel_content_1 & steel_content_2

```
# Require long table to display the figure

steel_content_1_long <- steel_content_1 %>%
  pivot_longer(cols = c("total", "AHSS", "UHSS"), names_to = "Materials", values_to = "values")

steel_content_1_long <- steel_content_1_long %>% select(-c(`AHSS/lb`, `UHSS/lb`))

steel_content_2_long <- steel_content_2 %>%
  pivot_longer(cols = c("total", "AHSS", "UHSS"), names_to = "Materials", values_to = "values")
```

```

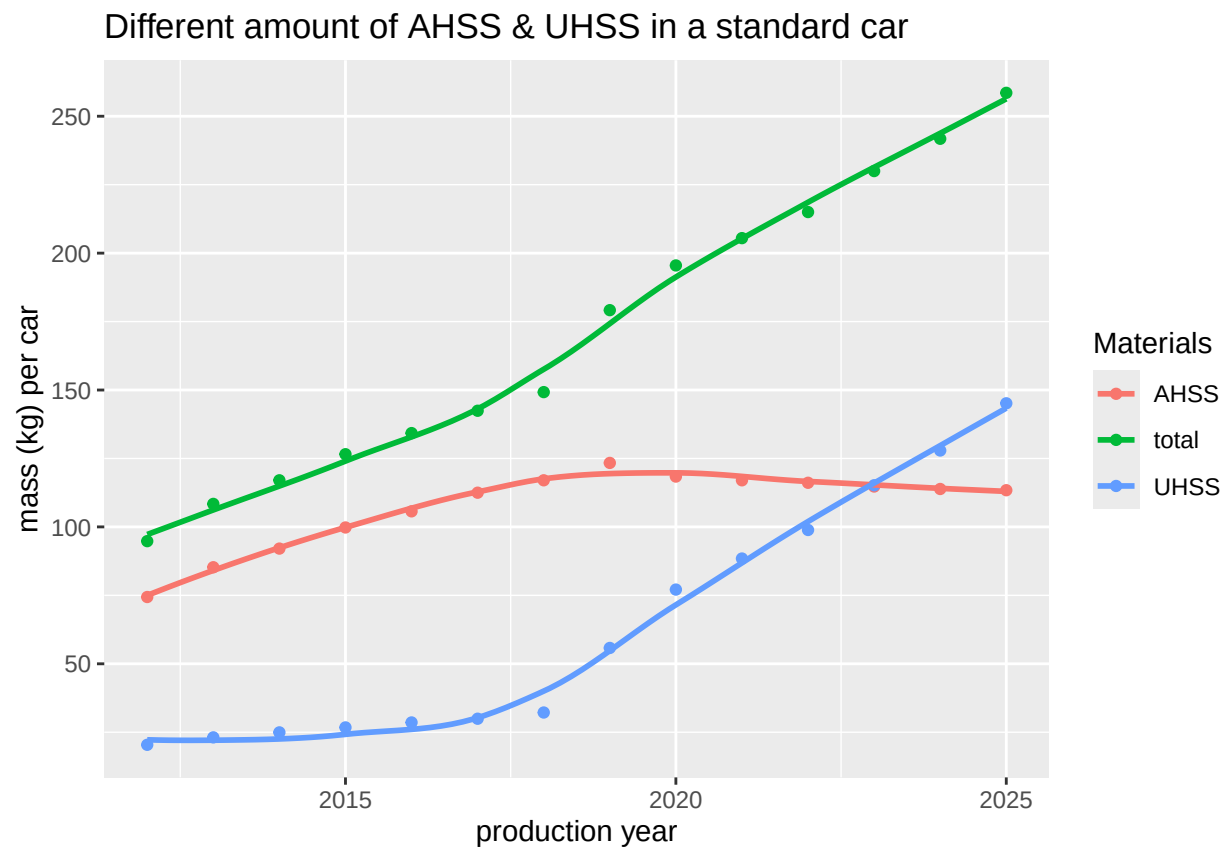
steel_content_2_long <- steel_content_2_long %>% select(-c(`AHSS/lb`, `UHSS/lb`))

## steel_content_1

g <- ggplot() +
  geom_point(data = steel_content_1_long, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = steel_content_1_long, mapping = aes(x = year, y = values, colour = Materials), method = "loess", se = FALSE) +
  ggtitle("Different amount of AHSS & UHSS in a standard car") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)

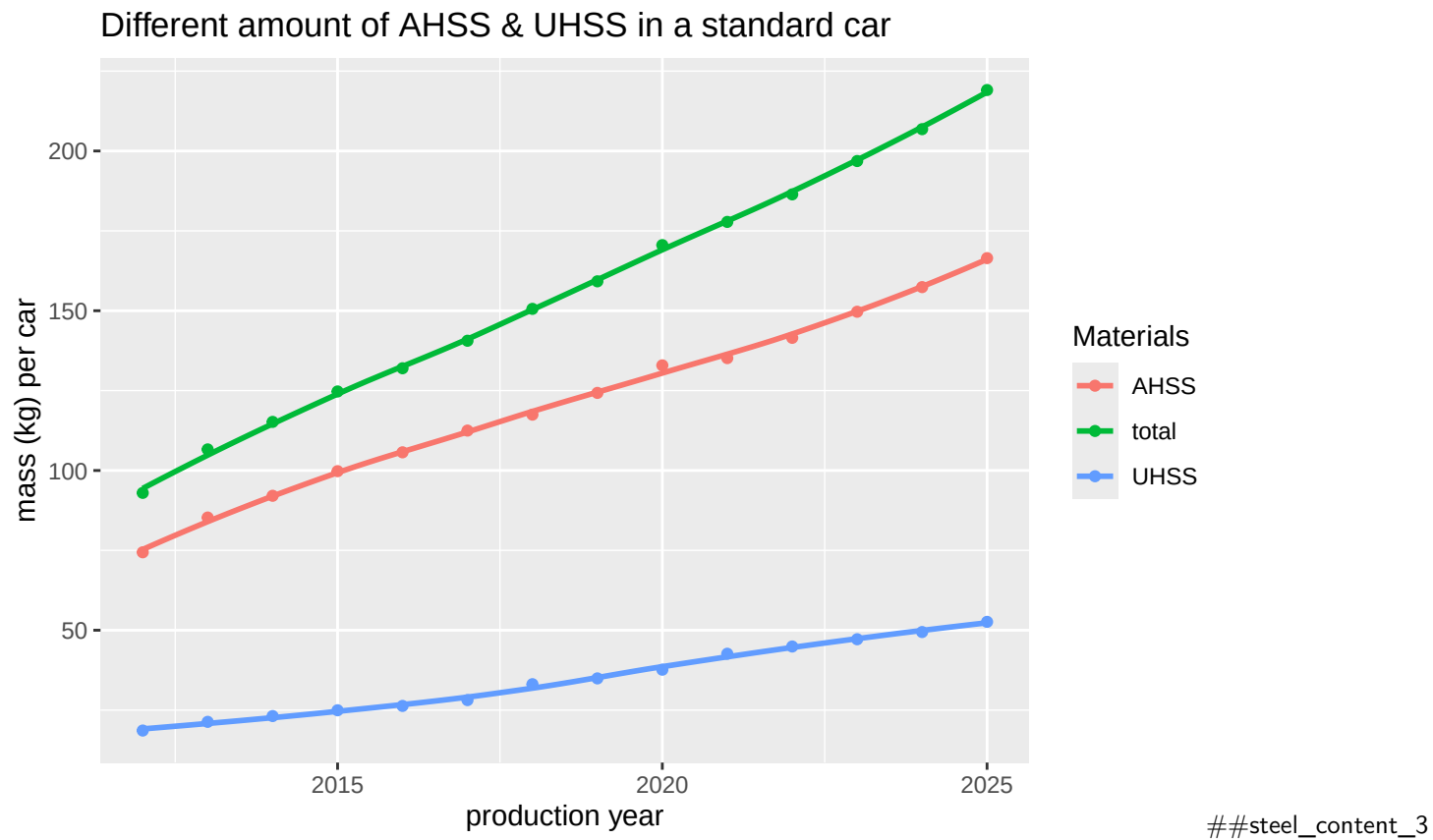
```



```
## steel_content_2

g <- ggplot() +
  geom_point(data = steel_content_2_long, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = steel_content_2_long, mapping = aes(x = year, y = values, colour = Materials), method = "loess", se = FALSE) +
  ggtitle("Different amount of AHSS & UHSS in a standard car") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```



Calculate the different masses based on the respective mass fractions

```
steel_content_3 <- steel_content_3 %>% select(-c(fraction_mildSteel_old, fraction_AHSS_old, fraction_UHSS_old))

# Step 1: Get the long table
steel_content_3_long <- steel_content_3 %>%
  pivot_longer(cols = c("fraction_mildSteel", "fraction_HSS", "fraction_AHSS", "fraction_UHSS"), names_to = "Material", names_prefix = "fraction_", values_to = "fraction")

# Step 2: Calculate the mass for each material
steel_content_3_long$mass <- steel_content_3_long$total * steel_content_3_long$fraction

# Step 3: Make the table wider
steel_content_3 <- steel_content_3_long %>%
  pivot_wider(names_from = Material, values_from = c(fraction, mass))

steel_content_3
```

```
## # A tibble: 6 x 12
##   year `total/lb` Scenario total fraction_mildSteel fraction_HSS fraction_AHSS
##   <dbl>     <dbl> <chr>    <dbl>         <dbl>         <dbl>         <dbl>
## 1  2013       1650 <NA>     748.         0.55          0.39          0.11
## 2  2015       1545 <NA>     701.         0.5           0.38          0.2
## 3  2018       1480 <NA>     671.         0.48          0.36          0.22
## 4  2020       1420 <NA>     644.         0.45          0.3           0.22
## 5  2025       1313 A       596.         0.4           0.15          0.22
## 6  2025       1137 B       516.         0.4           0.13          0.22
## # i 5 more variables: fraction_UHSS <dbl>, mass_mildSteel <dbl>,
## #   mass_HSS <dbl>, mass_AHSS <dbl>, mass_UHSS <dbl>
```

show the data as figures

```
# Require longer table to include total in figure

steel_content_3_long <- steel_content_3 %>%
  pivot_longer(cols = c("total", "mass_mildSteel", "mass_HSS", "mass_AHSS", "mass_UHSS"), names_to = "Materials", names_prefix = "mass_", values_to = "values")

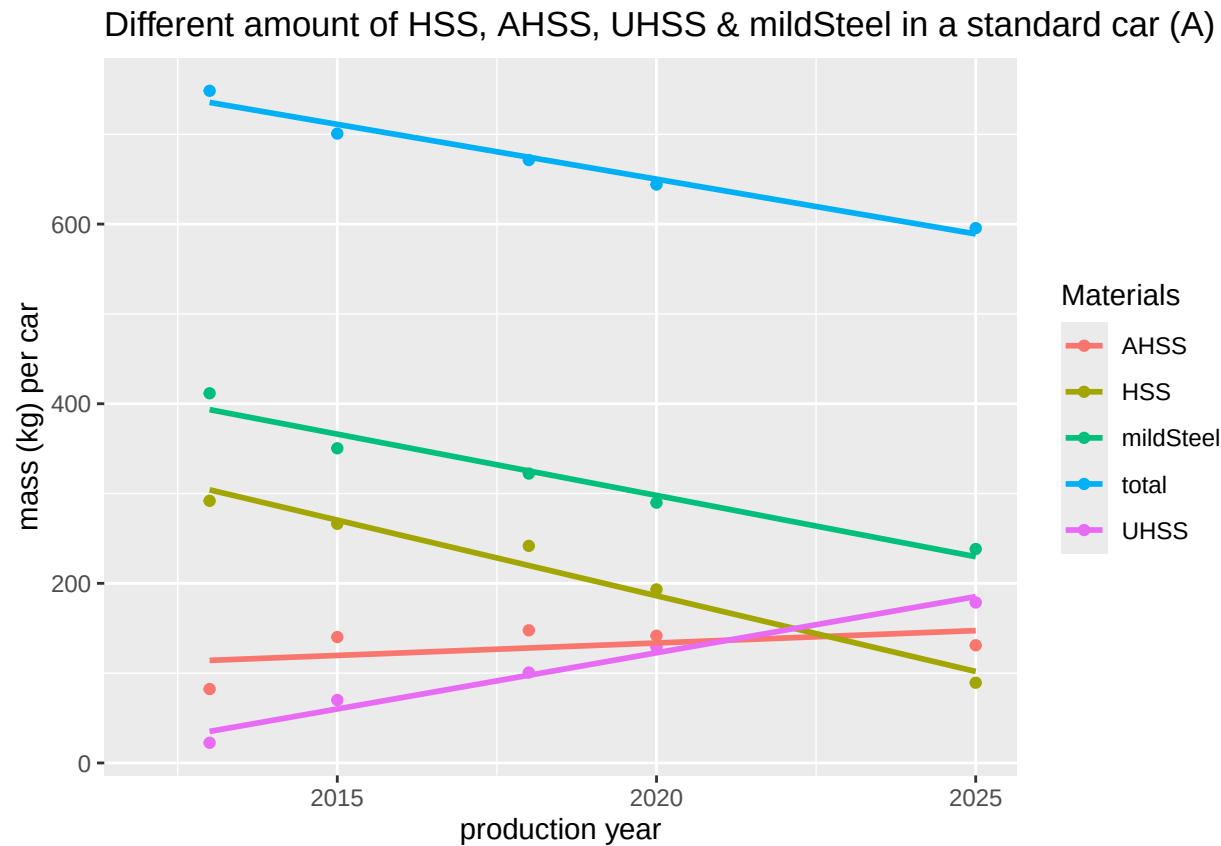
steel_content_3_long <- steel_content_3_long %>% select(-c(`total/lb`, fraction_mildSteel, fraction_HSS, fraction_AHSS, fraction_UHSS))

# Scenario A
```

```
temp <- steel_content_3_long %>% filter(is.na(Scenario) | Scenario == "A")

g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = temp, mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  ggtitle("Different amount of HSS, AHSS, UHSS & mildSteel in a standard car (A)") +
  scale_x_continuous(name = "production year", limits = c(2012, 2025)) +
  ylab("mass (kg) per car")

plot(g)
```



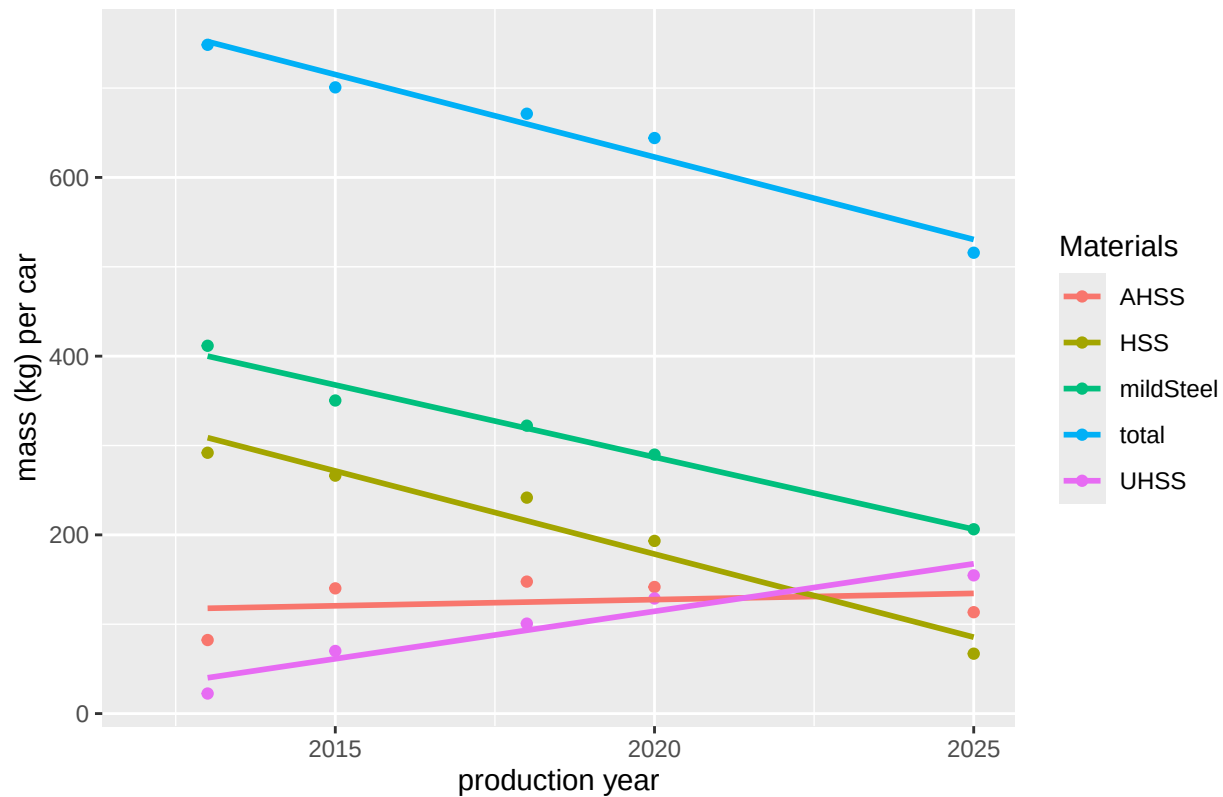
```
# Scenario B

temp <- steel_content_3_long %>%
  filter(is.na(Scenario) | Scenario == "B")

g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = temp, mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  ggtitle("Different amount of HSS, AHSS, UHSS & mildSteel in a standard car (B)") +
  scale_x_continuous(name = "production year", limits = c(2012, 2025)) +
  ylab("mass (kg) per car")

plot(g)
```

Different amount of HSS, AHSS, UHSS & mildSteel in a standard car (B)



steel_content_4

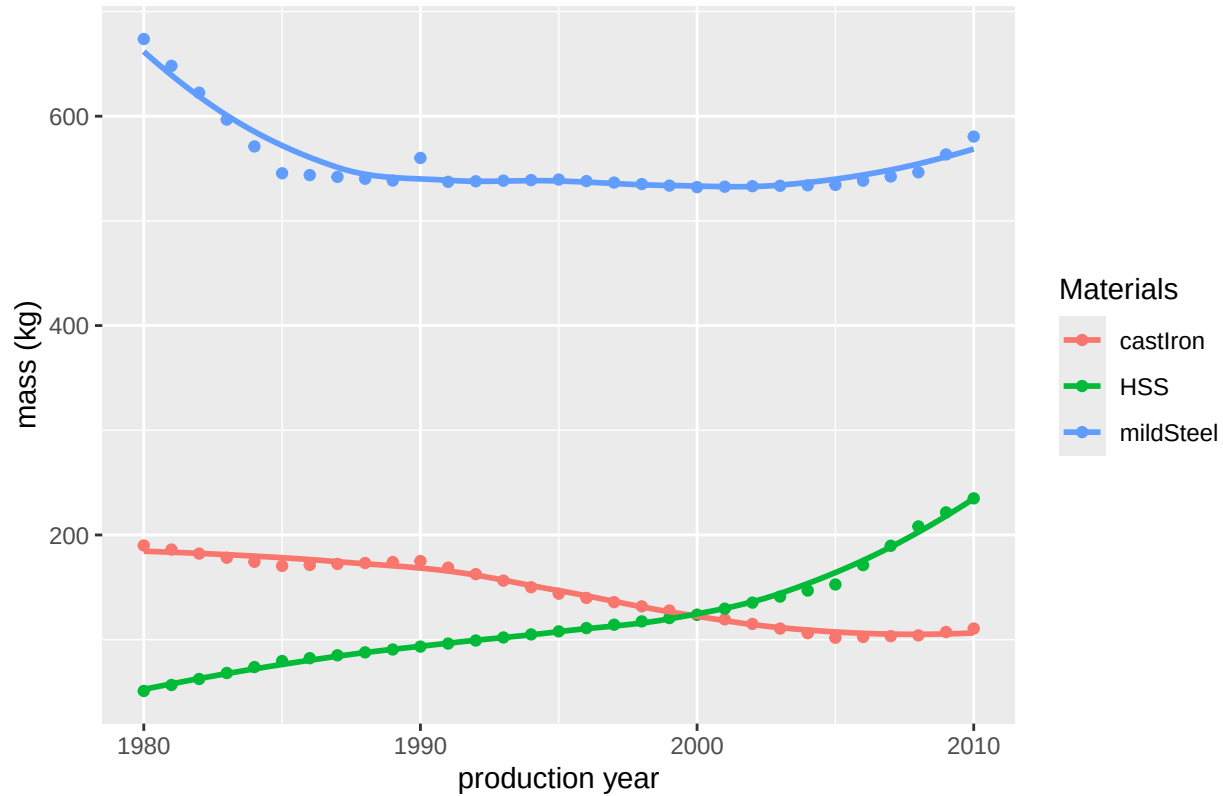
```
# require first the long table

steel_content_4_long <- steel_content_4 %>%
  pivot_longer(cols = c("castIron", "mildSteel", "HSS"), names_to = "Materials", values_to = "values")

g <- ggplot() +
  geom_point(data = steel_content_4_long, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = steel_content_4_long, mapping = aes(x = year, y = values, colour = Materials), method = "loess", se = FALSE) +
  ggtitle("Different amount of castIron, HSS & mildSteel in a standard car") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

Different amount of castIron, HSS & mildSteel in a standard car



Start combining the different steel_content data files

steel_content_1 and steel_content_2 are a similar dataset, but from different years. Therefore we combine them one after the other with steel_content_3

steel_content_1 & steel_content_3

```
## Combining the two long tables
# For combining and fitting them properly both tables have to have the same number of data points
```

```

steel_content_1_long <- steel_content_1_long %>% filter(year == 2013 | year == 2015 | year == 2018 | year == 2020 | year == 2025)

temp_long_1 <- bind_rows(steel_content_1_long, steel_content_3_long)

# remove total as there are not always all data sets complete for a given year
temp_long_1 <- temp_long_1 %>% filter(Materials != "total")

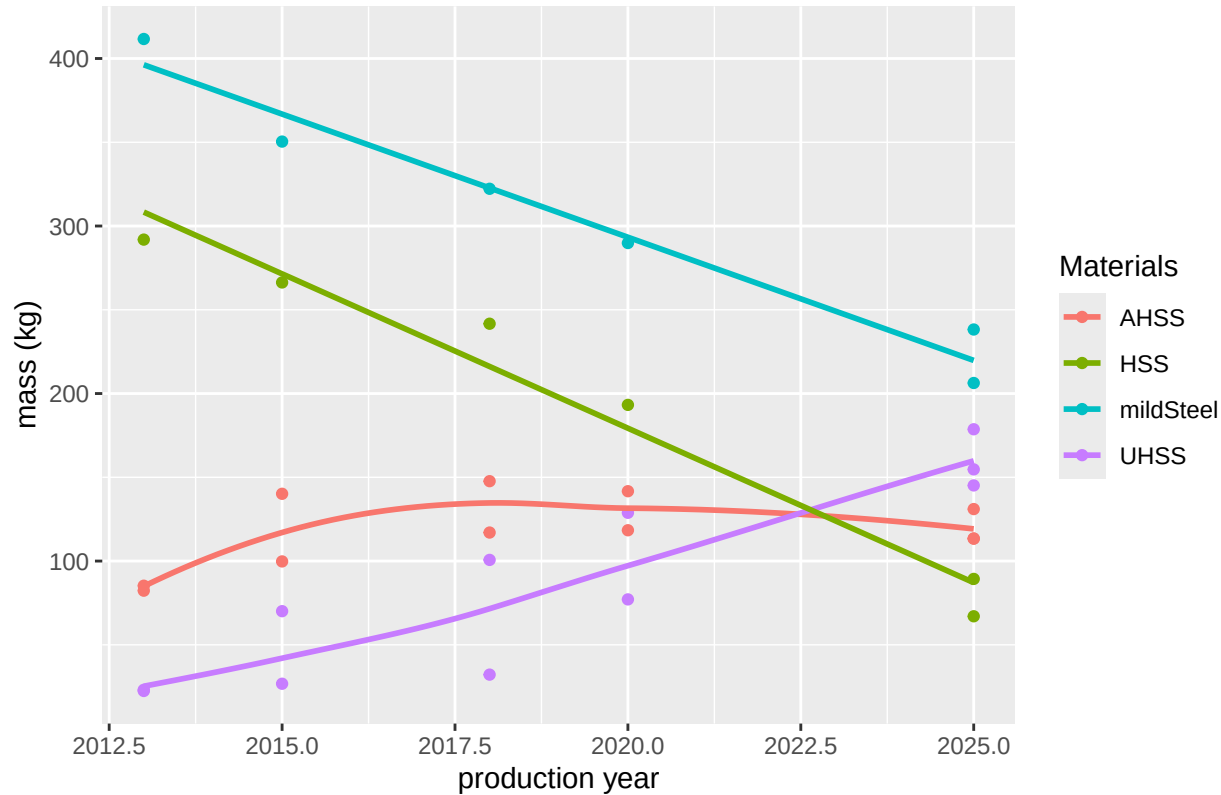
# Now display the figure

g <- ggplot() +
  geom_point(data = temp_long_1, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = filter(temp_long_1, Materials == "AHSS" | Materials == "UHSS"), mapping = aes(x = year, y = values, colour = Materials), method = "loess", span = 1, se = FALSE) +
  geom_smooth(data = filter(temp_long_1, Materials == "HSS" | Materials == "mildSteel"), mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  ggtitle("Different amount of AHSS, UHSS, HSS & mildSteel in a standard car") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

Different amount of AHSS, UHSS, HSS & mildSteel in a standard car



```
# The differences for the AHSS and the UHSS are to big therefore only the data of steel_content_3_long are used
```

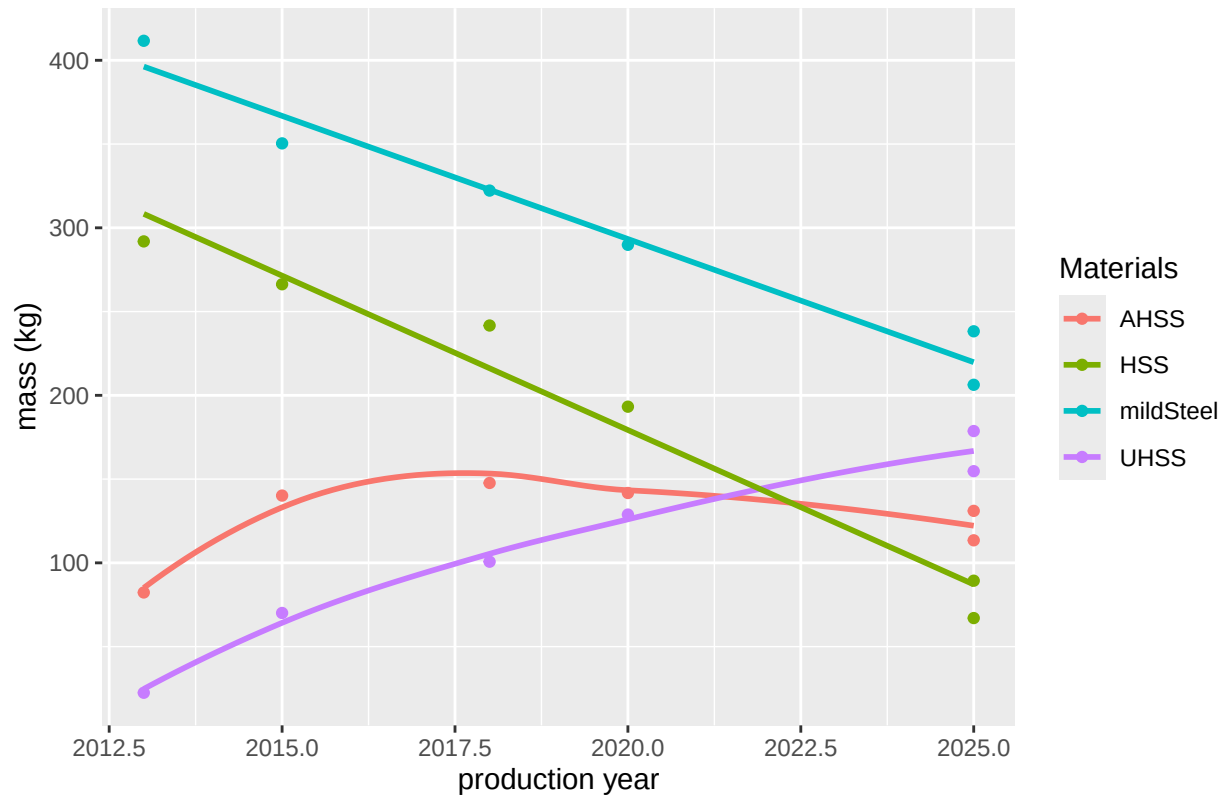
```
temp_long_1 <- steel_content_3_long %>% filter(Materials != "total")
```

```
# Now display the figure
```

```
g <- ggplot() +
  geom_point(data = temp_long_1, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = filter(temp_long_1, Materials == "AHSS" | Materials == "UHSS"), mapping = aes(x = year, y = values, colour = Materials), method = "loess", span = 1, se = FALSE) +
  geom_smooth(data = filter(temp_long_1, Materials == "HSS" | Materials == "mildSteel"), mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  ggtitle("Different amount of AHSS, UHSS, HSS & mildSteel in a standard car") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")
```

```
plot(g)
```

Different amount of AHSS, UHSS, HSS & mildSteel in a standard car



steel_content_2 & steel_content_3

```
## Combining the two long tables
```

```
# For combining and fitting them properly both tables have to have the same number of data points
```

```
steel_content_2_long <- steel_content_2_long %>% filter(year == 2013 | year == 2015 | year == 2018 | year == 2020 | year == 2025)
```

```

temp_long_2 <- bind_rows(steel_content_2_long, steel_content_3_long)

# remove total as there are not always all data sets complete for a given year
temp_long_2 <- temp_long_2 %>% filter(Materials != "total")

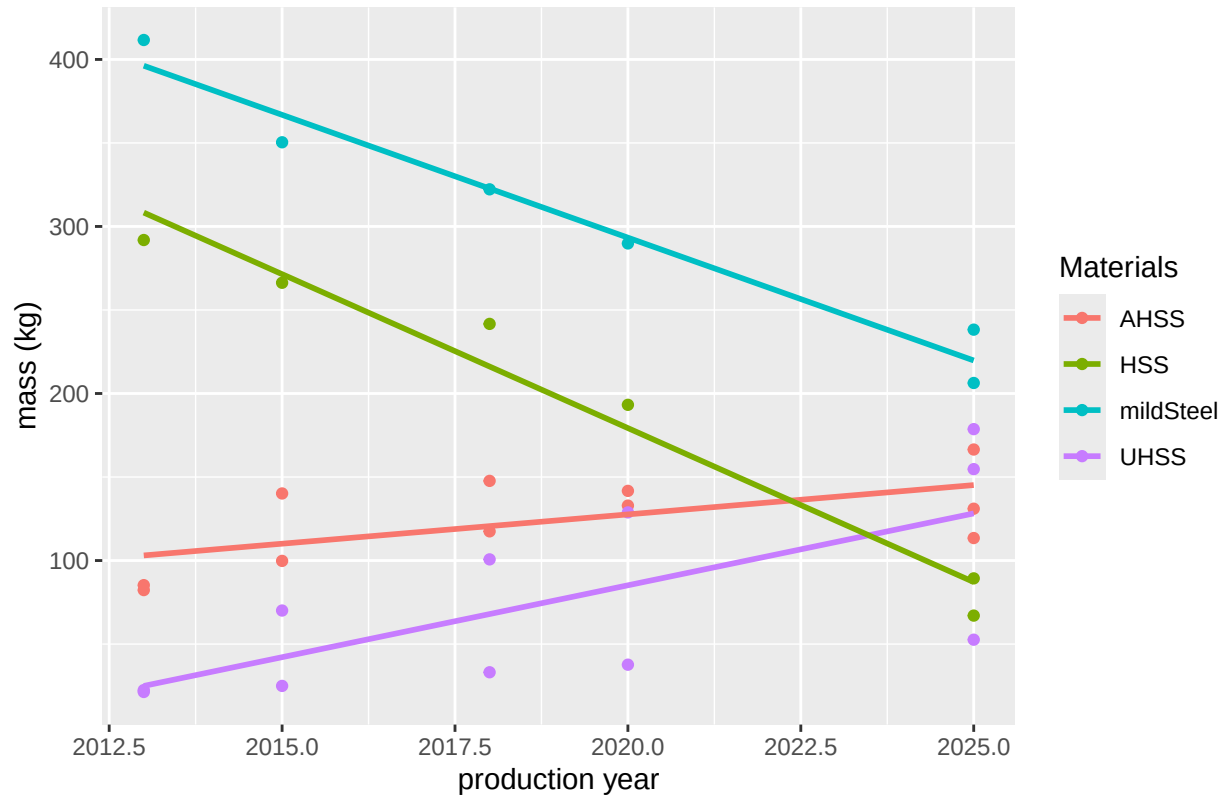
# Now display the figure

g <- ggplot() +
  geom_point(data = temp_long_2, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = filter(temp_long_2, Materials == "AHSS" | Materials == "UHSS"), mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  geom_smooth(data = filter(temp_long_2, Materials == "HSS" | Materials == "mildSteel"), mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  ggtitle("Different amount of AHSS, UHSS, HSS & mildSteel in a standard car") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

Different amount of AHSS, UHSS, HSS & mildSteel in a standard car



```
# The differences for the AHSS and the UHSS are to big therefore only the data of steel_content_3_long are used
```

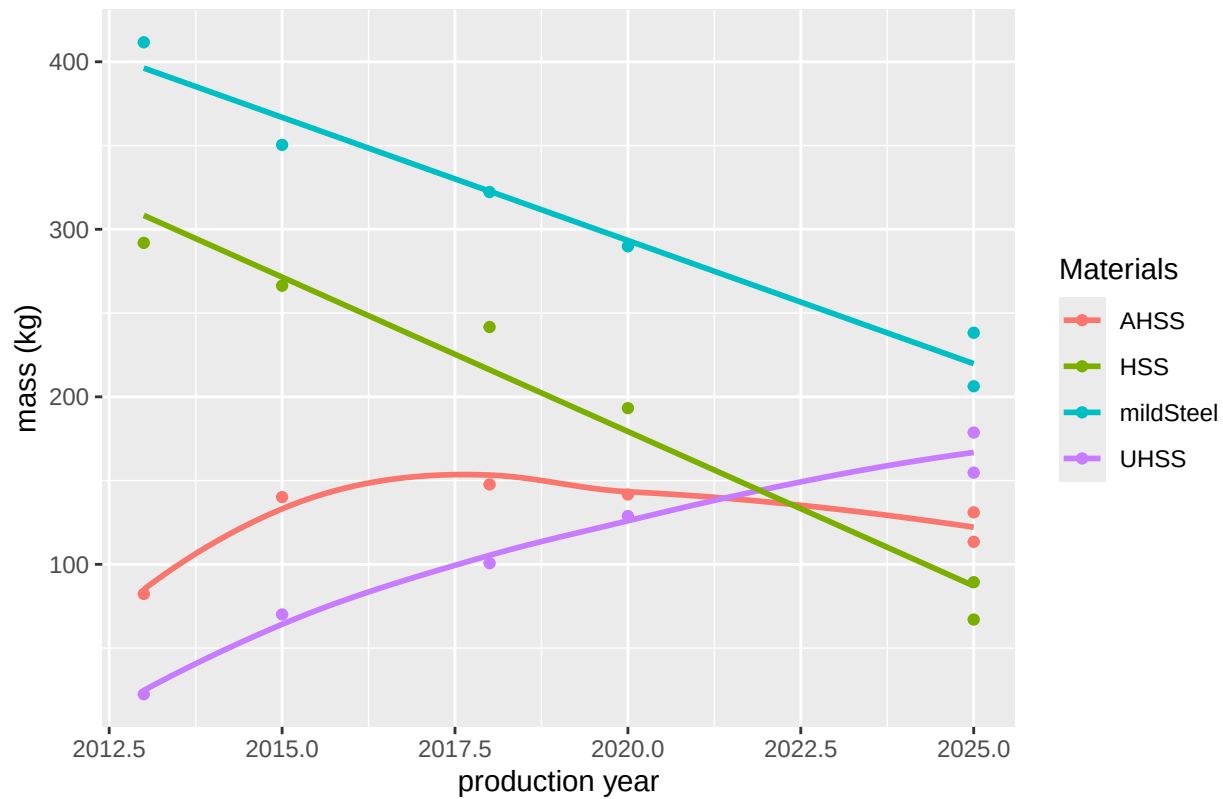
```
temp_long_2 <- steel_content_3_long %>% filter(Materials != "total")
```

```
# Now display the figure
```

```
g <- ggplot() +
  geom_point(data = temp_long_2, mapping = aes(x = year, y = values, colour = Materials)) +
  geom_smooth(data = filter(temp_long_2, Materials == "AHSS" | Materials == "UHSS"), mapping = aes(x = year, y = values, colour = Materials), method = "loess", span = 1, se = FALSE) +
  geom_smooth(data = filter(temp_long_2, Materials == "HSS" | Materials == "mildSteel"), mapping = aes(x = year, y = values, colour = Materials), method = "lm", se = FALSE) +
  ggtitle("Different amount of AHSS, UHSS, HSS & mildSteel in a standard car") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")
```

```
plot(g)
```

Different amount of AHSS, UHSS, HSS & mildSteel in a standard car



This data combination does not reflect each

other for the AHSS and UHSS steel

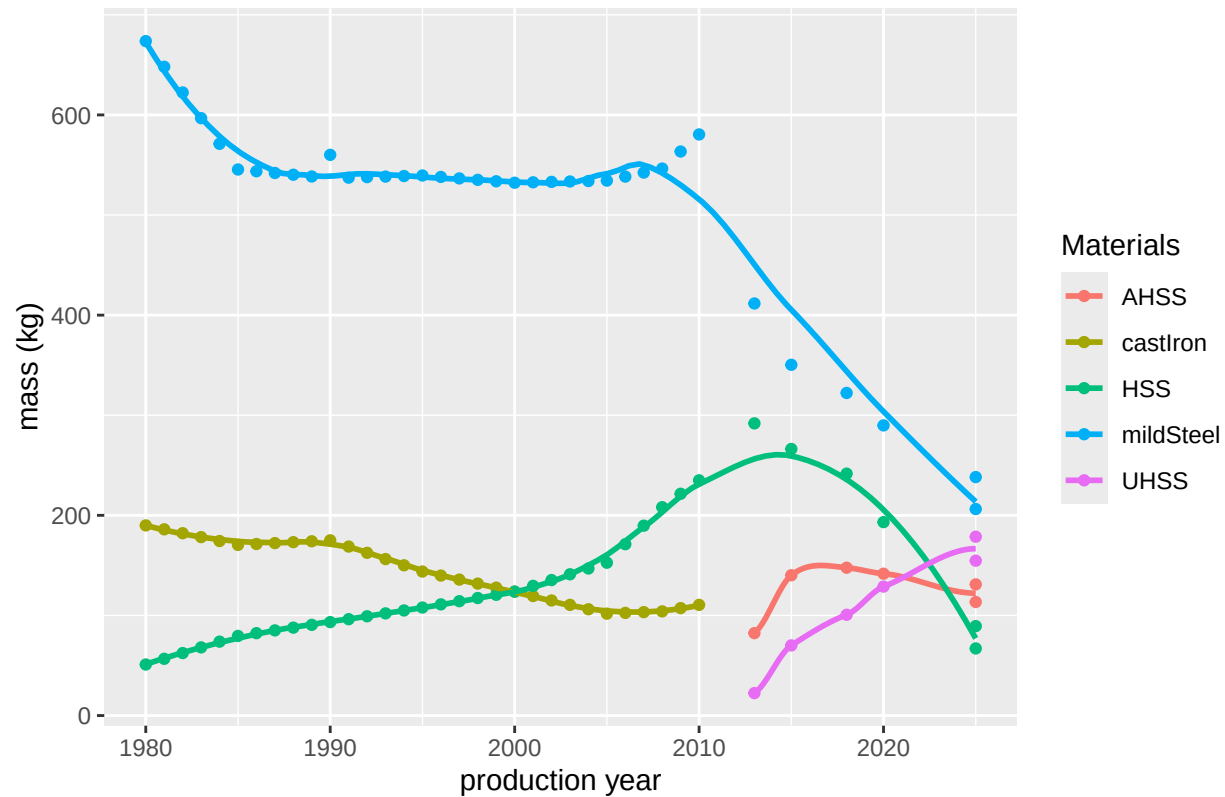
Combine all steel_contents (steel_content_3 & steel_content_4)

```
# Combining the two long tables
steel_combined_long <- bind_rows(temp_long_1, steel_content_4_long)
g <- ggplot() +
```

```
geom_point(data = steel_combined_long, mapping = aes(x = year, y = values, colour = Materials)) +
geom_smooth(data = steel_combined_long, mapping = aes(x = year, y = values, colour = Materials), method = "loess", span = 0.5, se = FALSE) +
ggtitle("Different amount of AHSS, UHSS, HSS, mildSteel & castIron in a standard car") +
scale_x_continuous(name = "production year") +
scale_y_continuous(name = "mass (kg)")
```

plot(g)

Different amount of AHSS, UHSS, HSS, mildSteel & castIron in a standard



Start combining the new data with the old data for each of the different steels

castIron

<https://metalcastinginstitute.com/cast-iron-automotive-applications> 8% of average weight. This is not a too interesting source <https://castimoo.com/what-are-the-properties-of-iron-castings-used-in-the-automotive-industry/>

Copilot text In modern cars, cast iron still plays a significant role, especially in components that require high strength and durability. On average, cast iron makes up about **8% of the total vehicle weight**¹. This includes parts like brake rotors, some brake caliper structures, and occasionally engine components¹².

The use of cast iron has slightly declined over the past decade due to the automotive industry's shift towards lighter materials like aluminum. However, it remains essential for parts where its properties, such as wear resistance and thermal conductivity, are crucial¹².

There is an interesting article concerning the average weight of cars depending on the continent <https://www.measuringknowhow.com/average-car-weight-key-facts-explained/> But this slight decrease is compensated by the overall size and weight increase.

Therefore it is essential to correct the JRC data for the steady increase of the car-weight for all segments and the standard cars, as they are not reflected in the JRC data. For this purpose we use the initial ratio of a given material between 1980 and 1990 towards the overall weight of a car. Based on this assumption we add the yearly increase of weight using this ratio towards the overall increase until 2010. Afterwards the JRC corrected data are combined with the new data.

```
# get the old data
temp_old <- JRC_Data %>% filter(materialKeyLevel2 == "castIron" & is.na(element) & productKeyLevel2 == "V0101010000")

#remove all unnecessary columns
temp_old <-temp_old %>% select(c(productionYear, materialKeyLevel2, value))

# transform a two variable back to numerals
temp_old$productionYear <- as.numeric(temp_old$productionYear)
temp_old$value <- as.numeric(temp_old$value)

# Calculate the average value
temp_old_average <- temp_old %>% filter((productionYear >= 1980) & (productionYear <= 1990)) %>% summarise(avg_value = mean(value))

#Get the ratio
ratiocastIron <- temp_old_average / startWeightCar

## Calculate the annual increase between 1980 and 2010 based on the ratio and annual increase of the weight
# Create a data frame
increasecastIron <- data.frame(year = 1980:2010, value = 0)

# Extract the slope (coefficient) from the model and calculate its ratio
slopecastIron <- coef(modelcarWeightStandard)["Year"] * ratiocastIron$avg_value
```

```

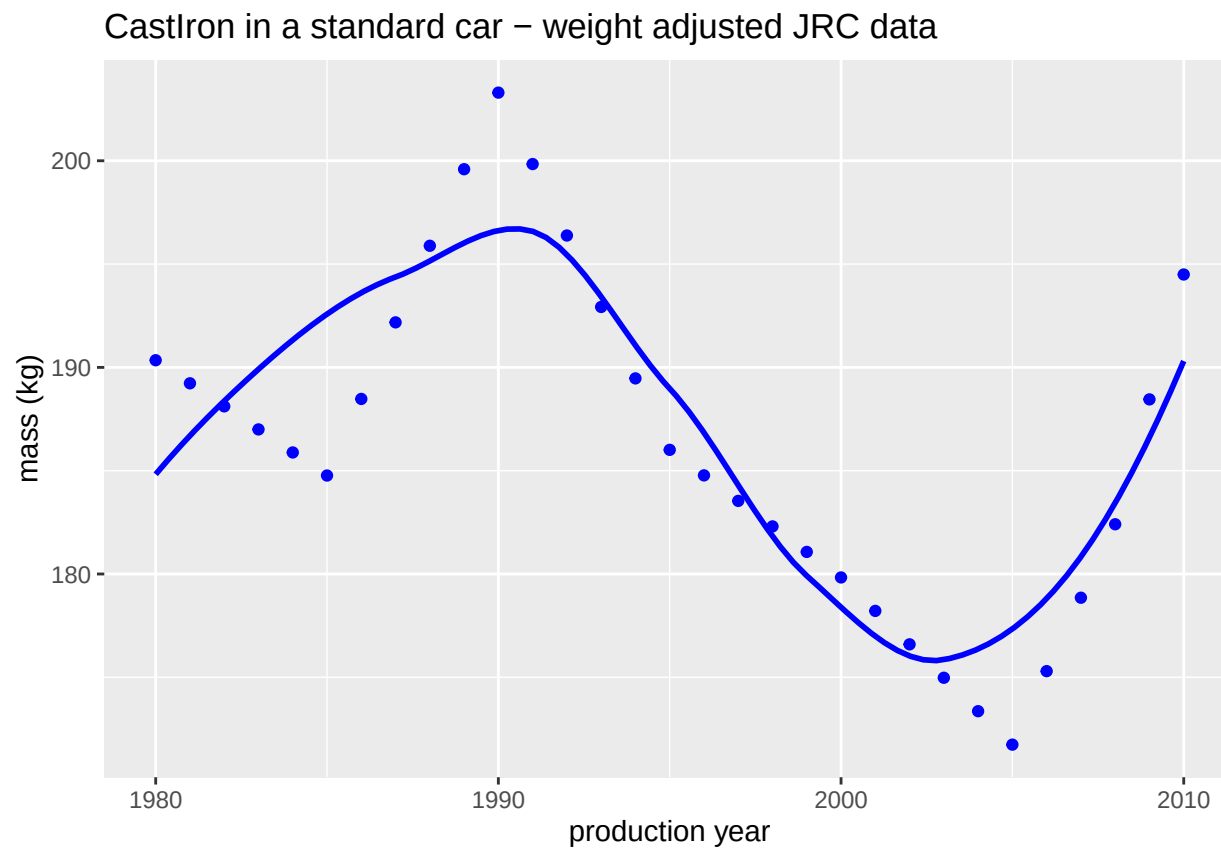
# Calculate the new values based on the slope
increasecastIron <- increasecastIron %>% mutate(new_value = value + slopecastIron * (year - 1980))

temp_old <- temp_old %>% mutate(value = value + increasecastIron$new_value)

# Display the corrected JRC data
g <- ggplot() +
  geom_point(data = temp_old, mapping = aes(x = productionYear, y = value), color = "blue") +
  geom_smooth(data = temp_old, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, color = "blue") +
  ggtitle("CastIron in a standard car - weight adjusted JRC data") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```



```

## The adjusted JRC data vary over time. It is best to obtain the average and the use this average values for the whole time-period (1980 to 2030)

# calculate the mean
temp <- temp_old %>% filter(productionYear >= 1980 & productionYear <= 2010)
castIronMean <- mean(temp$value)

# generate the dataset for castIron 2010:2030
year <- c(1980:2030)
values <- rep(castIronMean, times = 51)

castIronCalculate <- data.frame(yearCalculate = year, valueCalculate = values)

# display the combined data

g <- ggplot() +
  geom_point(data = temp_old, mapping = aes(x = productionYear, y = value), color = "black") +
  geom_smooth(data = temp_old, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, span = 0.4, color = "black") +
  geom_point(data = castIronCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), color = "blue") +
  geom_smooth(data = castIronCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), method = "lm", se = FALSE, color = "blue") +
  ggtitle("CastIron in a standard car (averaged value)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)", limits = c(0, 210))

plot(g)

```

CastIron in a standard car (averaged value)



```
## combine the two data sets into one and then obtain a smooth function for the overall time period 1980 - 2030
# use the average values
castIronCalculate <- castIronCalculate %>% rename(year = yearCalculate, castingIron = valueCalculate)
#castIronCalculate
```

mildSteel

One has also to correct in a similar way the JRC data of mild steel, as it was done for the castiron. The JRC data do not consider the overall weight increase of the cars.

```

# get the old data
temp_old <- JRC_Data %>% filter(materialKeyLevel2 == "mildSteel" & is.na(element) & productKeyLevel2 == "V0101010000")

#remove all unnecessary columns
temp_old <-temp_old %>% select(c(productionYear, materialKeyLevel2, value))

# transform a two variable back to numerals
temp_old$productionYear <- as.numeric(temp_old$productionYear)
temp_old$value <- as.numeric(temp_old$value)

# Calculate the average value
temp_old_average <- temp_old %>% filter((productionYear >= 1980) & (productionYear <= 1990)) %>% summarise(avg_value = mean(value))

#Get the ratio
ratiomildSteel <- temp_old_average / startWeightCar

## Calculate the annual increase between 1980 and 2010 based on the ratio and annual increase of the weight
# Create a data frame
increasemildSteel <- data.frame(year = 1980:2010, value = 0)

# Extract the slope (coefficient) from the model and calculate its ratio
slopemildSteel <- coef(modelcarWeightStandard)["Year"] * ratiomildSteel$avg_value

# Calculate the new values based on the slope
increasemildSteel <- increasemildSteel %>% mutate(new_value = value + slopemildSteel * (year - 1980))

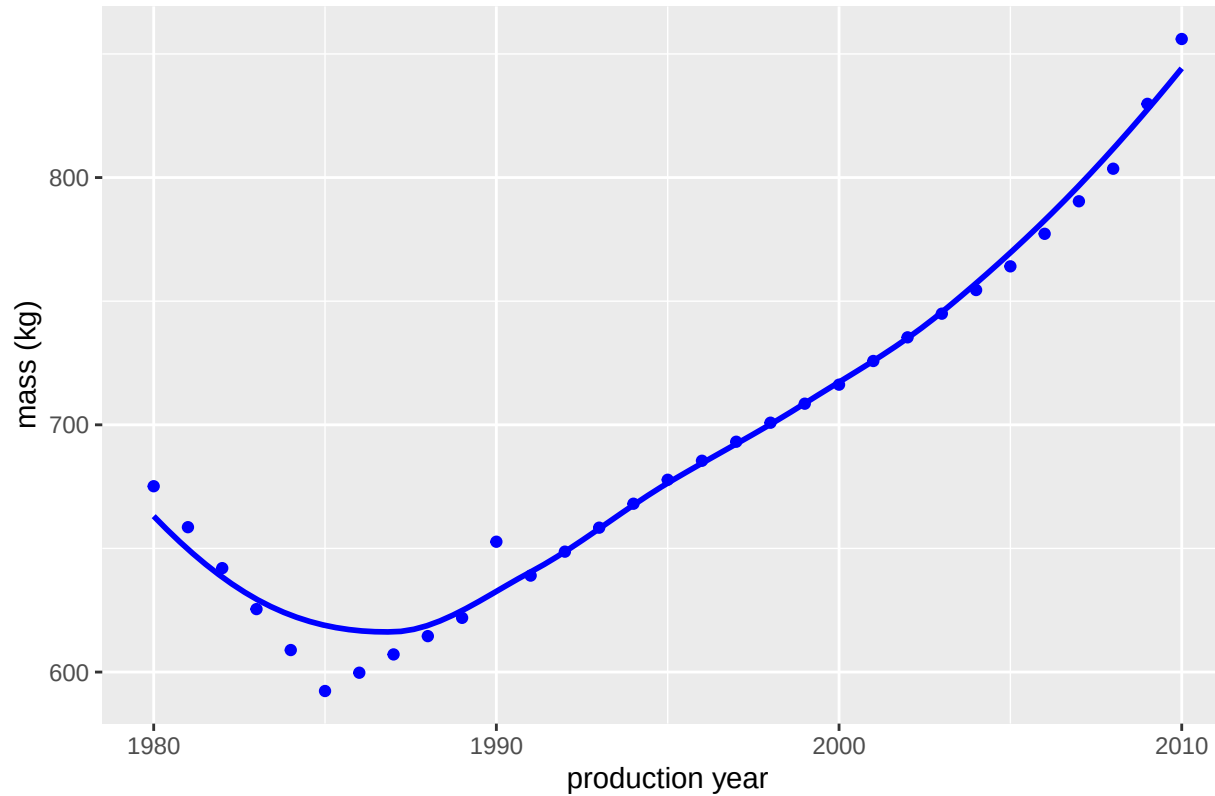
temp_old <- temp_old %>% mutate(value = value + increasemildSteel$new_value)

# Display the weight corrected JRC data
g <- ggplot() +
  geom_point(data = temp_old, mapping = aes(x = productionYear, y = value), color = "blue") +
  geom_smooth(data = temp_old, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, color = "blue") +
  ggtitle("CastIron in a standard car - weight adjusted JRC data") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

CastIron in a standard car – weight adjusted JRC data

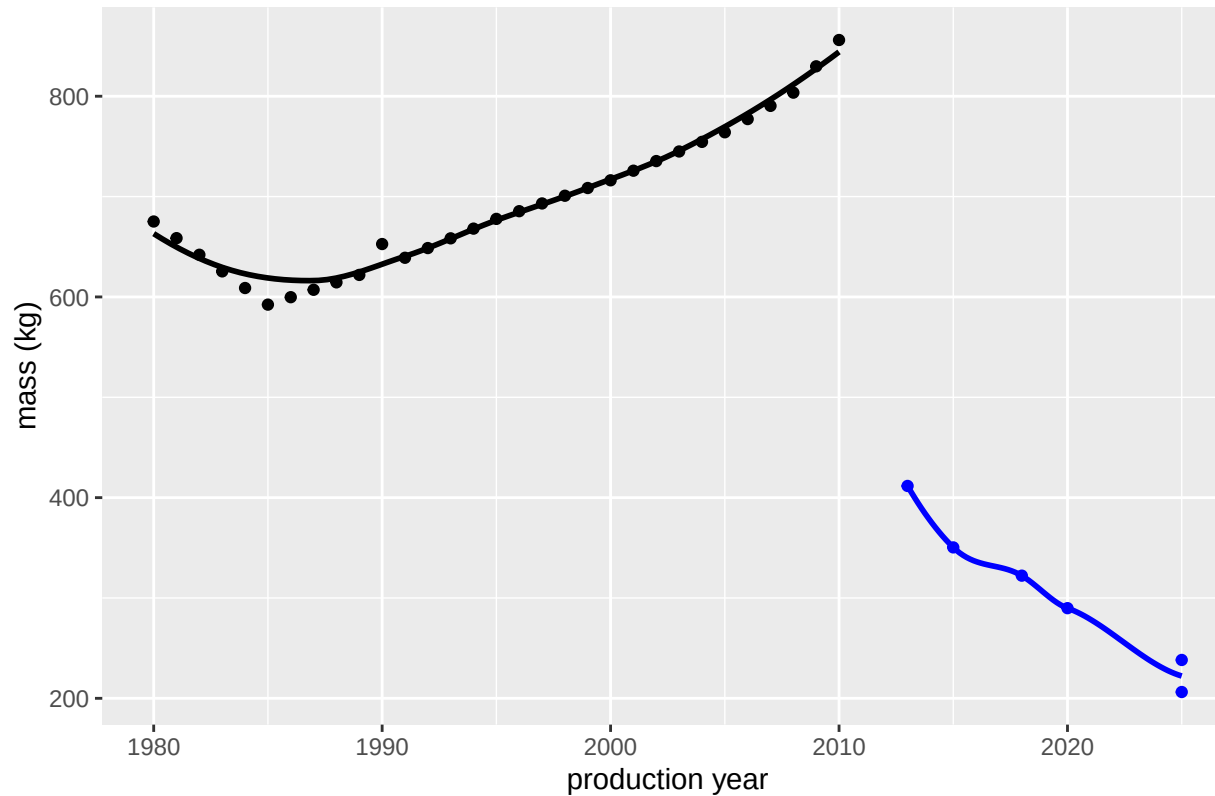


```
# get the new data
temp_new <- steel_combined_long %>% filter((Materials == "mildSteel") & (year > 2010))

g <- ggplot() +
  geom_point(data = temp_old, mapping = aes(x = productionYear, y = value), color = "black") +
  geom_smooth(data = temp_old, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, color = "black") +
  geom_point(data = temp_new, mapping = aes(x = year, y = values), color = "blue") +
  geom_smooth(data = temp_new, mapping = aes(x = year, y = values), method = "loess", se = FALSE, color = "blue") +
  ggtitle("MildSteel in a standard car - JRC data (black) & new data (blue)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

MildSteel in a standard car – JRC data (black) & new data (blue)



Looking at this figure. It becomes obvious that the old and new data fit together. Looking at the comparisons between the actual weight of the different car classes and the calculated one, it

```
# prepare temp_old and temp_new, so that they can be combined
temp_old <- temp_old %>% filter( productionYear <= 2005)
temp_new <- temp_new %>% select(-Scenario) %>% rename(productionYear = year, materialKeyLevel2 = Materials, value = values)

temp_new <- temp_new %>% filter((value > 280)) %>% mutate(value = value * 1.5)

temp <- bind_rows(temp_old, temp_new)
temp
```

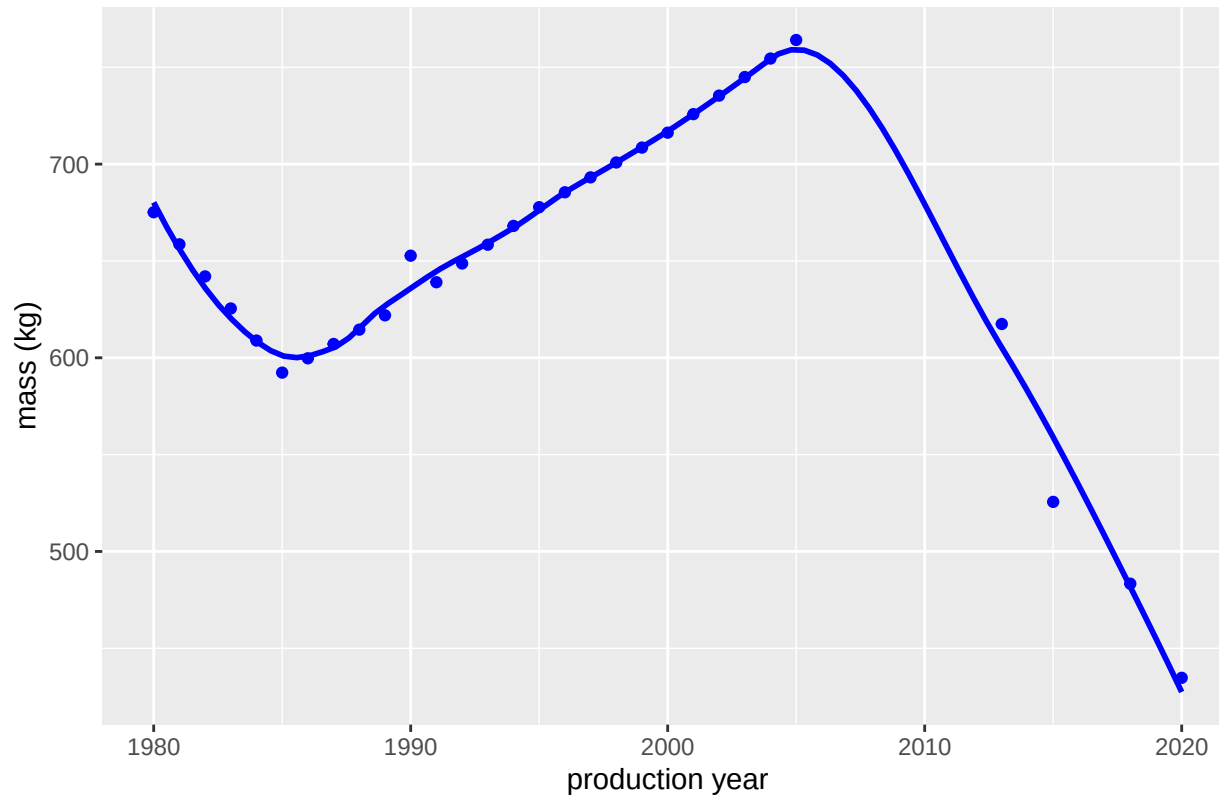
```
## # A tibble: 30 x 3
##   productionYear materialKeyLevel2 value
```

```
##           <dbl> <chr>           <dbl>
## 1         1980 mildSteel         675.
## 2         1981 mildSteel         659.
## 3         1982 mildSteel         642.
## 4         1983 mildSteel         625.
## 5         1984 mildSteel         609.
## 6         1985 mildSteel         592.
## 7         1986 mildSteel         600.
## 8         1987 mildSteel         607.
## 9         1988 mildSteel         615.
## 10        1989 mildSteel         622.
## # i 20 more rows
```

```
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = productionYear, y = value), color = "blue") +
  geom_smooth(data = temp, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, span = 0.4, color = "blue") +
  ggtitle("MildSteel in a standard car - old data (until 2005) & new data (starting 2010)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

MildSteel in a standard car – old data (until 2005) & new data (starting 2010)



```
# start model the new data (interpolate) for the whole time period with available data (1980 - 2025)
yearCalculate <- c(1980:2020)
mildSteelInterpolate <- data.frame(yearCalculate = yearCalculate)

mmildSteelCaculate <- loess(value ~ productionYear, data = temp, span = 0.4) #0.85
valueCalculate <- predict(mmildSteelCaculate, yearCalculate)

mildSteelCalculate <- as.data.frame(cbind(yearCalculate = yearCalculate, valueCalculate = valueCalculate))

# extrapolate to 2030 - it is assume as a first order, that the amount of mild steel only decreases, if it is replaced by other materials
temp <- mildSteelCalculate %>% filter(yearCalculate == 2020)

yearCalculate <- c(2021:2030)
```

```

# Create a vector with identical values (e.g., 100)
identical_values <- rep(temp$valueCalculate, length(yearCalculate))

# Combine the vectors into a data frame
df <- data.frame(yearCalculate = yearCalculate, valueCalculate = identical_values)
mildSteelCalculate <- bind_rows(mildSteelCalculate, df)

g <- ggplot() +
  geom_point(data = mildSteelCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), color = "blue") +
  geom_smooth(data = mildSteelCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), method = "loess", se = FALSE, span = 0.3, color = "blue") +
  ggtitle("MildSteel in a standard car - Calculated values (1980 - 2030)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

MildSteel in a standard car – Calculated values (1980 – 2030)



HSS

```
# get the old data
temp_old <- JRC_Data %>% filter(materialKeyLevel2 == "highStrengthSteel" & is.na(element) & productKeyLevel2 == "V0101010000")

#remove all unnecessary columns
temp_old <-temp_old %>% select(c(productionYear, materialKeyLevel2, value))

# transform a two variable back to numerals
temp_old$productionYear <- as.numeric(temp_old$productionYear)
temp_old$value <- as.numeric(temp_old$value)
```

```

# Calculate the average value
temp_old_average <- temp_old %>% filter((productionYear >= 1980) & (productionYear <= 1985)) %>% summarise(avg_value = mean(value))

#Get the ratio
ratioHSS <- temp_old_average / startWeightCar

## Calculate the annual increase between 1980 and 2010 based on the ratio and annual increase of the weight
# Create a data frame
increaseHSS <- data.frame(year = 1980:2010, value = 0)

# Extract the slope (coefficient) from the model and calculate its ratio
slopeHSS <- coef(modelcarWeightStandard)["Year"] * ratioHSS$avg_value

# Calculate the new values based on the slope
increaseHSS <- increaseHSS %>% mutate(new_value = value + slopeHSS * (year - 1980))

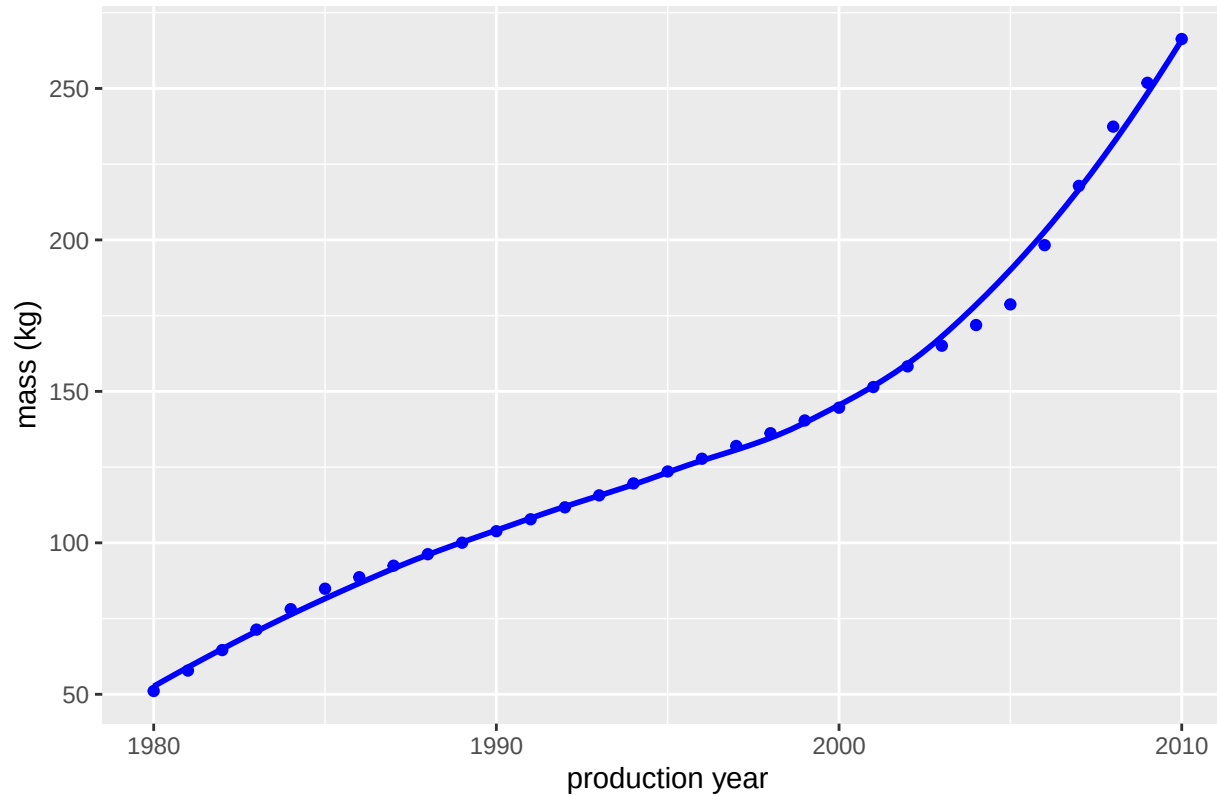
temp_old <- temp_old %>% mutate(value = value + increaseHSS$new_value)

g <- ggplot() +
  geom_point(data = temp_old, mapping = aes(x = productionYear, y = value), color = "blue") +
  geom_smooth(data = temp_old, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, color = "blue") +
  ggtitle("HSS in a standard car - JRC data (blue)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

HSS in a standard car – JRC data (blue)

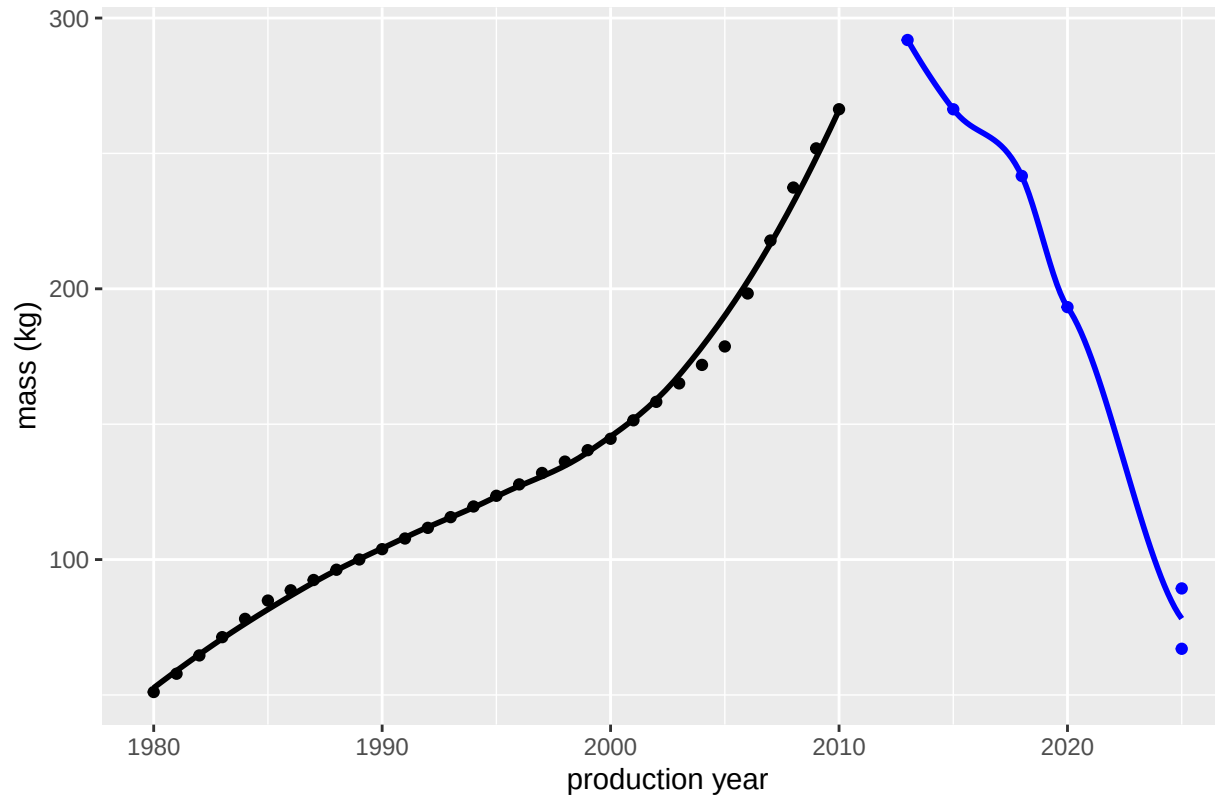


```
# get the new data
temp_new <- steel_combined_long %>% filter((Materials == "HSS") & (year > 2010))

g <- ggplot() +
  geom_point(data = temp_old, mapping = aes(x = productionYear, y = value), color = "black") +
  geom_smooth(data = temp_old, mapping = aes(x = productionYear, y = value), method = "loess", se = FALSE, color = "black") +
  geom_point(data = temp_new, mapping = aes(x = year, y = values), color = "blue") +
  geom_smooth(data = temp_new, mapping = aes(x = year, y = values), method = "loess", se = FALSE, color = "blue") +
  ggtitle("MildSteel in a standard car - JRC data (black) & new data (blue)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

MildSteel in a standard car – JRC data (black) & new data (blue)



```
# prepare temp_old and temp_new, so that they can be combined
temp_old <- temp_old %>% filter( productionYear <= 2010)
temp_new <- temp_new %>% select(-Scenario) %>% rename(productionYear = year, materialKeyLevel2 = Materials, value = values)

#remove the values from 2025, as they are unrealistically low
temp_new <- temp_new %>% filter(value > 150)

temp <- bind_rows(temp_old, temp_new)

# start model the new data (interpolate) - All the data are used this time
yearCalculate <- c(1980:2020)
HSSInterpolate <- data.frame(yearCalculate = yearCalculate)

mHSSCaculate <- loess(value ~ productionYear, data = temp, span = 0.5)
```

```

valueCalculate <- predict(mHSSCalculate, yearCalculate)
temp <- valueCalculate # this is necessary, but I have no idea why

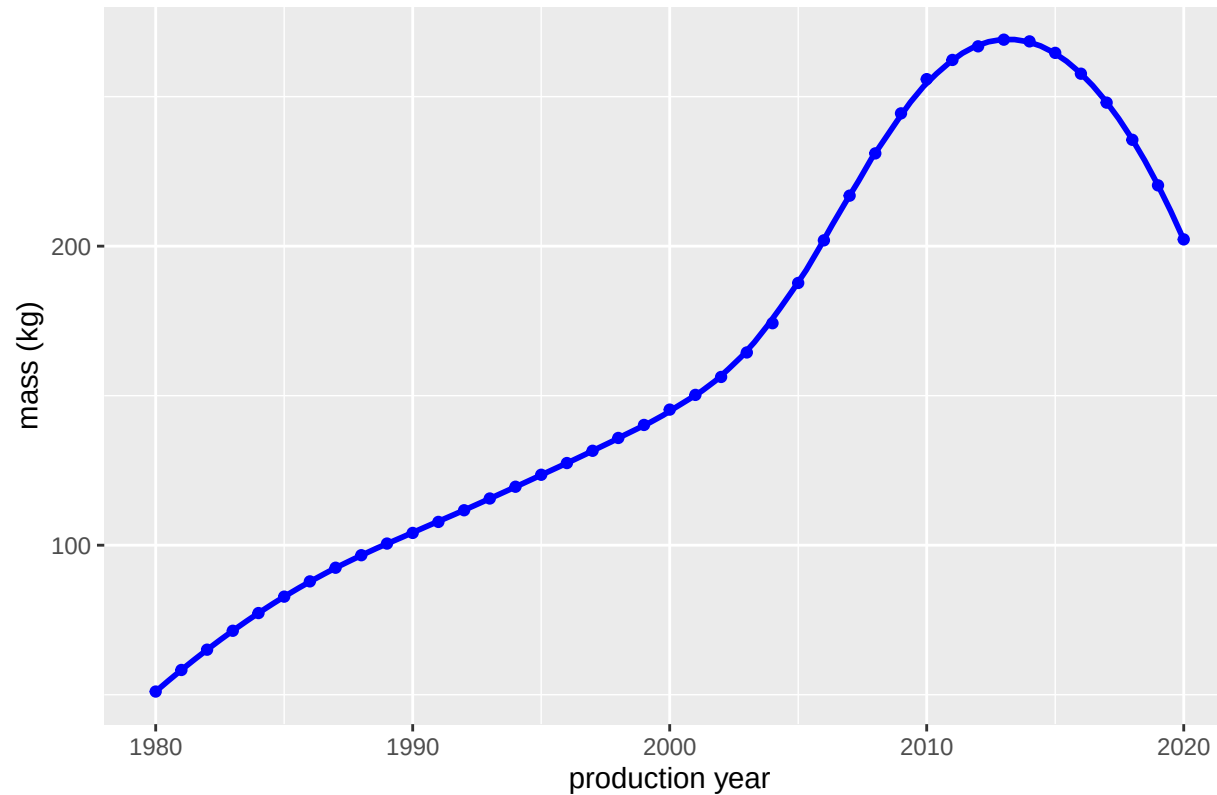
HSSCalculate <- as.data.frame(cbind(year = yearCalculate, valueCalculate = temp))

g <- ggplot() +
  geom_point(data = HSSCalculate, mapping = aes(x = year, y = valueCalculate), color = "blue") +
  geom_smooth(data = HSSCalculate, mapping = aes(x = year, y = valueCalculate), method = "loess", se = FALSE, span = 0.3, color = "blue") +
  ggtitle("HSS in a standard car - new data caluated") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

HSS in a standard car – new data caluated



```

# extrapolate to 2030 - it is assume as a first order, that the amount of mild steel only decreases, if it is replaced by other materials
temp <- HSSCalculate %>% filter(yearCalculate == 2020)

year <- c(2021:2030)

# Create a vector with identical values (e.g., 100)
identical_values <- rep(temp$valueCalculate, length(year))

# Combine the vectors into a data frame
df <- data.frame(year = year, valueCalculate = identical_values)
HSSCalculate <- bind_rows(HSSCalculate, df)

# extrapolate to 2030
#temp <- HSSCalculate %>% filter(year > 2017)

#mtempLinear <- lm(valueCalculate ~ year, data = temp)
#summary(mtempLinear)

#yearCalculate <- c(2020:2030)

#valueCalculate <- yearCalculate * mtempLinear$coefficients[2] + mtempLinear$coefficients[1]
#HSSCalculate_2 <- as.data.frame(cbind(yearCalculate, valueCalculate))

# Replace negative values with 0
#HSSCalculate_2 <- HSSCalculate_2 %>% mutate(across(everything(), ~ if_else(. < 0, 0, .)))

# rename the column headings
HSSCalculate <- HSSCalculate %>% rename(yearCalculate = year)
#HSSCalculate <- bind_rows(HSSCalculate, HSSCalculate_2)

g <- ggplot() +
  geom_point(data = HSSCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), color = "blue") +
  geom_smooth(data = HSSCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), method = "loess", se = FALSE, span = 0.3, color = "blue") +
  ggtitle("HSS in a standard car - Calculated values (2000:2030)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

HSS in a standard car – Calculated values (2000:2030)



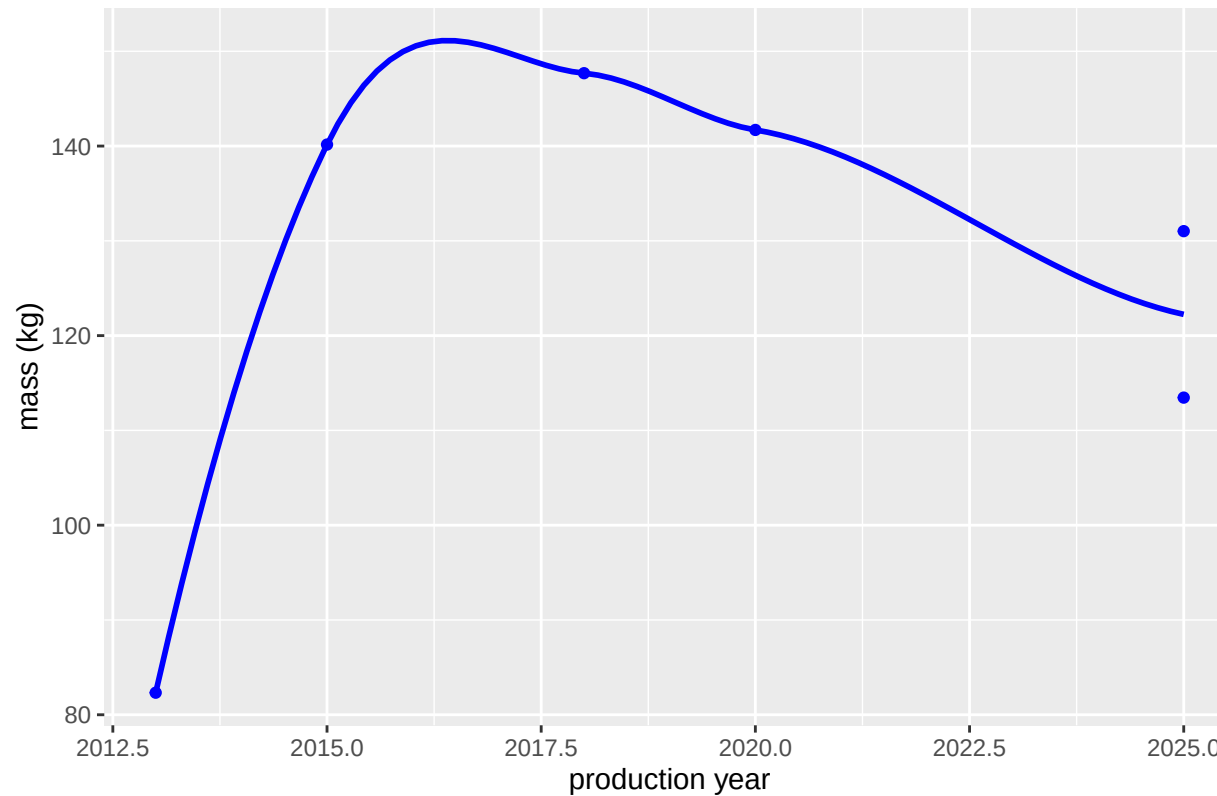
AHSS

```
# get the new data
temp_new <- steel_combined_long %>% filter(Materials == "AHSS")
temp_new <- temp_new %>% arrange(year)

g <- ggplot() +
  geom_point(data = temp_new, mapping = aes(x = year, y = values), color = "blue") +
  geom_smooth(data = temp_new, mapping = aes(x = year, y = values), method = "loess", se = FALSE, span = 0.75, color = "blue") +
  ggtitle("AHSS in a standard car - new data (blue)") +
  scale_x_continuous(name = "production year") +
```

```
scale_y_continuous(name = "mass (kg)")
plot(g)
```

AHSS in a standard car – new data (blue)



```
# remove the lower data point in 2025
temp_new <- temp_new %>% filter(!(year == 2025 & values < 120))

# start model the new data (interpolate) - All the data are used this time
yearCalculate <- c(2013:2025)
AHSSInterpolate <- data.frame(yearCalculate = yearCalculate)

mAHSSCaculate <- loess(values ~ year, data = temp_new, span = 0.85)
```

```

valueCalculate <- predict(mAHSSCalculate, yearCalculate)
temp <- valueCalculate # this is necessary, but I have no idea why

# interpolation in the time period, where data are available
AHSSCalculate <- as.data.frame(cbind(year = yearCalculate, valueCalculate = temp))

g <- ggplot() +
  geom_point(data = AHSSCalculate, mapping = aes(x = year, y = valueCalculate), color = "blue") +
  geom_smooth(data = AHSSCalculate, mapping = aes(x = year, y = valueCalculate), method = "loess", se = FALSE, span = 0.75, color = "blue") +
  ggtitle("AHSS in a standard car - new data caluated") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)

```

AHSS in a standard car – new data caluated



```
## extrapolate from the time point, where AHSS became available, to 2030
```

```
# extrapolate for earlier years
```

```
temp <- AHSSCalculate %>% filter(year < 2017)
```

```
mtempLinear1 <- lm(valueCalculate ~ year, data = temp)
```

```
summary(mtempLinear1)
```

```
##
```

```
## Call:
```

```
## lm(formula = valueCalculate ~ year, data = temp)
```

```
##
```

```
## Residuals:
```

```
##      1      2      3      4  
## -5.839  5.560  6.396 -6.117
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)  
## (Intercept) -45806.351   7626.923  -6.006   0.0266 *  
## year         22.799      3.786    6.022   0.0265 *  
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## Residual standard error: 8.466 on 2 degrees of freedom
```

```
## Multiple R-squared:  0.9477, Adjusted R-squared:  0.9216
```

```
## F-statistic: 36.26 on 1 and 2 DF,  p-value: 0.02649
```

```
yearCalculate <- c(2000:2012)
```

```
valueCalculate <- yearCalculate * mtempLinear1$coefficients[2] + mtempLinear1$coefficients[1]
```

```
AHSSCalculate_2 <- as.data.frame(cbind(yearCalculate, valueCalculate))
```

```
# valueCalculate need to be >0
```

```
AHSSCalculate_2 <- AHSSCalculate_2 %>% filter(valueCalculate > 0)
```

```
# extrapolate into the future
```

```
temp <- AHSSCalculate %>% filter(year > 2020)
```

```
mtempLinear2 <- lm(valueCalculate ~ year, data = temp)
```

```
summary(mtempLinear2)
```

```
##
## Call:
## lm(formula = valueCalculate ~ year, data = temp)
##
## Residuals:
```

	1	2	3	4	5
	0.08499	0.04763	-0.14507	-0.19269	0.20515

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	4673.33599	122.35980	38.19	3.95e-05 ***
year	-2.24322	0.06048	-37.09	4.31e-05 ***

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1913 on 3 degrees of freedom
## Multiple R-squared:  0.9978, Adjusted R-squared:  0.9971
## F-statistic: 1375 on 1 and 3 DF, p-value: 4.312e-05
```

```
yearCalculate <- c(2026:2030)

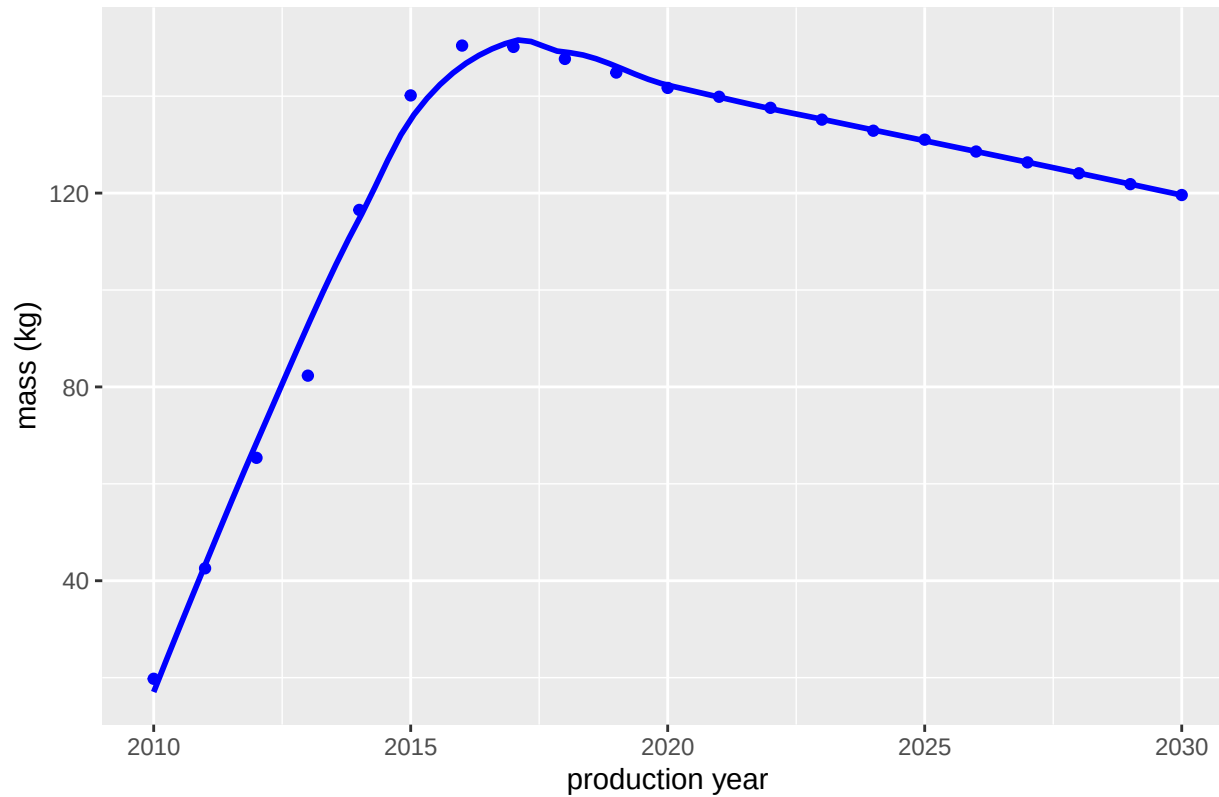
valueCalculate <- yearCalculate * mtempLinear2$coefficients[2] + mtempLinear2$coefficients[1]
AHSSCalculate_3 <- as.data.frame(cbind(yearCalculate, valueCalculate))

# rename the column headings
AHSSCalculate <- AHSSCalculate %>% rename(yearCalculate = year)
AHSSCalculate <- bind_rows(list(AHSSCalculate, AHSSCalculate_2, AHSSCalculate_3)) %>% arrange(yearCalculate)

g <- ggplot() +
  geom_point(data = AHSSCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), color = "blue") +
  geom_smooth(data = AHSSCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), method = "loess", se = FALSE, span = 0.5, color = "blue") +
  ggtitle("AHSS in a standard car - Calculated values (2003 - 2030)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

AHSS in a standard car – Calculated values (2003 – 2030)



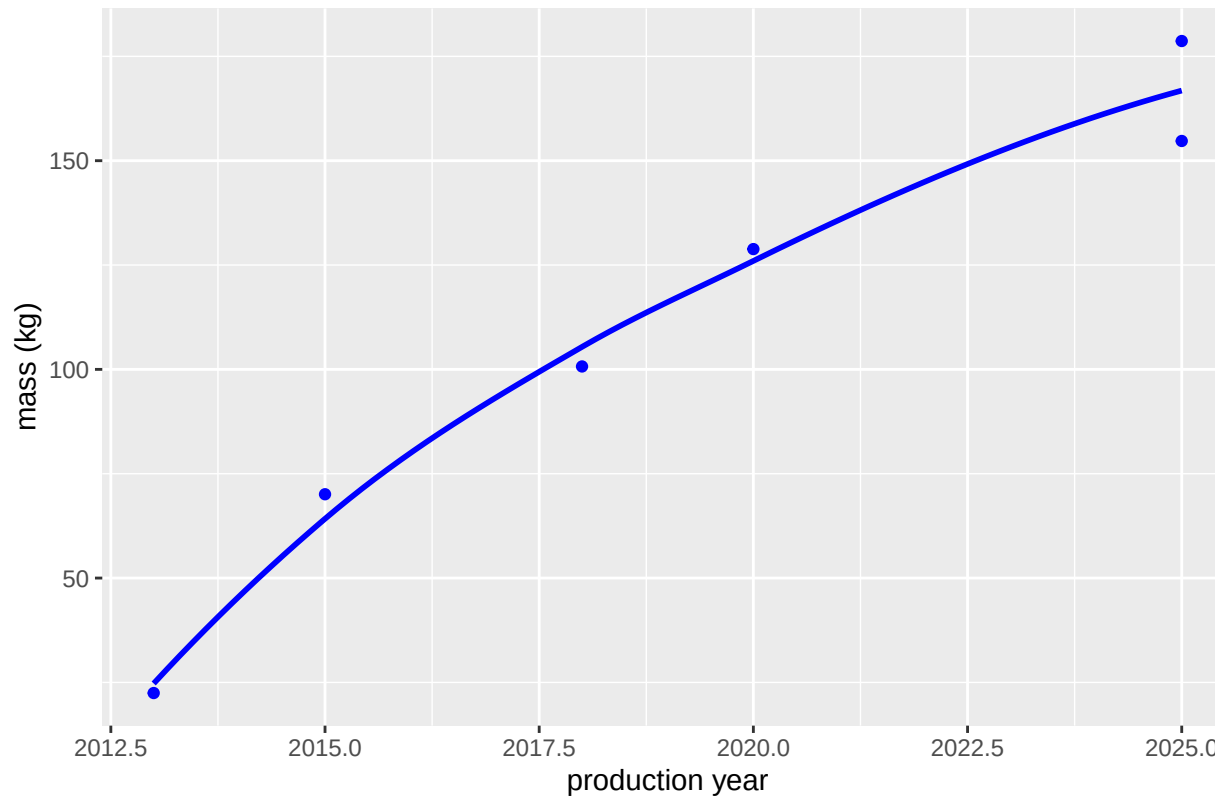
UHSS

```
# get the new data
temp_new <- steel_combined_long %>% filter(Materials == "UHSS")
temp_new <- temp_new %>% arrange(year)

g <- ggplot() +
  geom_point(data = temp_new, mapping = aes(x = year, y = values), color = "blue") +
  geom_smooth(data = temp_new, mapping = aes(x = year, y = values), method = "loess", se = FALSE, span = 1.0, color = "blue") +
  ggtitle("UHSS in a standard car - new data (blue)") +
  scale_x_continuous(name = "production year") +
```

```
scale_y_continuous(name = "mass (kg)")
plot(g)
```

UHSS in a standard car – new data (blue)



```
# start model the new data (interpolate) - All the data are used this time
yearCalculate <- c(2012:2025)
AHSSInterpolate <- data.frame(yearCalculate = yearCalculate)

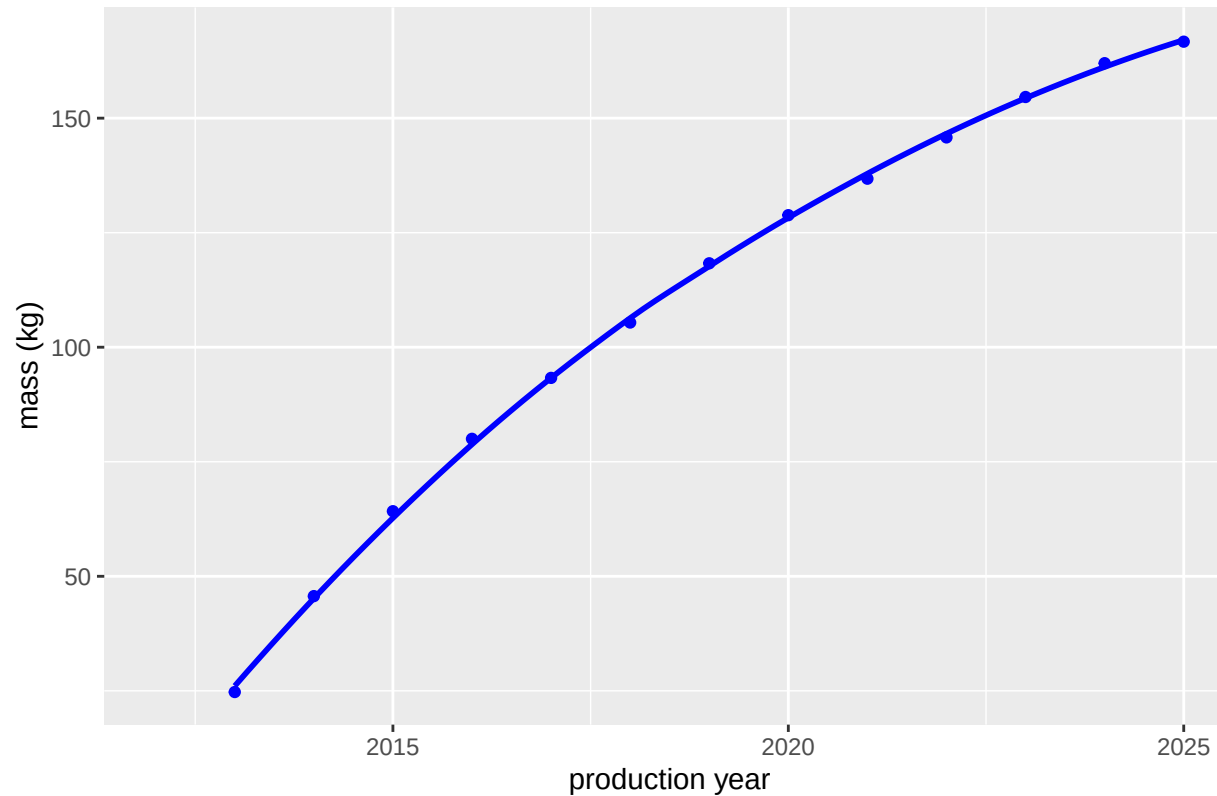
mUHSSCaculate <- loess(values ~ year, data = temp_new, span = 0.9)
valueCalculate <- predict(mUHSSCaculate, yearCalculate)
temp <- valueCalculate # this is necessary, but I have no idea why

UHSSCalculate <- as.data.frame(cbind(year = yearCalculate, valueCalculate = temp))
```

```
g <- ggplot() +
  geom_point(data = UHSSCalculate, mapping = aes(x = year, y = valueCalculate), color = "blue") +
  geom_smooth(data = UHSSCalculate, mapping = aes(x = year, y = valueCalculate), method = "loess", se = FALSE, span = 1, color = "blue") +
  ggtitle("UHSS in a standard car - new data caluated") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

UHSS in a standard car – new data caluated



```
# extrapolate to 2010
temp <- UHSSCalculate %>% filter(year < 2019)

mtempLinear <- lm(valueCalculate ~ year, data = temp)
```

```
summary(mtempLinear)
```

```
##
## Call:
## lm(formula = valueCalculate ~ year, data = temp)
##
## Residuals:
##      2      3      4      5      6      7
## -4.0072  0.8684  3.3478  3.0900  0.3387 -3.6378
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -3.229e+04  1.719e+03  -18.79 4.73e-05 ***
## year         1.606e+01  8.529e-01   18.83 4.69e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.568 on 4 degrees of freedom
## (1 observation deleted due to missingness)
## Multiple R-squared:  0.9888, Adjusted R-squared:  0.9861
## F-statistic: 354.4 on 1 and 4 DF,  p-value: 4.688e-05
```

```
yearCalculate <- c(2000:2011)
```

```
valueCalculate <- yearCalculate * mtempLinear$coefficients[2] + mtempLinear$coefficients[1]
UHSSCalculate_2 <- as.data.frame(cbind(yearCalculate, valueCalculate))
UHSSCalculate_2 <- UHSSCalculate_2 %>% filter(valueCalculate > 0)
```

```
# extrapolate to 2030
```

```
temp <- UHSSCalculate %>% filter(year > 2017)
```

```
mtempLinear <- lm(valueCalculate ~ year, data = temp)
summary(mtempLinear)
```

```
##
## Call:
## lm(formula = valueCalculate ~ year, data = temp)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
--	-----	----	--------	----	-----

```
## -3.8343 -0.6598  0.8561  1.6496  2.1229
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.752e+04  8.103e+02  -21.63 6.38e-07 ***
## year         8.738e+00  4.008e-01   21.80 6.09e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.598 on 6 degrees of freedom
## Multiple R-squared:  0.9875, Adjusted R-squared:  0.9855
## F-statistic: 475.2 on 1 and 6 DF,  p-value: 6.087e-07
```

```
yearCalculate <- c(2026:2030)

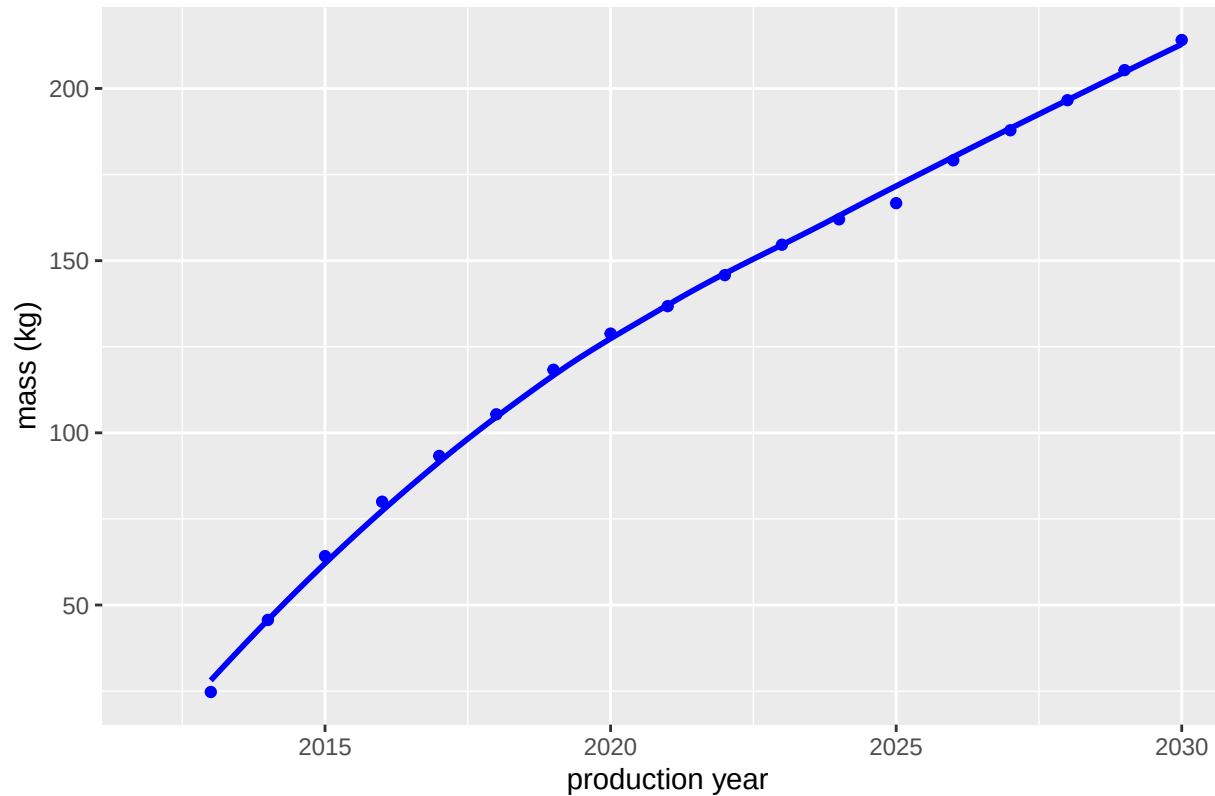
valueCalculate <- yearCalculate * mtempLinear$coefficients[2] + mtempLinear$coefficients[1]
UHSSCalculate_3 <- as.data.frame(cbind(yearCalculate, valueCalculate))

# rename the column headings
UHSSCalculate <- UHSSCalculate %>% rename(yearCalculate = year)
UHSSCalculate <- bind_rows(UHSSCalculate, UHSSCalculate_2, UHSSCalculate_3) %>% arrange()

g <- ggplot() +
  geom_point(data = UHSSCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), color = "blue") +
  geom_smooth(data = UHSSCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), method = "loess", se = FALSE, span= 1, color = "blue") +
  ggtitle("UHSS in a standard car - Calculated values (2010:2030)") +
  scale_x_continuous(name = "production year") +
  scale_y_continuous(name = "mass (kg)")

plot(g)
```

UHSS in a standard car – Calculated values (2010:2030)



Combine all the new data of iron and steel

```
castIronCalculate <- castIronCalculate %>% rename(yearCalculate = year)

FeCalculate.standard <- full_join(castIronCalculate, mildSteelCalculate, by = c("yearCalculate"))
FeCalculate.standard <- full_join(FeCalculate.standard, HSSCalculate, by = c("yearCalculate"))
FeCalculate.standard <- full_join(FeCalculate.standard, AHSSCalculate, by = c("yearCalculate"))
FeCalculate.standard <- full_join(FeCalculate.standard, UHSSCalculate, by = c("yearCalculate"))

FeCalculate.standard <- FeCalculate.standard %>% rename("castIronCalculate" = "castingIron", "mildSteelCalculate" = "valueCalculate.x", "HSSCalculate" = "valueCalculate.y",
  "AHSSCalculate" = "valueCalculate.x.x", "UHSSCalculate" = "valueCalculate.y.y")
```

```

# Replace NA with 0
FeCalculate.standard <- FeCalculate.standard %>% mutate(across(everything(), ~ replace_na(., 0)))

# Generate the total of all iron and steel
FeCalculate.standard <- FeCalculate.standard %>% mutate(totalIronSteelCalculate = castIronCalculate + mildSteelCalculate + HSSCalculate + AHSSCalculate + UHSSCalculate)
FeCalculate.standard <- FeCalculate.standard %>% rename("year" = "yearCalculate") %>% drop_na()

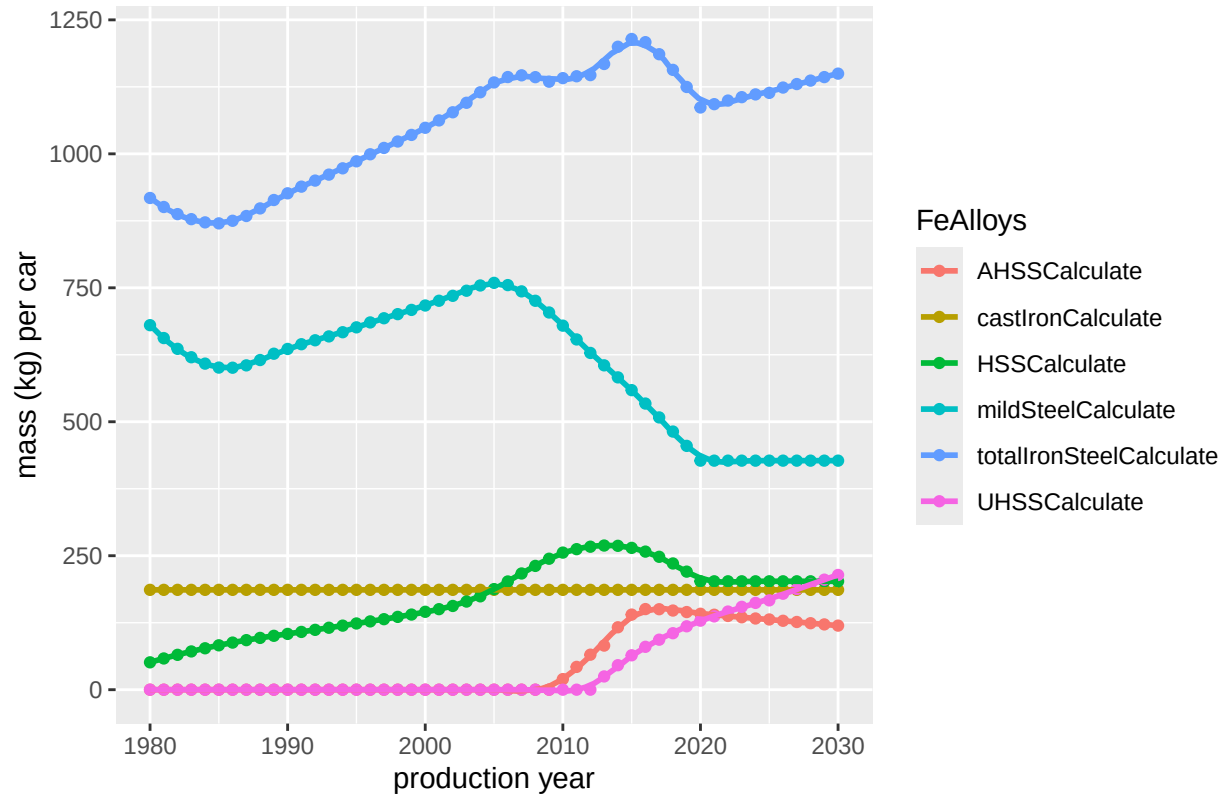
# generate long table
FeCalculate.standard_long <- FeCalculate.standard %>% pivot_longer(-year, names_to = "FeAlloys", values_to = "values")

g <- ggplot() +
  geom_point(data = FeCalculate.standard_long, mapping = aes(x = year, y = values, color = FeAlloys)) +
  geom_smooth(data = FeCalculate.standard_long, mapping = aes(x = year, y = values, color = FeAlloys), method = "loess", se = FALSE, span= 0.2) +
  ggtitle("Iron and steel alloys for standard car (calculated from 1980 to 2030)") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)

```

Iron and steel alloys for standard car (calculated from 1980 to 2030)



Derive the composition of all the car segments based on the standard car (V0101010000)

```
# For all the different segments based on the factors documented in derivedFeComposition

FeCalculate <- crossing(FeCalculate.standard, derivedFeComposition) %>%
  mutate(
    castIronCalculate = castIronCalculate * fraction,
    mildSteelCalculate = mildSteelCalculate * fraction,
    HSSCalculate = HSSCalculate * fraction,
    AHSSCalculate = AHSSCalculate * fraction,
    UHSSCalculate = UHSSCalculate * fraction,
```

```

    totalIronSteelCalculate = totalIronSteelCalculate * fraction
  ) %>%
  select(-fraction)

# for all the different drivetrains does NOT yet include the differences due to the different drivetrains

FeCalculate <- FeCalculate %>% mutate(productKeyLevel1 = "elvPetrol")

additionalproductKeyLevel1 <- c("elvDiesel", "elvBEV", "elvHEV", "elvPHEV", "elvOther")

# Duplicate the dataset 5 times, each time with a new description
FeCalculate <- bind_rows(FeCalculate, # Keep the original dataset
  map_dfr(additionalproductKeyLevel1, ~ FeCalculate %>%
    mutate(productKeyLevel1 = .x))
)

FeCalculate <- FeCalculate %>% left_join(productKeys, by = c("productKeyLevel1", "segment"))
#FeCalculate

```

Magnesium content. (Still missing - move to front and then reduce amount of Al by 1/3 more than just weight to weight)

```

# bring it to longer format, so that handling becomes easier
magnesium_content <- magnesium_content %>% pivot_longer(cols = -c(year, segment), names_to = "drivetrain", values_to = "value")

# calculate the standard magnesium content as mean between c and d segment and the relocate columns, so that it be combined with magnesium_content
magnesium_content_standard <- magnesium_content %>% filter(segment == "C" | segment == "D") %>% group_by(drivetrain, year) %>% summarise(value = mean(value), segment = "standard")
  relocate(year) %>% relocate(segment, .after = year)

# combine to get everything
magnesium_content <- bind_rows(magnesium_content, magnesium_content_standard)

magnesiumCalculate <- data.frame(yearCalculate = numeric(), valueCalculate = numeric(), drivetrain = character(), segment = character())

unique_drivetrain <- unique(magnesium_content$drivetrain)
for (i in 1:length(unique_drivetrain)) {
  temp_drivetrain <- magnesium_content %>% filter(drivetrain == unique_drivetrain[i])
  unique_segment <- unique(temp_drivetrain$segment)
  for (j in 1:length(unique_segment)) {
    yearCalculate <- c(1980:2050)
    temp_model <- temp_drivetrain %>% filter((segment == unique_segment[j]) & (value > 0))
    if(dim(temp_model)[1] == 0) {
      valueCalculate <- rep(0, (2050-1980+1))
      tempCalculate <- data.frame(yearCalculate = yearCalculate, valueCalculate = valueCalculate, drivetrain = unique_drivetrain[i],

```

```

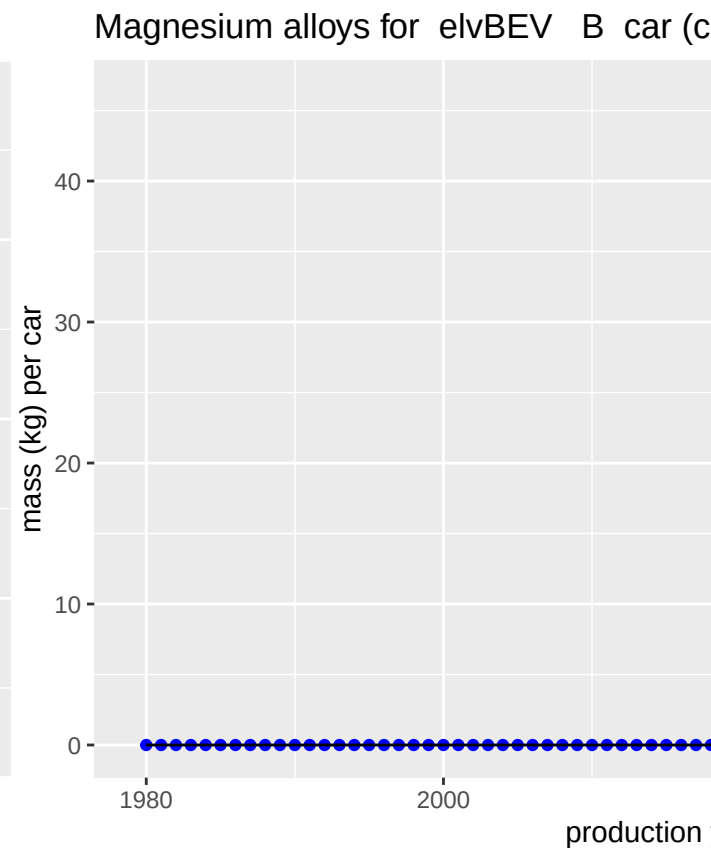
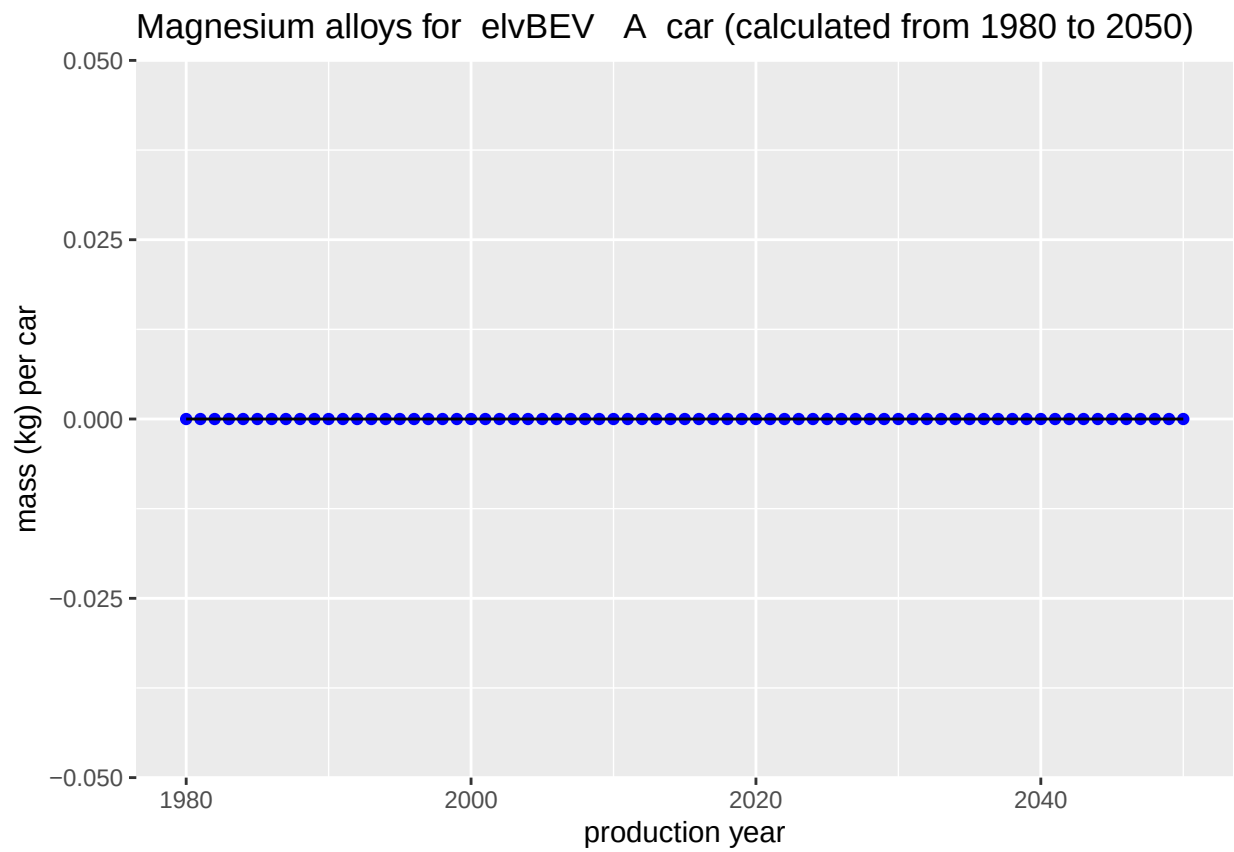
        segment = unique_segment[j])
    }
    else {
      mtempLinear <- lm(value ~ year, data = temp_model)
      summary(mtempLinear)

      valueCalculate <- yearCalculate * mtempLinear$coefficients[2] + mtempLinear$coefficients[1]
      tempCalculate <- as.data.frame(cbind(yearCalculate, valueCalculate))
      tempCalculate <- tempCalculate %>% mutate(valueCalculate = replace(valueCalculate, valueCalculate < 0, 0)) %>% mutate(drivetrain = unique_drivetrain[i]) %>%
        mutate(segment = unique_segment[j])
    }

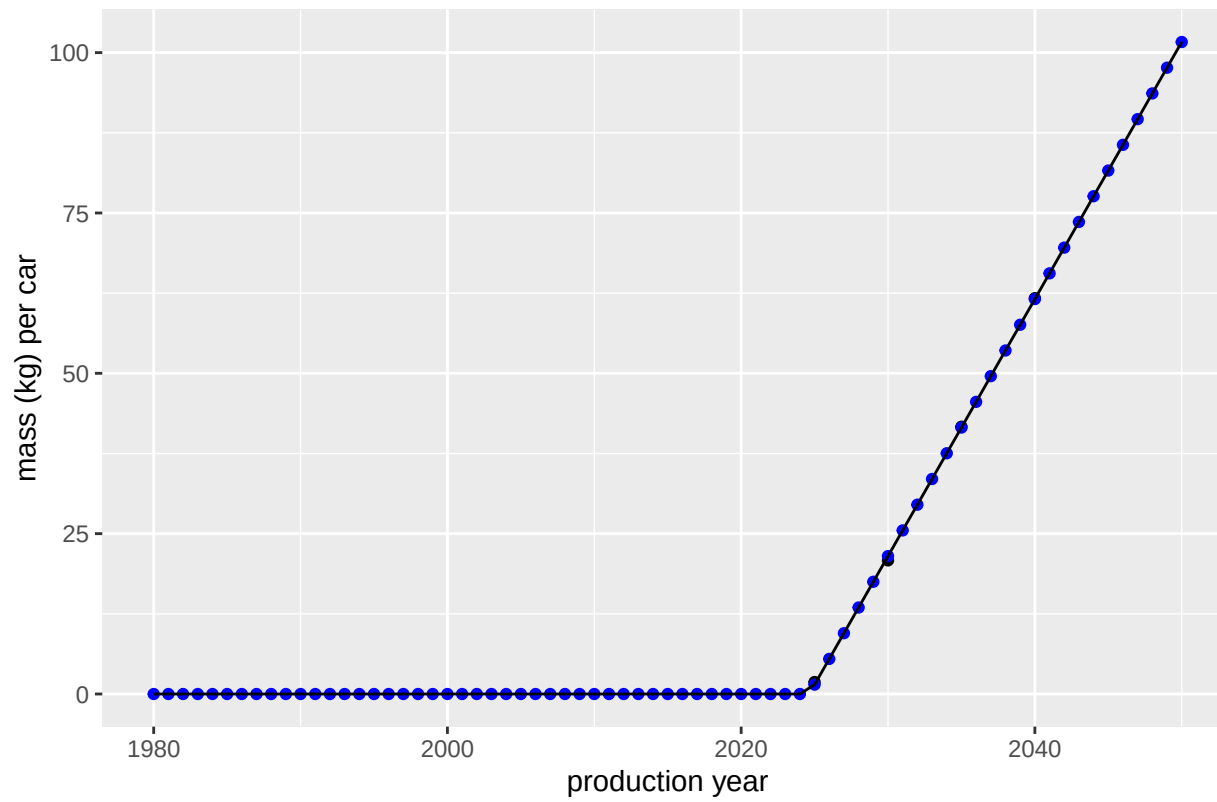
    g <- ggplot() +
      geom_point(data = temp_model, mapping = aes(x = year, y = value)) +
      geom_point(data = tempCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), color = "blue") +
      geom_line(data = tempCalculate, mapping = aes(x = yearCalculate, y = valueCalculate), method = "lm", se = FALSE) +
      ggtitle(paste("Magnesium alloys for ", magnesium_content$drivetrain[i], " ", unique_segment[j], " car (calculated from 1980 to 2050)")) +
      xlab("production year") +
      ylab("mass (kg) per car")

    plot(g)
    magnesiumCalculate <- bind_rows(magnesiumCalculate, tempCalculate)
  }
}

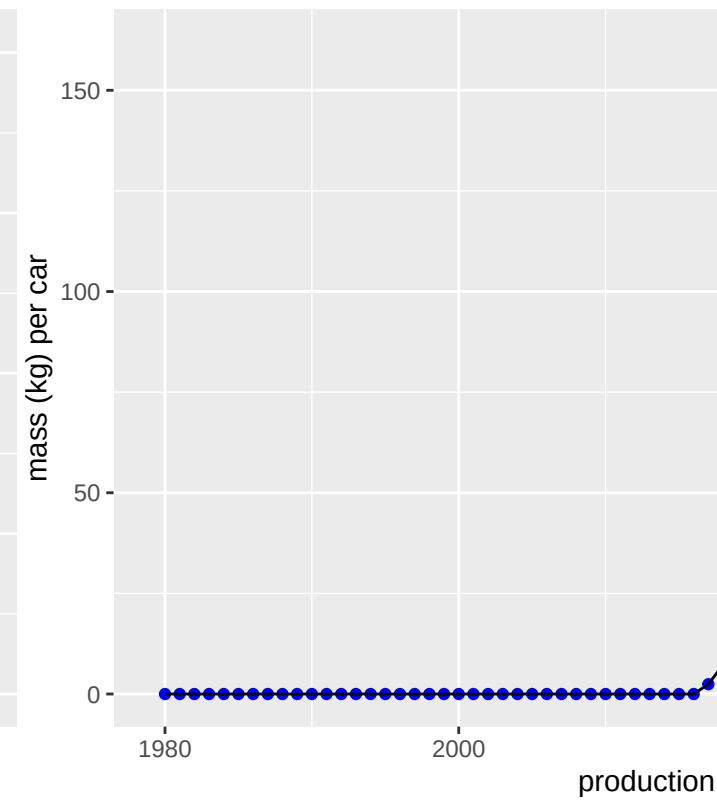
```



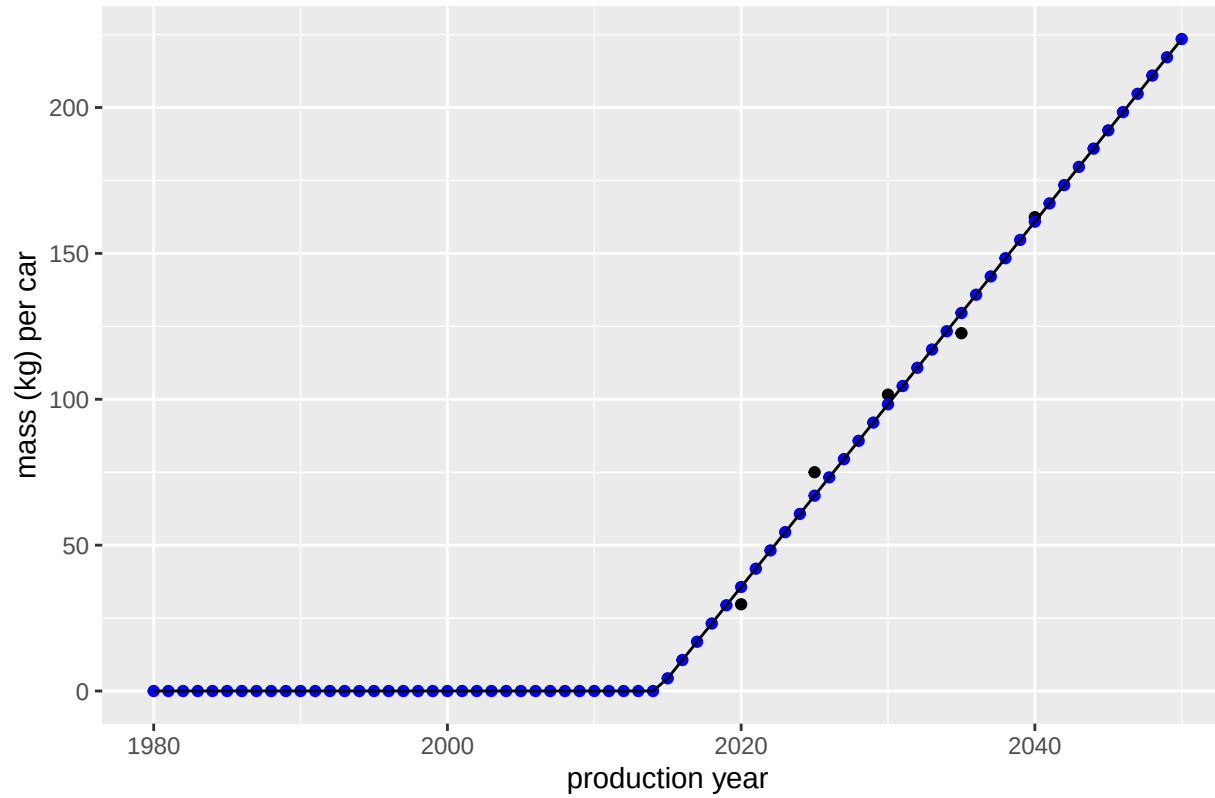
Magnesium alloys for elvBEV C car (calculated from 1980 to 2050)



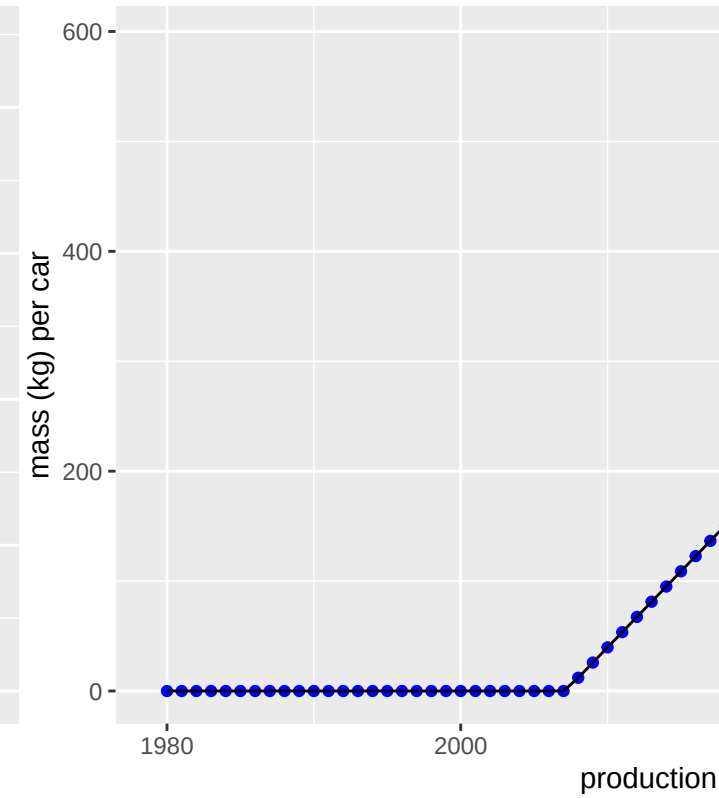
Magnesium alloys for elvBEV D car (calculated from 1980 to 2050)

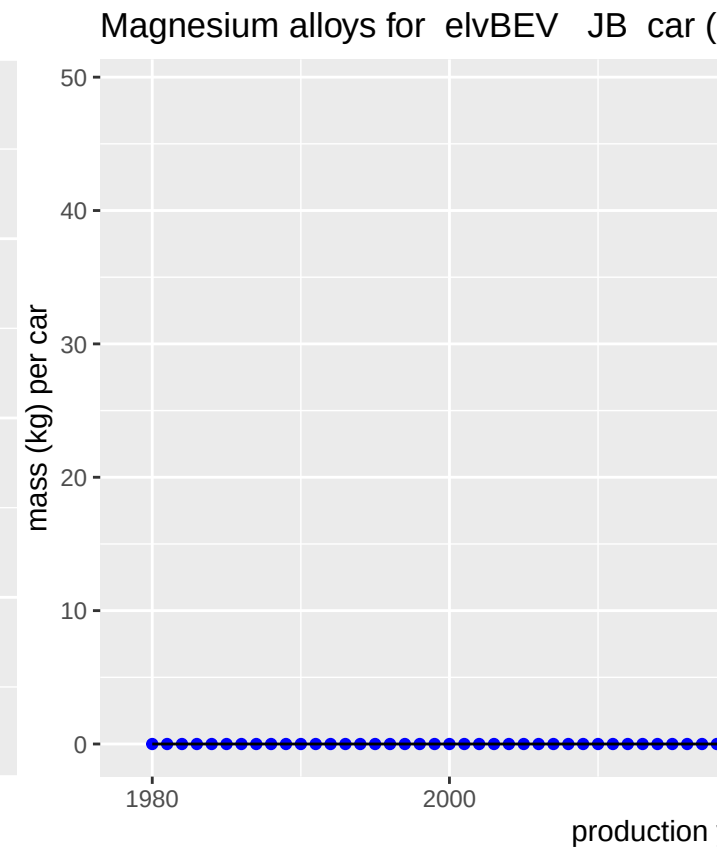
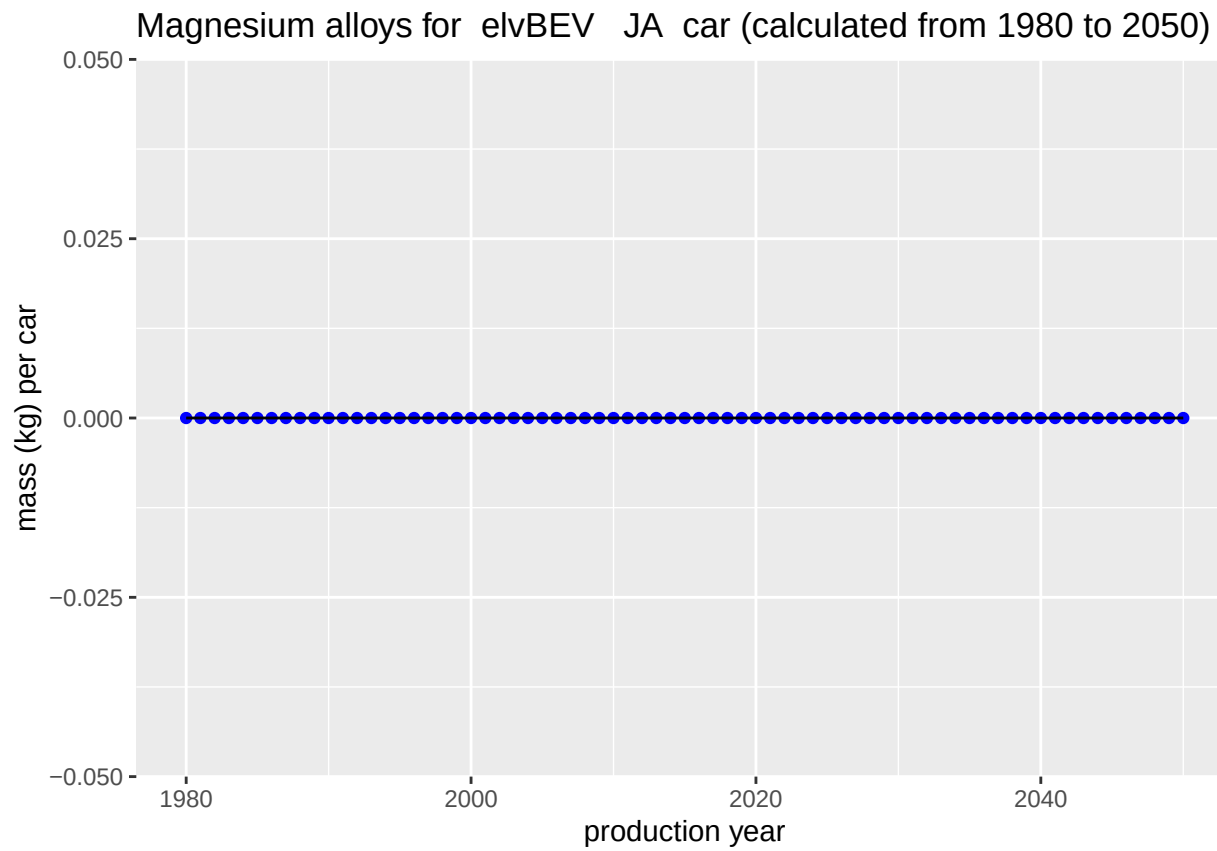


Magnesium alloys for elvBEV E car (calculated from 1980 to 2050)

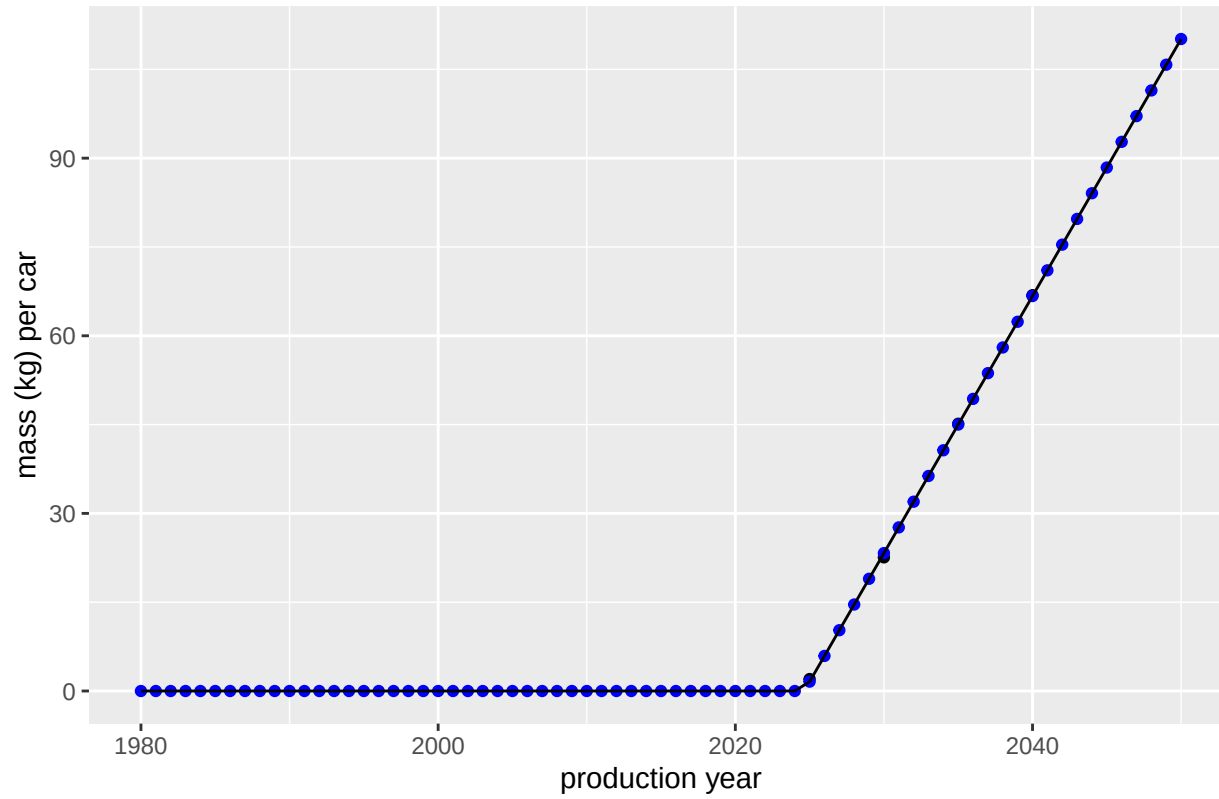


Magnesium alloys for elvBEV F car (calculated from 1980 to 2050)

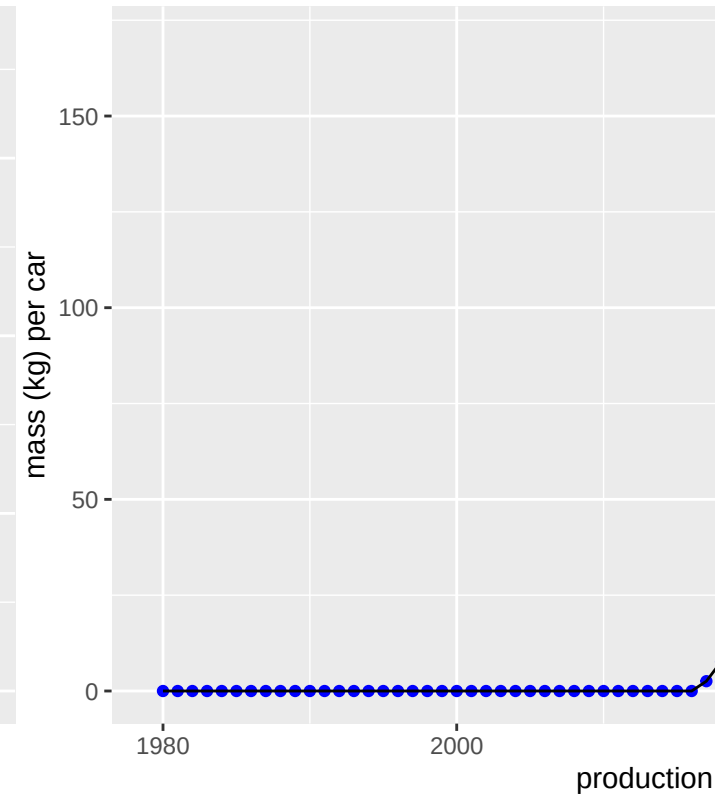




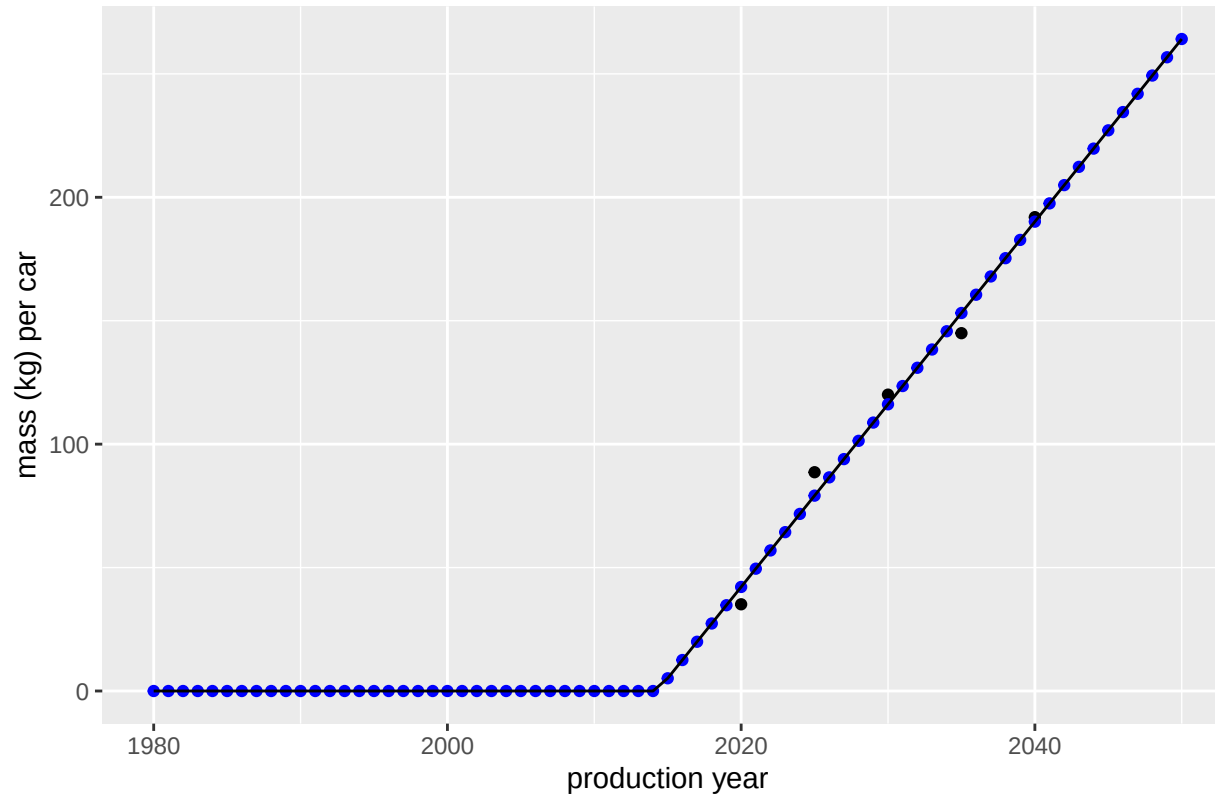
Magnesium alloys for elvBEV JC car (calculated from 1980 to 2050)



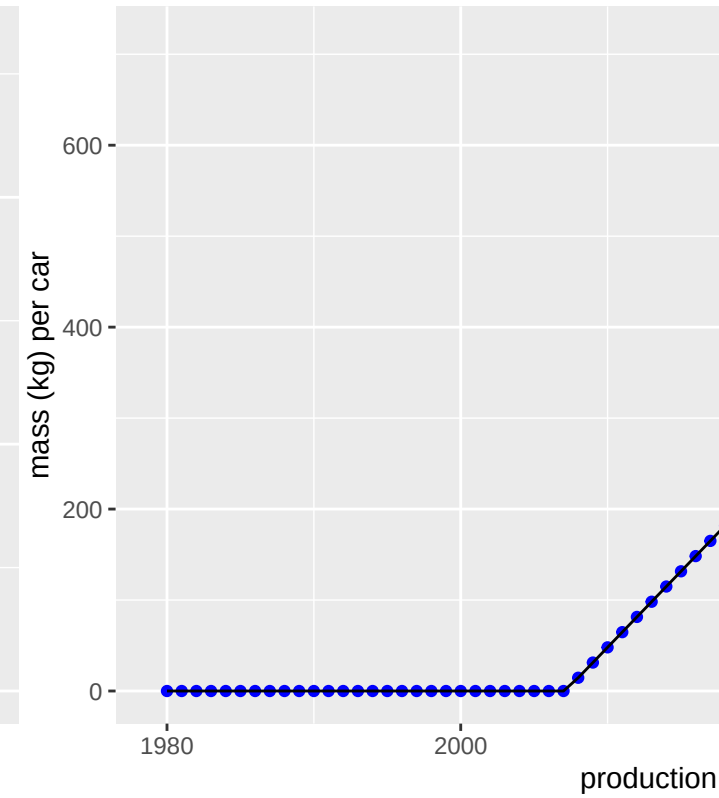
Magnesium alloys for elvBEV JD car



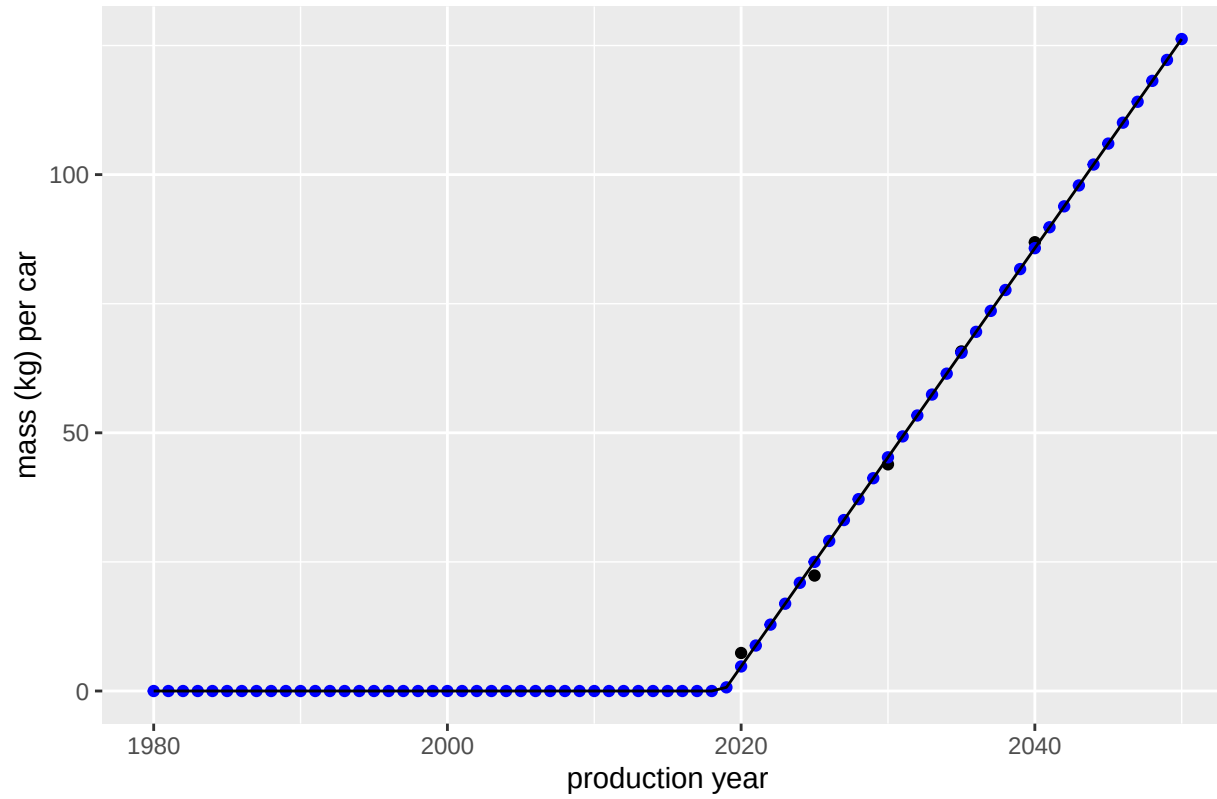
Magnesium alloys for elvBEV JE car (calculated from 1980 to 2050)



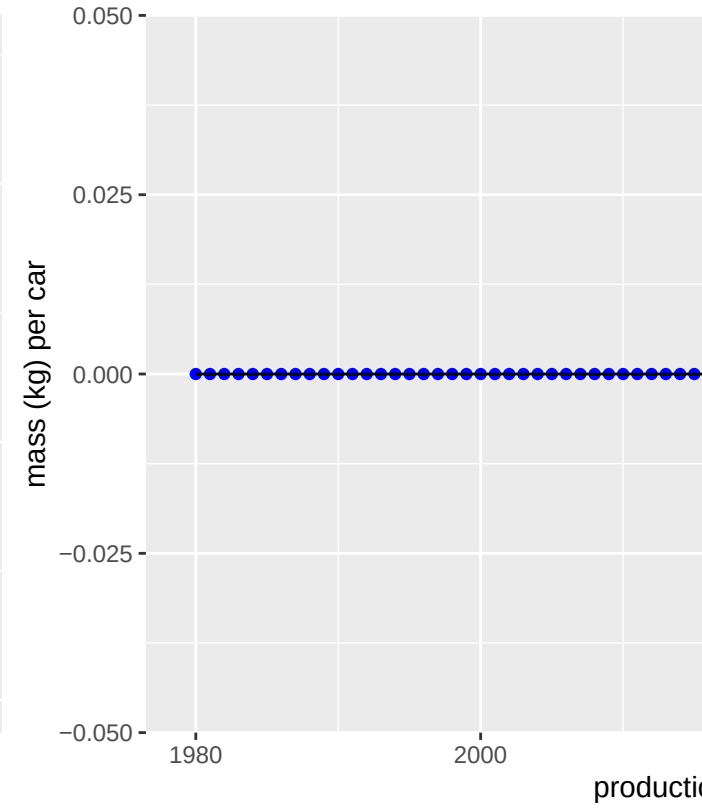
Magnesium alloys for elvBEV JF car



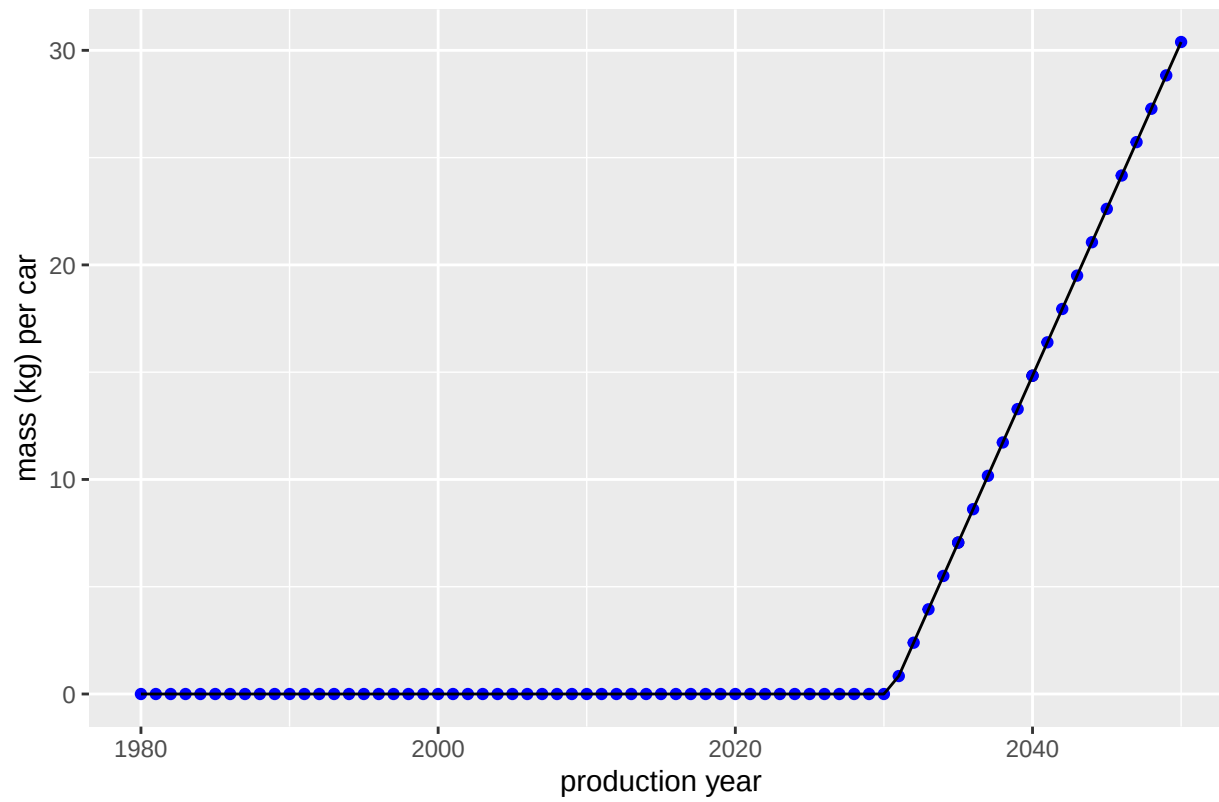
Magnesium alloys for elvBEV standard car (calculated from 1980 to 2050)



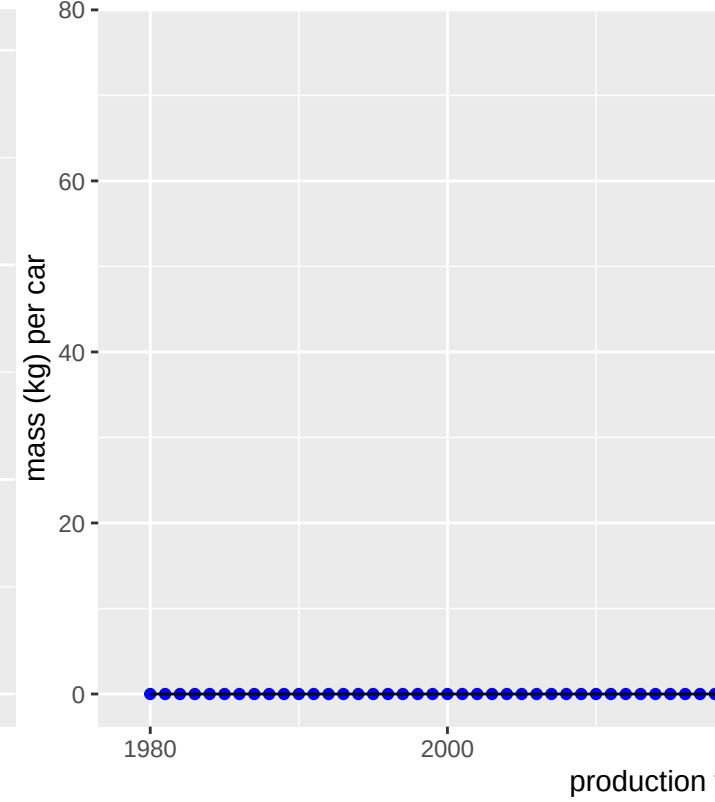
Magnesium alloys for elvDiesel A



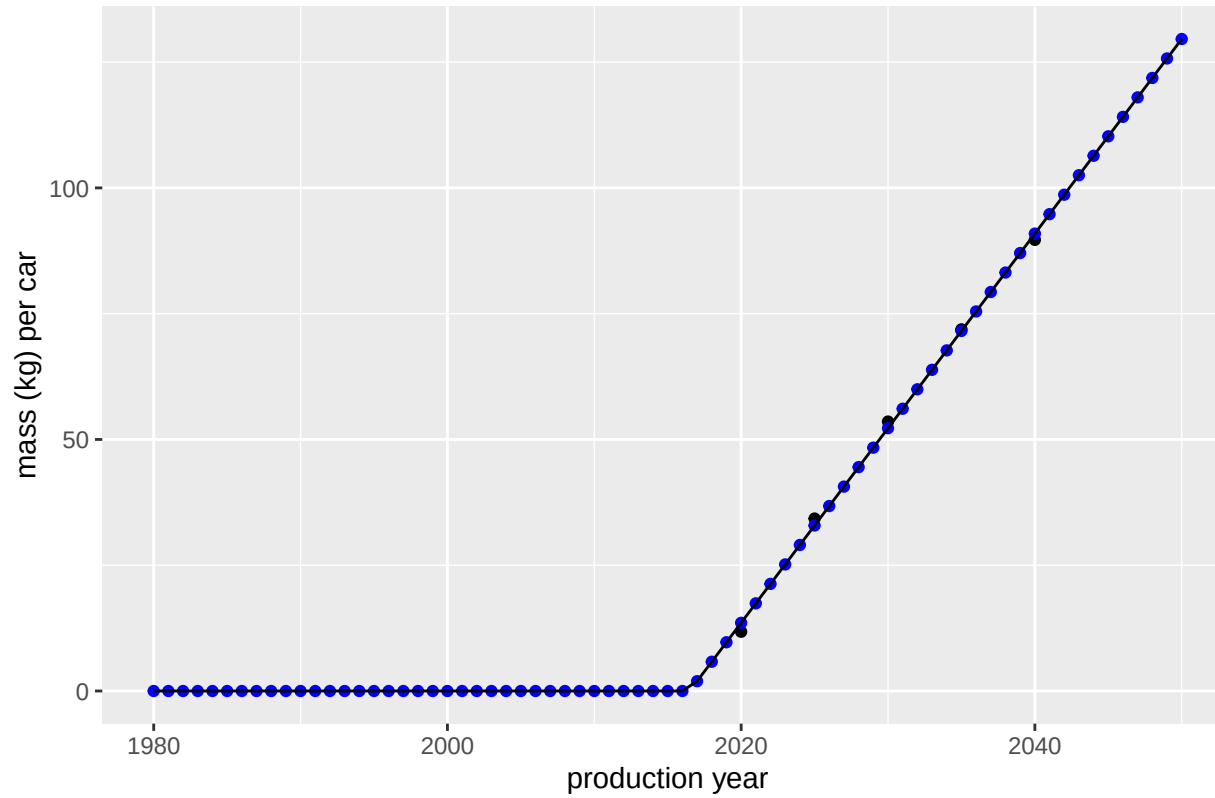
Magnesium alloys for elvDiesel B car (calculated from 1980 to 2050)



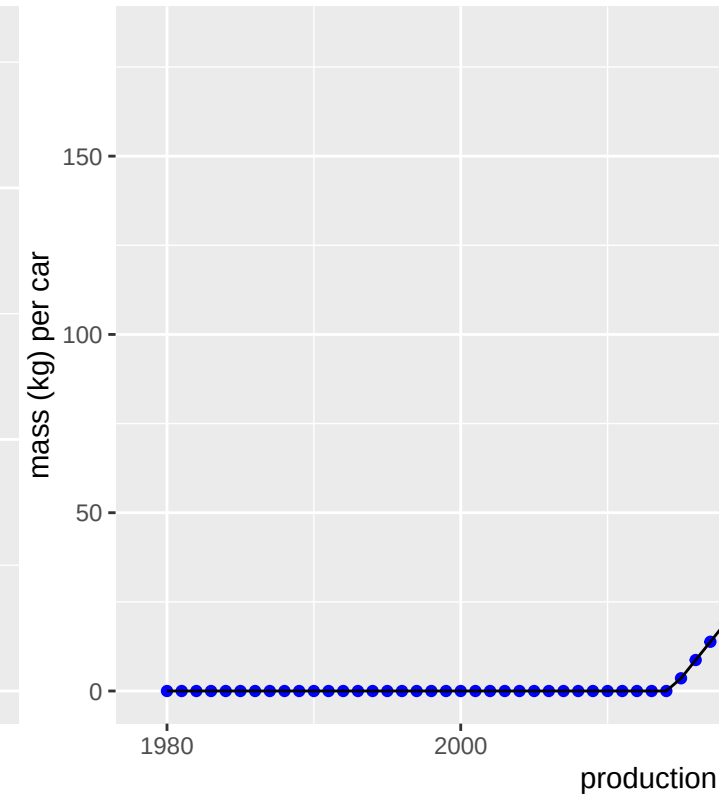
Magnesium alloys for elvDiesel C car



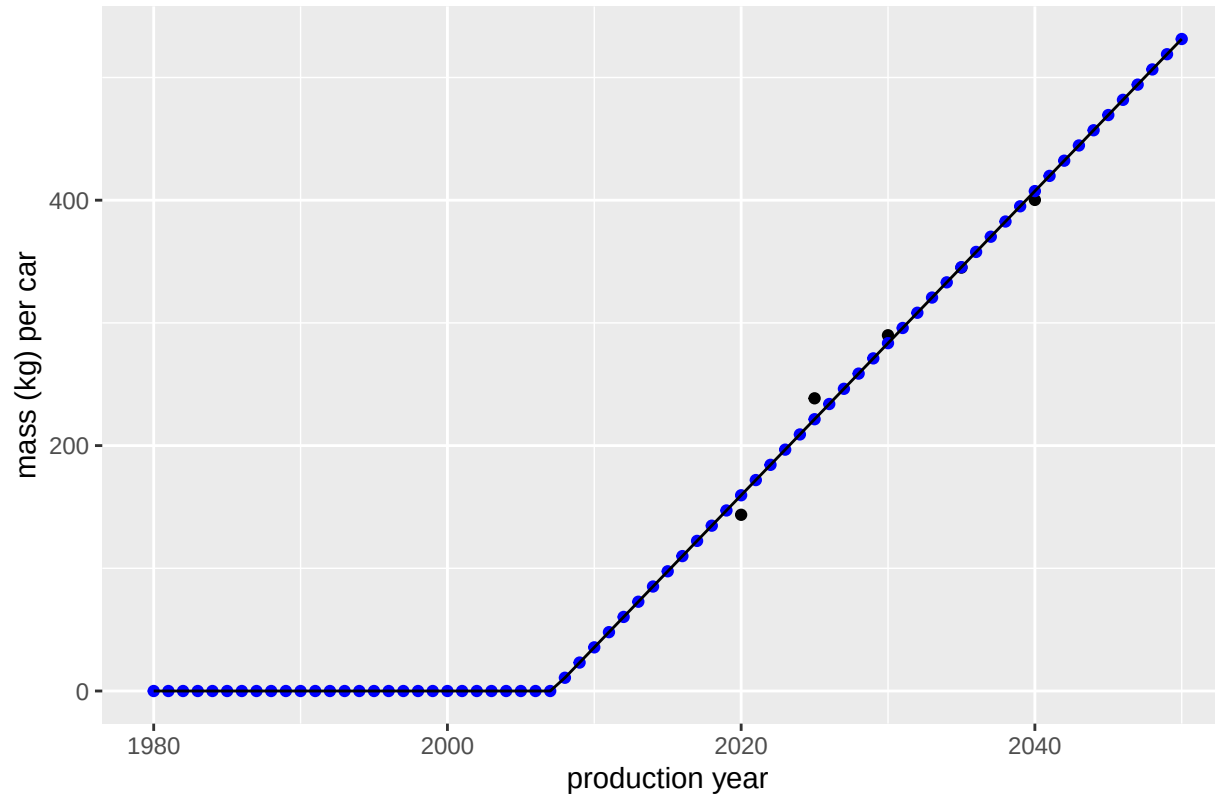
Magnesium alloys for elvDiesel D car (calculated from 1980 to 2050)



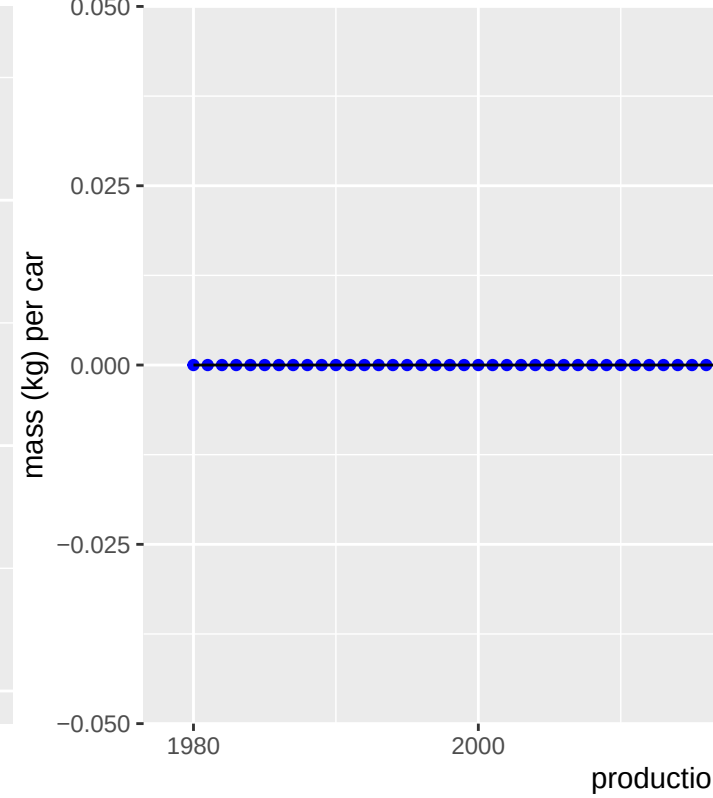
Magnesium alloys for elvDiesel E car



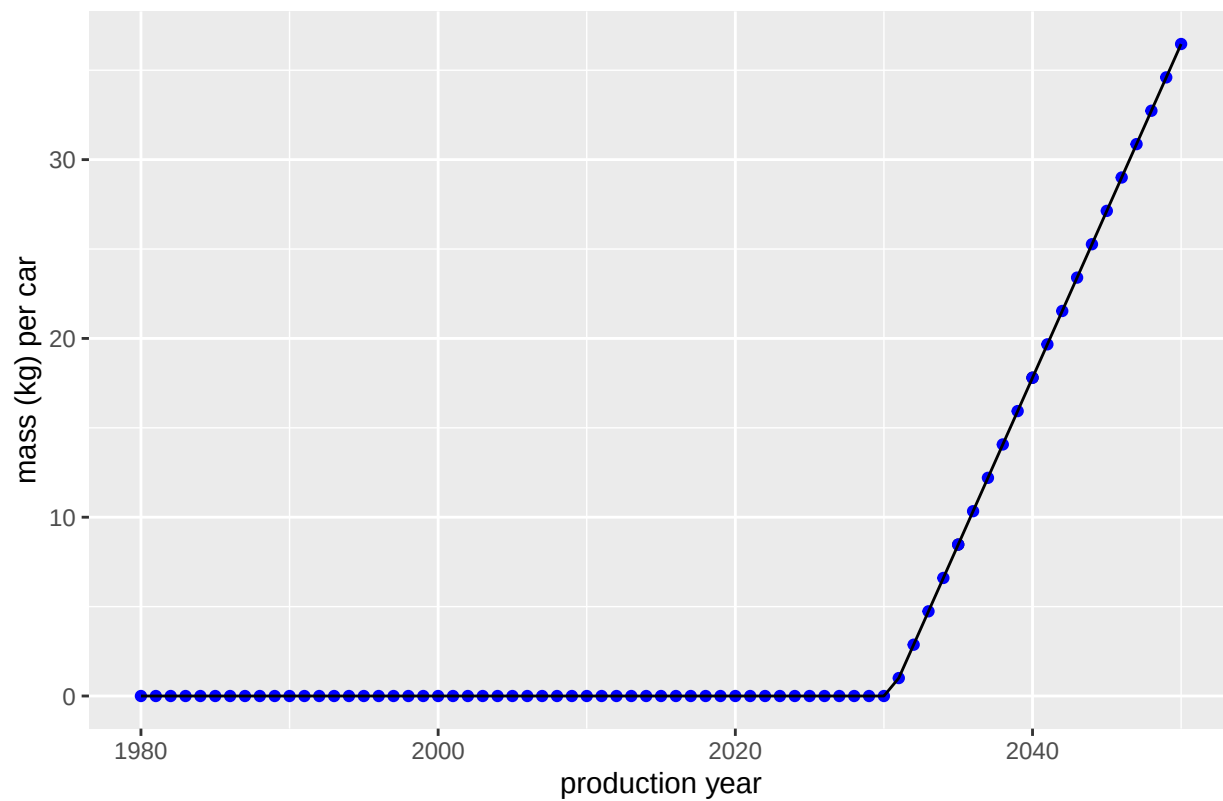
Magnesium alloys for elvDiesel F car (calculated from 1980 to 2050)



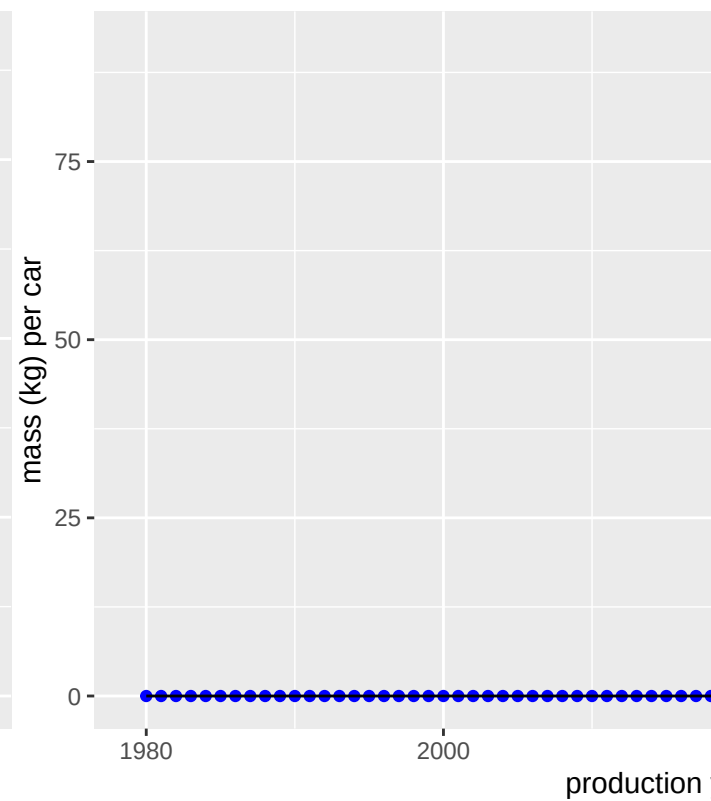
Magnesium alloys for elvDiesel JA



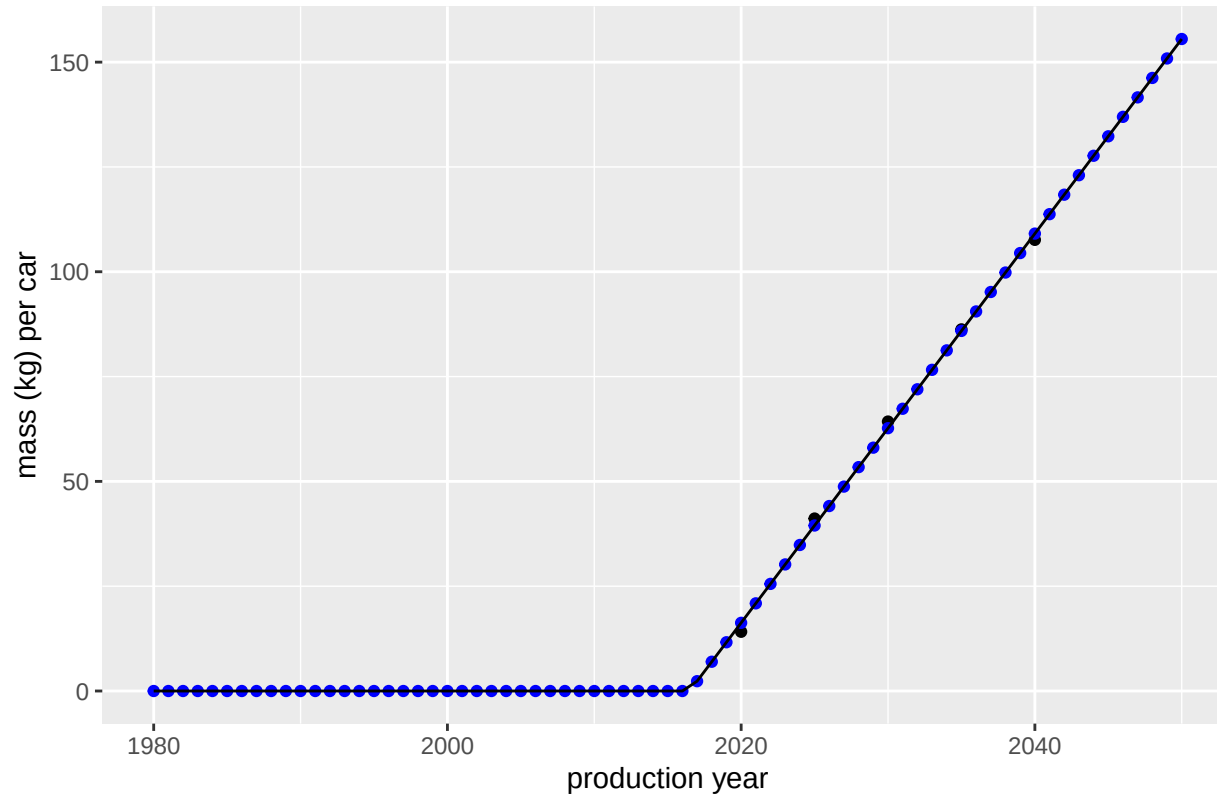
Magnesium alloys for elvDiesel JB car (calculated from 1980 to 2050)



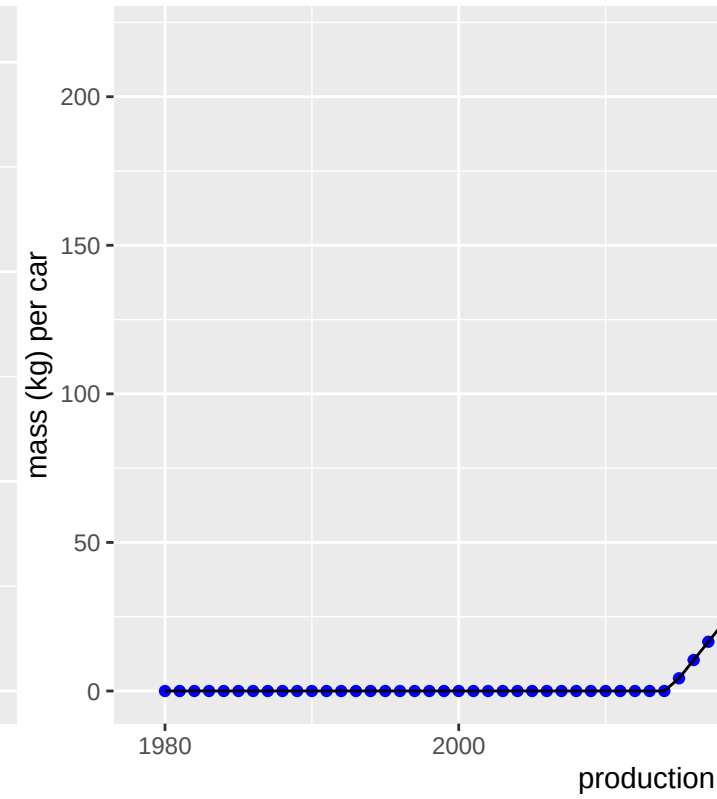
Magnesium alloys for elvDiesel JC car



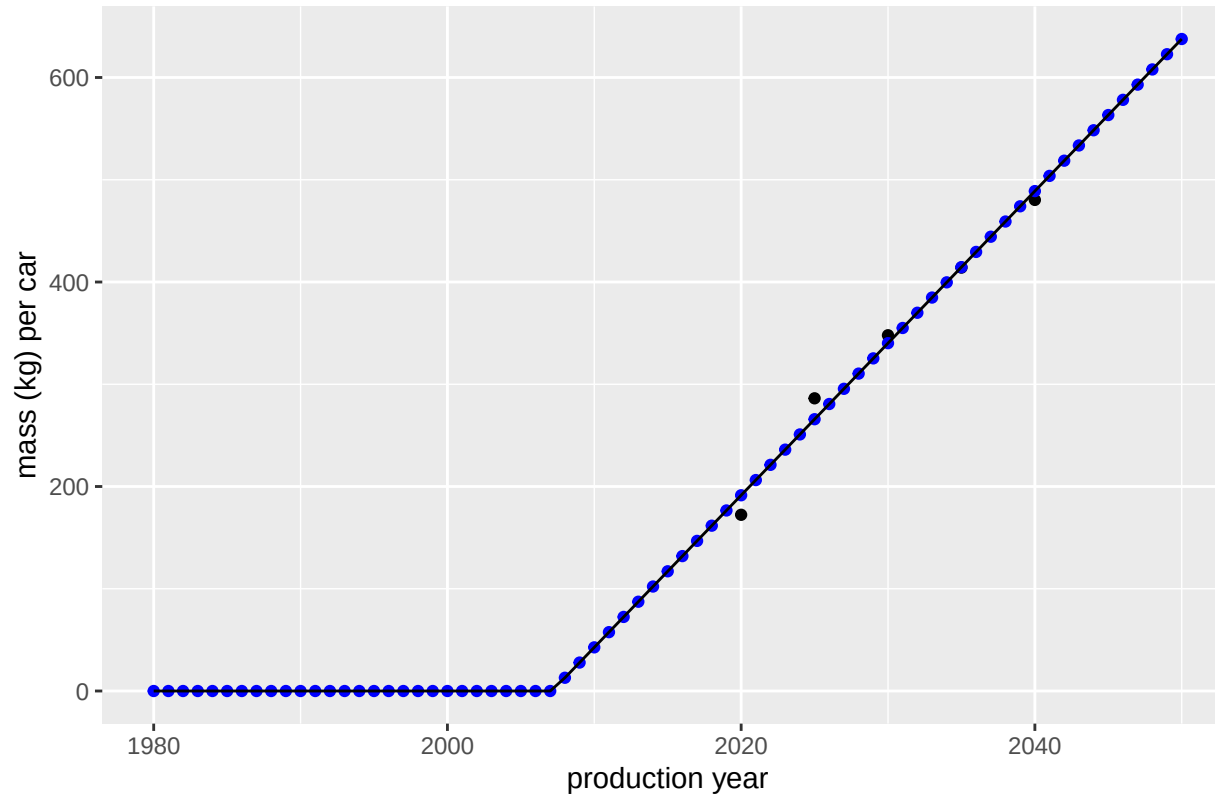
Magnesium alloys for elvDiesel JD car (calculated from 1980 to 2050)



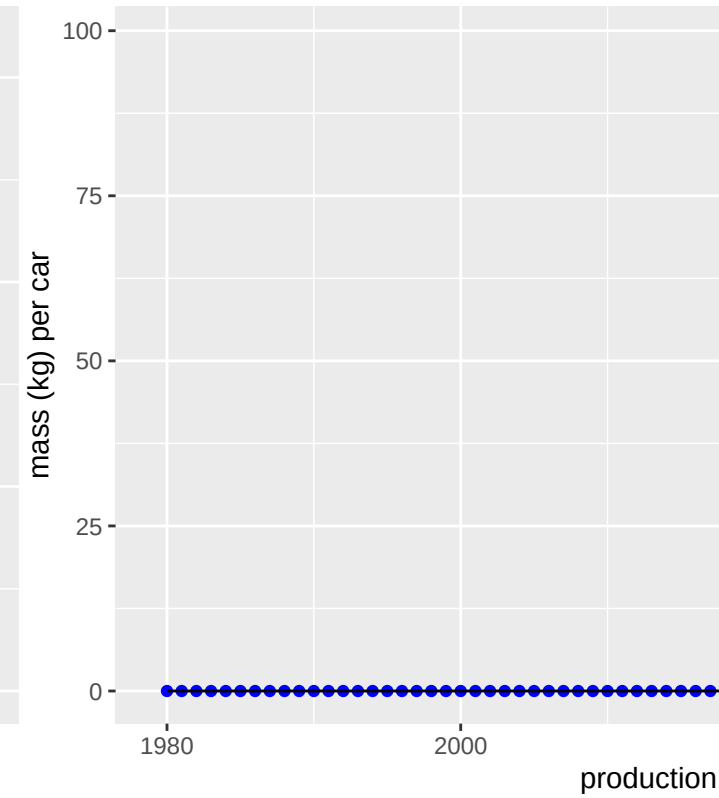
Magnesium alloys for elvDiesel JE car

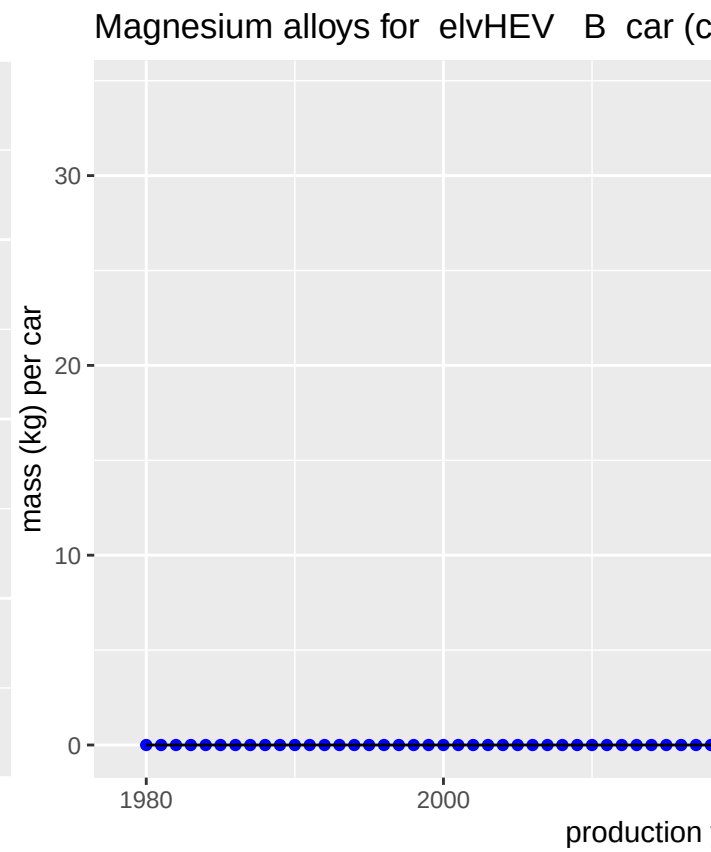
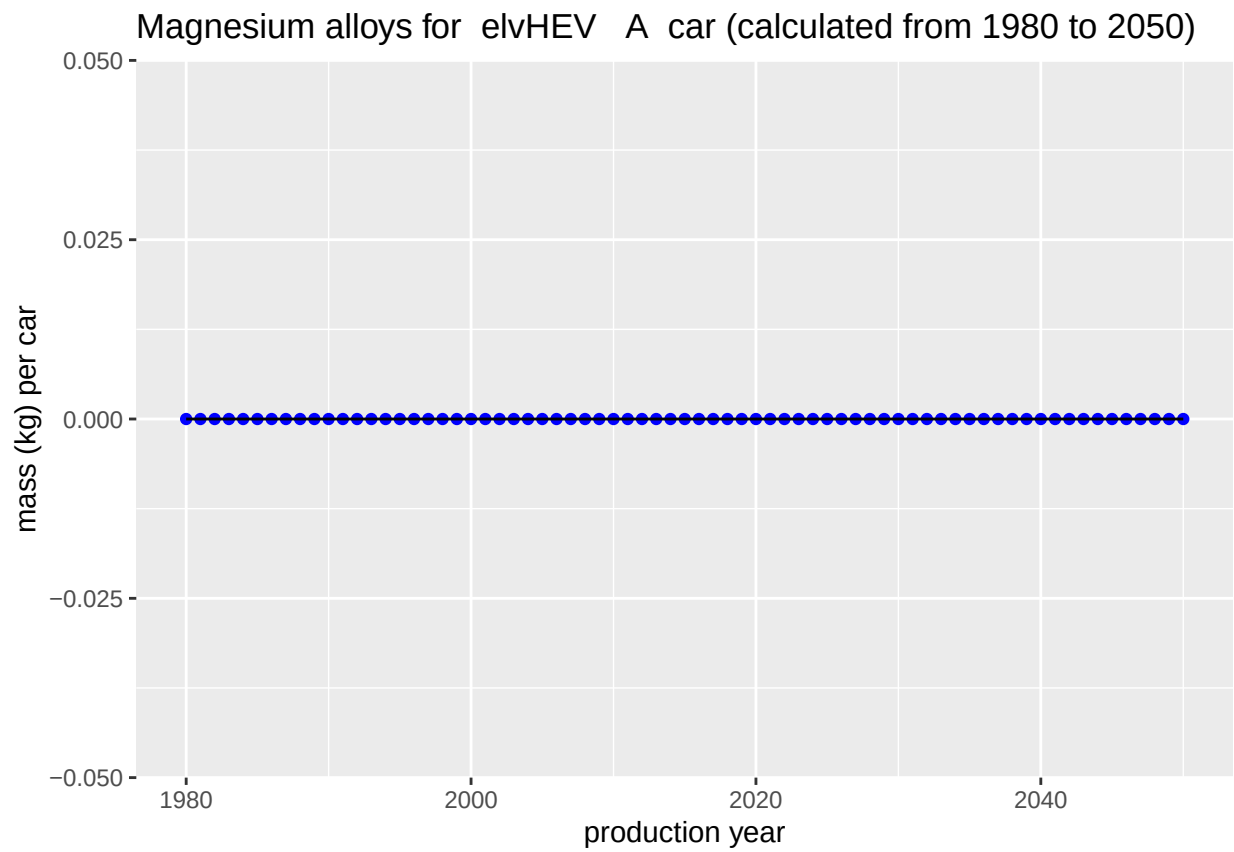


Magnesium alloys for elvDiesel JF car (calculated from 1980 to 2050)

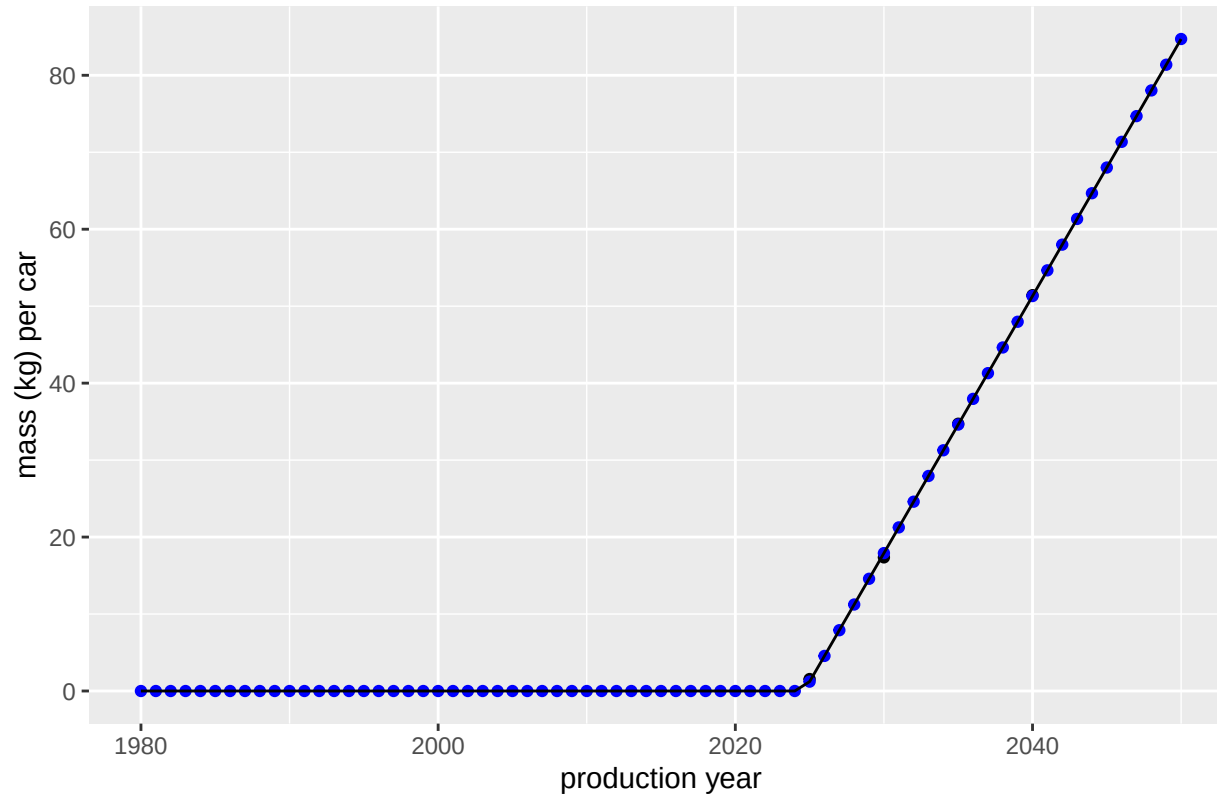


Magnesium alloys for elvDiesel stand

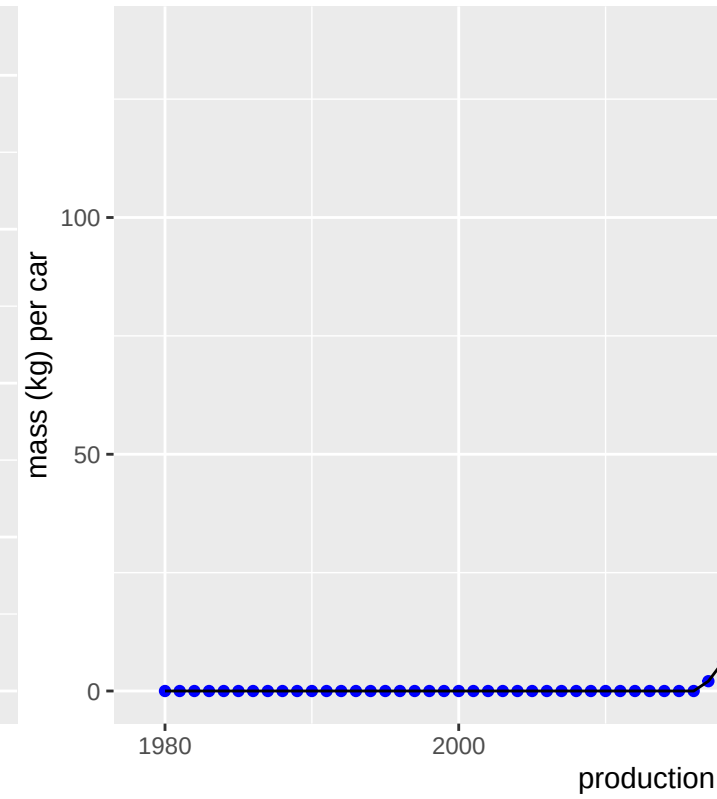




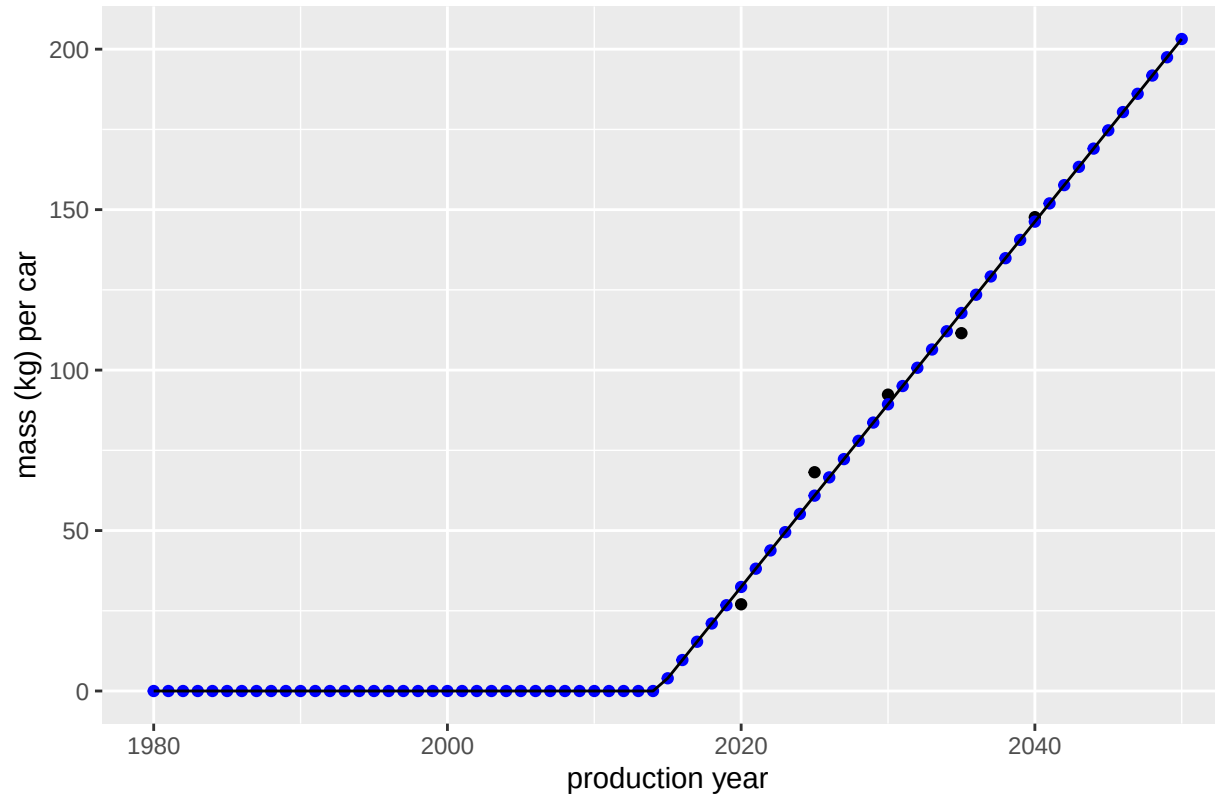
Magnesium alloys for elvHEV C car (calculated from 1980 to 2050)



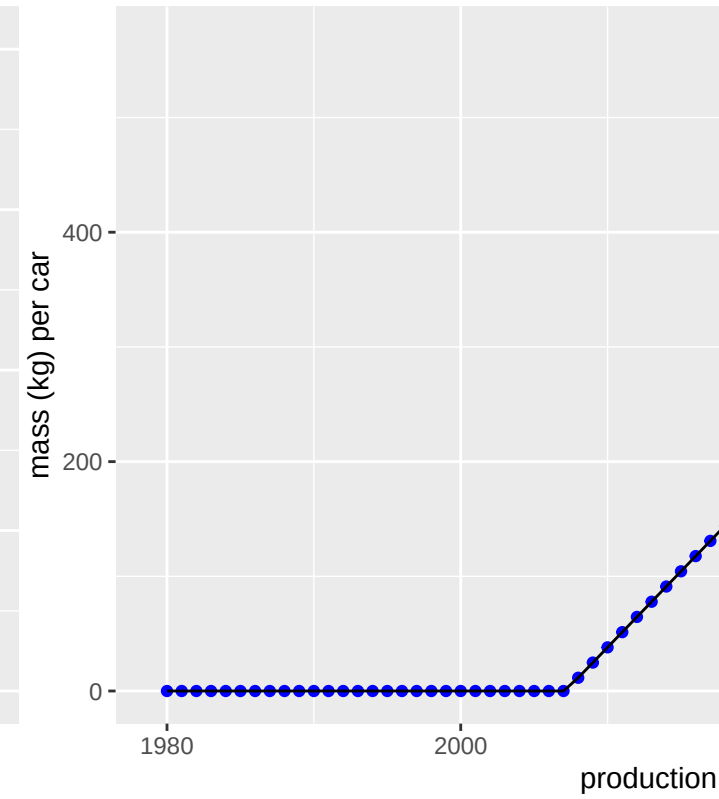
Magnesium alloys for elvHEV D car (

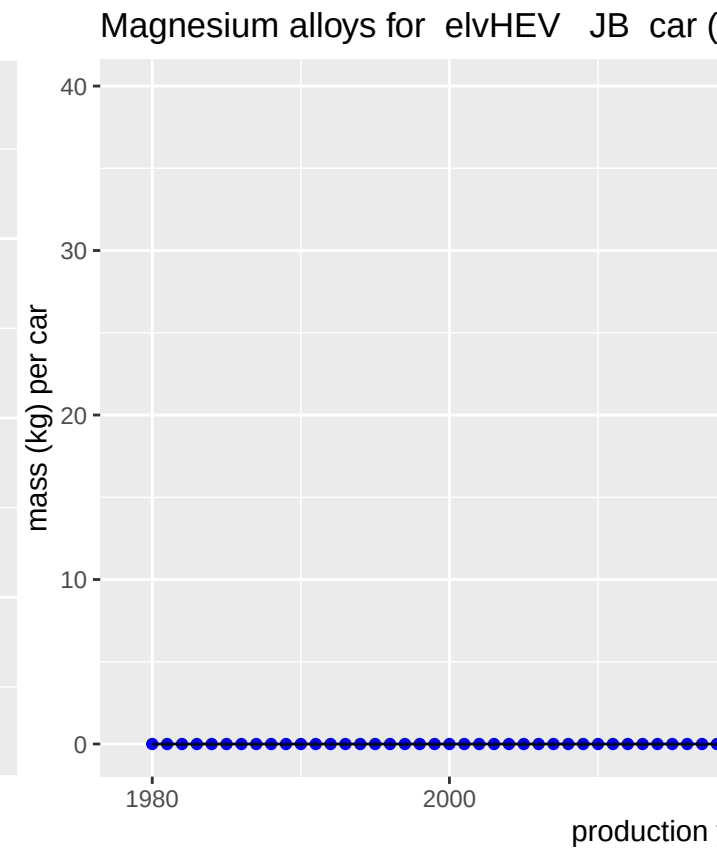
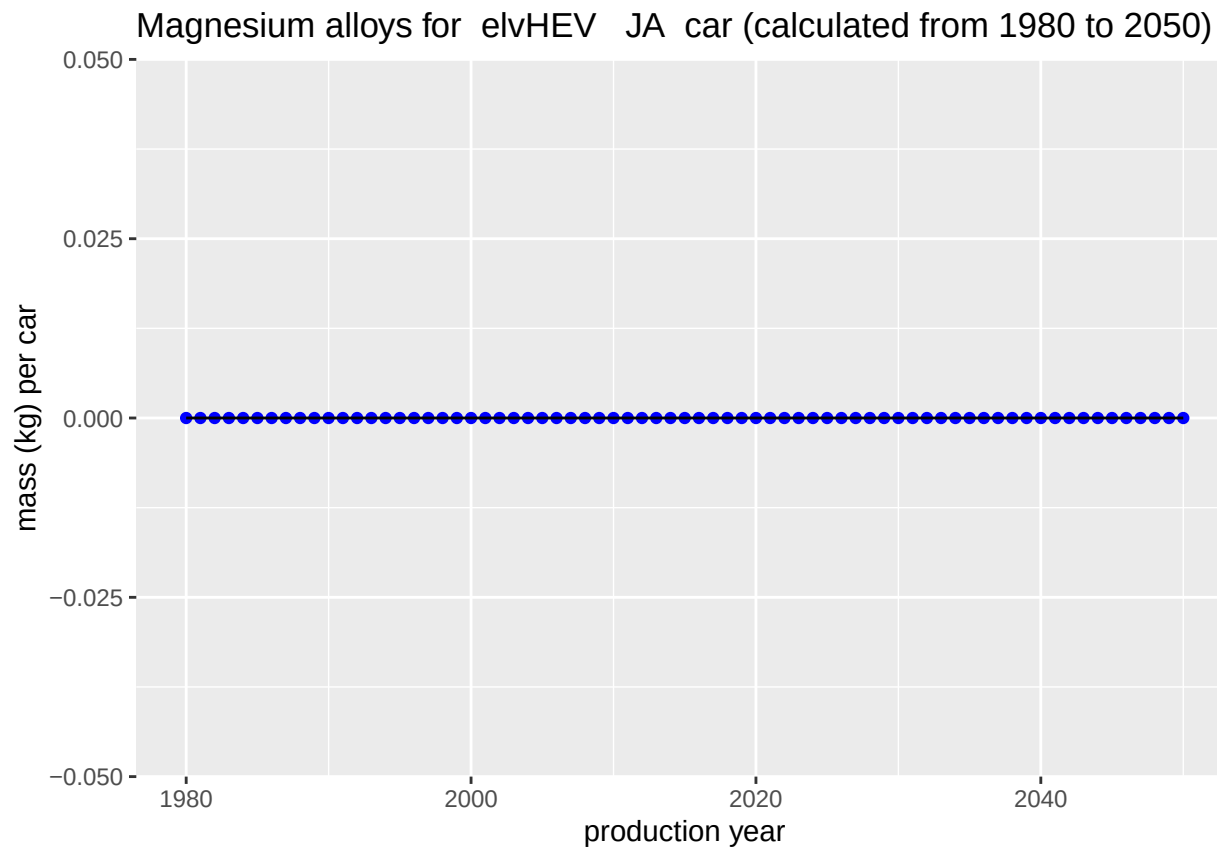


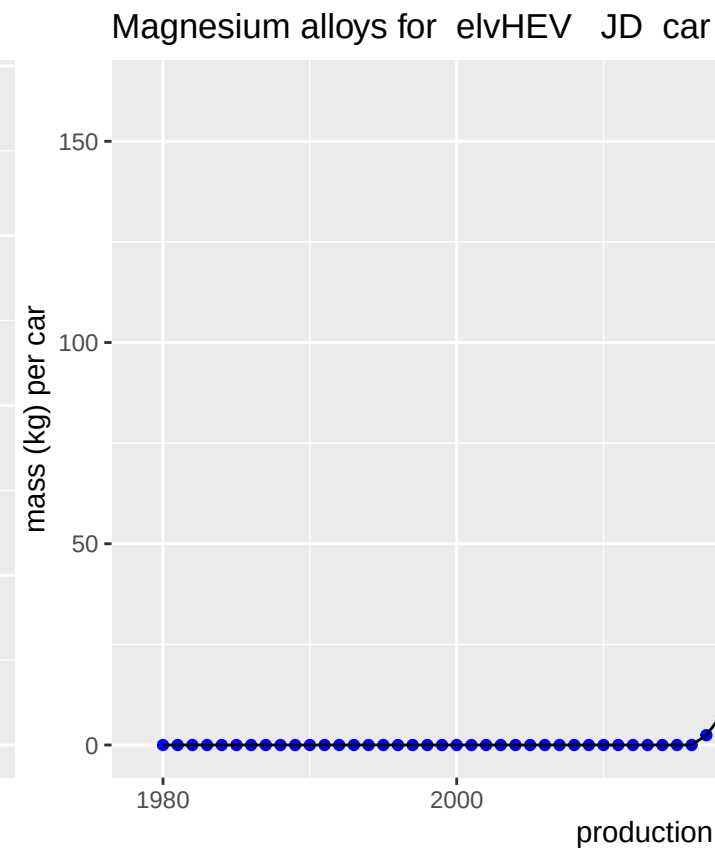
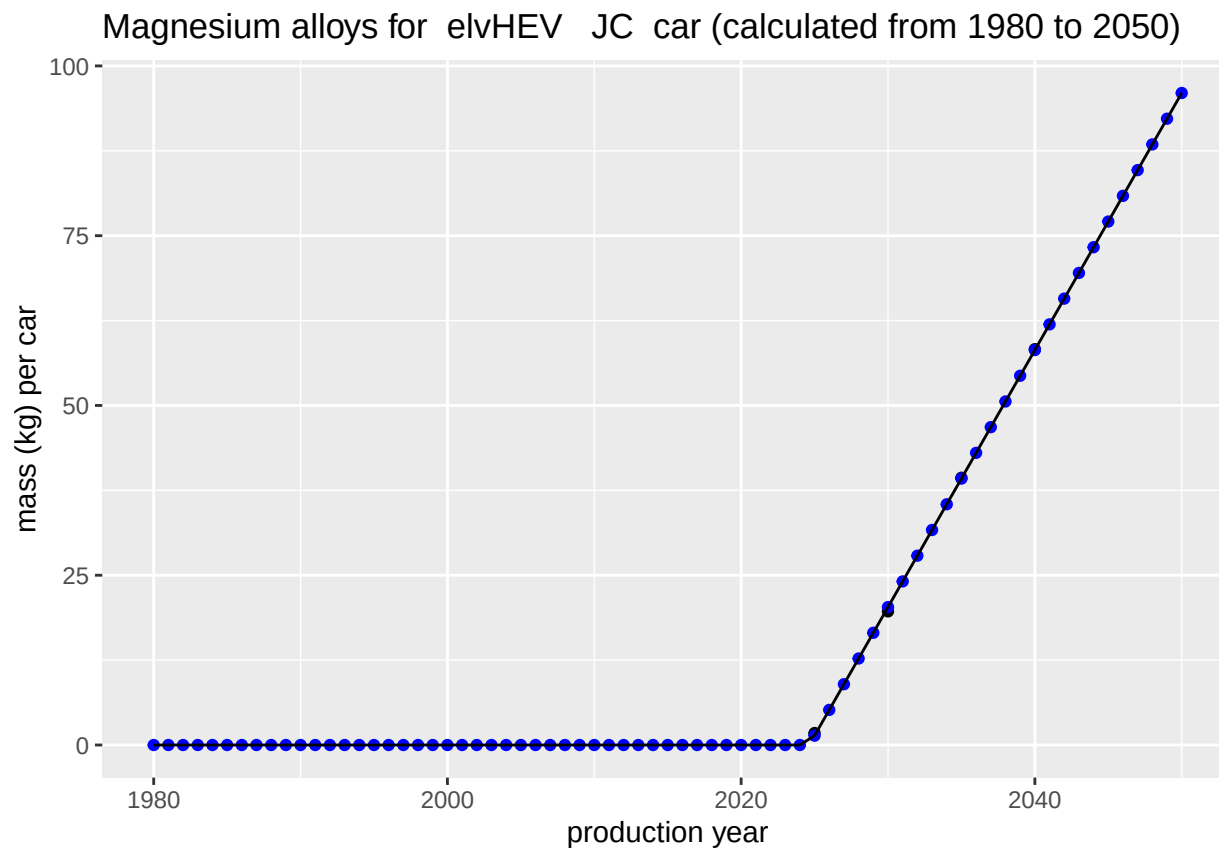
Magnesium alloys for elvHEV E car (calculated from 1980 to 2050)



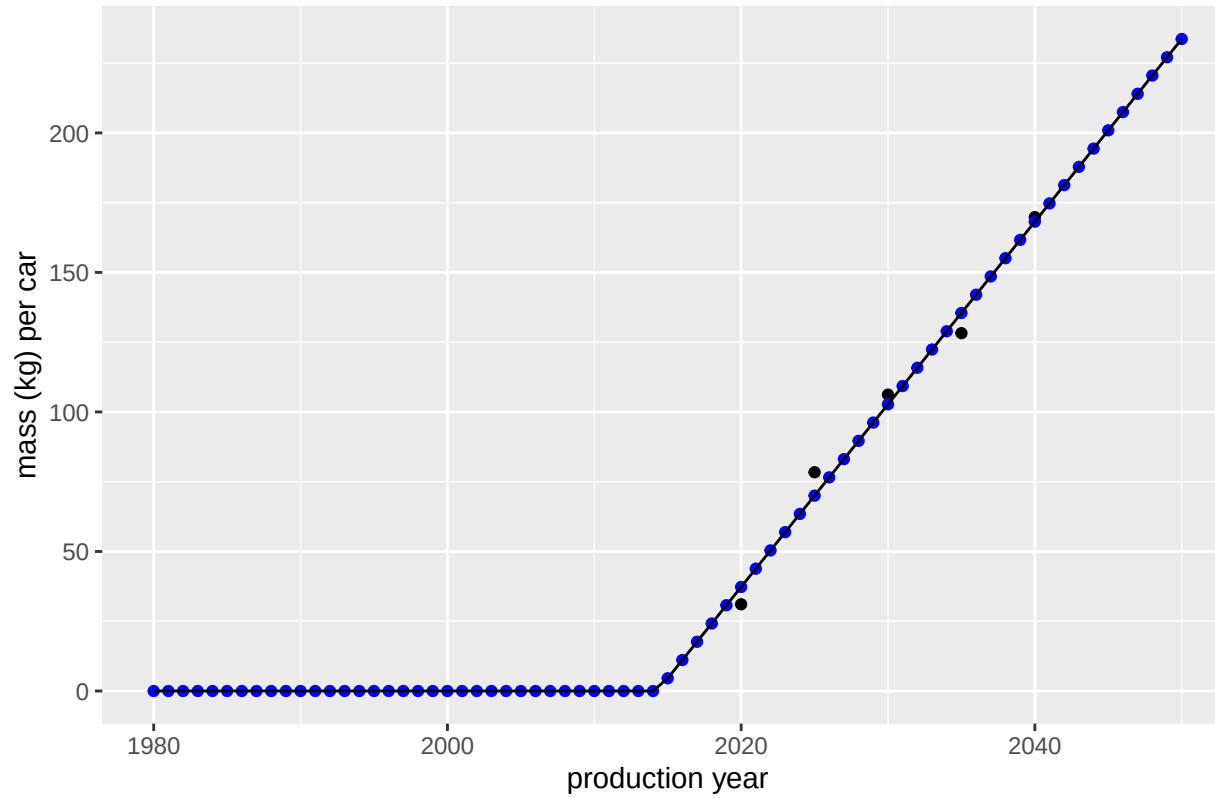
Magnesium alloys for elvHEV F car (calculated from 1980 to 2050)



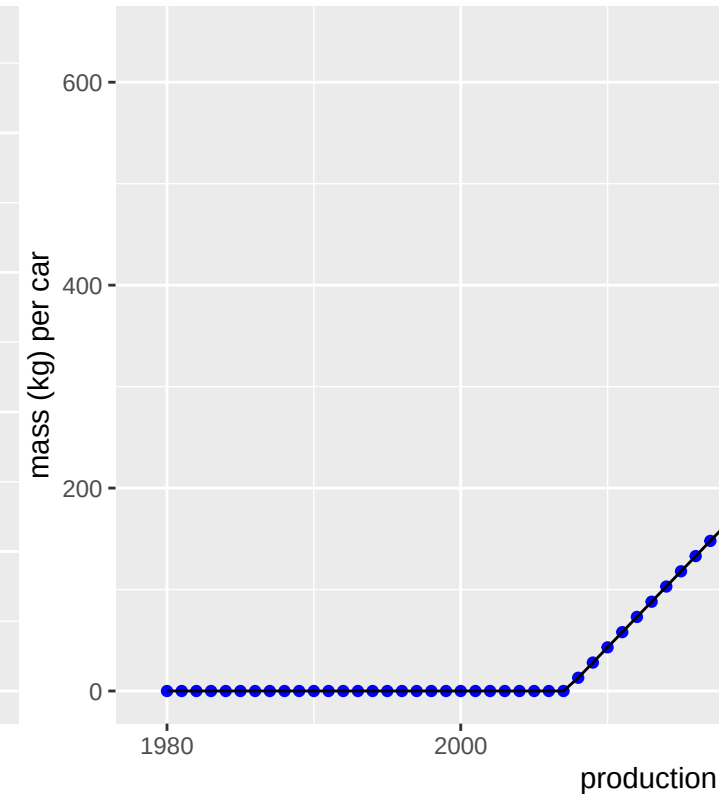




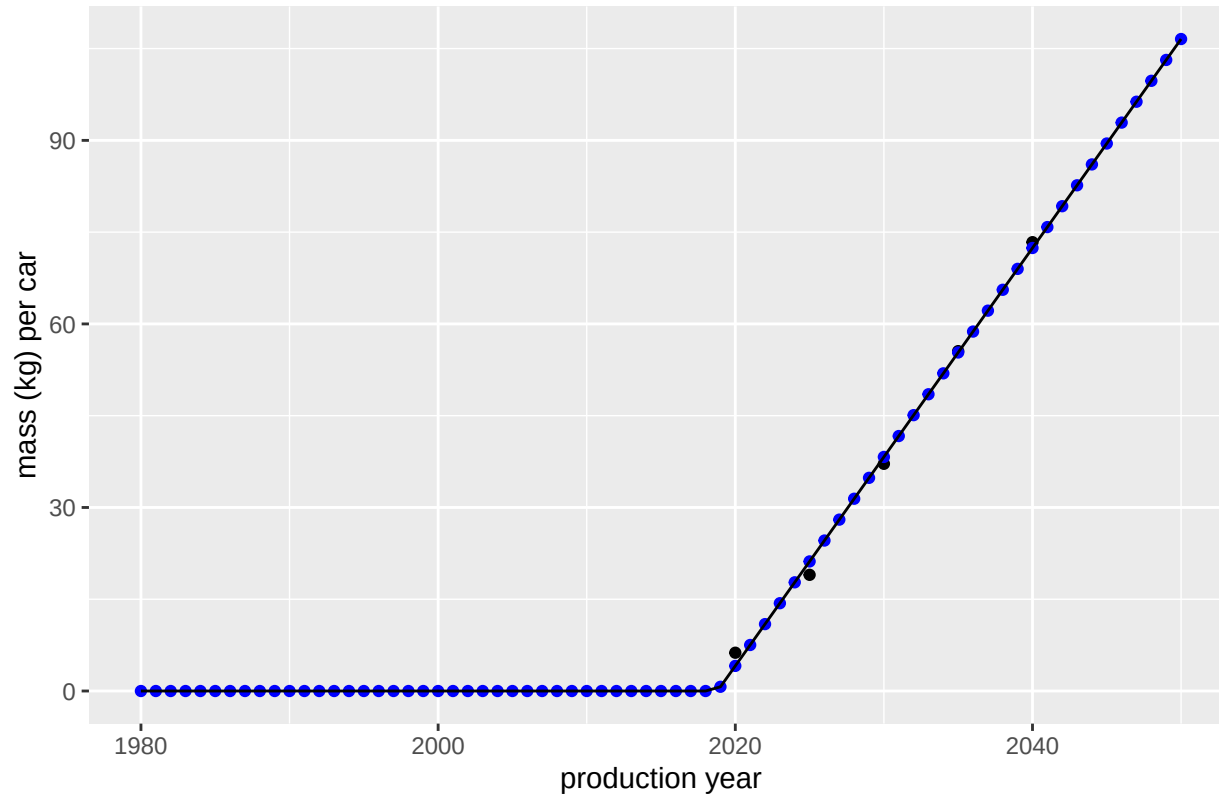
Magnesium alloys for elvHEV JE car (calculated from 1980 to 2050)



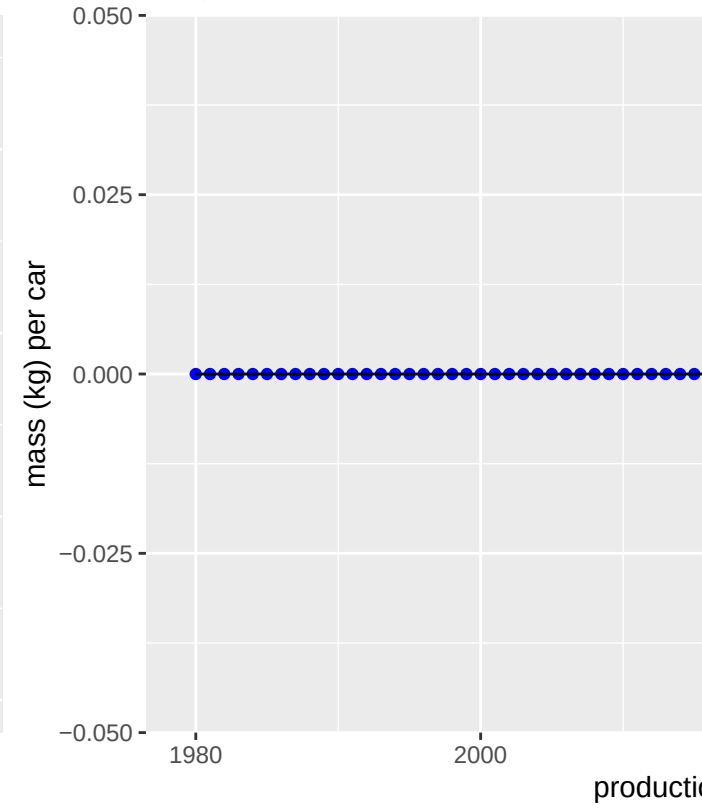
Magnesium alloys for elvHEV JF car



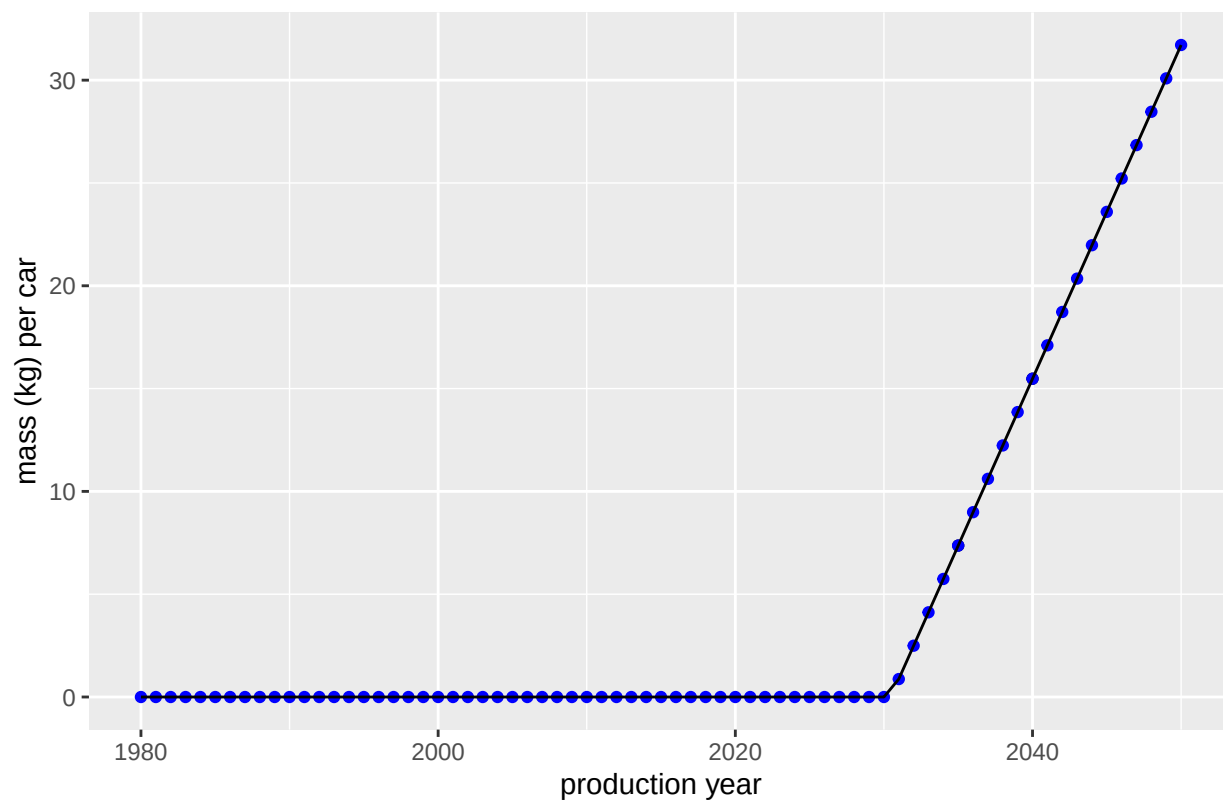
Magnesium alloys for elvHEV standard car (calculated from 1980 to 2050)



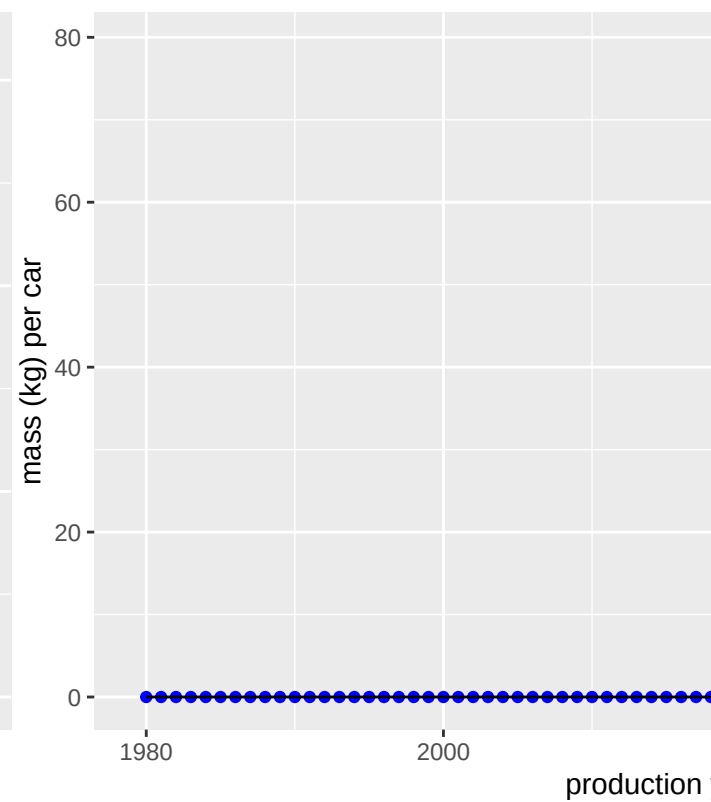
Magnesium alloys for elvOther A car



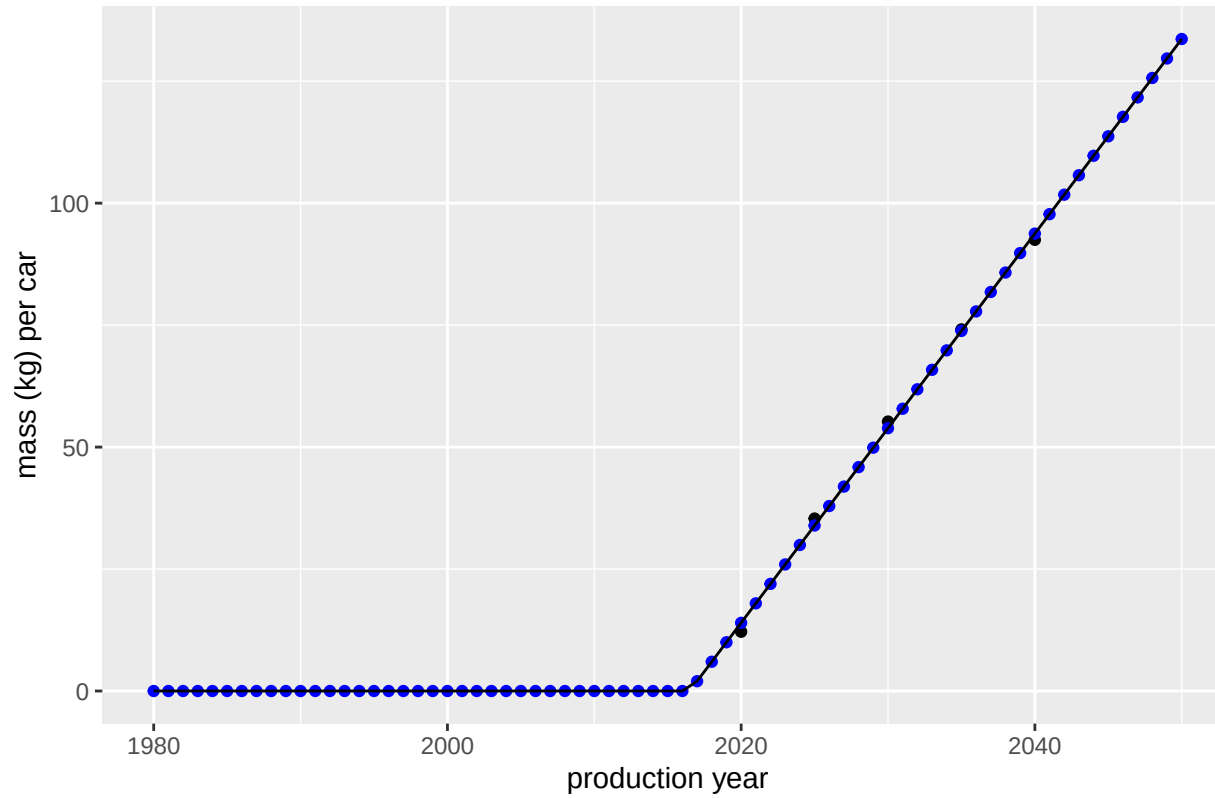
Magnesium alloys for elvOther B car (calculated from 1980 to 2050)



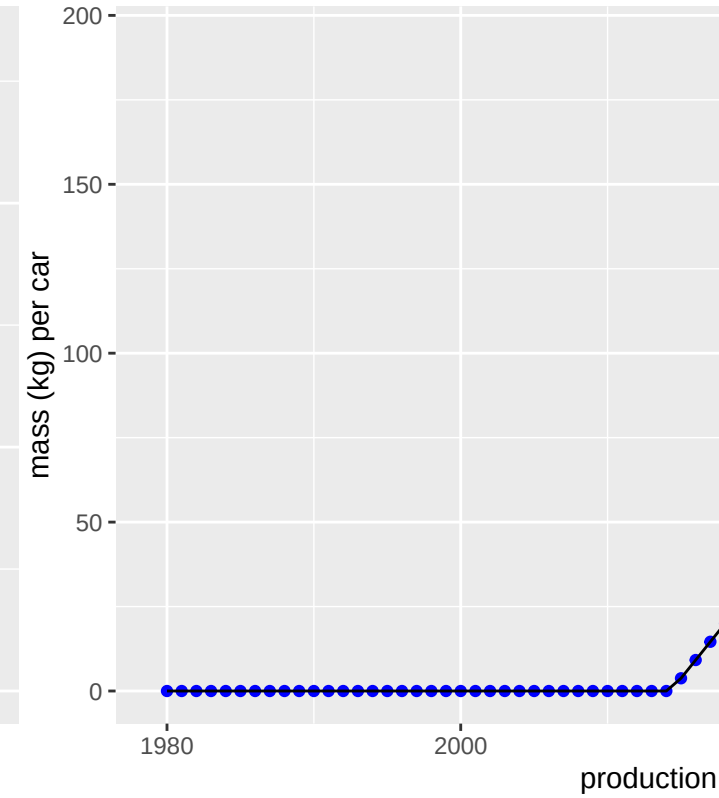
Magnesium alloys for elvOther C car (calculated from 1980 to 2050)



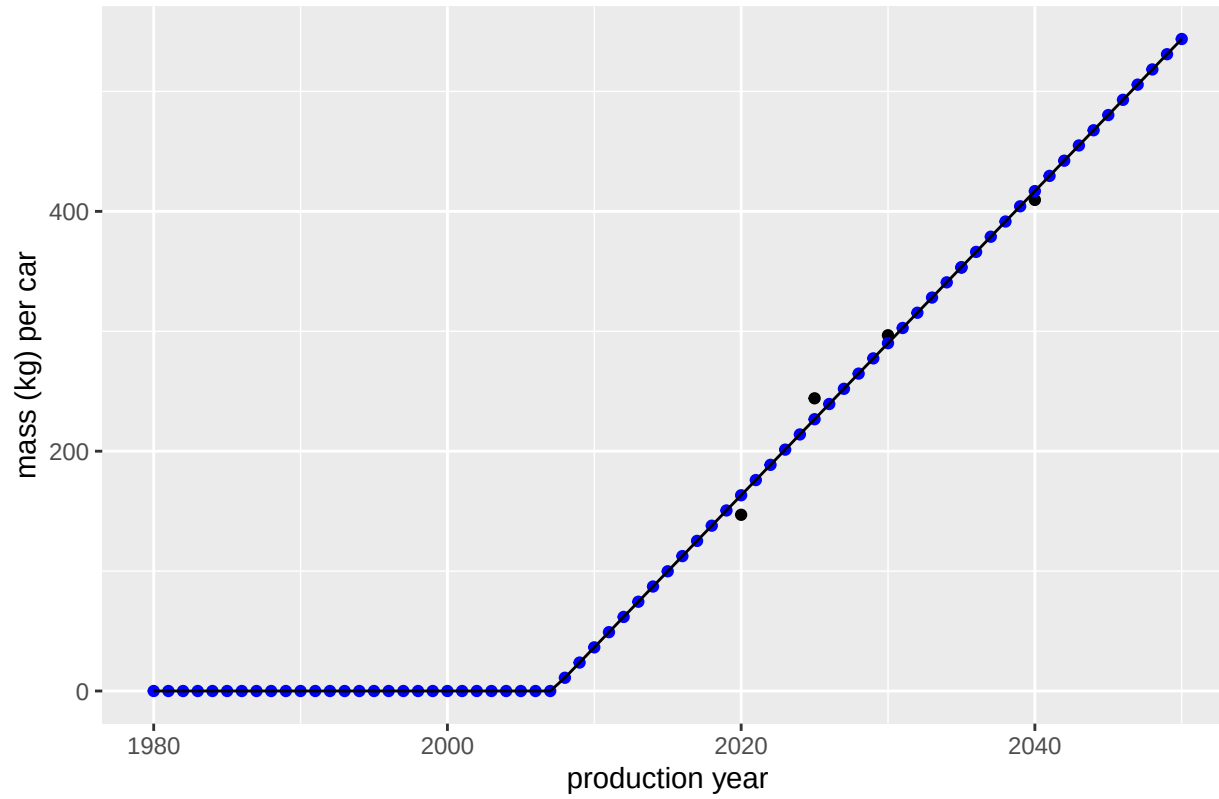
Magnesium alloys for elvOther D car (calculated from 1980 to 2050)



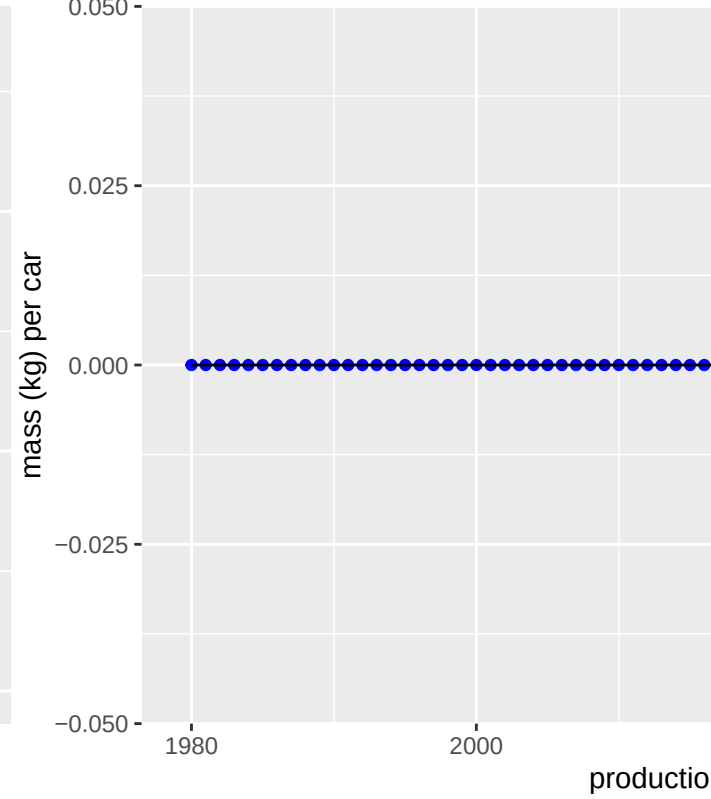
Magnesium alloys for elvOther E car



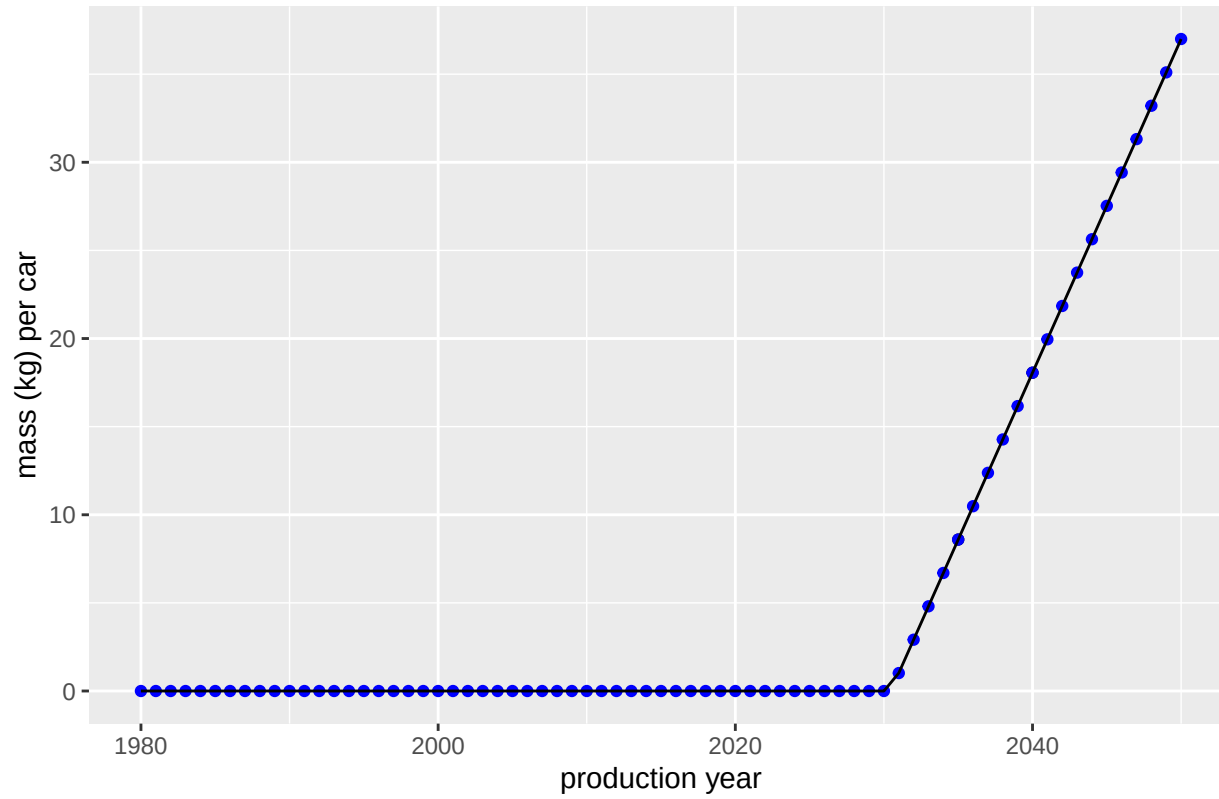
Magnesium alloys for elvOther F car (calculated from 1980 to 2050)



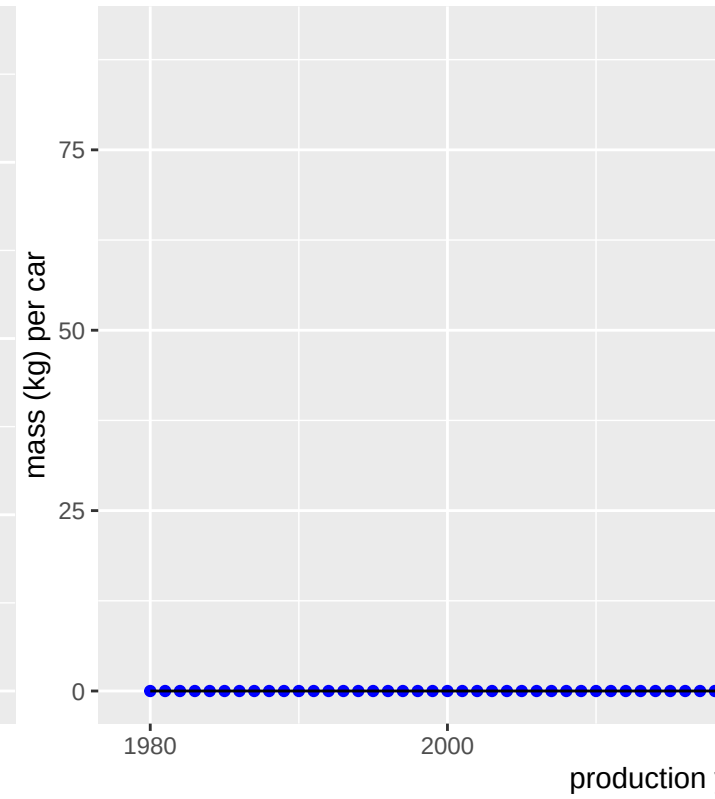
Magnesium alloys for elvOther JA c



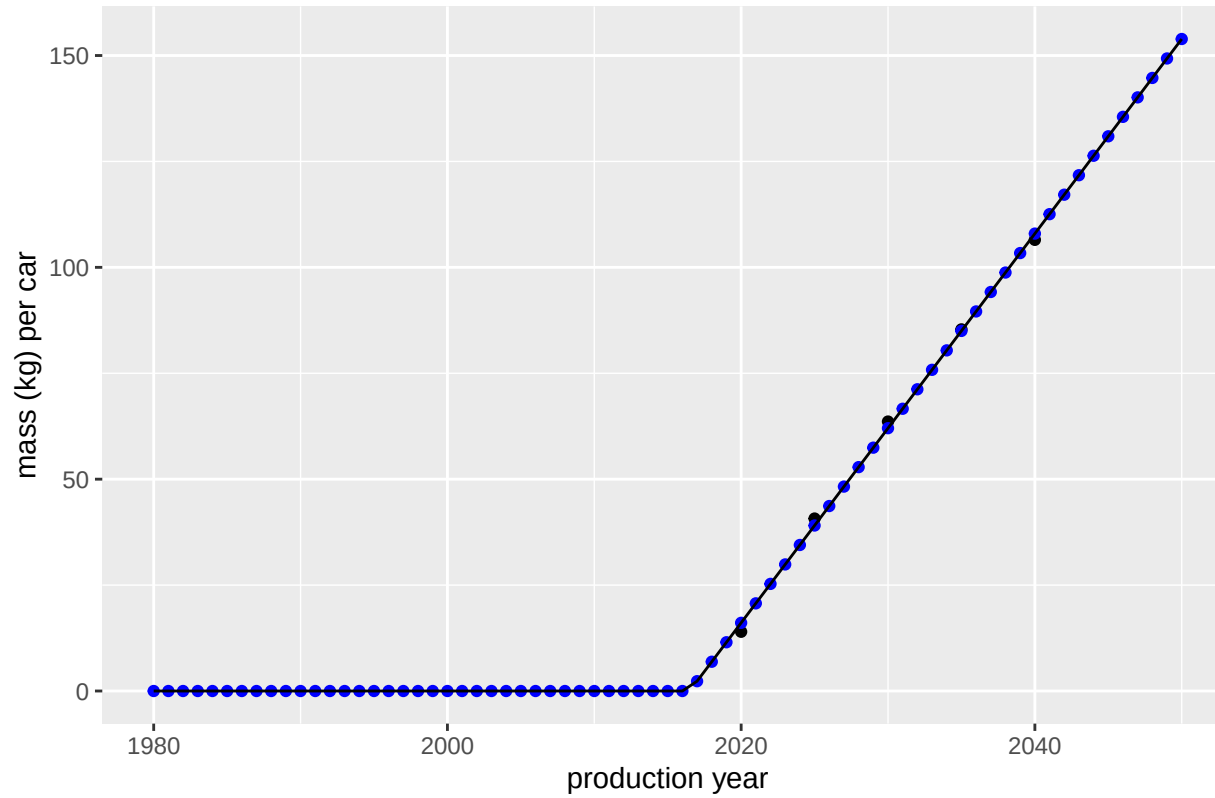
Magnesium alloys for elvOther JB car (calculated from 1980 to 2050)



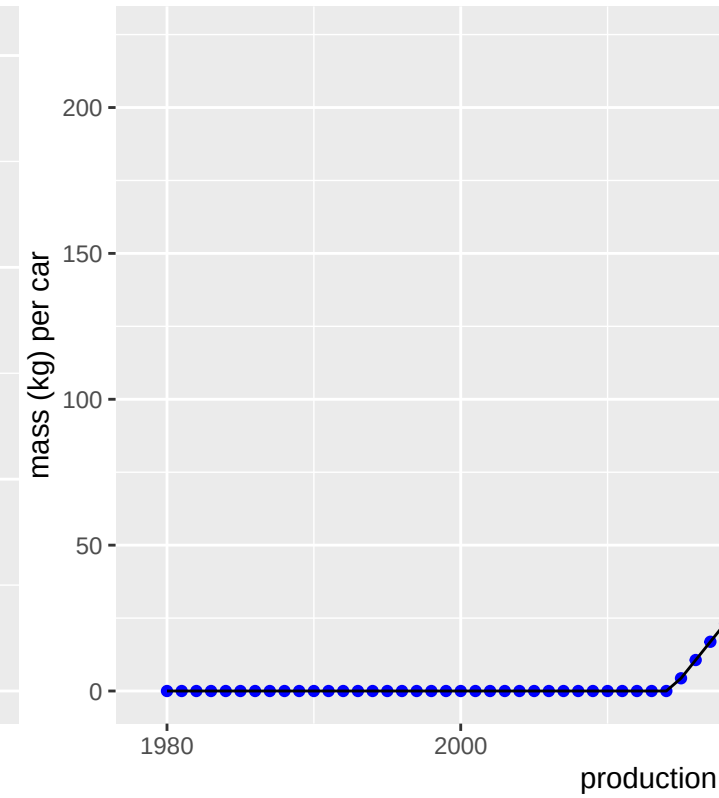
Magnesium alloys for elvOther JC car



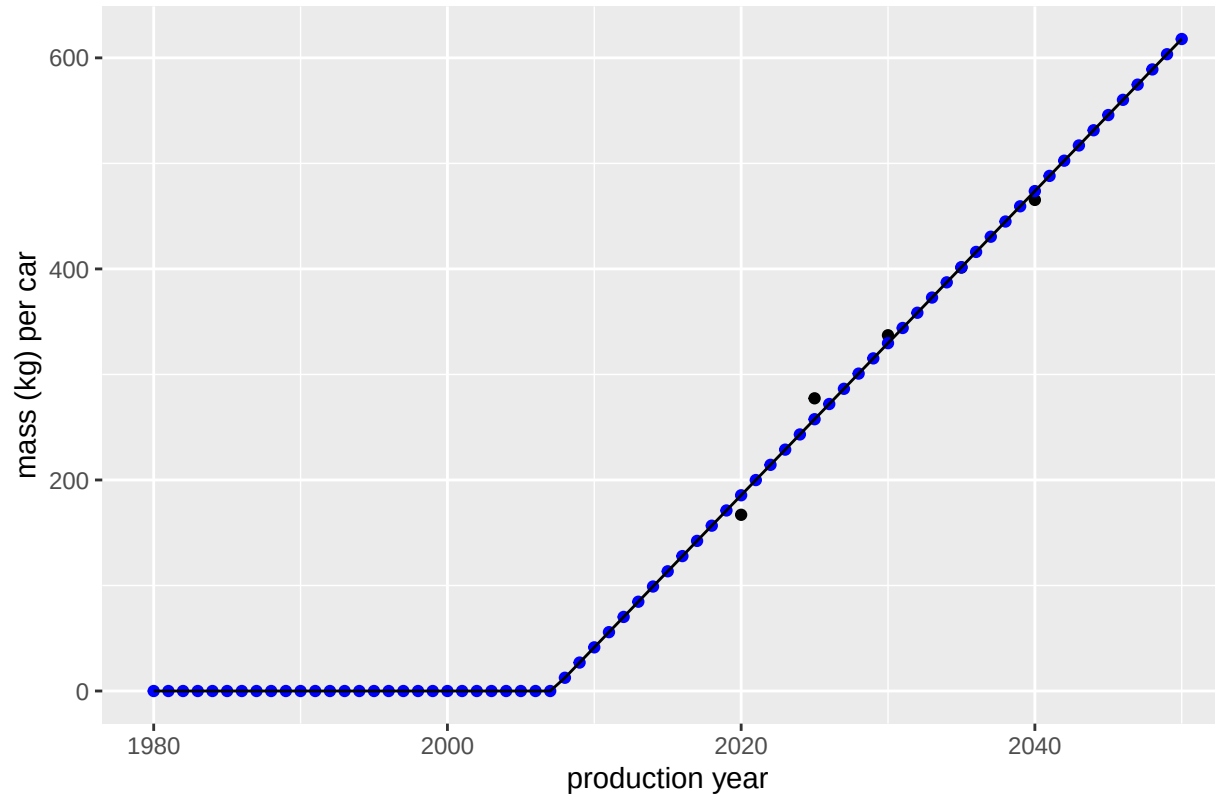
Magnesium alloys for elvOther JD car (calculated from 1980 to 2050)



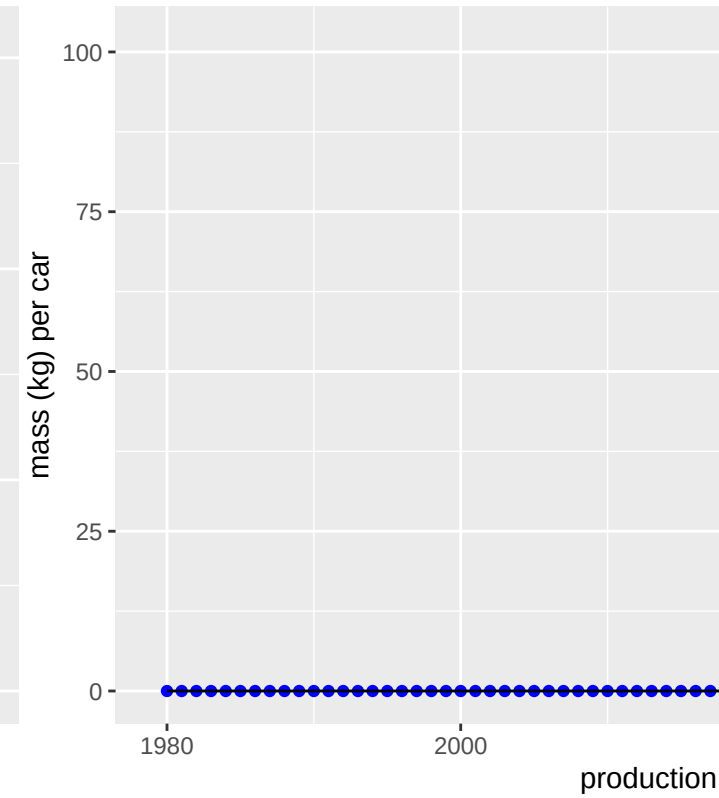
Magnesium alloys for elvOther JE car

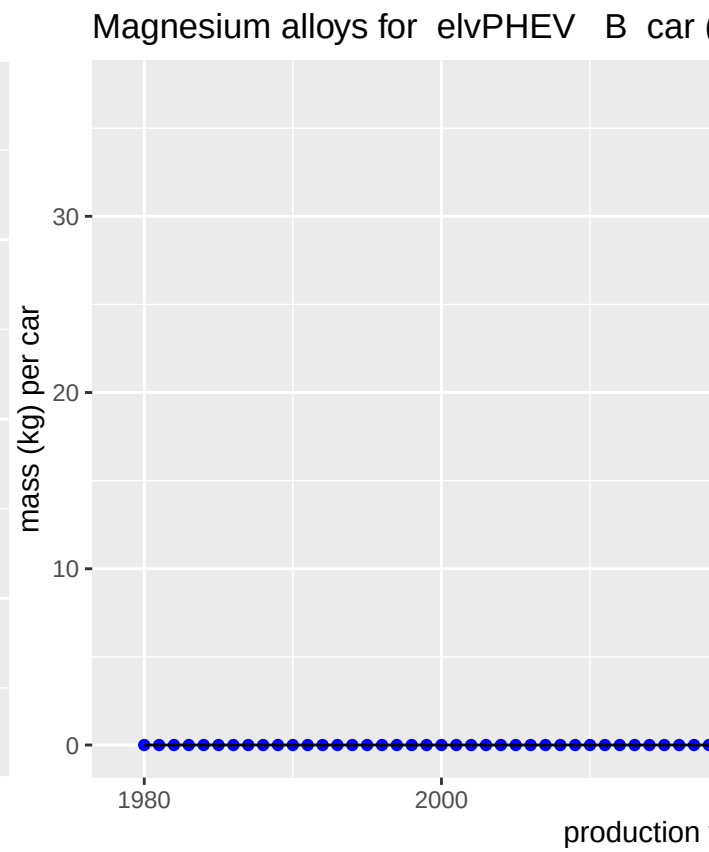
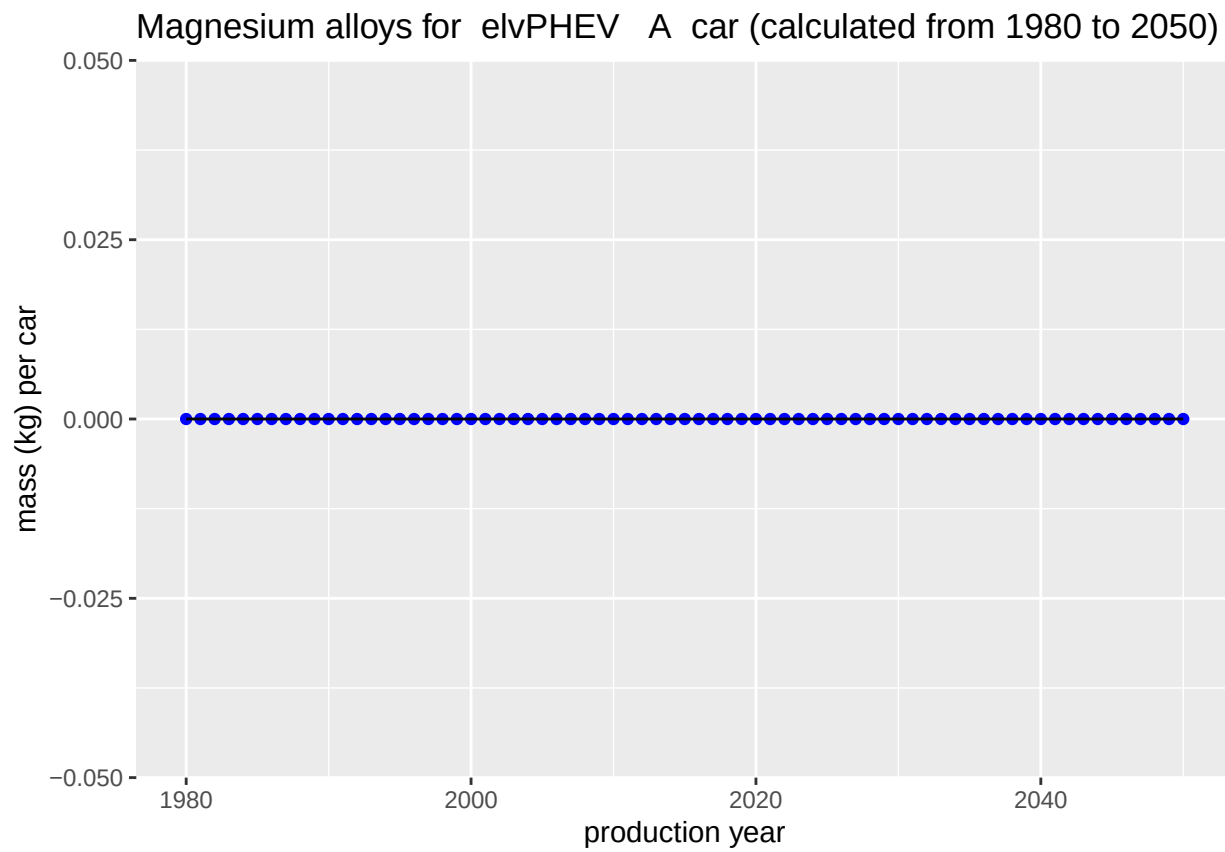


Magnesium alloys for elvOther JF car (calculated from 1980 to 2050)

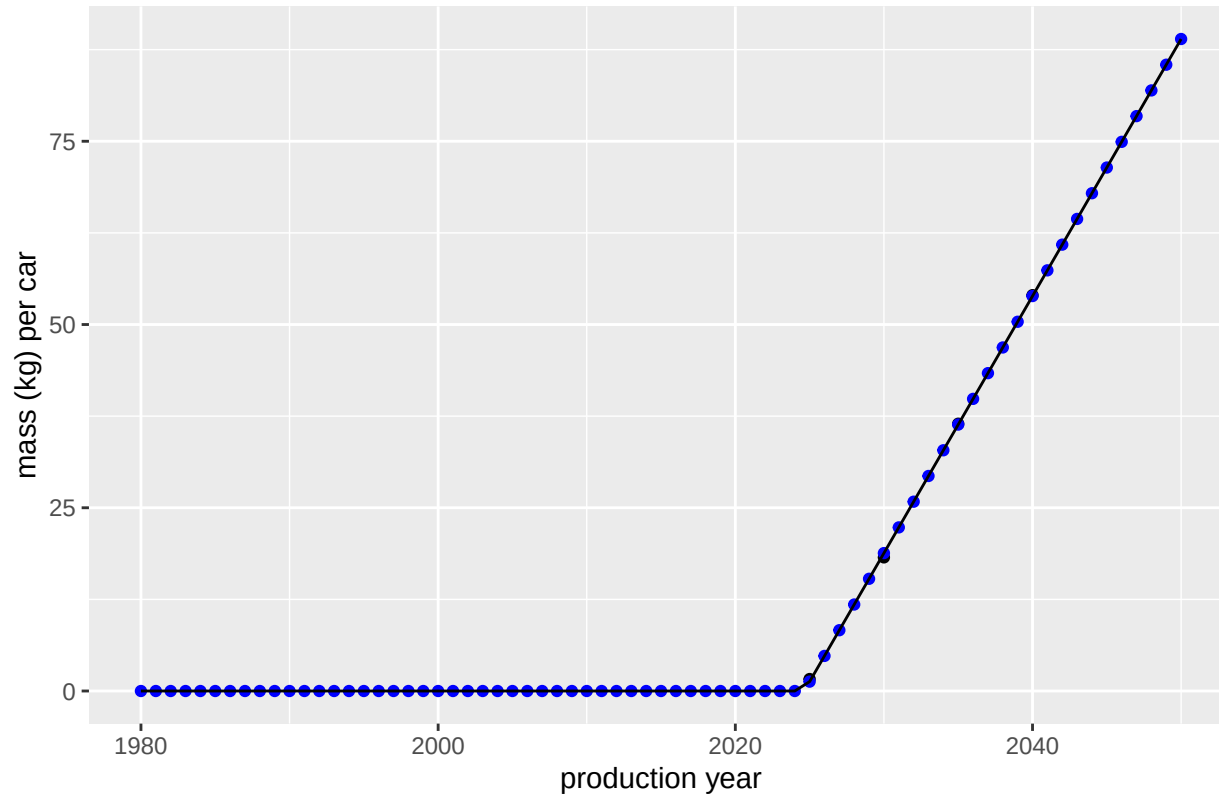


Magnesium alloys for elvOther standa

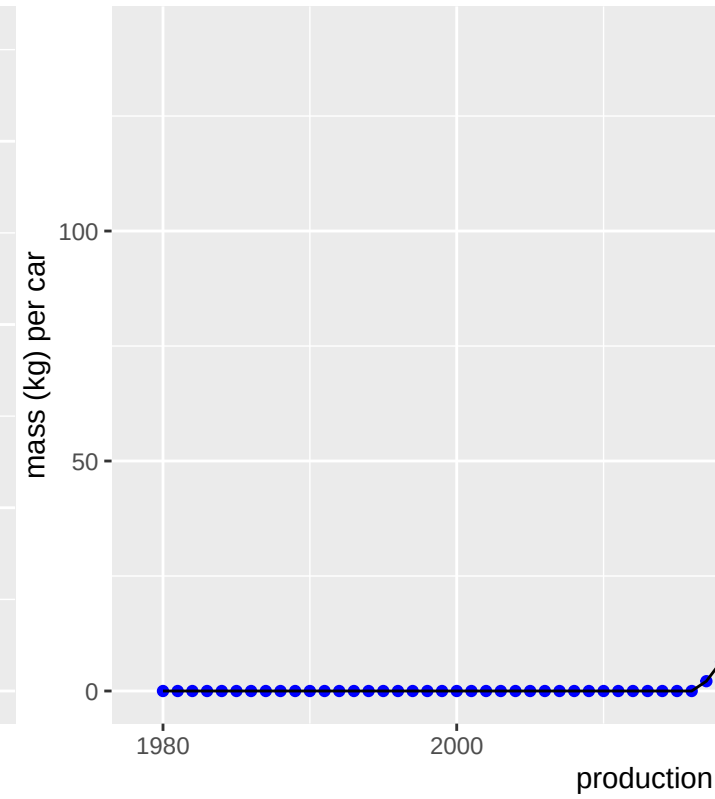




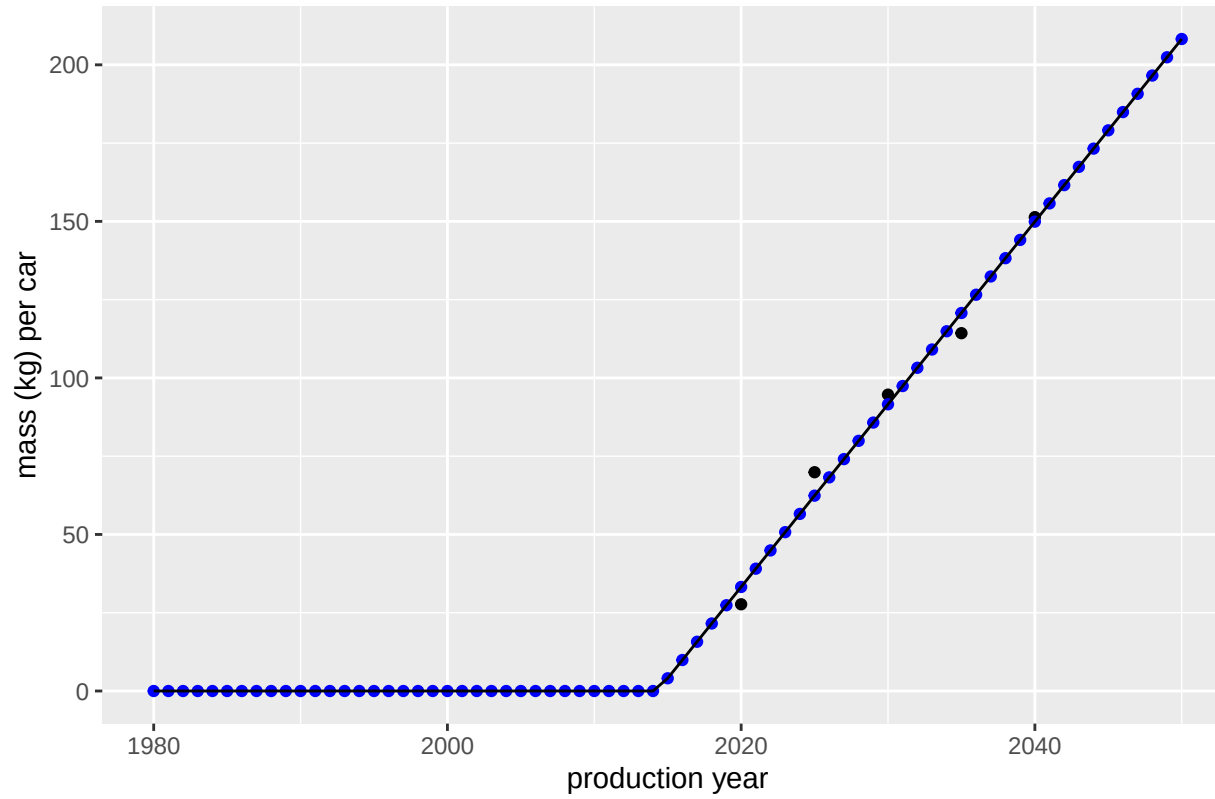
Magnesium alloys for elvPHEV C car (calculated from 1980 to 2050)



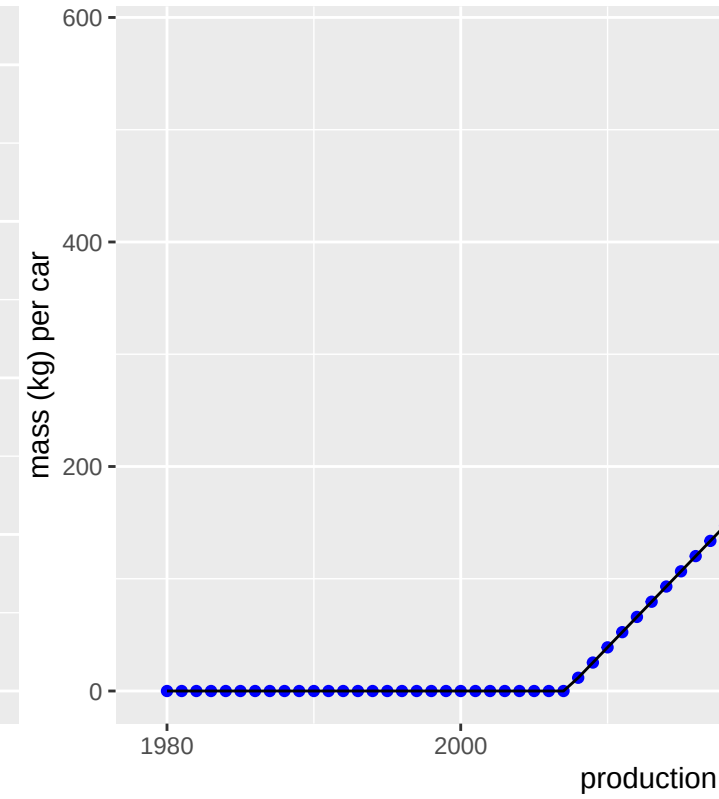
Magnesium alloys for elvPHEV D car

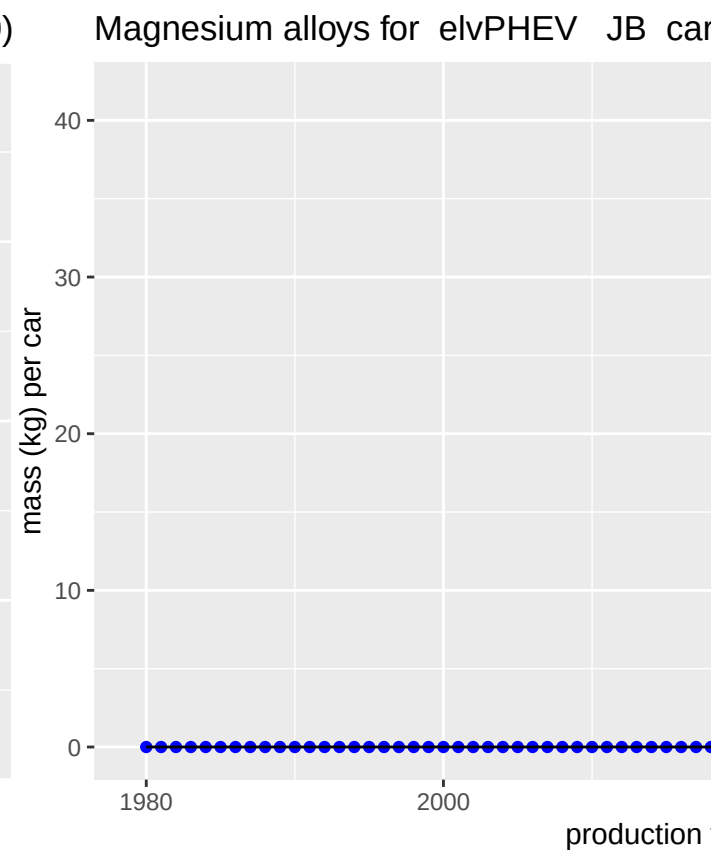
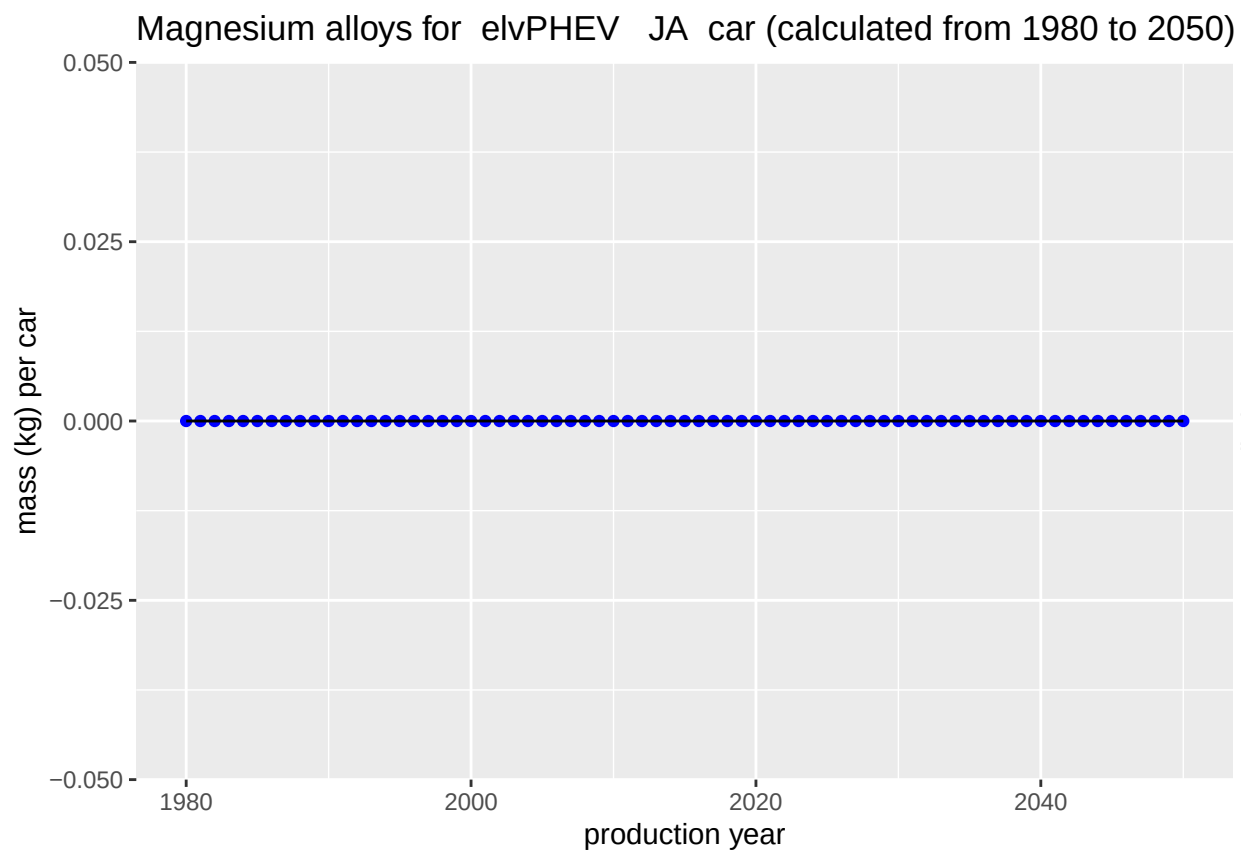


Magnesium alloys for elvPHEV E car (calculated from 1980 to 2050)

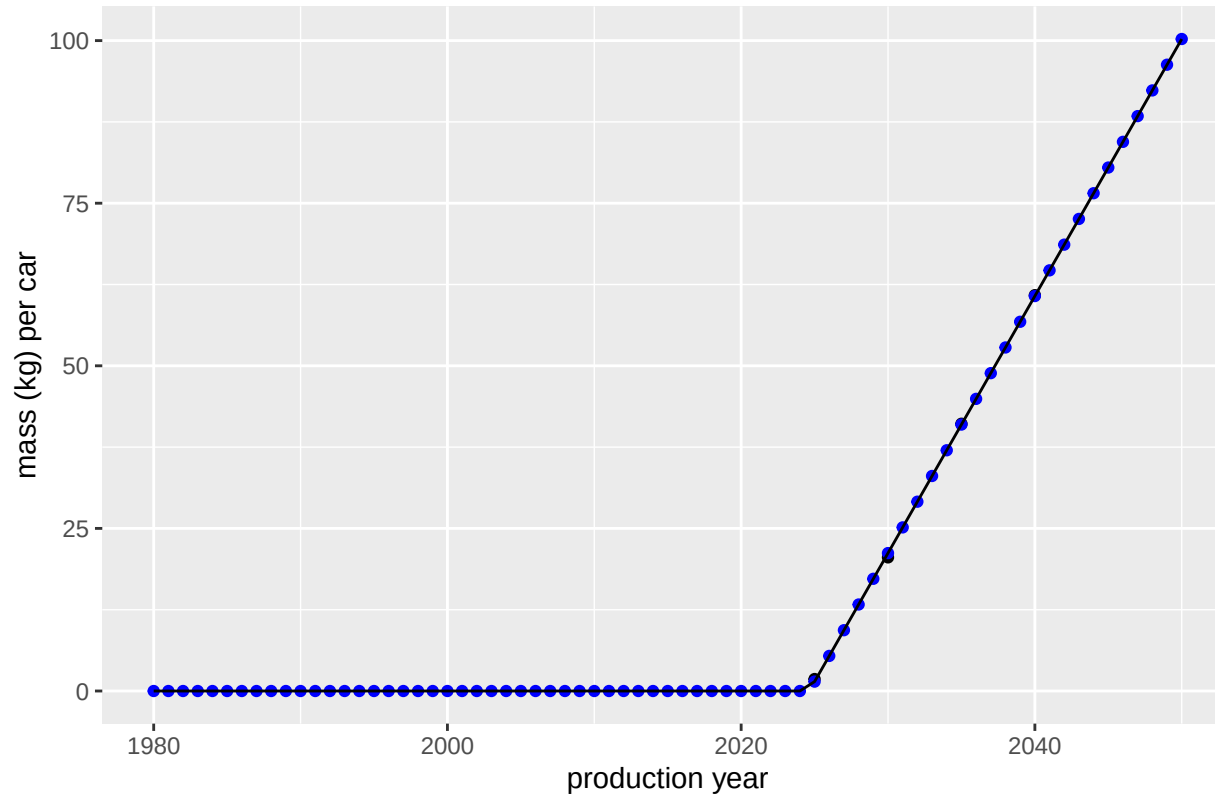


Magnesium alloys for elvPHEV F car

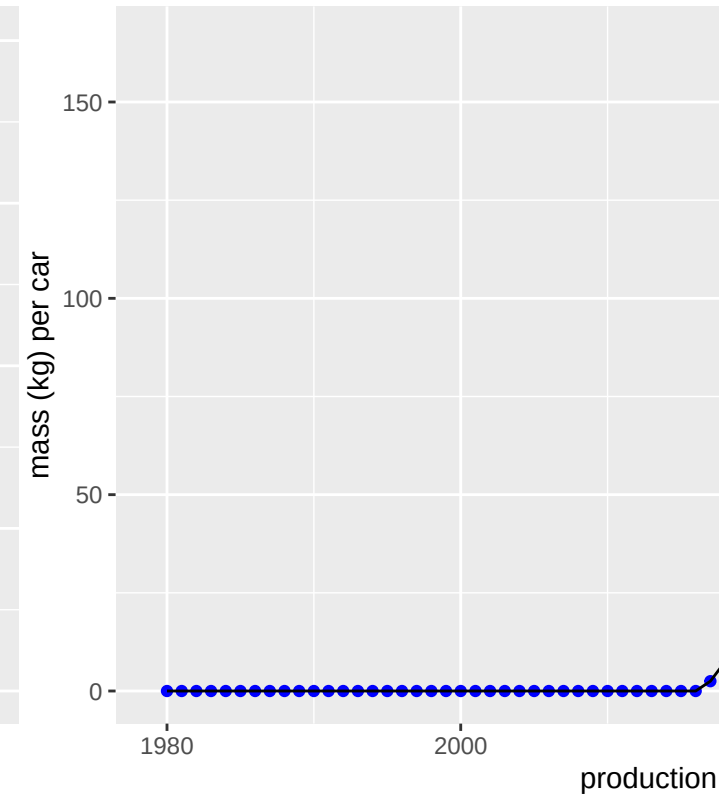




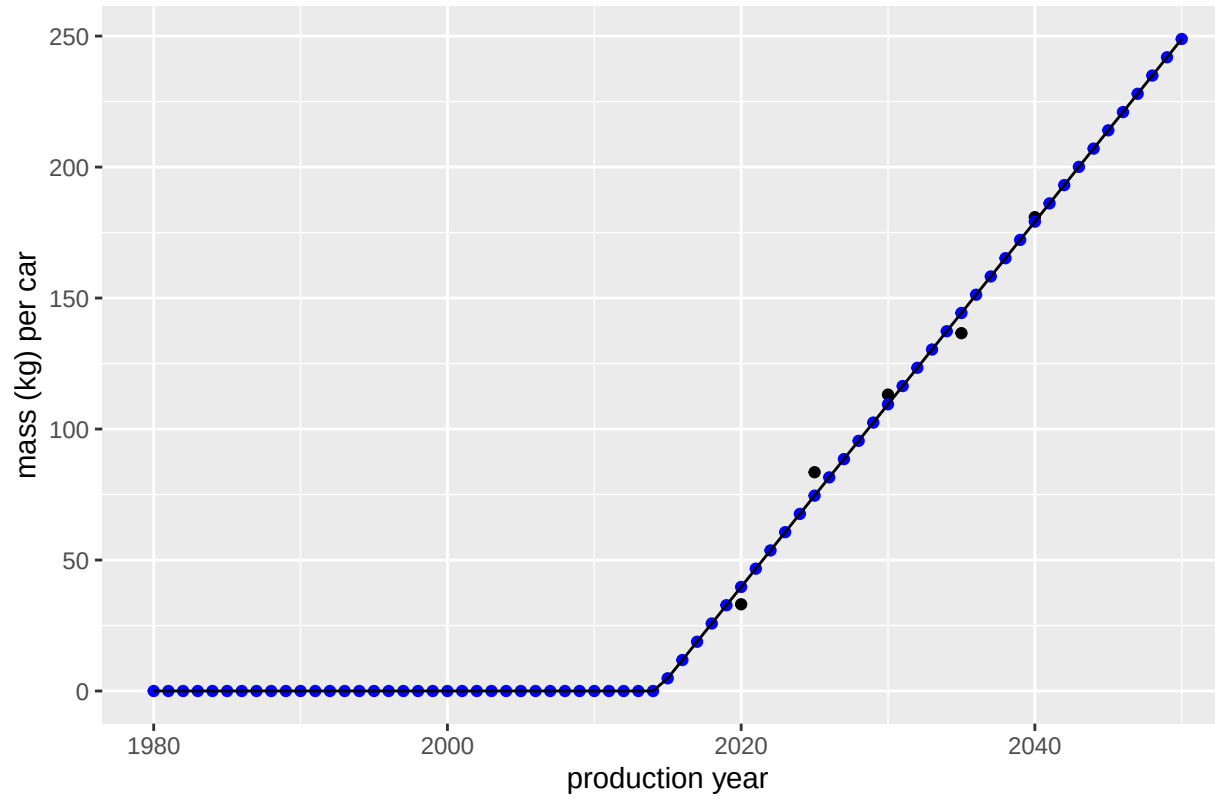
Magnesium alloys for elvPHEV JC car (calculated from 1980 to 2050)



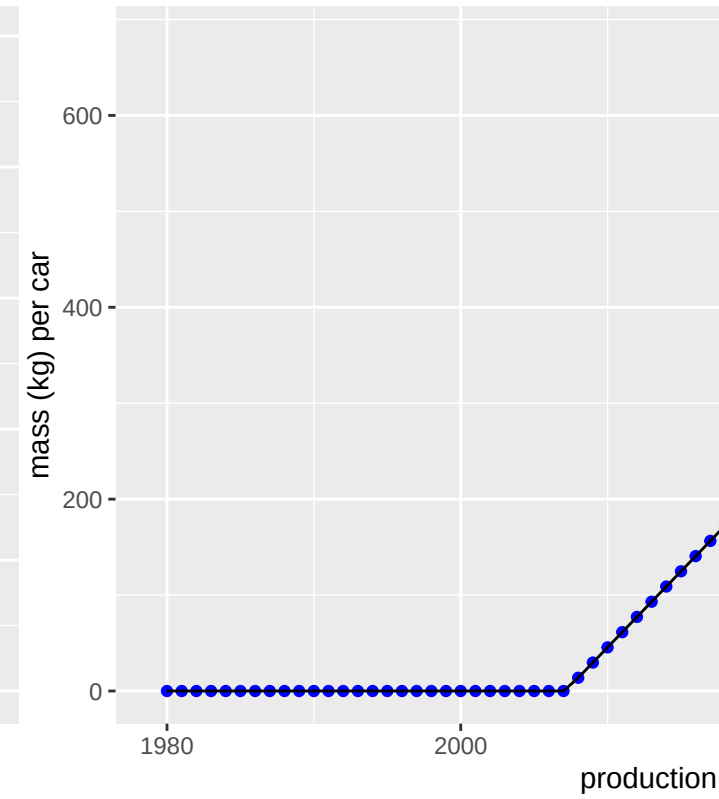
Magnesium alloys for elvPHEV JD car



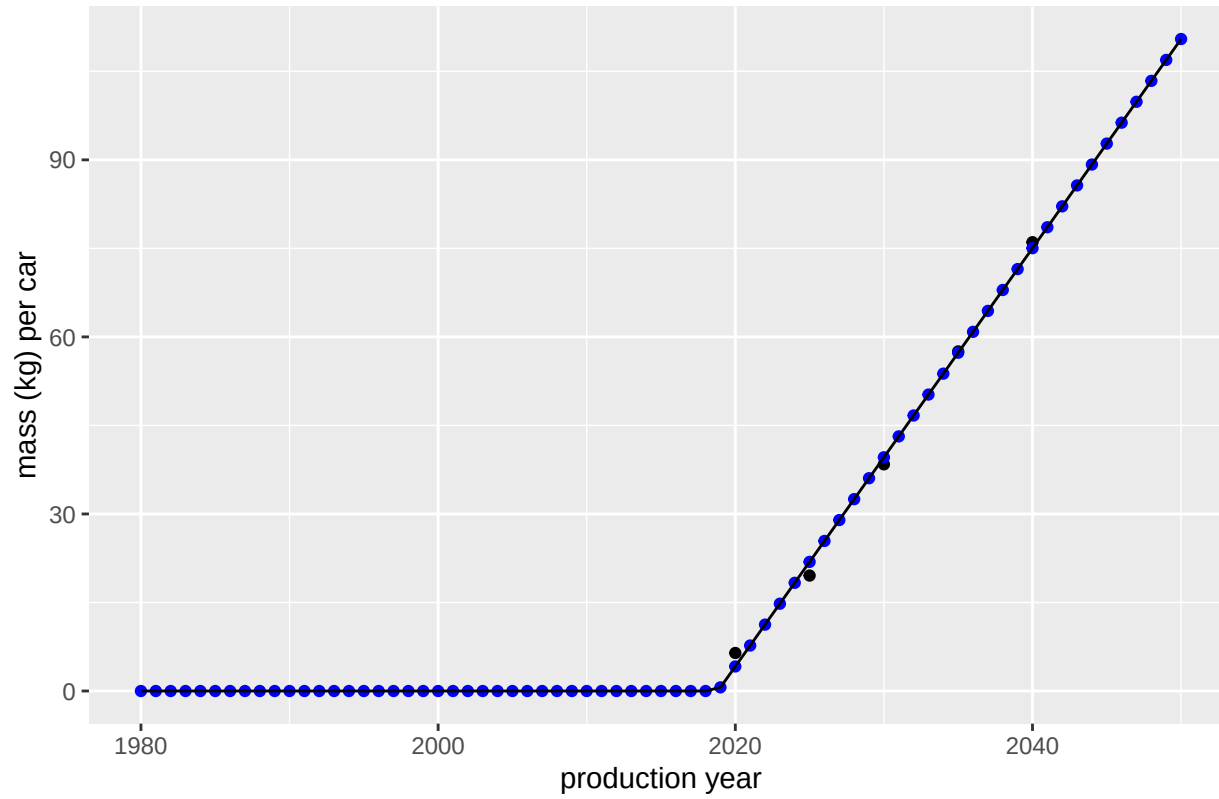
Magnesium alloys for elvPHEV JE car (calculated from 1980 to 2050)



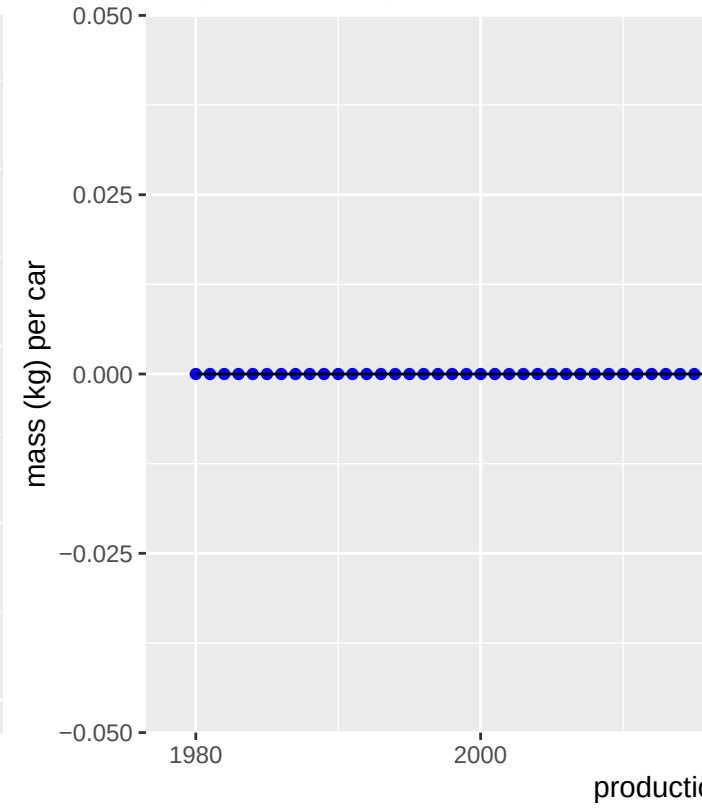
Magnesium alloys for elvPHEV JF car



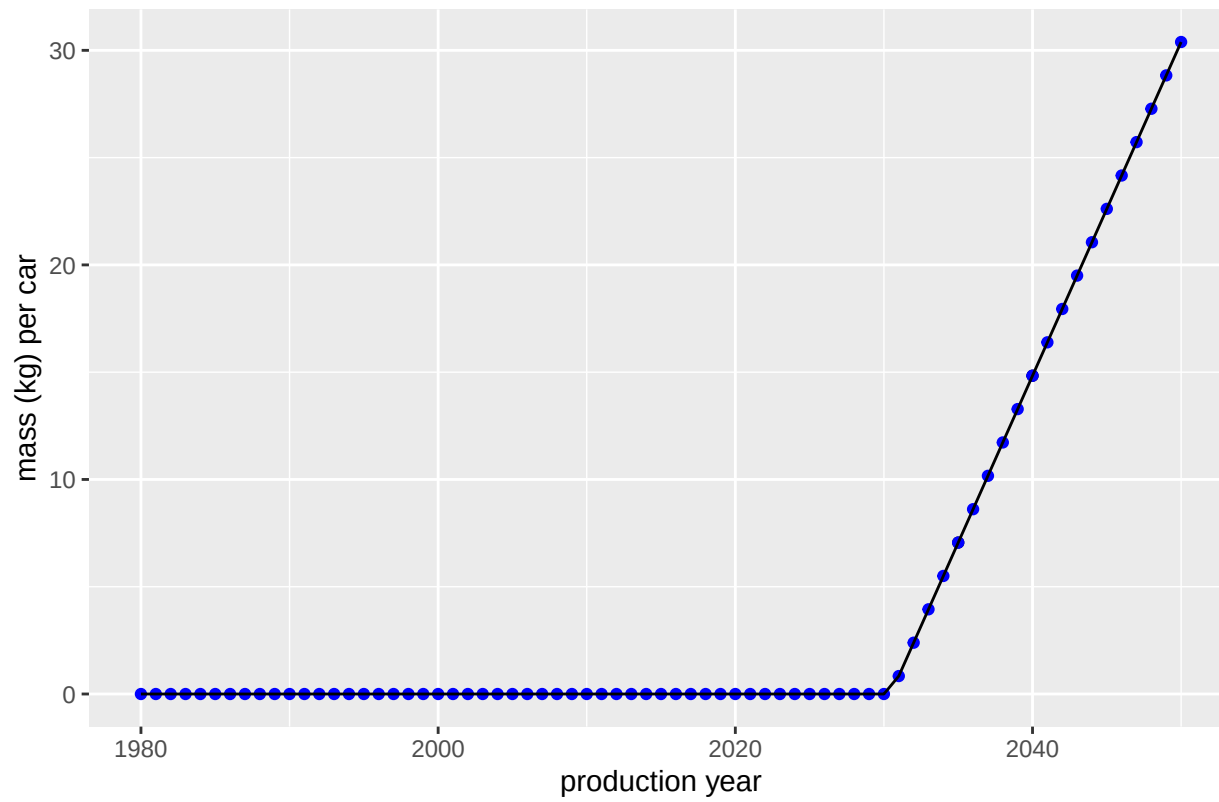
Magnesium alloys for elvPHEV standard car (calculated from 1980 to 205)



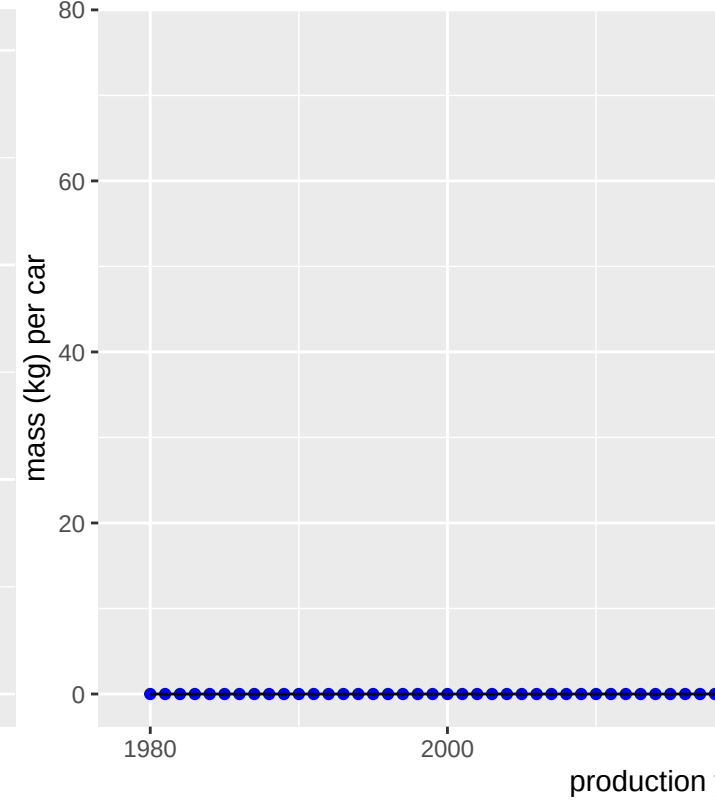
Magnesium alloys for elvPetrol A car



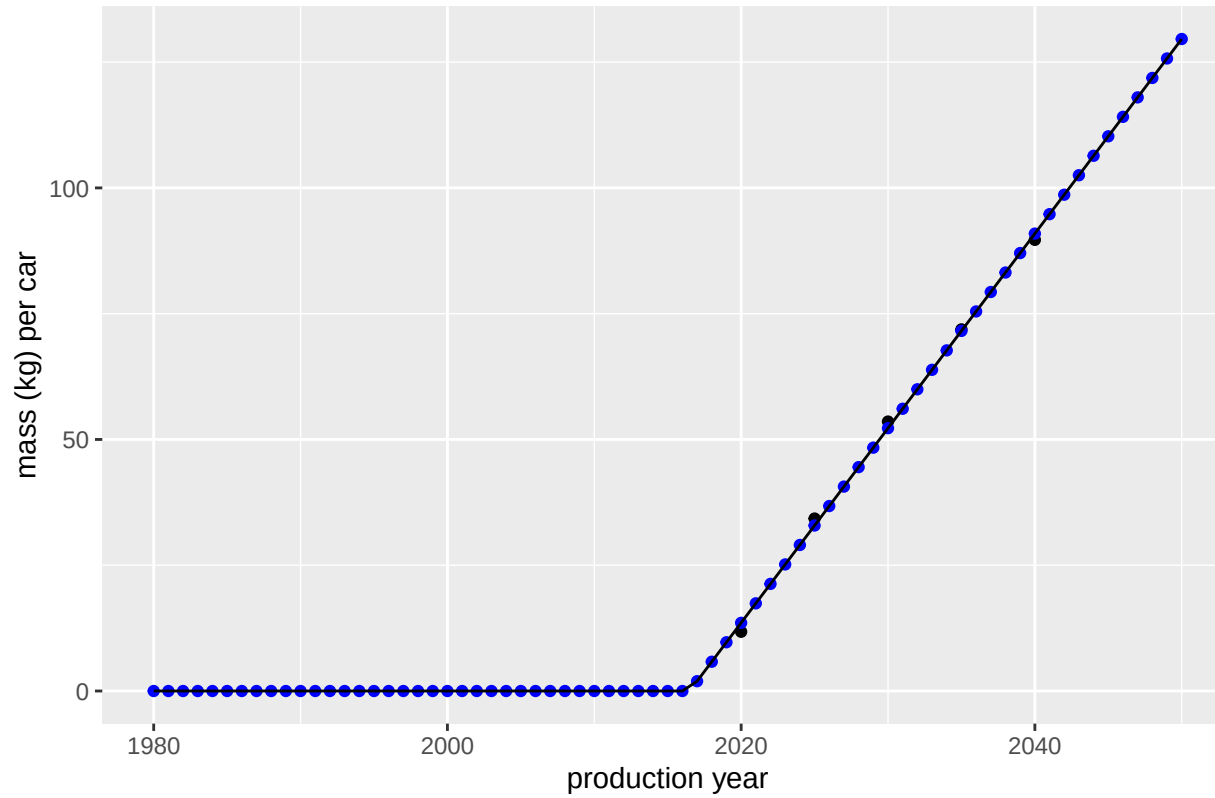
Magnesium alloys for elvPetrol B car (calculated from 1980 to 2050)



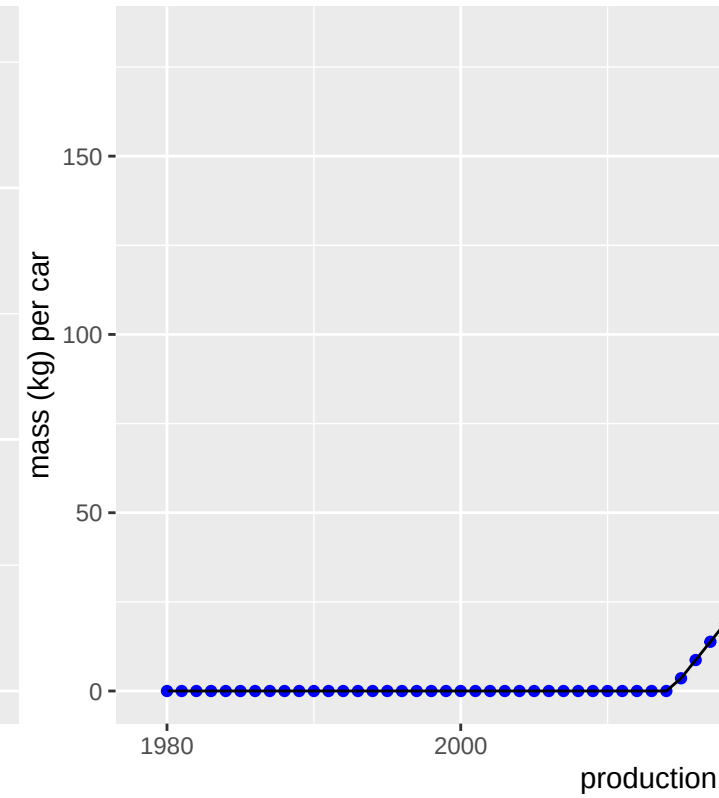
Magnesium alloys for elvPetrol C car (calculated from 1980 to 2050)



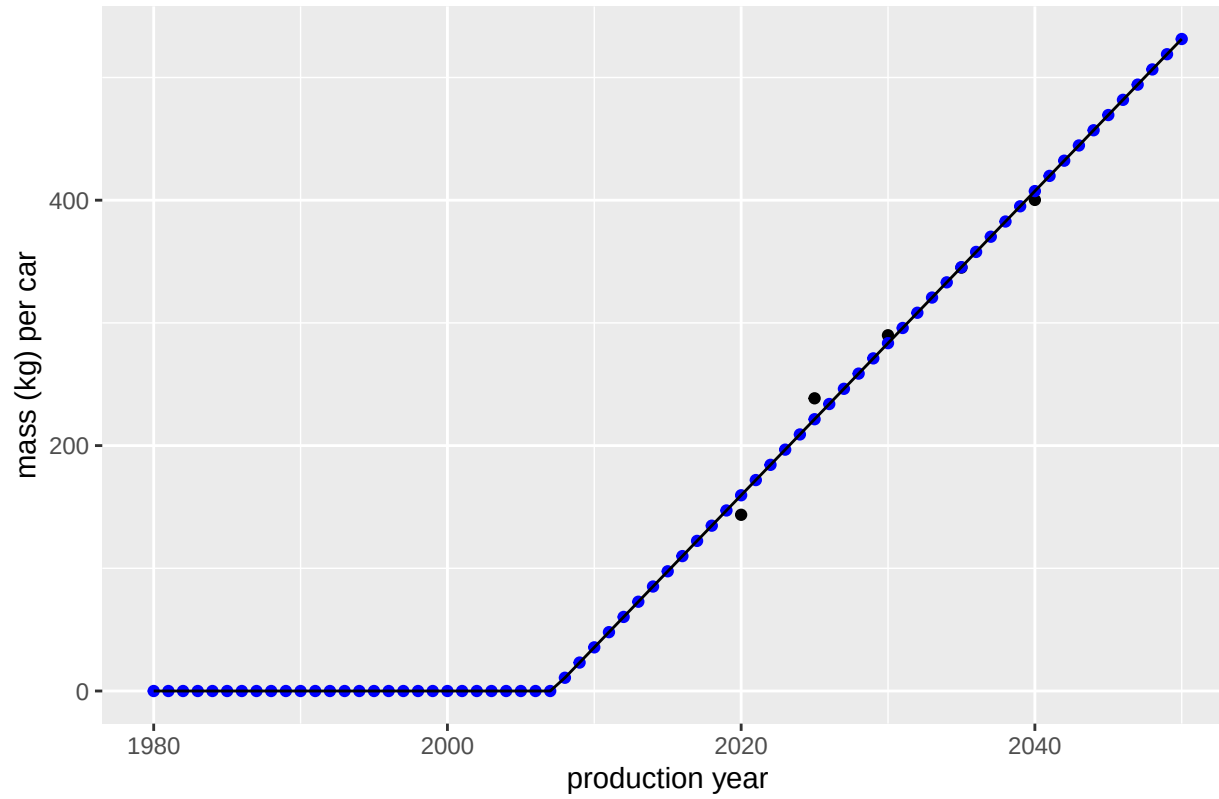
Magnesium alloys for elvPetrol D car (calculated from 1980 to 2050)



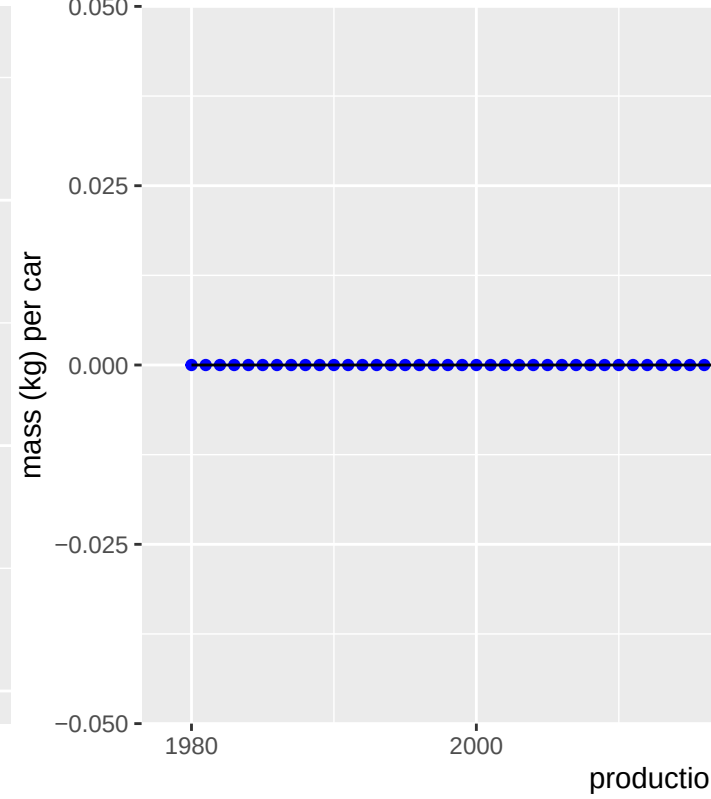
Magnesium alloys for elvPetrol E car



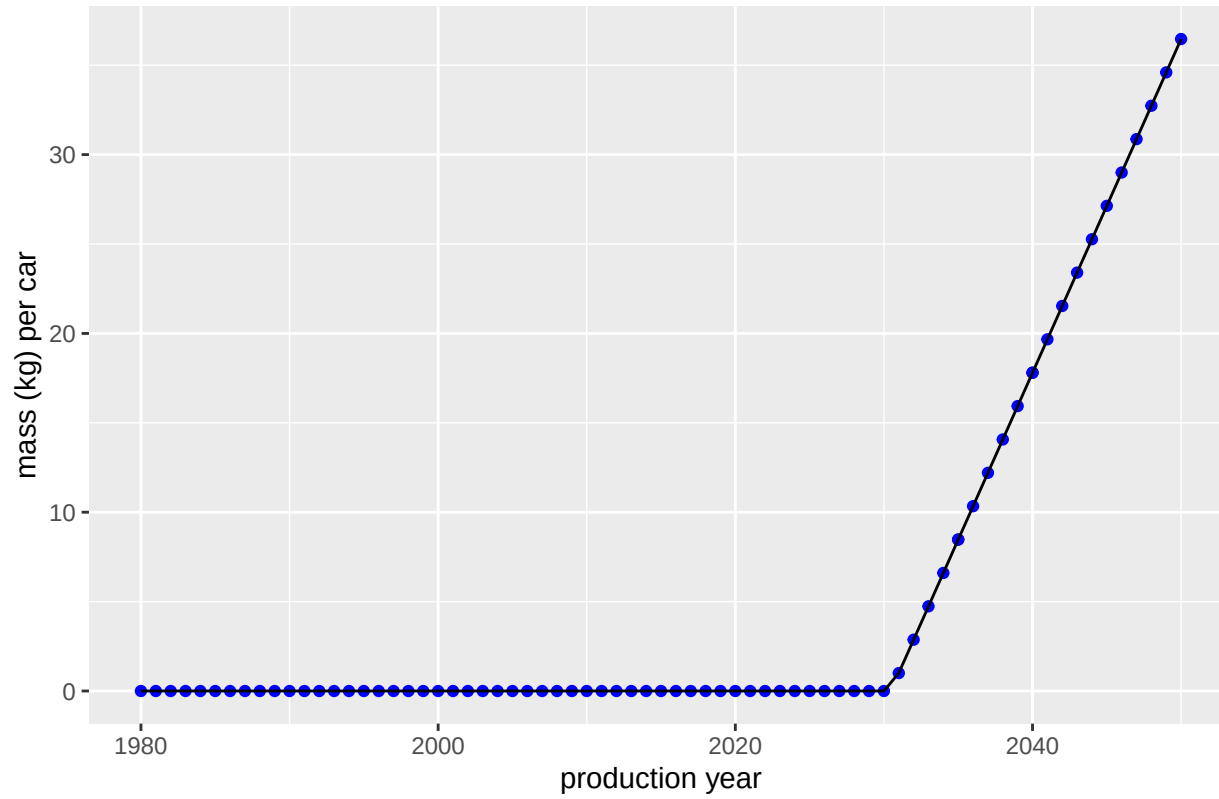
Magnesium alloys for elvPetrol F car (calculated from 1980 to 2050)



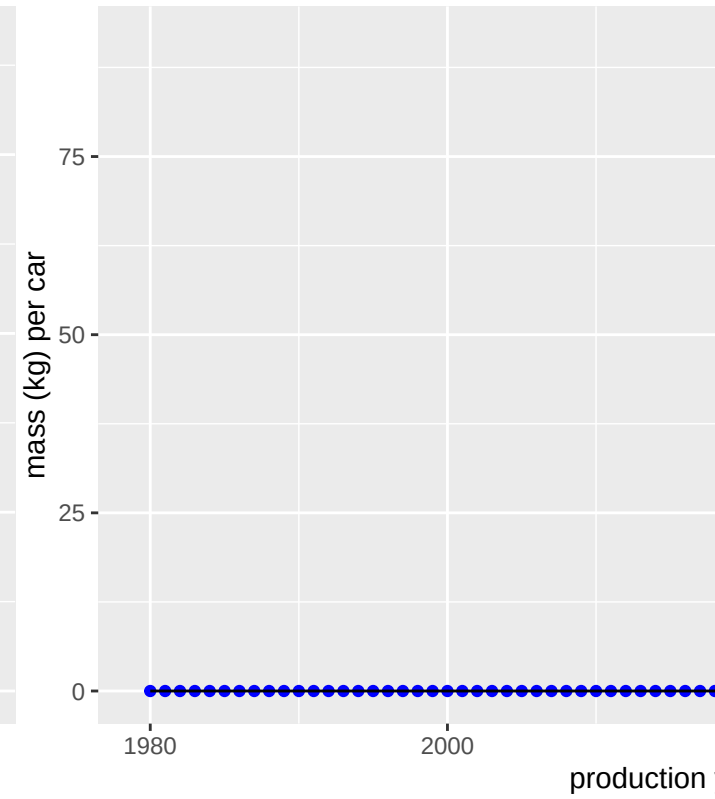
Magnesium alloys for elvPetrol JA car (calculated from 1980 to 2050)



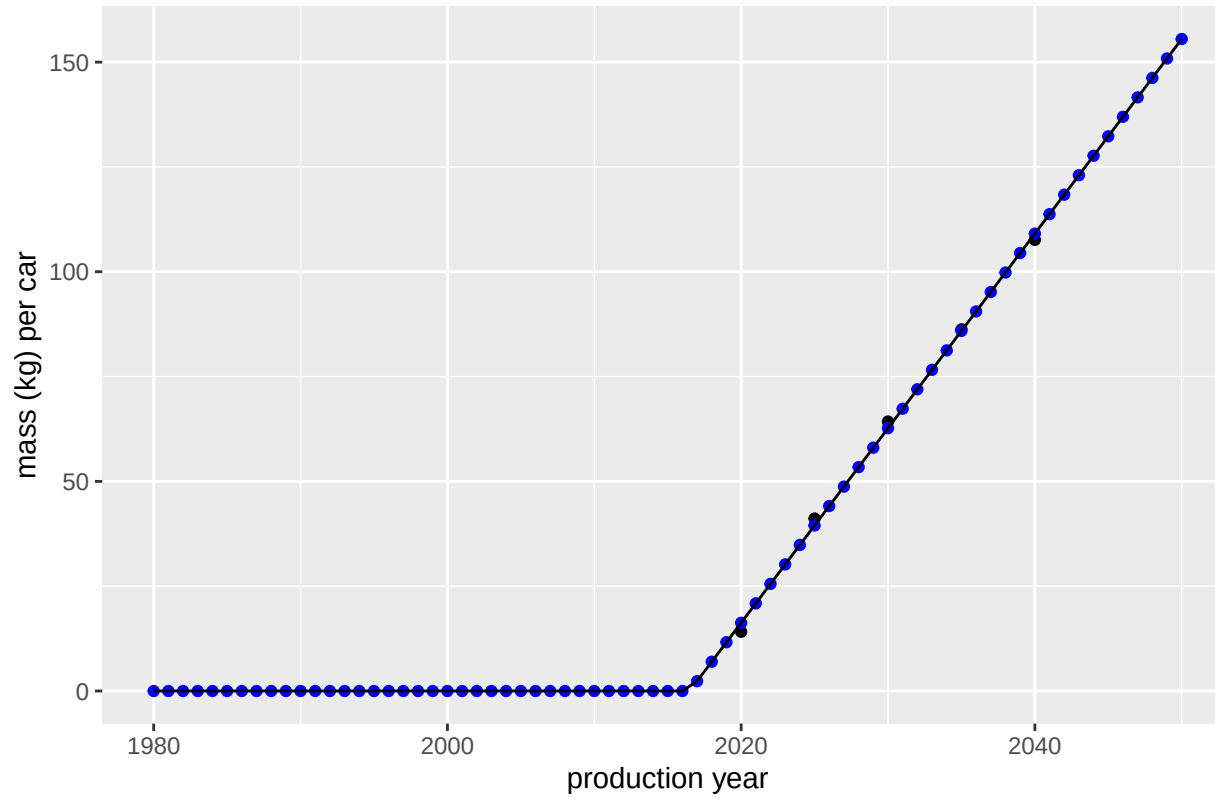
Magnesium alloys for elvPetrol JB car (calculated from 1980 to 2050)



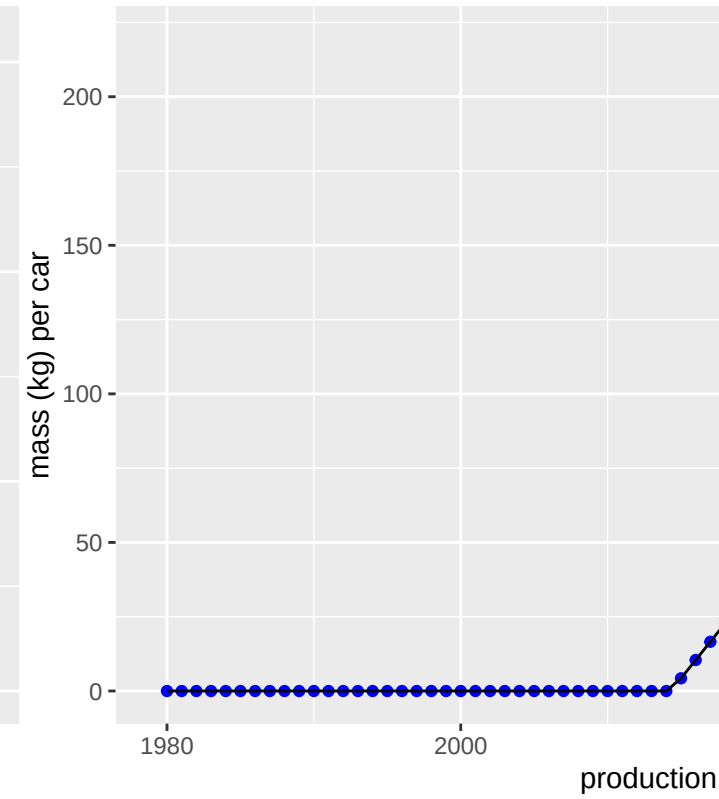
Magnesium alloys for elvPetrol JC car



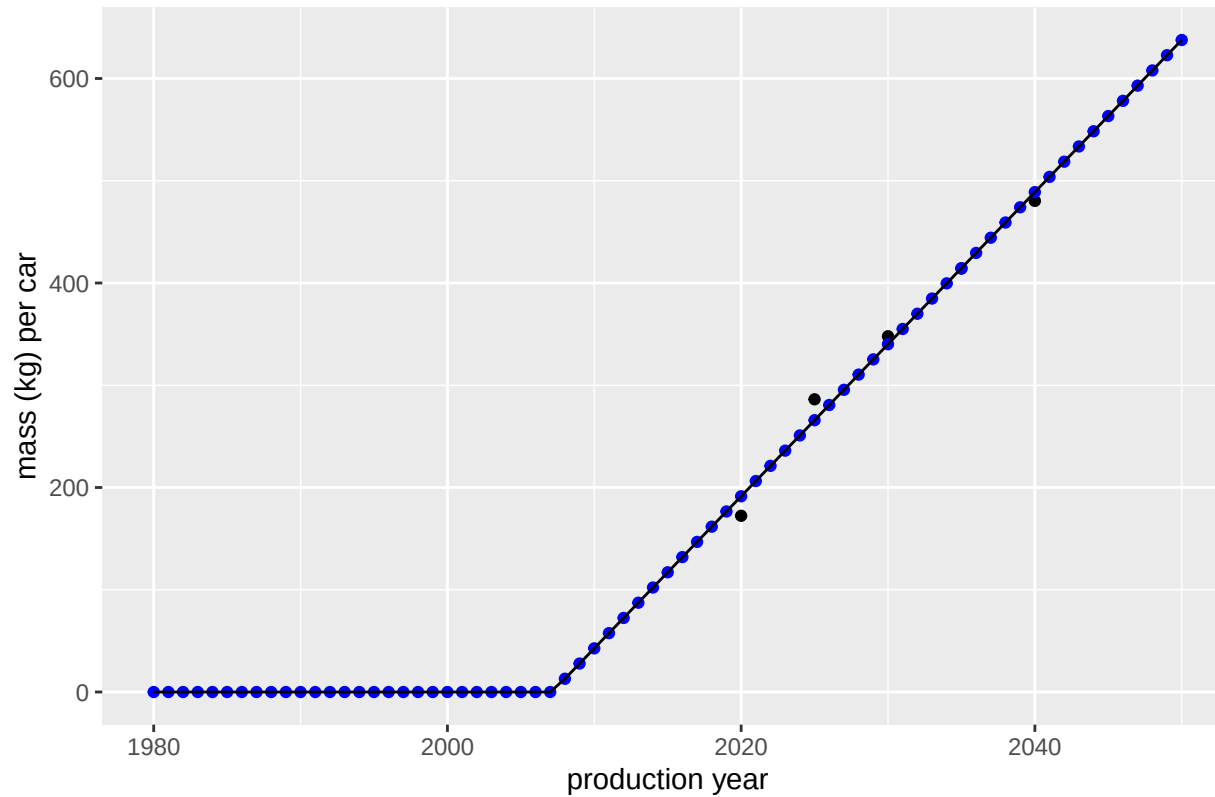
Magnesium alloys for elvPetrol JD car (calculated from 1980 to 2050)



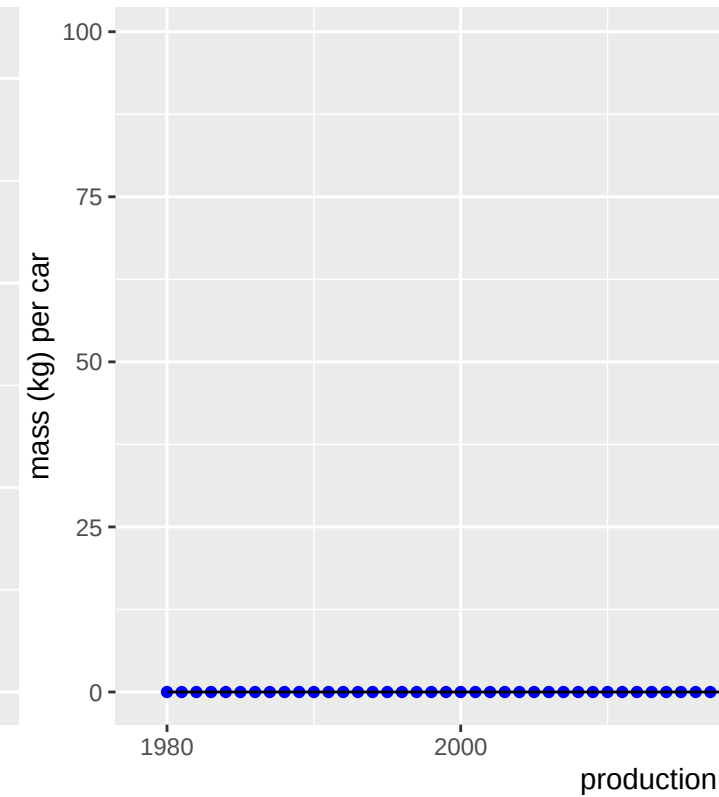
Magnesium alloys for elvPetrol JE car



Magnesium alloys for elvPetrol JF car (calculated from 1980 to 2050)



Magnesium alloys for elvPetrol standa



```
MgCalculate <- magnesiumCalculate %>% rename(productKeyLevel1 = drivetrain) %>% left_join(productKeys, by = c("productKeyLevel1", "segment")) %>% rename(MgCalculate = valueCalculate)
rename(year = yearCalculate)
```

Overall composition and weight of a car

Combine all the data

```
#AlCalculate
#FeCalculate
#MgCalculate

# combine Al and iron, steel
Calculate <- AlCalculate %>% left_join(FeCalculate, by = join_by(year, productKeyLevel1, productKeyLevel2, segment))

test <- Calculate

# combine with Mg
Calculate <- Calculate %>% left_join(MgCalculate, by = join_by(year, productKeyLevel1, productKeyLevel2, segment))

# the Al and Fe alloys have to be reduced by the amount of Mg alloys used in stead

Calculate <- Calculate %>% mutate(
  ratioAl = MgCalculate / totalAlCalculate,
  ratioFe = MgCalculate / totalIronSteelCalculate)

Calculate <- Calculate %>% mutate(
  castingAlCalculate = castingAlCalculate * (1 - ratioAl),
  extrusionAlCalculate = extrusionAlCalculate * (1 - ratioAl),
  forgingAlCalculate = forgingAlCalculate * (1 - ratioAl),
  sheetAlCalculate = sheetAlCalculate * (1 - ratioAl),
  totalAlCalculate = castingAlCalculate + extrusionAlCalculate + forgingAlCalculate + sheetAlCalculate,
  castIronCalculate = castIronCalculate * (1 - ratioFe),
  mildSteelCalculate = mildSteelCalculate * (1 - ratioFe),
  HSSCalculate = HSSCalculate * (1 - ratioFe),
  AHSSCalculate = AHSSCalculate * (1 - ratioFe),
  UHSSCalculate = UHSSCalculate * (1 - ratioFe),
  totalIronSteelCalculate = castIronCalculate + mildSteelCalculate + HSSCalculate + AHSSCalculate + UHSSCalculate
) %>% select(-c(ratioAl, ratioFe))

# calculate the total amount of alloys
Calculate <- Calculate %>% mutate(totalAlloys = totalAlCalculate + totalIronSteelCalculate + MgCalculate)

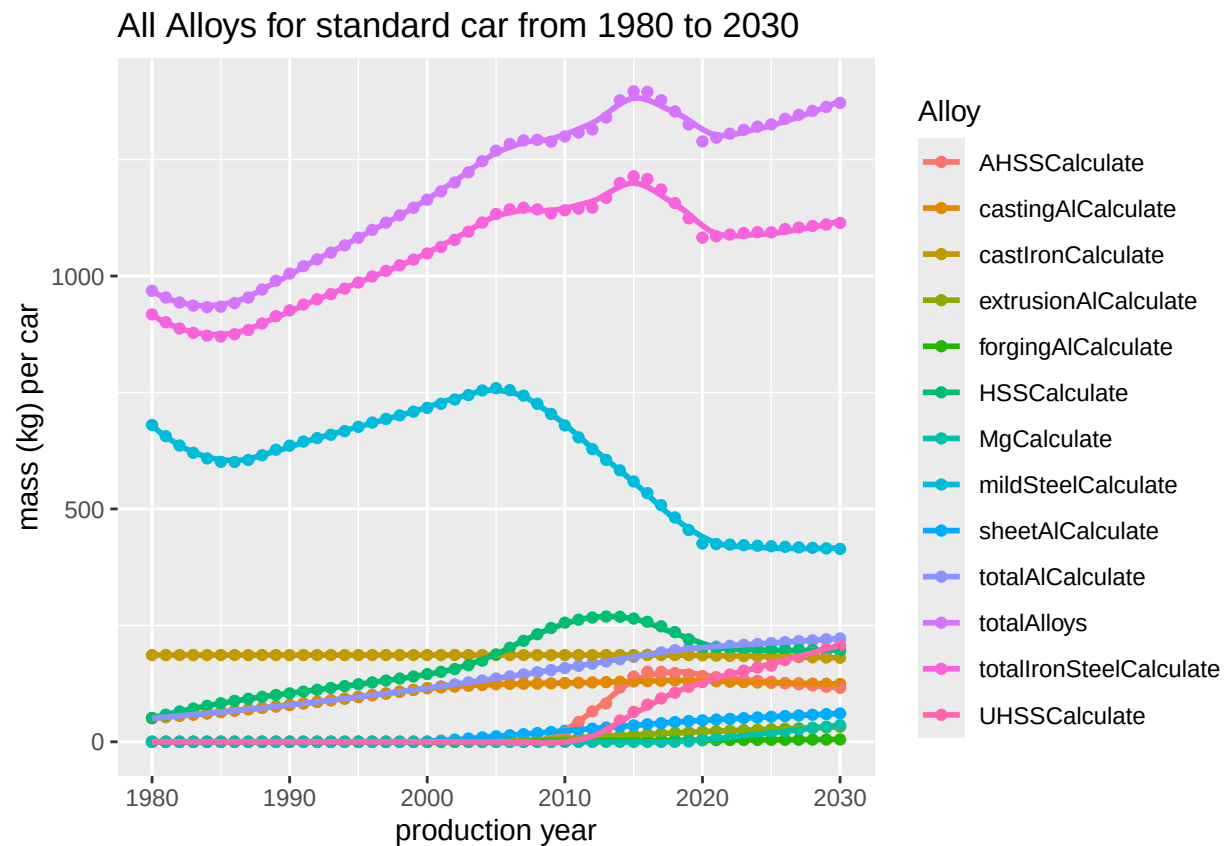
## Display material composition of standard cars

# generate a long data.frame for the display
Calculate_long <- pivot_longer(Calculate, cols = !c(year, productKeyLevel2, productKeyLevel1, segment), names_to = "Alloy", values_to = "values")

# select the standard car
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010000")
```

```
g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span= 0.3) +
  ggtitle("All Alloys for standard car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```



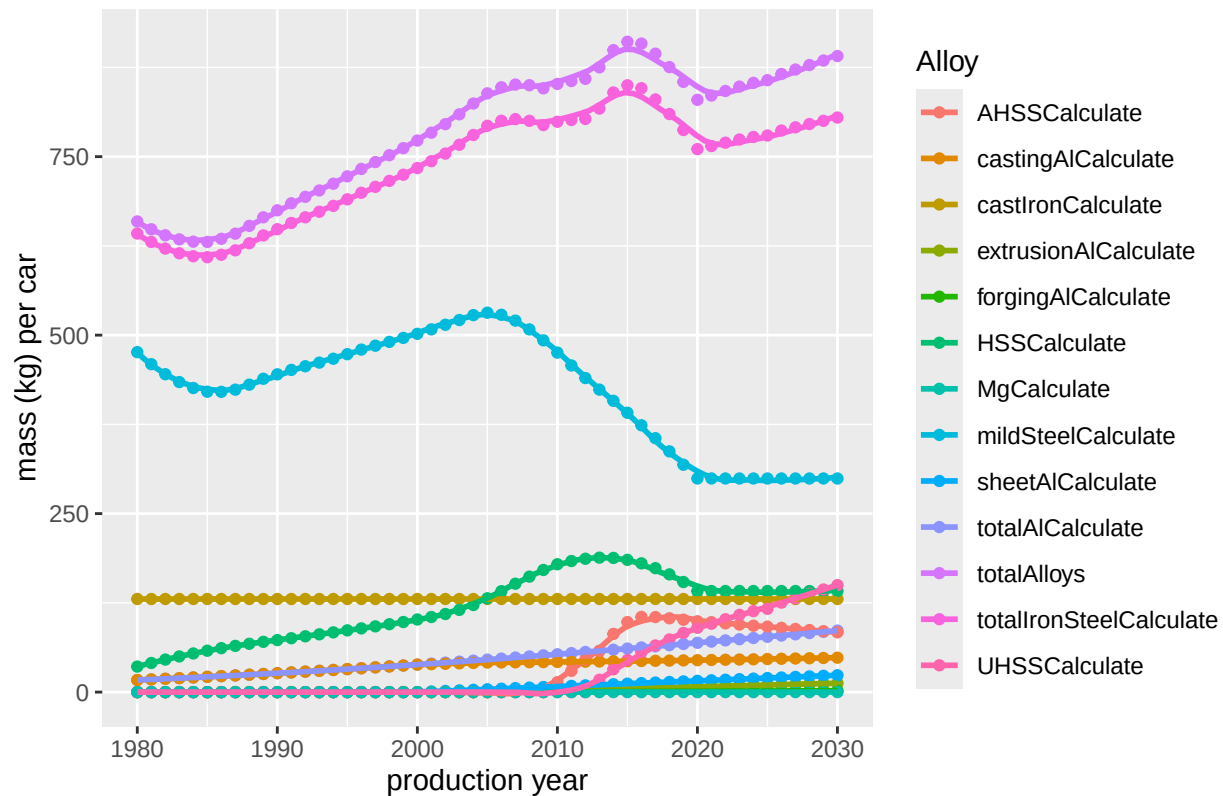
```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010101")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
```

```
geom_smooth(method = "loess", se = FALSE, span= 0.3) +
ggtitle("All Alloys for V0101010101 car from 1980 to 2030") +
xlab("production year") +
ylab("mass (kg) per car")
```

```
plot(g)
```

All Alloys for V0101010101 car from 1980 to 2030



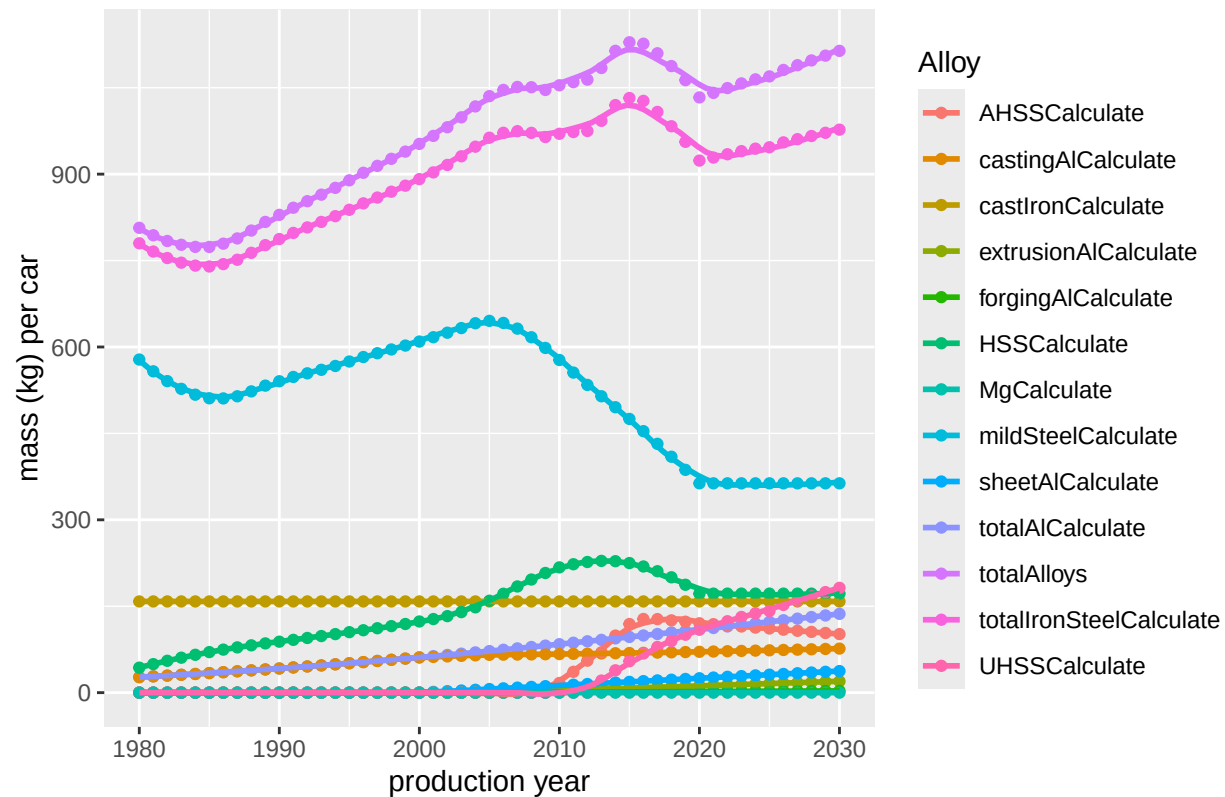
```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010102")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span= 0.3) +
  ggtitle("All Alloys for V0101010102 car from 1980 to 2030") +
```

```
xlab("production year") +  
ylab("mass (kg) per car")
```

```
plot(g)
```

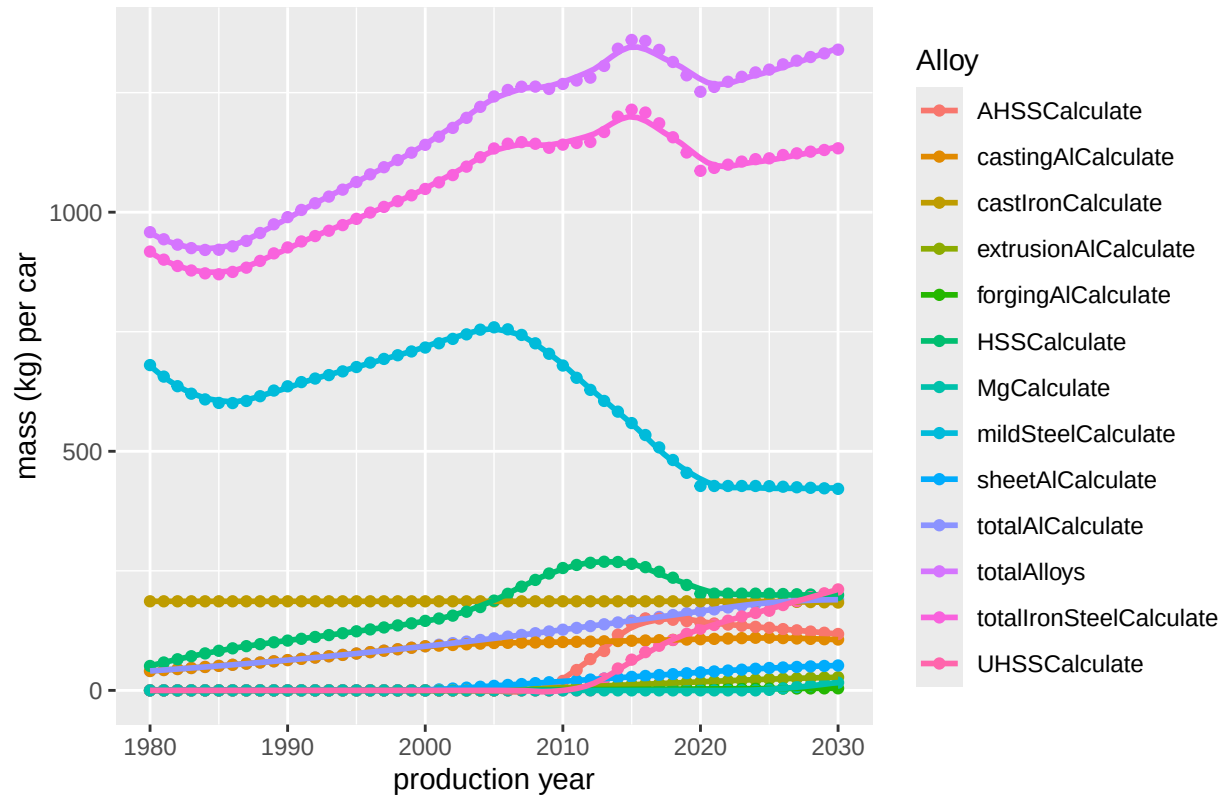
All Alloys for V0101010102 car from 1980 to 2030



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010103")  
  
g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +  
  geom_point() +  
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +  
  ggtitle("All Alloys for V0101010103 car from 1980 to 2030") +  
  xlab("production year") +  
  ylab("mass (kg) per car")
```

```
plot(g)
```

All Alloys for V0101010103 car from 1980 to 2030

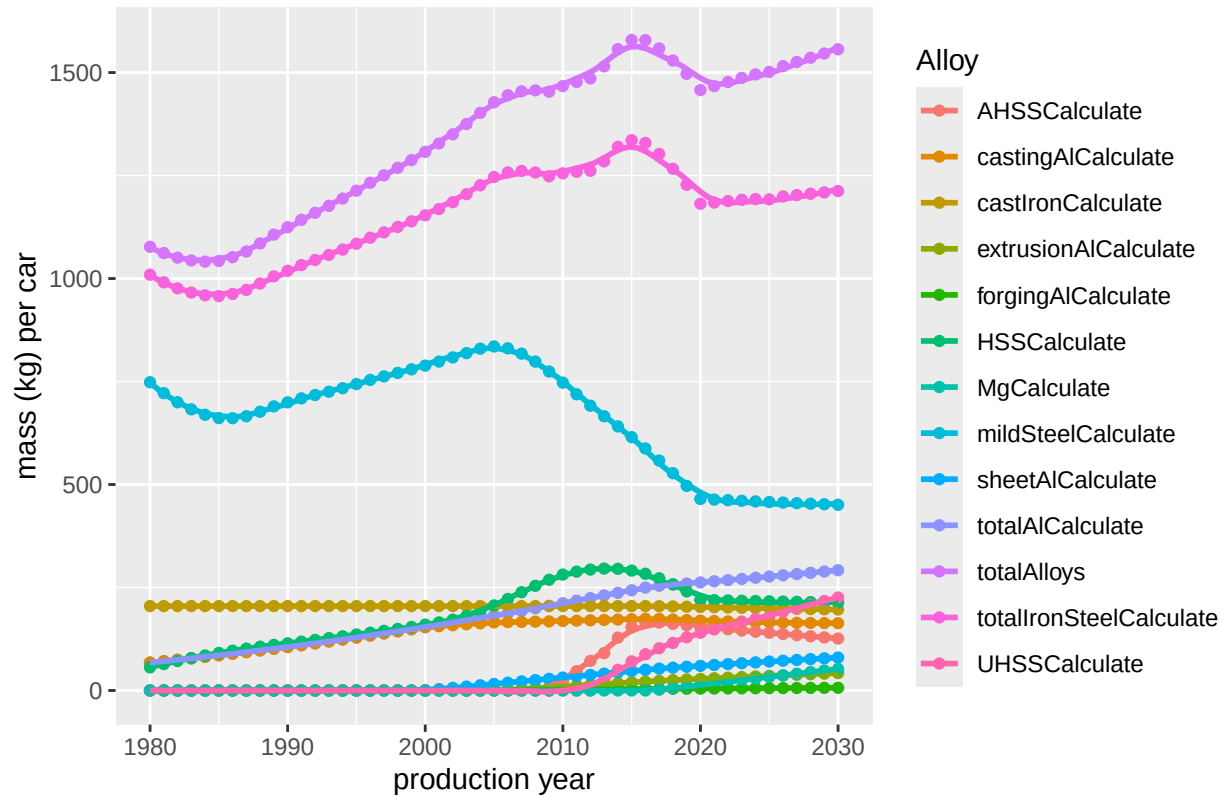


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010104")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010104 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010104 car from 1980 to 2030

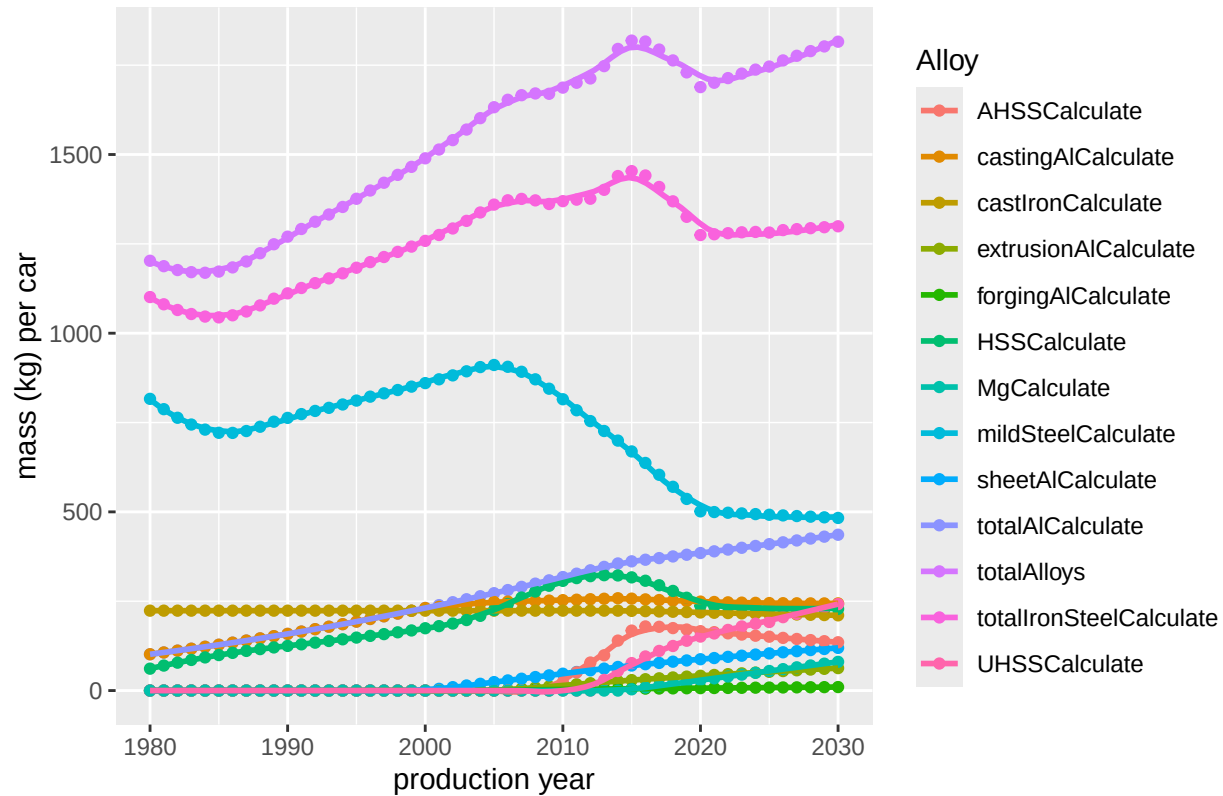


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010105")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010105 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010105 car from 1980 to 2030

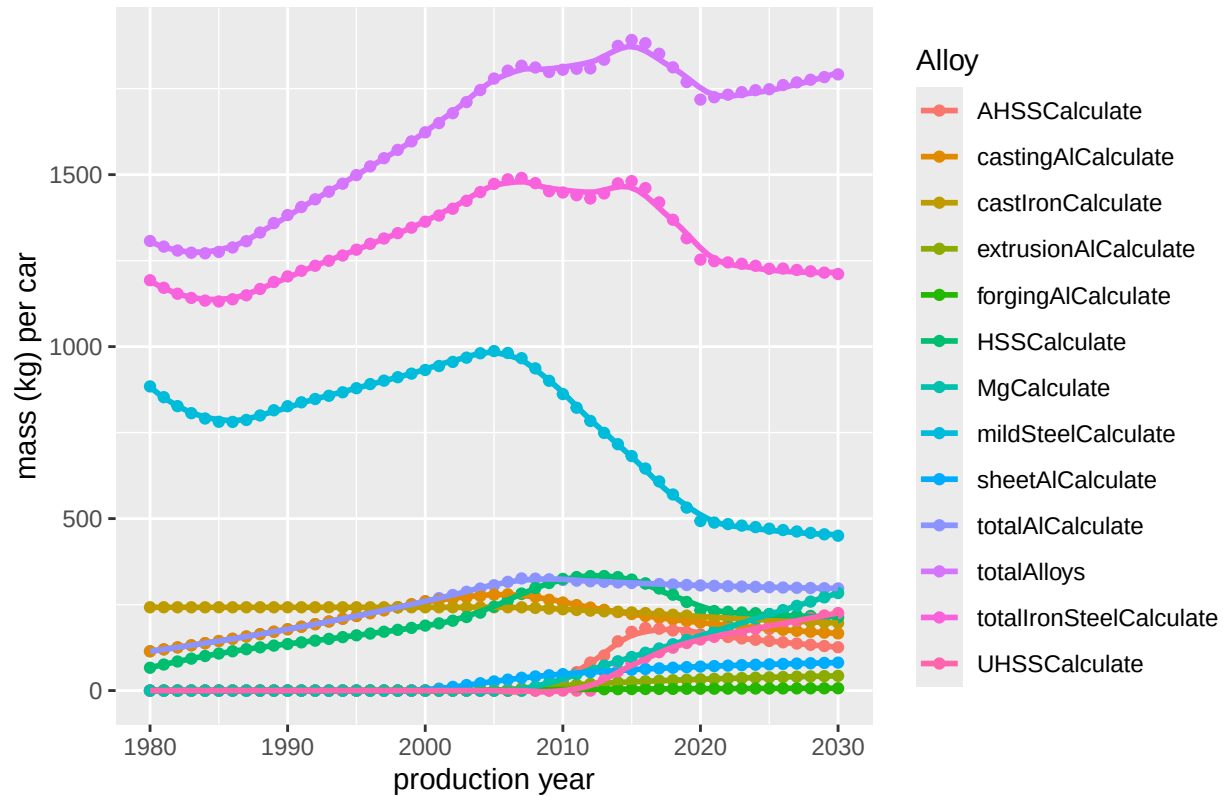


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010106")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010106 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010106 car from 1980 to 2030

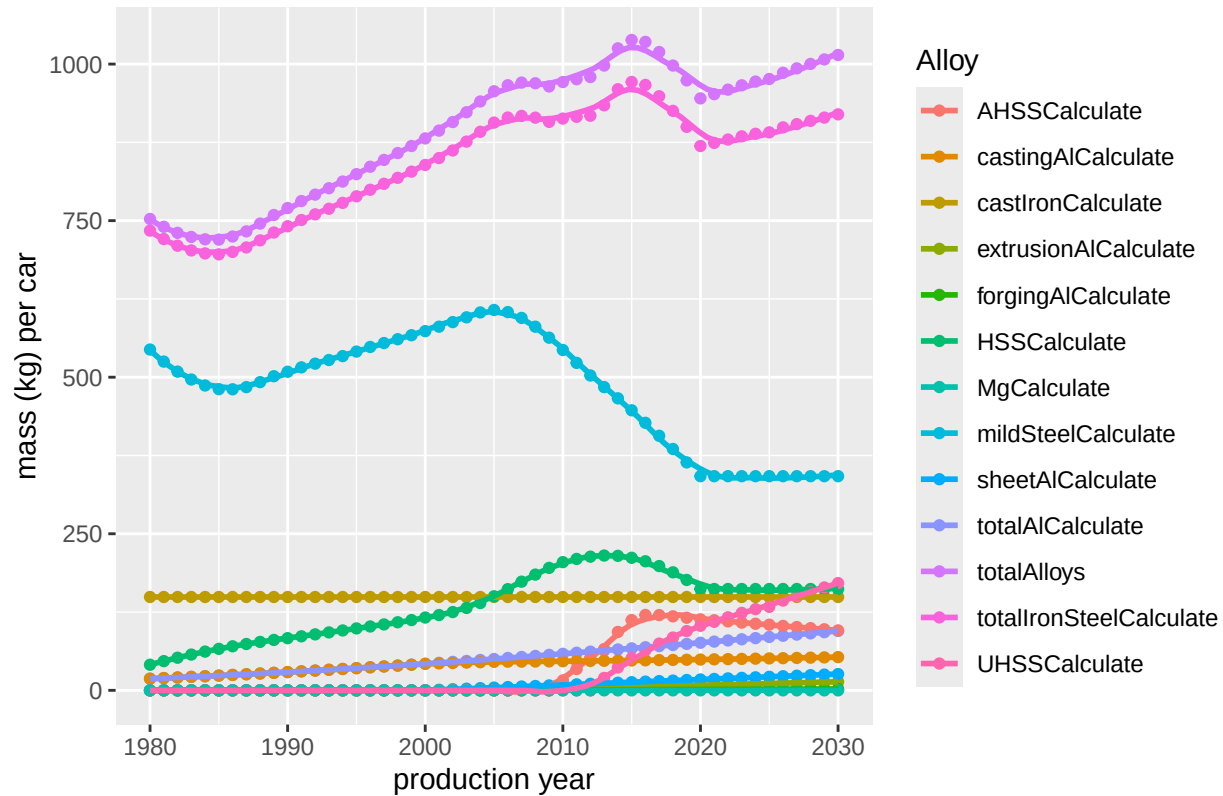


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010201")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010201 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010201 car from 1980 to 2030

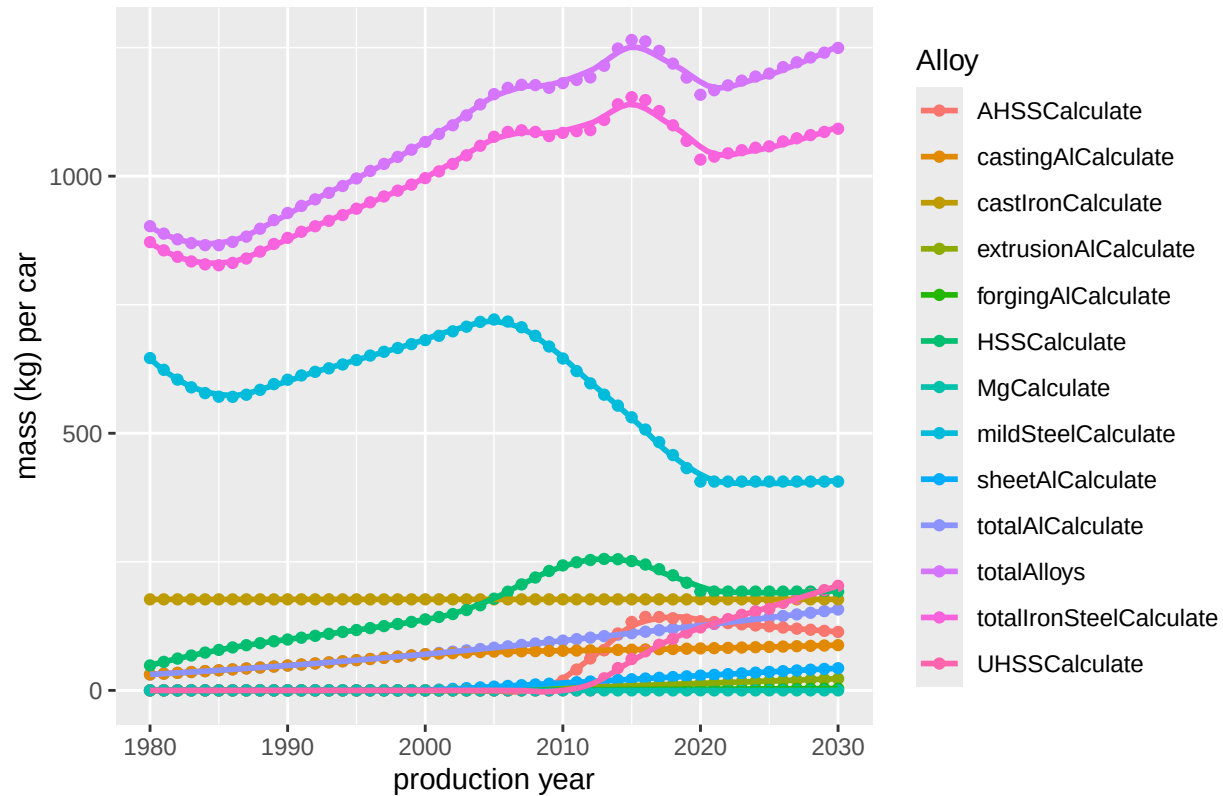


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010202")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010202 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010202 car from 1980 to 2030

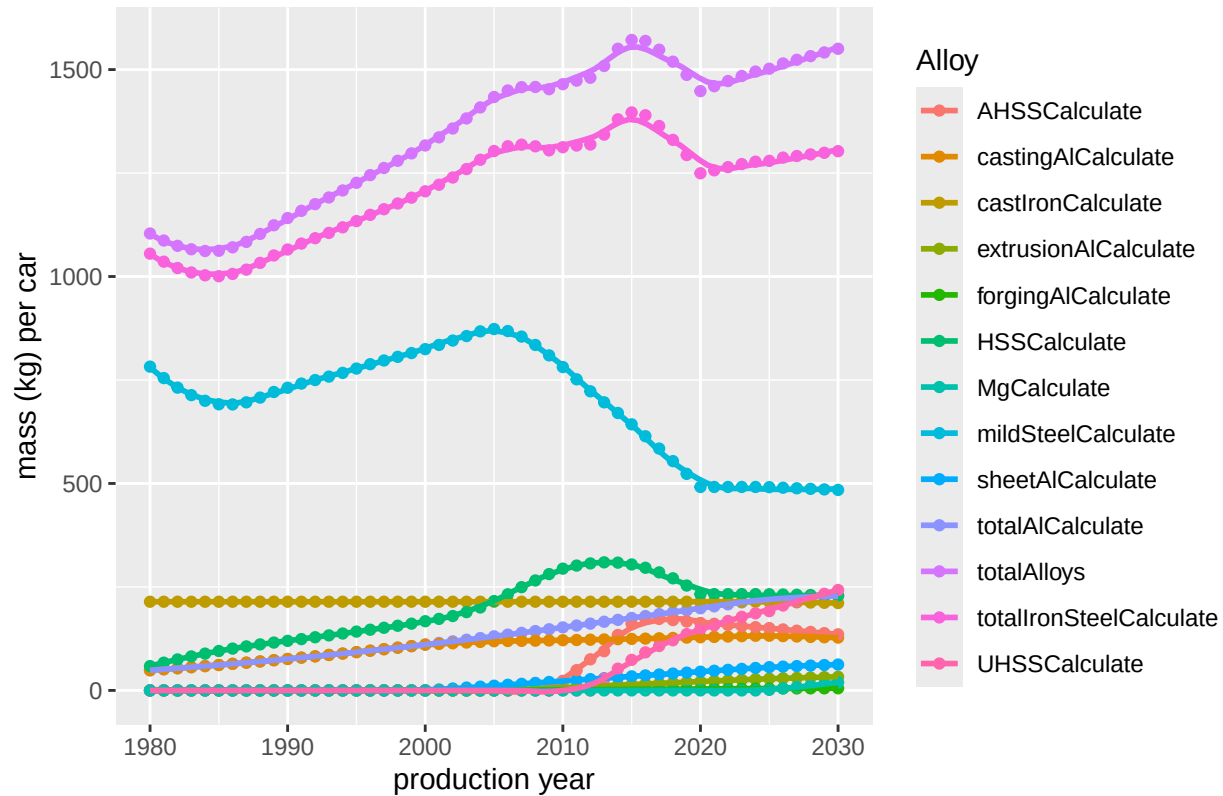


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010203")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010203 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010203 car from 1980 to 2030

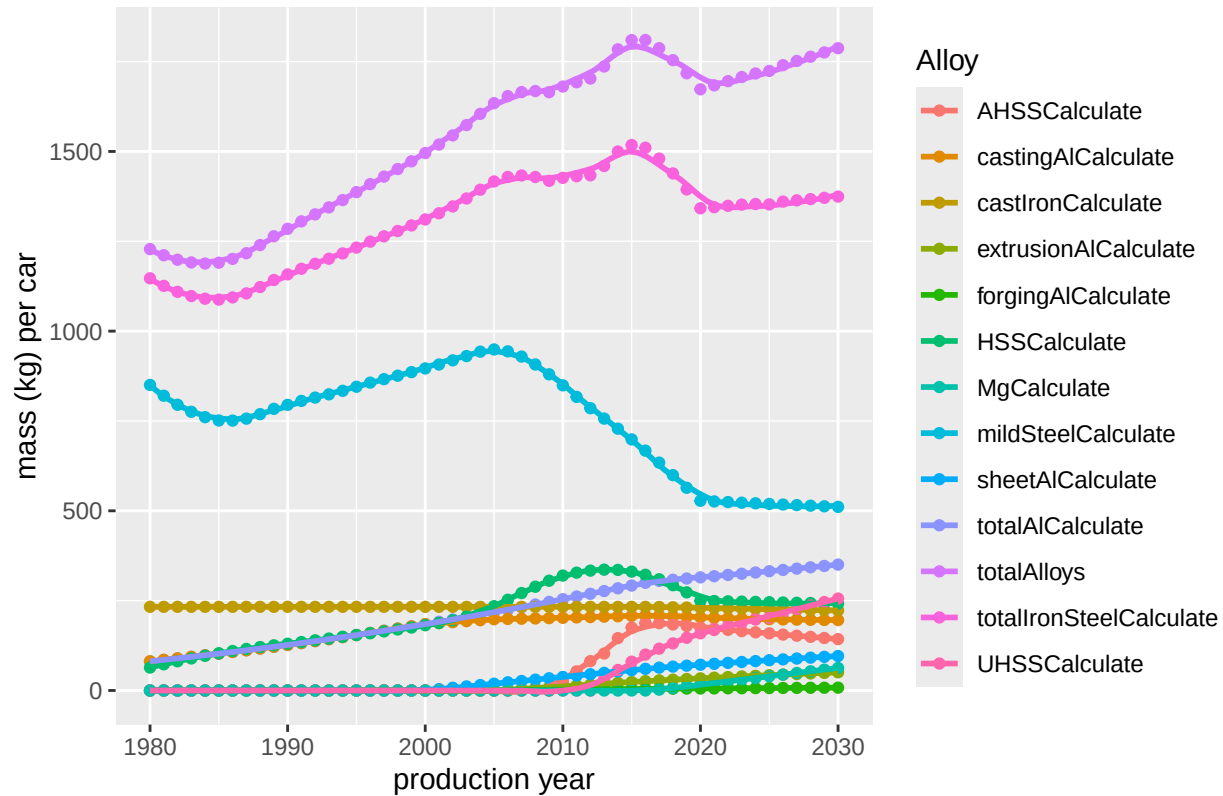


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010204")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010204 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010204 car from 1980 to 2030

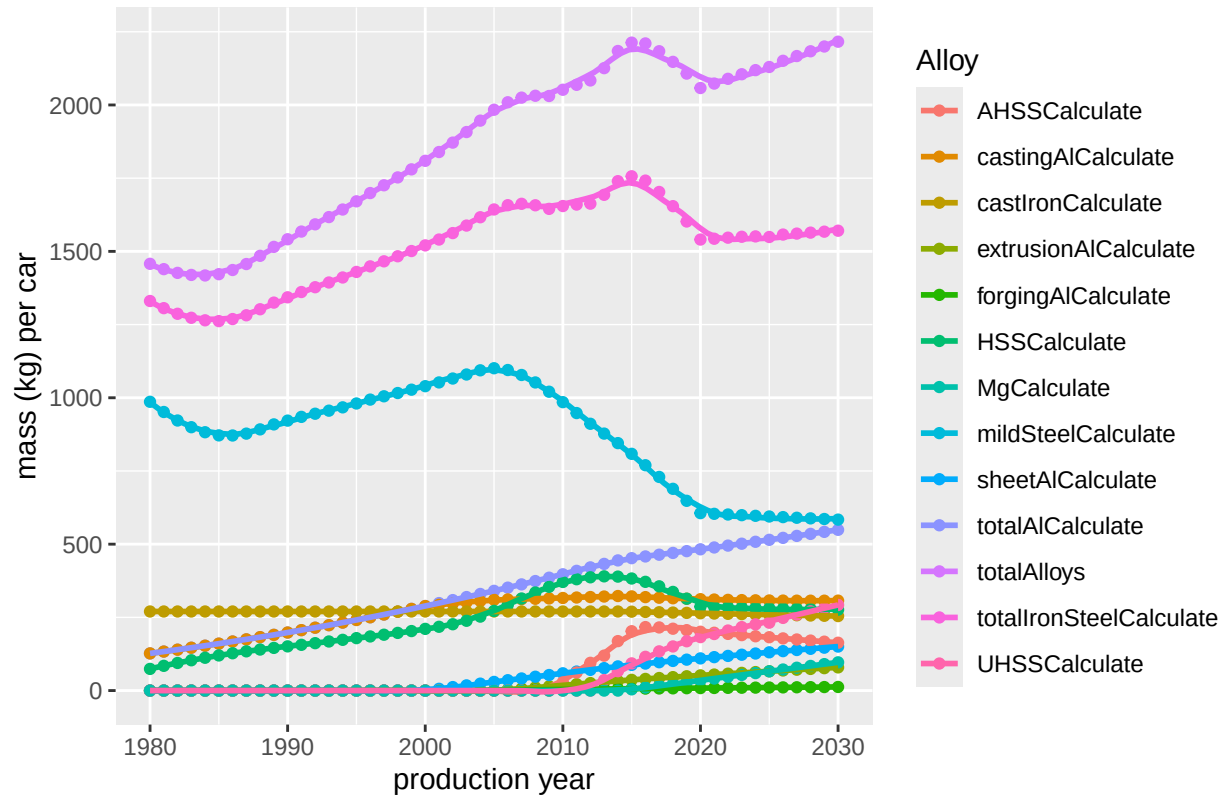


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010205")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010205 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010205 car from 1980 to 2030

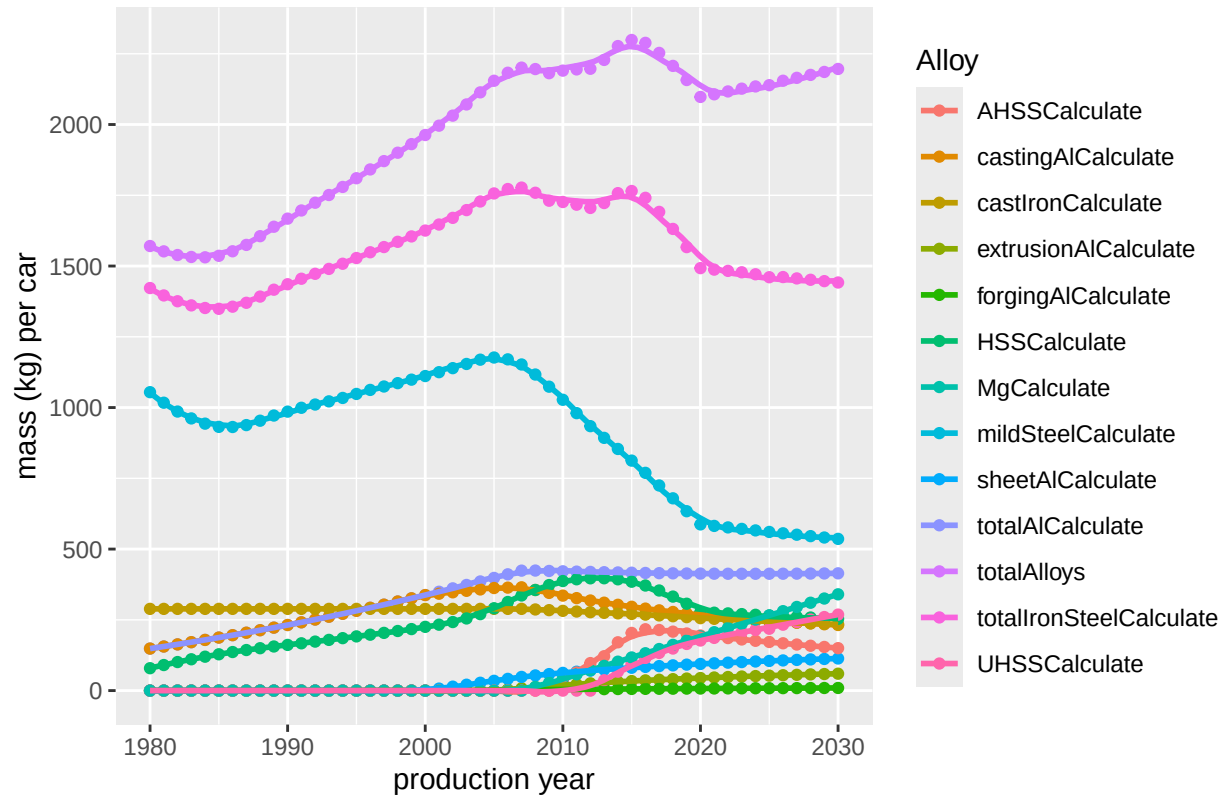


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010206")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Alloy)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("All Alloys for V0101010206 car from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

All Alloys for V0101010206 car from 1980 to 2030

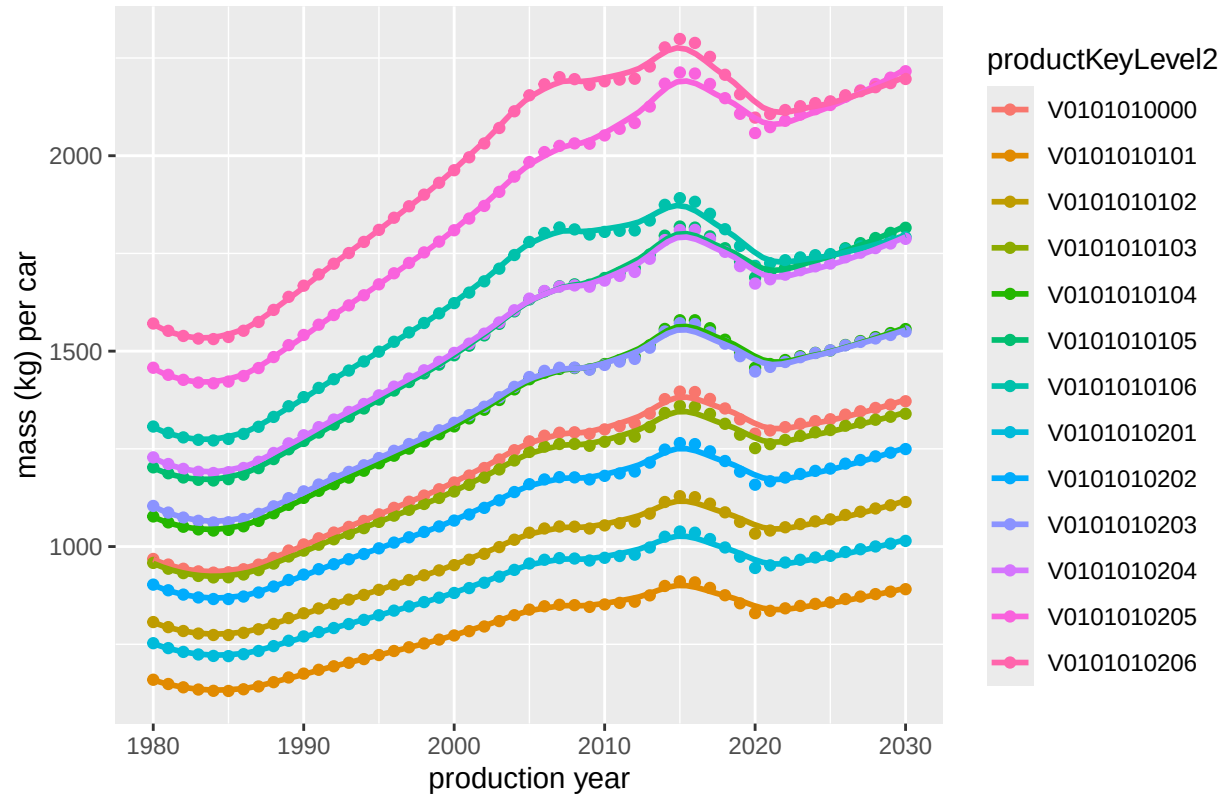


```
temp <- Calculate_long %>% filter(Alloy == "totalAlloys" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = productKeyLevel2)) +
  geom_point() +
  geom_smooth(method = "loess", se = FALSE, span = 0.3) +
  ggtitle("Total Mass for different cars from 1980 to 2030") +
  xlab("production year") +
  ylab("mass (kg) per car")

plot(g)
```

Total Mass for different cars from 1980 to 2030



Catalyst Content - Derive PGE composition for catalyst

Chinese data and cover area between 2010 to 2030

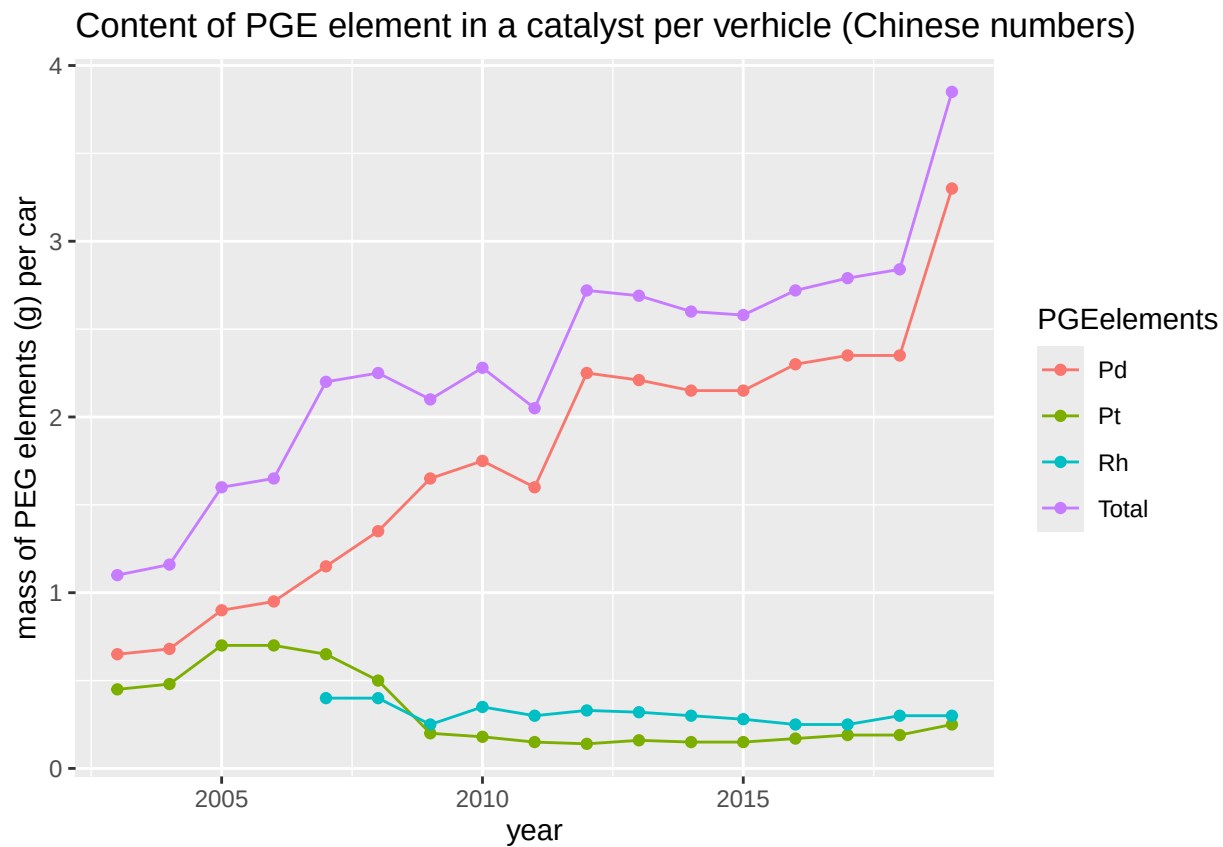
```
#catalyst_content_1

catalyst_content_1_long <- catalyst_content_1 %>% pivot_longer(cols = c("Pd", "Pt", "Rh", "Total"), names_to = "PGEelements", values_to = "values")

g <- ggplot(data = catalyst_content_1_long, mapping = aes(x = year, y = values, color = PGEelements)) +
  geom_point() +
```

```
geom_line() +
ggtitle("Content of PGE element in a catalyst per vehicle (Chinese numbers)") +
xlab("year") +
ylab("mass of PEG elements (g) per car")
```

```
plot(g)
```



```
#-----
#
# Look at Pt, Rh & Pd
# get linear behavior between 2009 and 2019
```

```
temp <- catalyst_content_1 %>% filter(year > 2008)
```

```
calculate.Pt <- mean(temp$Pt)
calculate.Rh <- mean(temp$Rh)
```

```
temp <- catalyst_content_1 %>% filter(year > 2009)
tempPd <- temp$Pd
```

```
temp <- catalyst_content_1 %>% filter(year > 2012 & year < 2019)
mtempLinear <- lm(Pd ~ year, data = temp)
summary(mtempLinear)
```

```
##
## Call:
## lm(formula = Pd ~ year, data = temp)
##
## Residuals:
##      1      2      3      4      5      6
## 0.061905 -0.039524 -0.080952  0.027619  0.036190 -0.005238
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -81.24762    28.55192   -2.846   0.0466 *
## year         0.04143     0.01417    2.924   0.0431 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.05926 on 4 degrees of freedom
## Multiple R-squared:  0.6813, Adjusted R-squared:  0.6017
## F-statistic: 8.553 on 1 and 4 DF,  p-value: 0.04305
```

```
Pd.2019 <- tempPd[length(tempPd)]
```

```
for(i in 1:11) {
  tempPd <- append(tempPd, Pd.2019 + i * mtempLinear$coefficients[2])
}
```

```
yearCalculate <- c(2010:2030)
```

```
PGEcalucalute.V0101010000 <- data.frame(year = yearCalculate, PdCalculate = tempPd/1000, PtCalculate = calculate.Pt/1000, RhCalculate = calculate.Rh/1000, productKeyLevel2 = "V0101010000")
```

Look at the overall data between 1980 to 2030

```
#catalyst_content_2_1
#catalyst_content_2_2
#catalyst_content_2_3

## prepare data sets, so that the amount of CC per vehicle can be calculated depending on the drive train and car segment (unknown = standard, <1400 cm = A & B, 1400-1900 = C & D, >1900 = E & F)

# generate all the required columns, so each segment has its own
catalyst_content_2_2complet <- catalyst_content_2_2 %>% mutate(B = A_B, D = C_D, F = E_F) %>% rename(A = A_B, C = C_D, E = E_F) %>% select(year, standard, A, B, C, D, E, F)

# generate the additional columns for the HEV and the PEHV
catalyst_content_2_3_long <- catalyst_content_2_3 %>% mutate(elvHEV = Petrol, elvPHEV = Petrol) %>% rename(elvPetrol = Petrol, elvDiesel = Diesel) %>% pivot_longer(cols = -Element, names_to = "drivetrain", values_to = "values")

# rename the columns properly
catalyst_content_2_1 <- catalyst_content_2_1 %>% rename(elvPetrol = petrol, elvDiesel = diesel, elvHEV = HEV, elvPHEV = PHEV)

catalyst_content_2 <- catalyst_content_2_1 %>% left_join(catalyst_content_2_2complet, by = "year")
catalyst_content_2_long <- catalyst_content_2 %>% pivot_longer(cols = c(elvPetrol, elvDiesel, elvHEV, elvPHEV), names_to = "drivetrain", values_to = "values") %>% mutate(standard = standard * values, A = A * values, B = B * values, C = C * values, D = D * values, E = E * values, F = F * values) %>% select(-values)

catalyst_content_2 <- catalyst_content_2_long %>% left_join(catalyst_content_2_3_long, by = "drivetrain") %>% mutate(standard = standard * values, A = A * values, B = B * values, C = C * values, D = D * values, E = E * values, F = F * values) %>% select(-values)

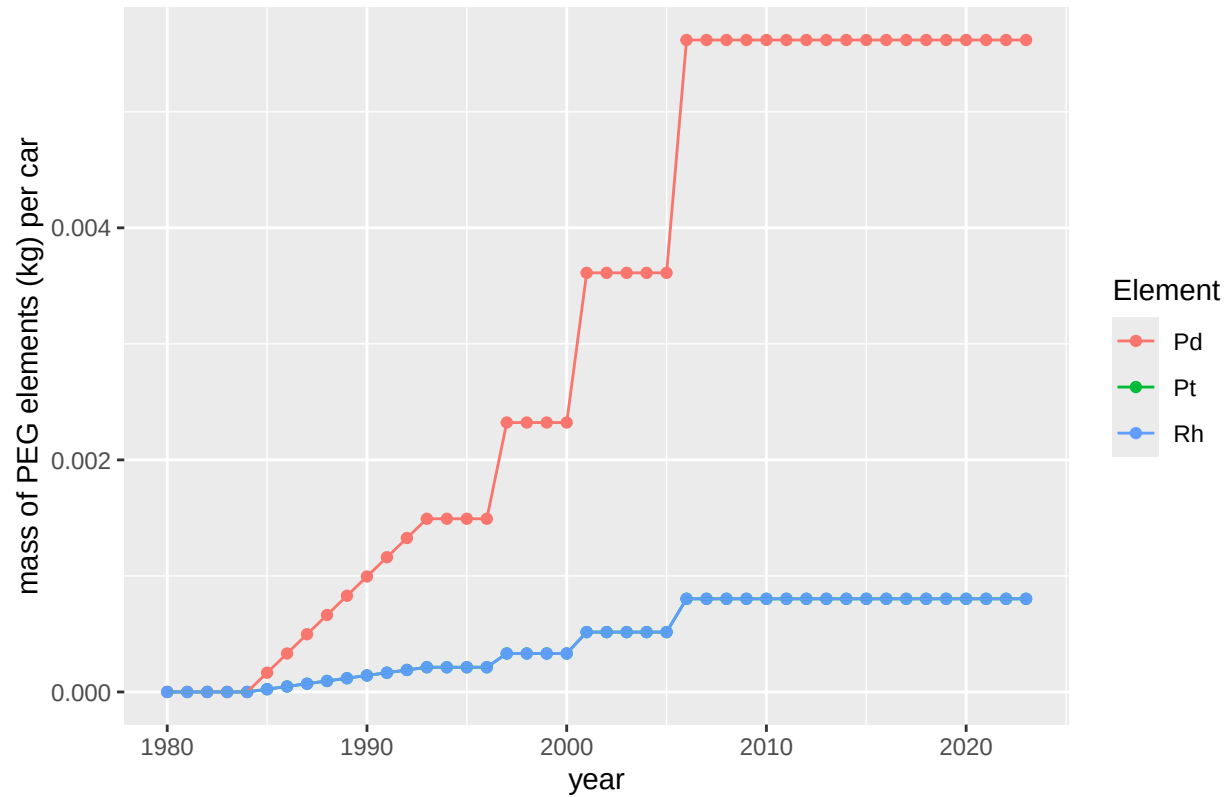
#calalyst_content_2

temp <- catalyst_content_2 %>% filter(drivetrain == "elvPetrol", (Element != "La") & (Element != "Ce"))

g <- ggplot(data = temp, mapping = aes(x = year, y = C, colour = Element)) +
  geom_point() +
  geom_line() +
  ggtitle("Content of PGE element in a catalyst per verhicle (Estimates)") +
  xlab("year") +
  ylab("mass of PEG elements (kg) per car")

plot(g)
```

Content of PGE element in a catalyst per vehicle (Estimates)



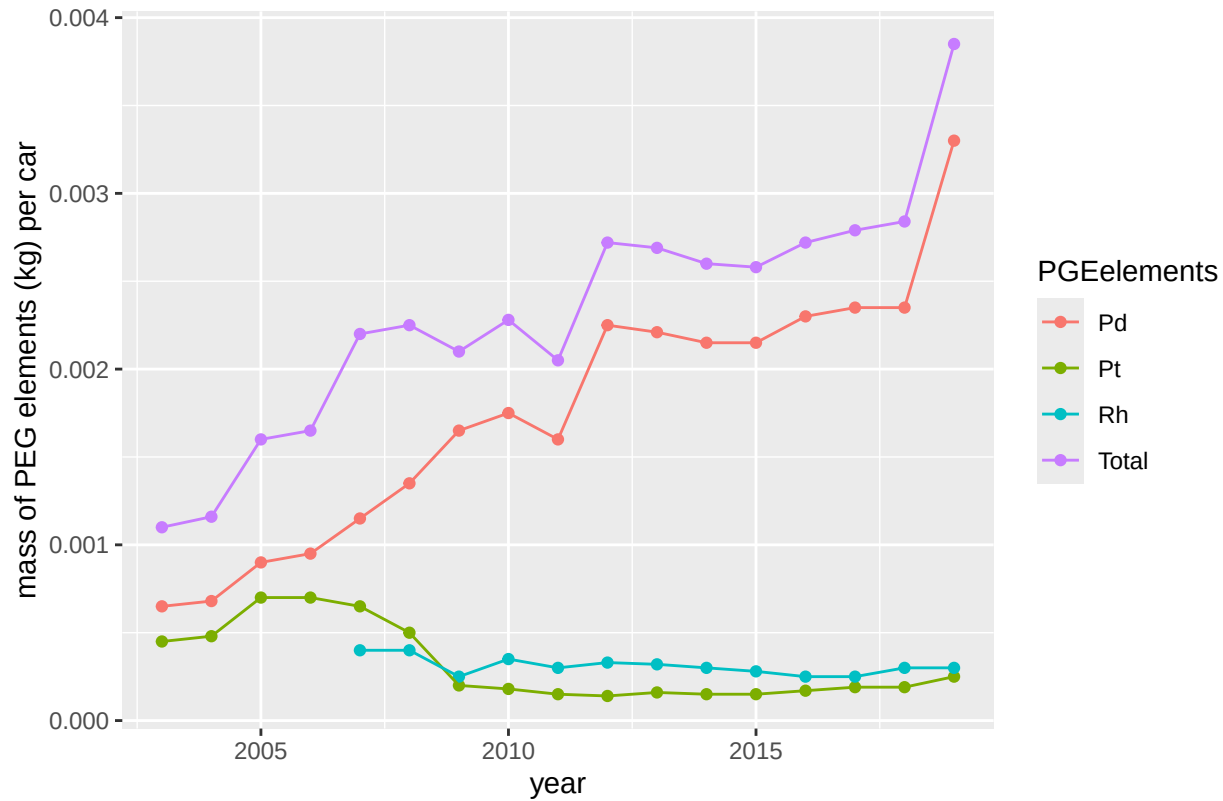
```
# Add the Chinese data

catalyst_content_1_long <- catalyst_content_1 %>% pivot_longer(cols = c("Pd", "Pt", "Rh", "Total"), names_to = "PGEelements", values_to = "values")

g <- ggplot(data = catalyst_content_1_long, mapping = aes(x = year, y = values/1000, color = PGEelements)) +
  geom_point() +
  geom_line() +
  ggtitle("Content of PGE element in a catalyst per vehicle (Chinese numbers)") +
  xlab("year") +
  ylab("mass of PEG elements (kg) per car")

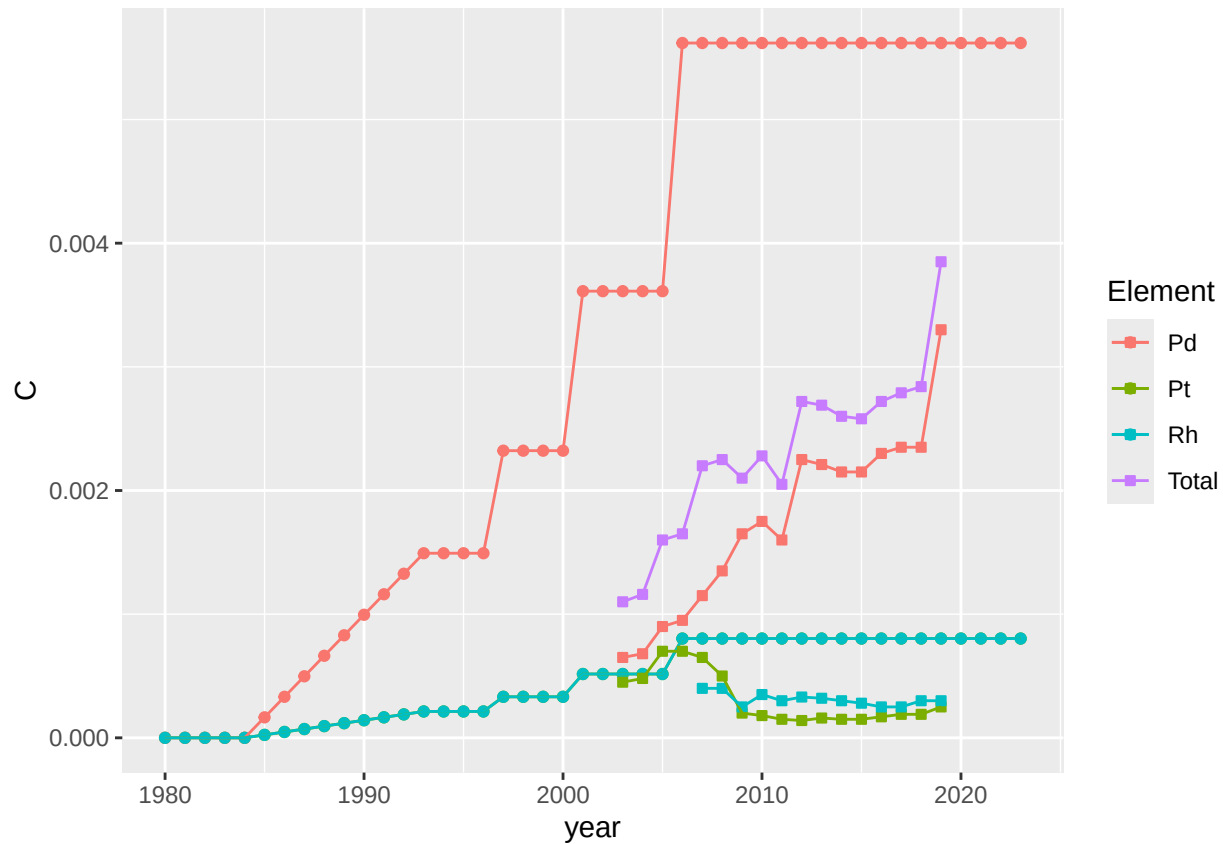
plot(g)
```

Content of PGE element in a catalyst per vehicle (Chinese numbers)



```
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = C, fill = Element, colour = Element), shape = 21) +
  geom_line(data = temp, mapping = aes(x = year, y = C, colour = Element)) +
  geom_point(data = catalyst_content_1_long, mapping = aes(x = year, y = values/1000, fill = PGEelements, colour = PGEelements), shape = 22) +
  geom_line(data = catalyst_content_1_long, mapping = aes(x = year, y = values/1000, fill = PGEelements, colour = PGEelements))

plot(g)
```



Based on the last figures the data, which were derived by Manuele were selected (catalyst_content_2). These values have now to be expanded until 2050

```
last_row <- catalyst_content_2 %>% filter(year == 2023)

years <- 2024:2050
newData <- data.frame(year = rep(years, each = dim(last_row)[1]),
  standard = rep(last_row$standard, length(years)),
  A = rep(last_row$A, length(years)),
  B = rep(last_row$B, length(years)),
  C = rep(last_row$C, length(years)),
  D = rep(last_row$D, length(years)),
  E = rep(last_row$E, length(years)),
  F = rep(last_row$F, length(years)),
  drivetrain = rep(last_row$drivetrain, length(years)),
  Element = rep(last_row$Element, length(years)))
```

```

    )
calalyst_content_2 <- bind_rows(calalyst_content_2, newData)

calalyst_content_2 <- calalyst_content_2 %>% mutate(JA = A, JB = B, JC = C, JD = D, JE = E, JF = F) %>% rename(productKeyLevel1 = drivetrain)

calalyst_content_2_long <- calalyst_content_2 %>% pivot_longer(cols = -c(year, productKeyLevel1, Element), names_to = "segment", values_to = "values")
calalyst_content_2_long <- calalyst_content_2_long %>% left_join(productKeys, by = join_by(productKeyLevel1, segment))

calalyst_content_2 <- calalyst_content_2_long %>% pivot_wider(names_from = Element, values_from = values, names_prefix = "calculate")
calalyst_content_2 <- calalyst_content_2 %>% mutate(totalPPR = calculatePt + calculatePd + calculateRh + calculateCe + calculateLa)

Calculate <- Calculate %>% left_join(calalyst_content_2, by = join_by(year, productKeyLevel1, productKeyLevel2, segment))
#Calculate

```

EEC Contribuitons

```

#EEC_ICEV_content_1

# Combine to component families ()
componentFamilyEEC_ICEV <- EEC_ICEV_content_1 %>% group_by(Type) %>% summarise(total = sum(Mass/1000))
componentFamilyEEC_ICEV <- componentFamilyEEC_ICEV %>% pivot_wider(names_from = Type, values_from = total)

componentFamilyEEC_BEV <- EEC_BEV_content_1 %>% group_by(Type) %>% summarise(total = sum(Mass/1000))
componentFamilyEEC_BEV <- componentFamilyEEC_BEV %>% pivot_wider(names_from = Type, values_from = total)

componentFamilyEEC_HEV <- EEC_HEV_content_1 %>% group_by(Type) %>% summarise(total = sum(Mass/1000))
componentFamilyEEC_HEV <- componentFamilyEEC_HEV %>% pivot_wider(names_from = Type, values_from = total)

componentFamilyEEC_PHEV <- EEC_PHEV_content_1 %>% group_by(Type) %>% summarise(total = sum(Mass/1000))
componentFamilyEEC_PHEV <- componentFamilyEEC_PHEV %>% pivot_wider(names_from = Type, values_from = total)

EED_weight <- EED_weight %>% mutate(elvDiesel = ICEV) %>% rename(elvPetrol = ICEV) %>% rename(elvBEV = BEV) %>% rename(elvPHEV = PHEV) %>% rename(elvHEV = HEV)
EED_weight_long <- EED_weight %>% pivot_longer(cols = -componentFamily, names_to = "productKeyLevel1", values_to = "values")

developmentEEC <- developmentEEC %>% mutate(elvDiesel = ICEV) %>% rename(elvPetrol = ICEV) %>% rename(elvBEV = BEV) %>% rename(elvPHEV = PHEV) %>% rename(elvHEV = HEV)
developmentEEC_long <- developmentEEC %>% pivot_longer(cols = -componentFamily, names_to = "productKeyLevel1", values_to = "change")

# Define the range of years (1980 to 2030)
years <- 1980:2030

EED_long <- EED_weight_long %>% left_join(developmentEEC_long, by = c("componentFamily", "productKeyLevel1")) %>% expand_grid(year = years) %>%

```

```

mutate(value = case_when(
  # For years before 2015
  year < 2015 ~ values - (2015 - year) * change * values,
  # For years after 2015 (including 2015)
  year >= 2015 ~ values + (year - 2015) * change * values
))

EED_long <- EED_long %>% select(-c(values, change)) %>% mutate(value = ifelse(value < 0, 0, value)) %>%
  mutate(value = ifelse(productKeyLevel1 %in% c("elvBEV", "elvHEV", "elvPHEV") & year < 2000, 0, value))

## display the different components over the years

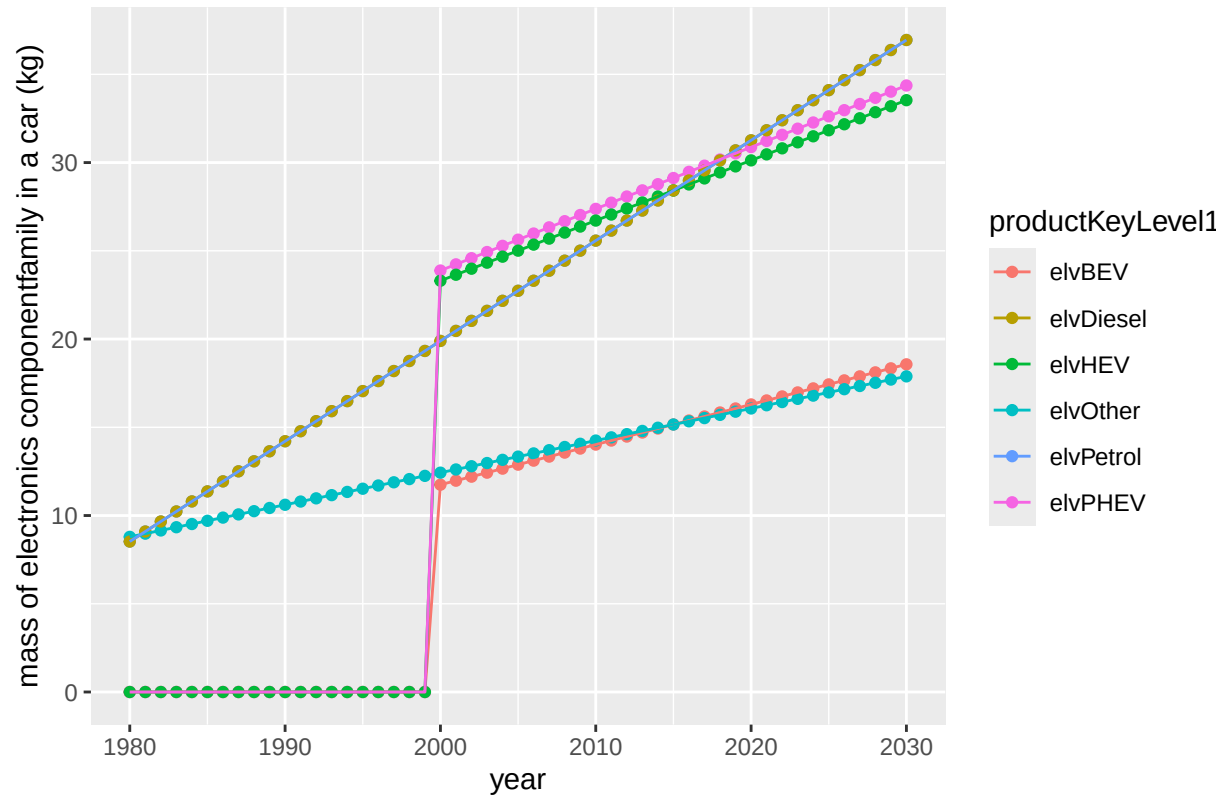
# actuators
temp <- EED_long %>% filter(componentFamily == "actuators")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = productKeyLevel1)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of acutators in a car (EVA2)") +
  ylab("mass of electronics componentfamily in a car (kg)")

plot(g)

```

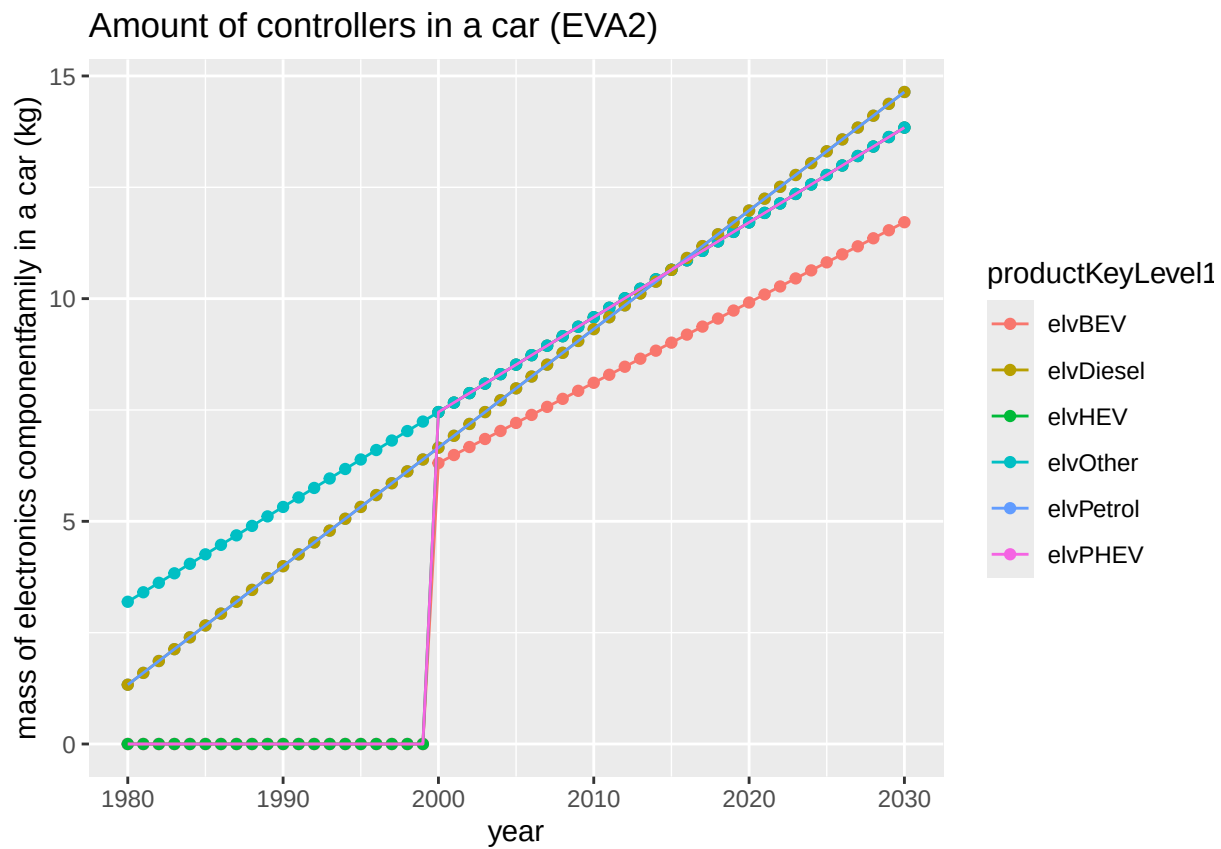
Amount of acutators in a car (EVA2)



```
# controllers
temp <- EED_long %>% filter(componentFamily == "controllers")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = productKeyLevel1)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of controllers in a car (EVA2)") +
  ylab("mass of electronics componentfamily in a car (kg)")

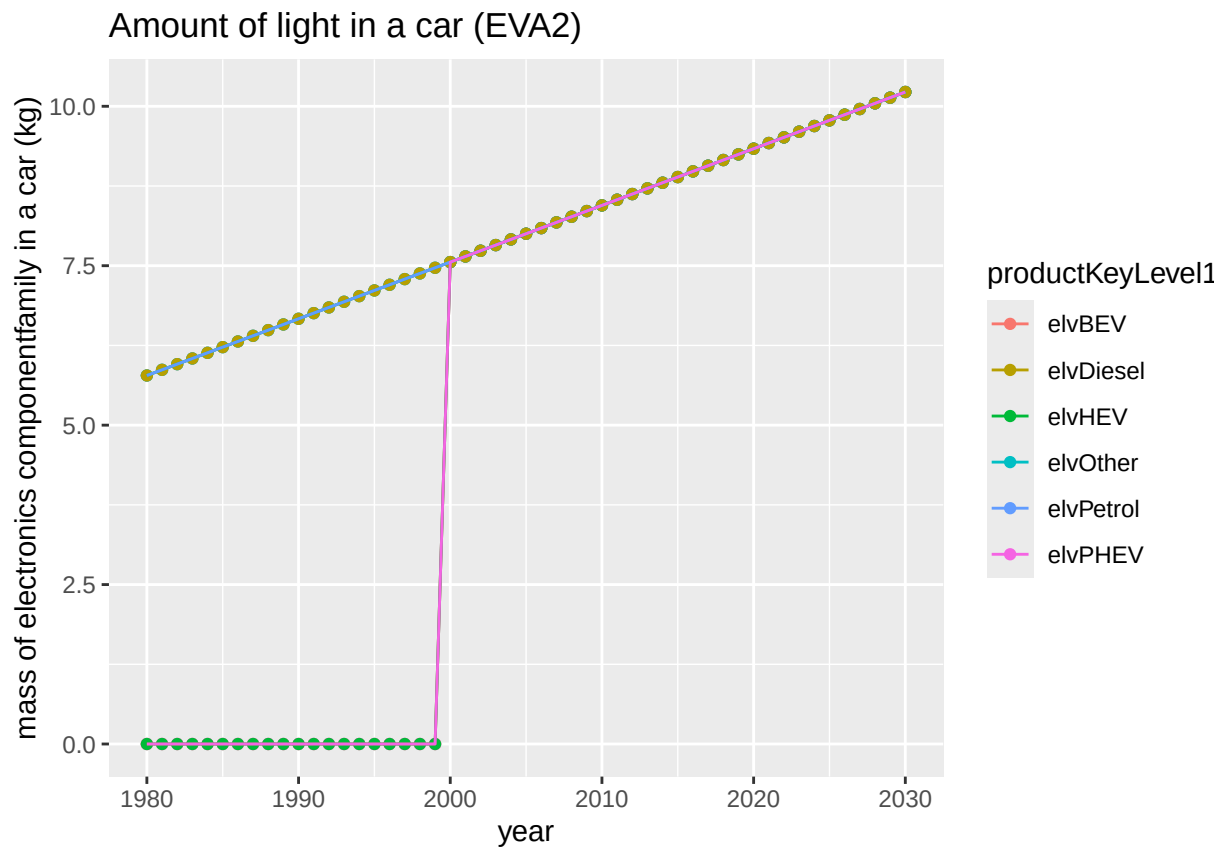
plot(g)
```



```
# light
temp <- EED_long %>% filter(componentFamily == "light")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = productKeyLevel1)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of light in a car (EVA2)") +
  ylab("mass of electronics componentfamily in a car (kg)")

plot(g)
```

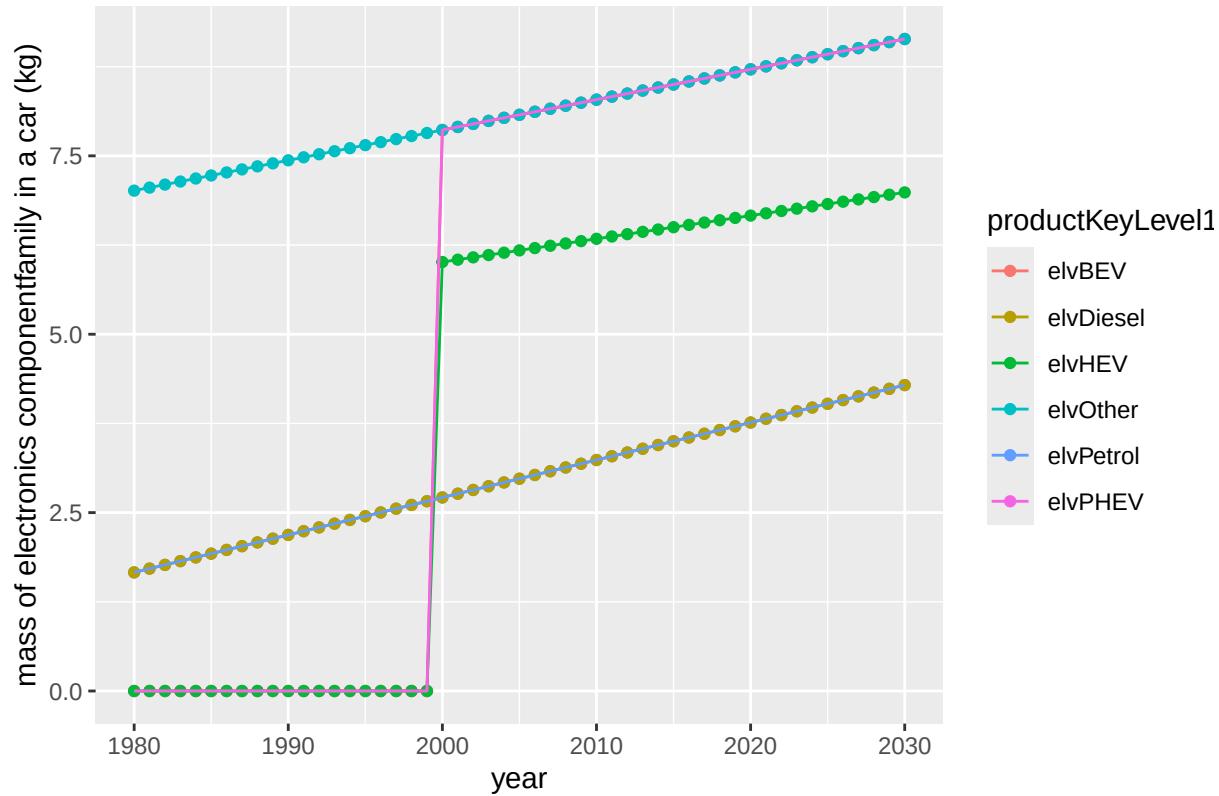


```
# cableLike
temp <- EED_long %>% filter(componentFamily == "cableLike")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = productKeyLevel1)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of cableLike in a car (EVA2)") +
  ylab("mass of electronics componentfamily in a car (kg)")

plot(g)
```

Amount of cableLike in a car (EVA2)

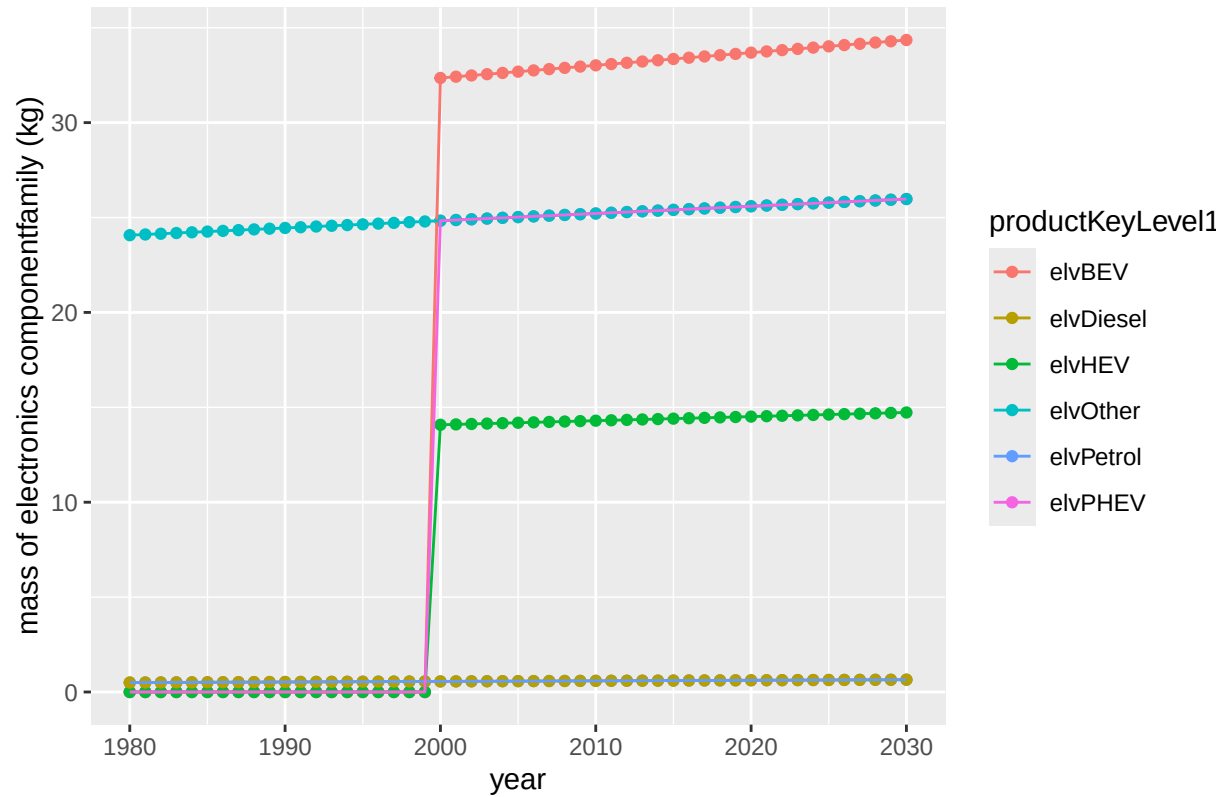


```
# power electronics
temp <- EED_long %>% filter(componentFamily == "powerElectronics")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = productKeyLevel1)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of power electronic in a car (EVA2)") +
  ylab("mass of electronics componentfamily (kg)")

plot(g)
```

Amount of power electronic in a car (EVA2)

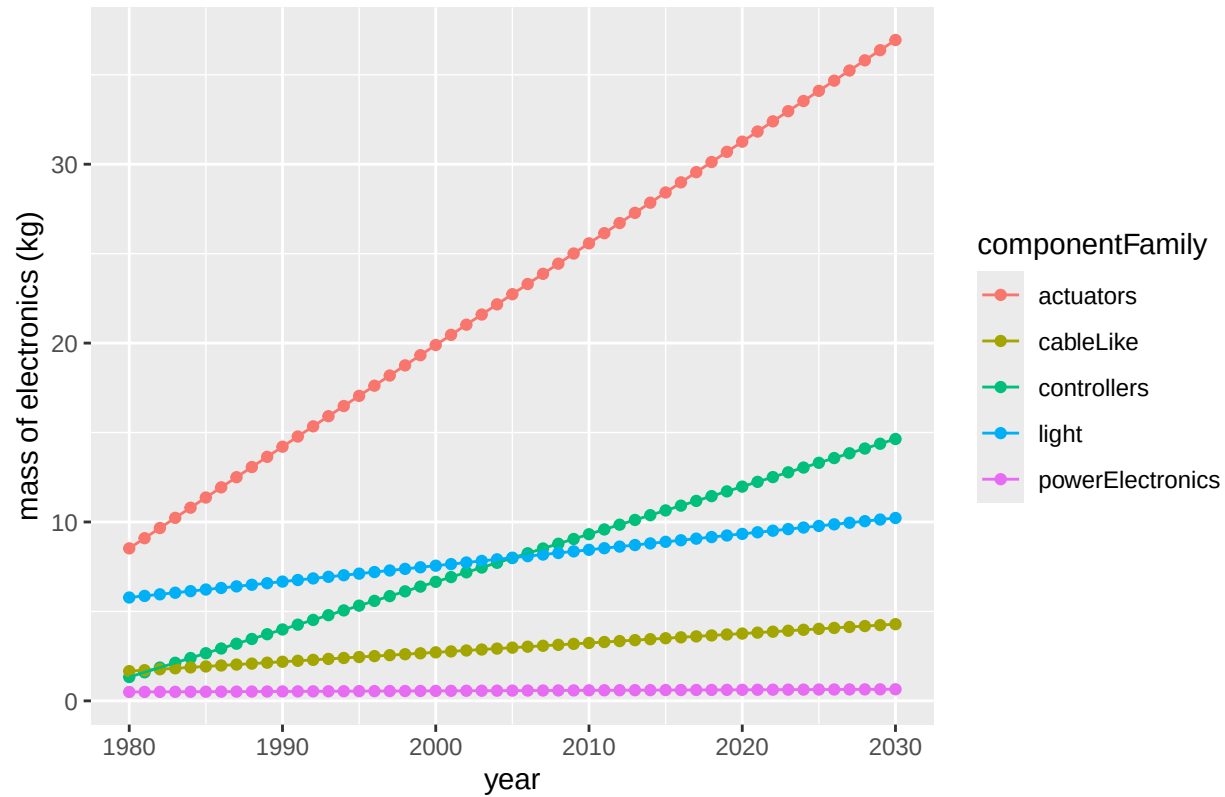


```
# elvPetrol, elvDiesel & elvOther
temp <- EED_long %>% filter(productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = componentFamily)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of electronics in a elvPetrol car (EVA2)") +
  ylab("mass of electronics (kg)")

plot(g)
```

Amount of electronics in a elvPetrol car (EVA2)

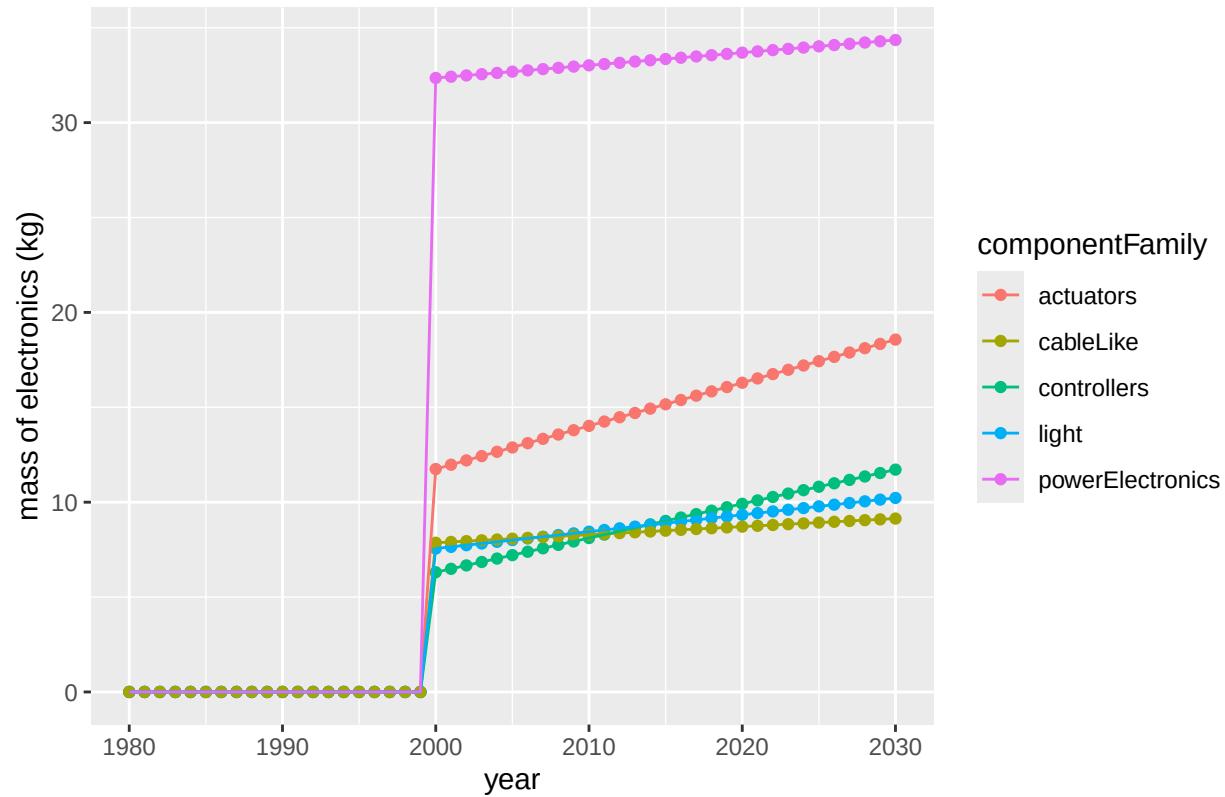


```
# elvBEV
temp <- EED_long %>% filter(productKeyLevel1 == "elvBEV")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = componentFamily)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of electronics in a elvBEV car (EVA2)") +
  ylab("mass of electronics (kg)")

plot(g)
```

Amount of electronics in a elvBEV car (EVA2)

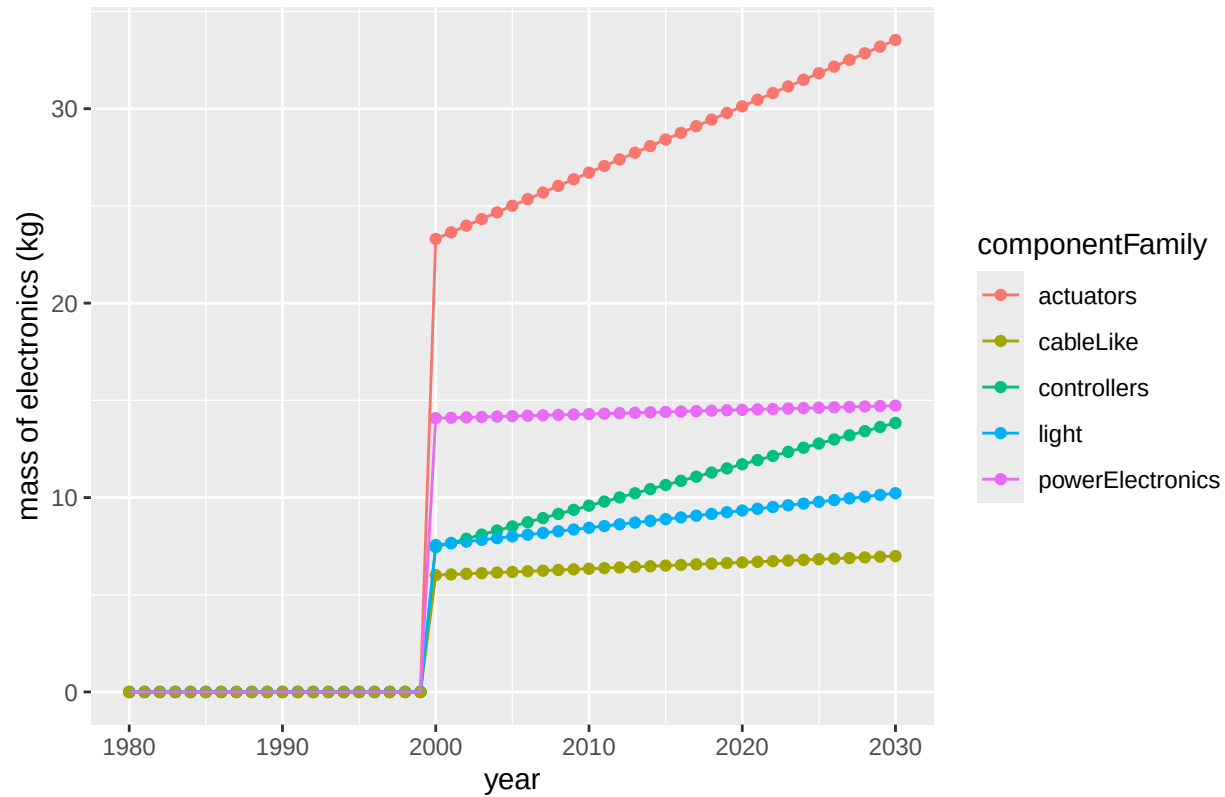


```
# elvHEV
temp <- EED_long %>% filter(productKeyLevel1 == "elvHEV")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = componentFamily)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of electronics in a elvHEV car (EVA2)") +
  ylab("mass of electronics (kg)")

plot(g)
```

Amount of electronics in a elvHEV car (EVA2)

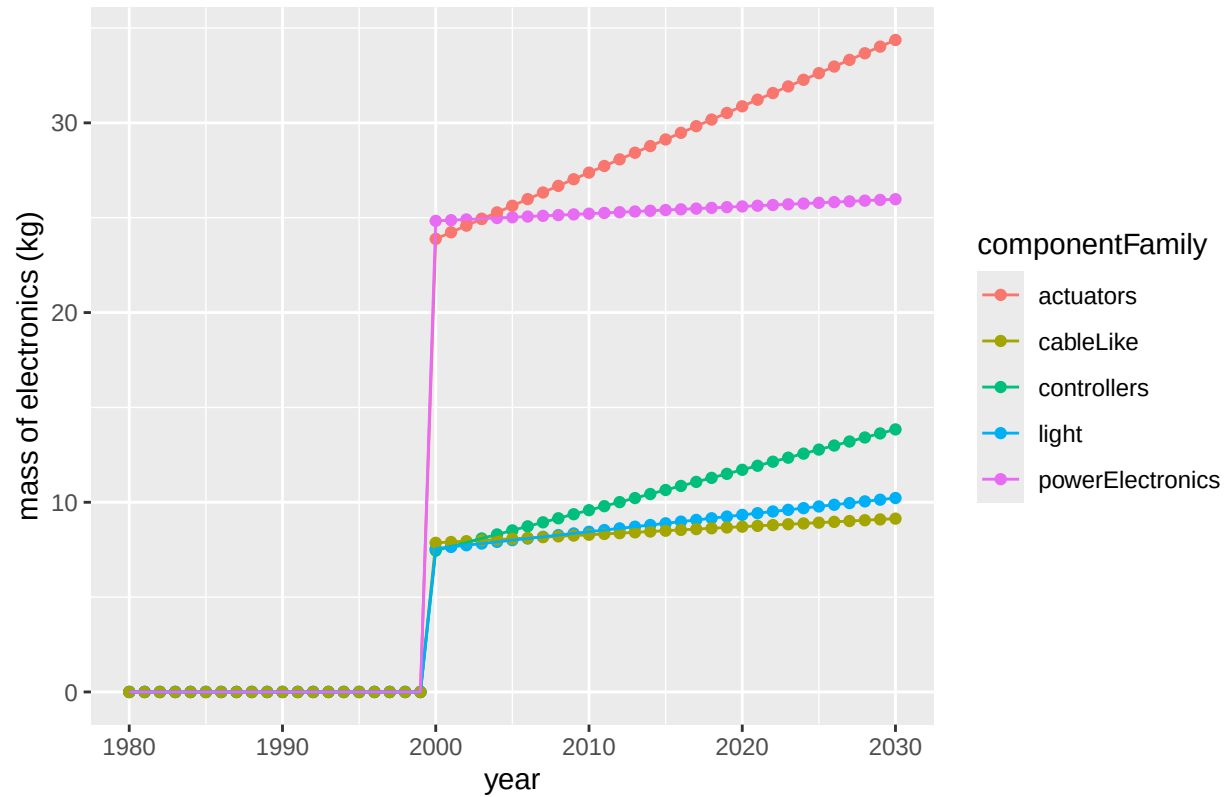


```
# elvPHEV
temp <- EED_long %>% filter(productKeyLevel1 == "elvPHEV")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = componentFamily)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of electronics in a elvPHEV car (EVA2)") +
  ylab("mass of electronics (kg)")

plot(g)
```

Amount of electronics in a elvPHEV car (EVA2)

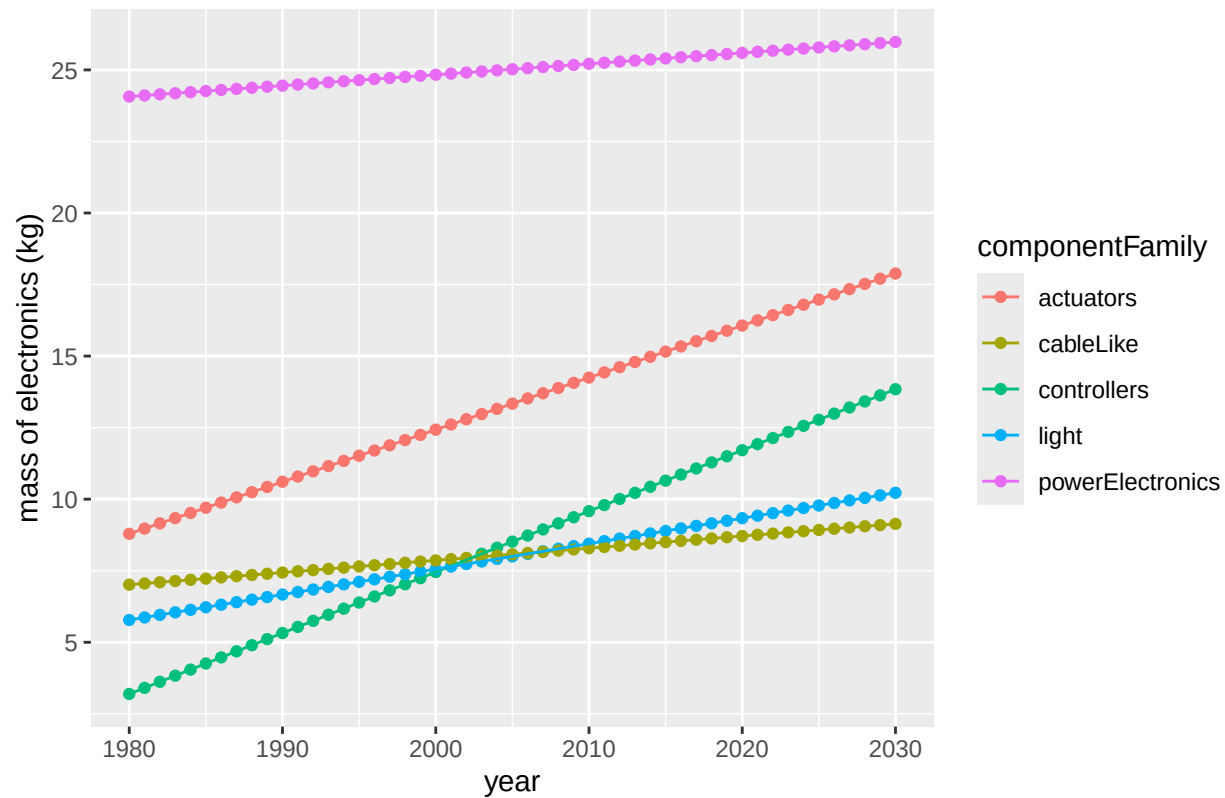


```
# elvOther
temp <- EED_long %>% filter(productKeyLevel1 == "elvOther")

g <- ggplot(data = temp, mapping = aes(x = year, y = value, colour = componentFamily)) +
  geom_point() +
  geom_line() +
  ggtitle("Amount of electronics in a elvOther car (EVA2)") +
  ylab("mass of electronics (kg)")

plot(g)
```

Amount of electronics in a elvOther car (EVA2)



```
EED <- EED_long %>% pivot_wider(names_from = componentFamily, values_from = value)

EED <- crossing(EED, derivedEECCComposition) %>%
  mutate(
    powerElectronics = powerElectronics * fraction,
    actuators = actuators * fraction,
    controllers = controllers * fraction,
    light = light * fraction,
    cableLike = cableLike * fraction
  ) %>%
  select(-fraction) %>%
  mutate(EED_total = powerElectronics + actuators + controllers + light + cableLike)

EED <- EED %>% left_join(productKeys, by = c("productKeyLevel1", "segment"))
```

```
#EED
```

Add all the data together and get an overview

```
# Update Calculate

Calculate <- Calculate %>% left_join(EED, by = join_by(year, productKeyLevel1, productKeyLevel2, segment))
Calculate <- Calculate %>% mutate(totalMetals = totalAlloys + EED_total)
#Calculate

## Add the rest material which is given by the data.frame rest - interpolate, so that there are values for each year

# Reshape to long format
rest_long <- rest %>% pivot_longer(cols = starts_with("elv"), names_to = "value_column", values_to = "value")

# Interpolate missing years for each label and value column
rest_interpolated <- rest_long %>%
  group_by(segment, value_column) %>% # Treat each label and value column independently
  complete(year = full_seq(rest$year, 1)) %>% # Fill in all missing years
  mutate(
    value = approx(x = year[!is.na(value)],
                  y = value[!is.na(value)],
                  xout = year)$y
  ) %>%
  ungroup()

# get the complete dataset
restcomplet <- rest_interpolated %>% pivot_wider(names_from = value_column, values_from = value)

restcomplet_long <- restcomplet %>% pivot_longer(cols = starts_with("elv"), names_to = "productKeyLevel1", values_to = "restFactor")

# Calculate the total amount of different materials
Calculate <- Calculate %>% left_join(restcomplet_long, by = c("year", "productKeyLevel1", "segment")) %>% mutate(totalOtherMaterialas = restFactor * totalMetals) %>%
  select(-restFactor) %>% mutate(total = totalMetals + totalOtherMaterialas)
#Calculate

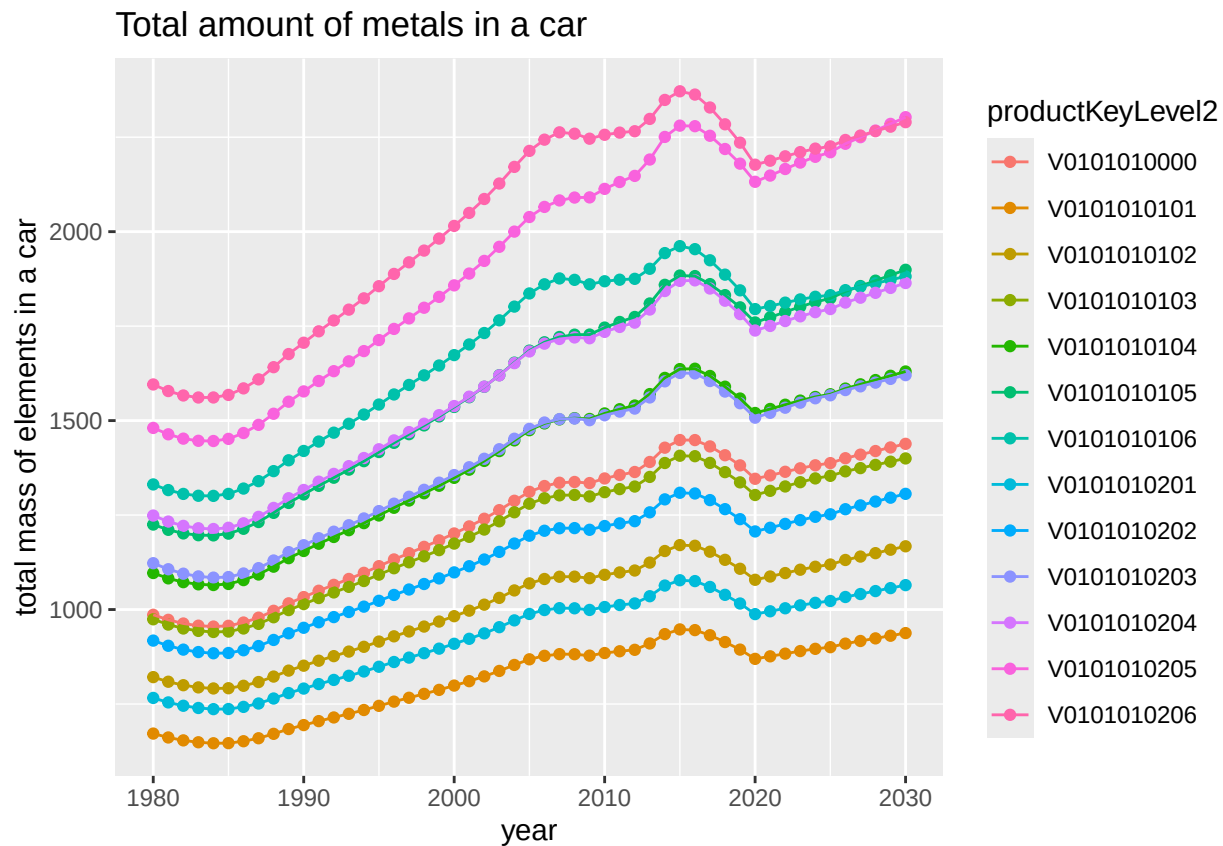
Calculate_long <- Calculate %>% pivot_longer(cols = !c(year, productKeyLevel1, productKeyLevel2, segment), names_to = "Materials", values_to = "values")

# by Materials

temp <- Calculate_long %>% filter(Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = productKeyLevel2)) +
  geom_point() +
  geom_line() +
```

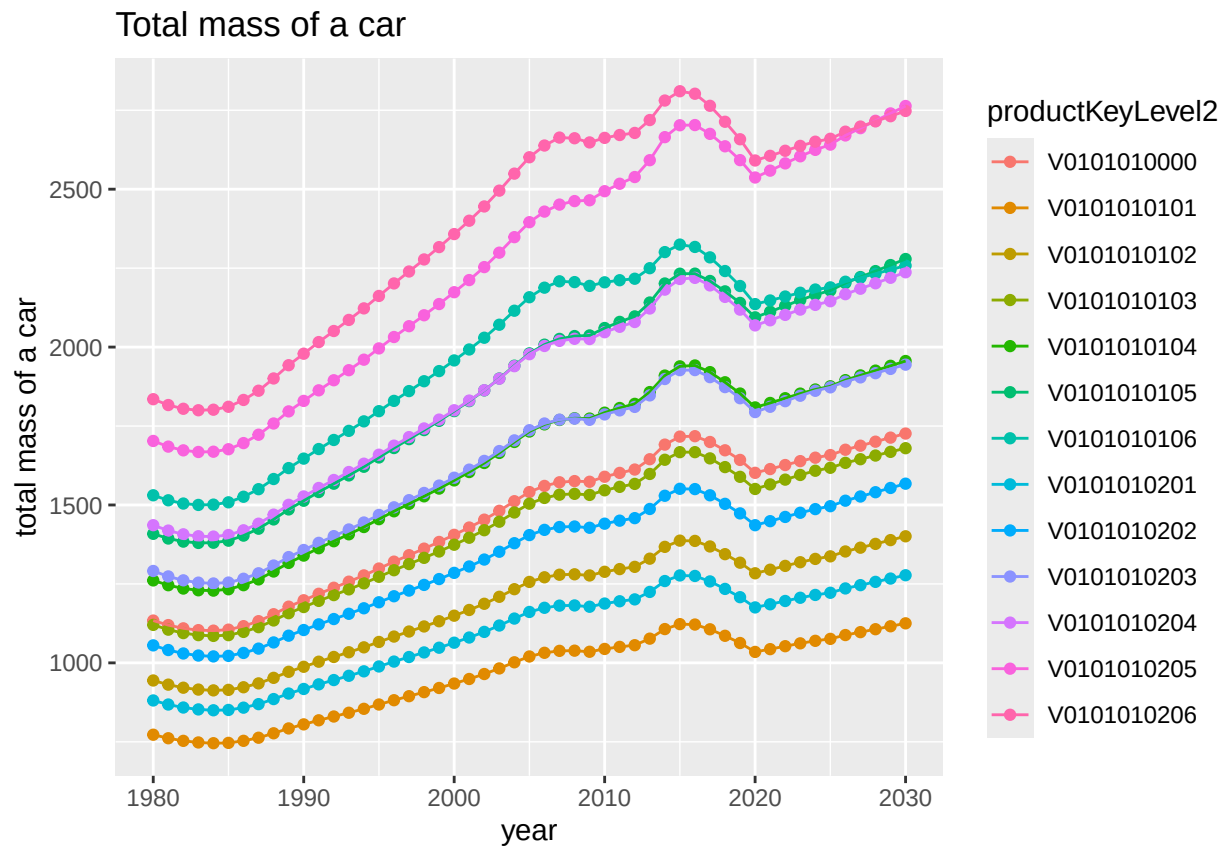
```
ggtitle("Total amount of metals in a car") +
xlab("year") +
ylab("total mass of elements in a car")
plot(g)
```



```
temp <- Calculate_long %>% filter(Materials == "total" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = productKeyLevel2)) +
  geom_point() +
  geom_line() +
  ggtitle("Total mass of a car") +
  xlab("year") +
  ylab("total mass of a car")
```

```
plot(g)
```



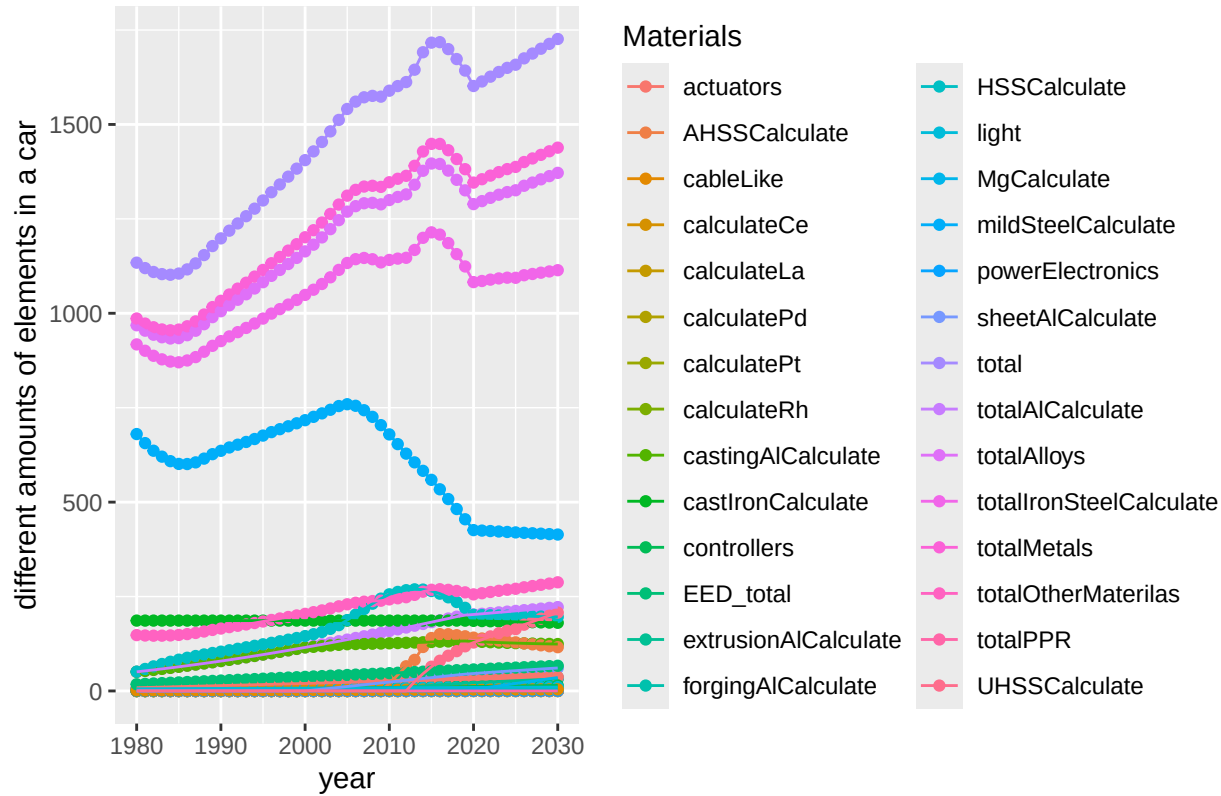
```
# By productKeyLevel2

temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010000" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010000)") +
  xlab("year") +
  ylab("different amounts of elements in a car")

plot(g)
```

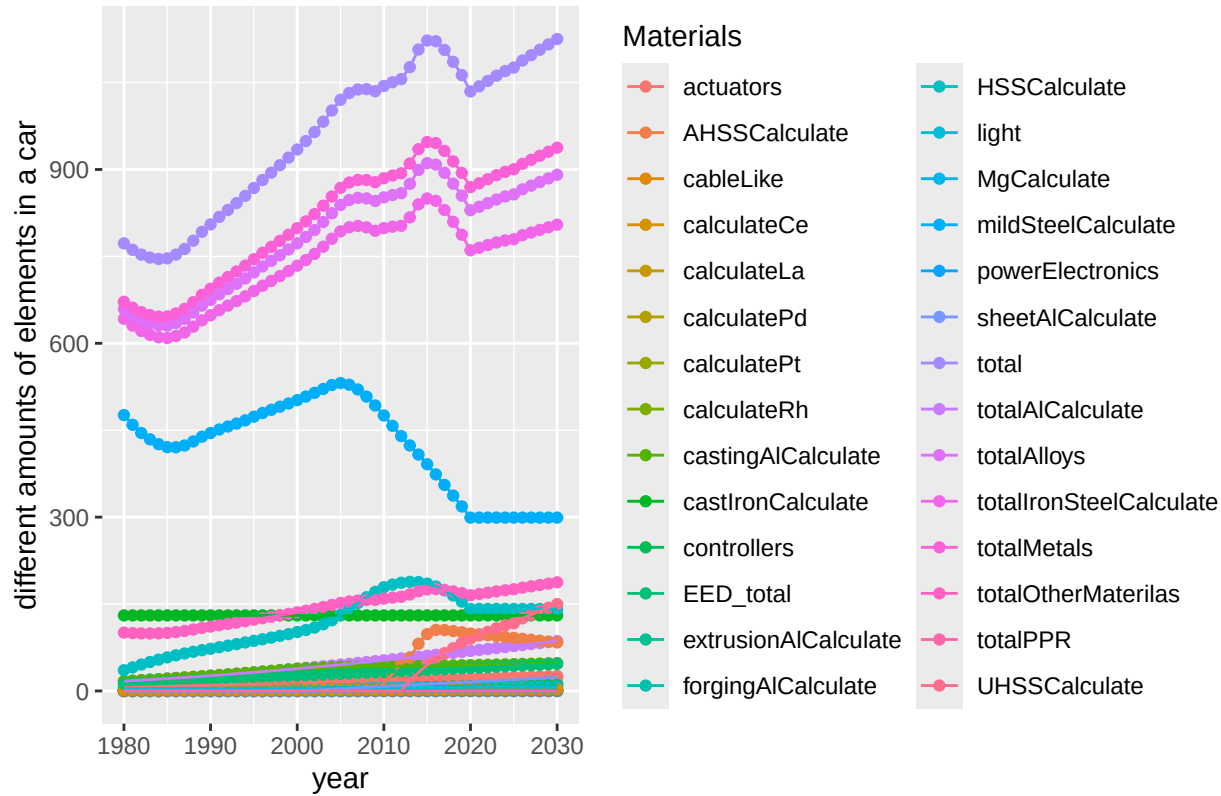
Total amount of metals in a car (V0101010000)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010101" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010101)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

Total amount of metals in a car (V0101010101)

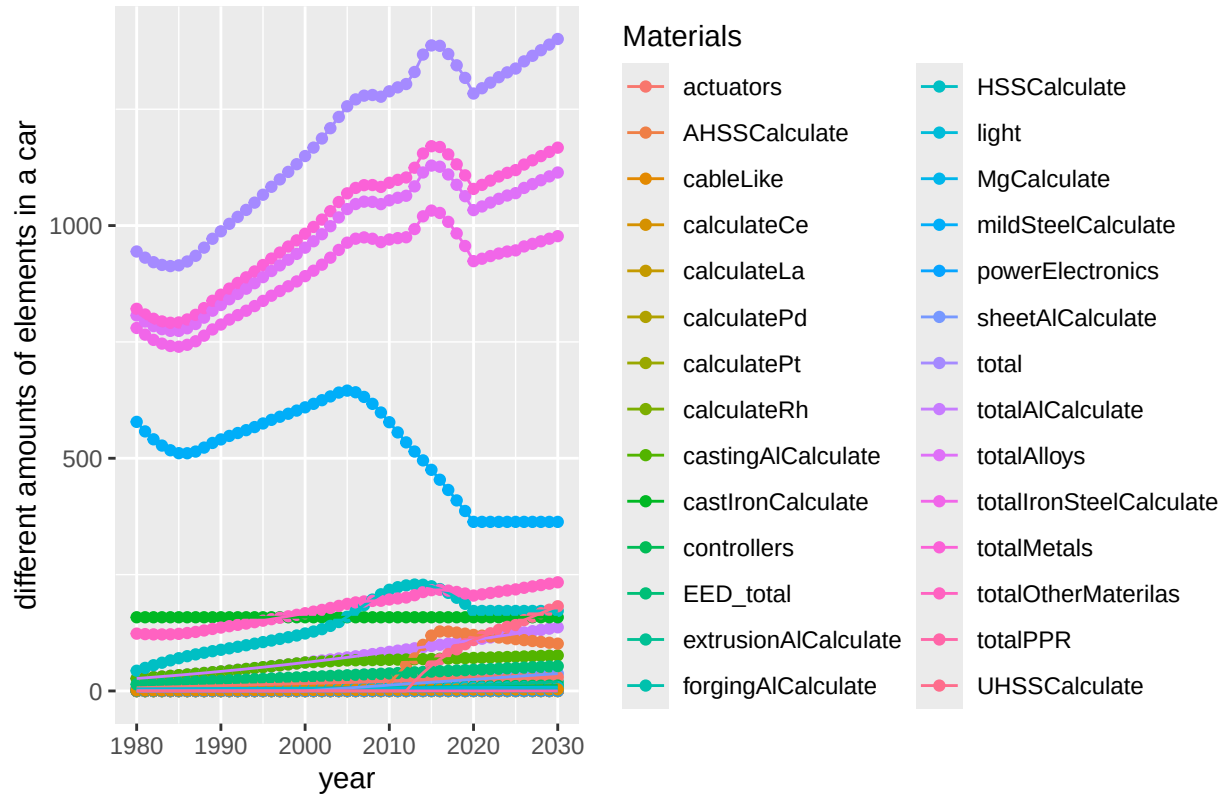


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010102" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010102)") +
  xlab("year") +
  ylab("different amounts of elements in a car")

plot(g)
```

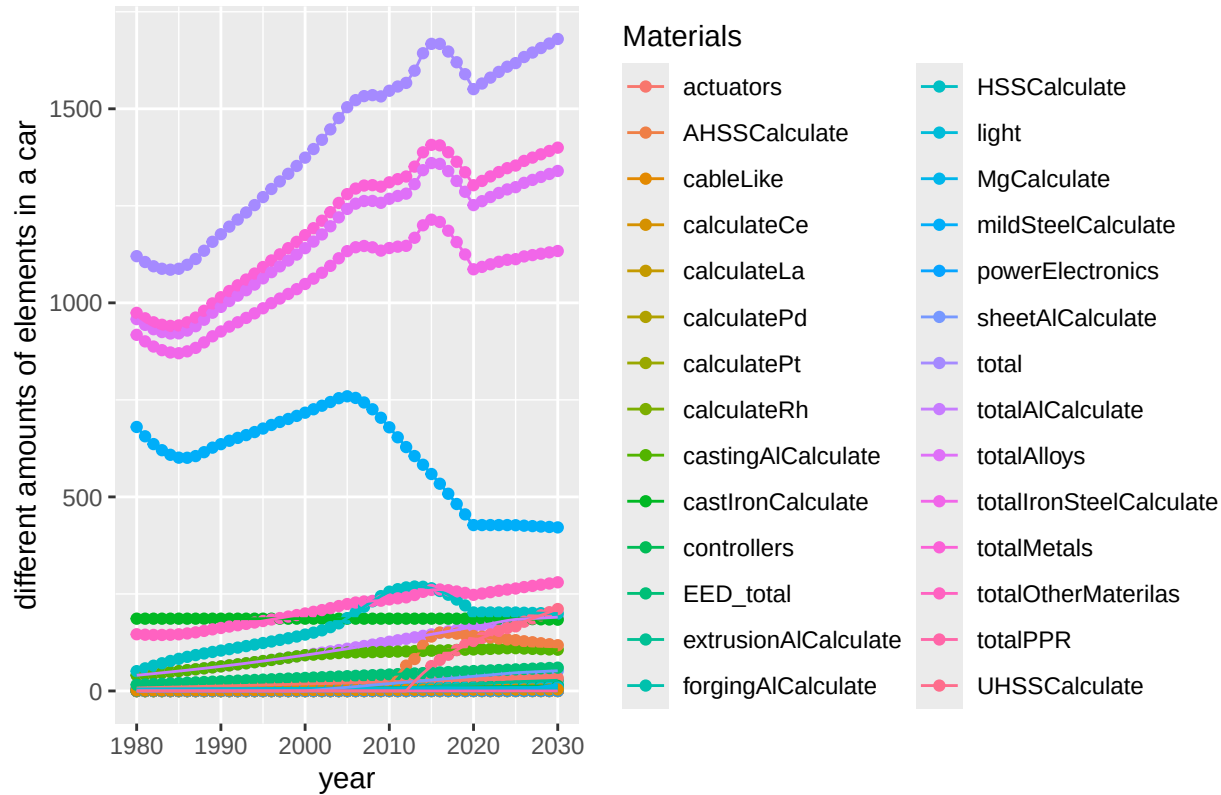
Total amount of metals in a car (V0101010102)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010103" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010103)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

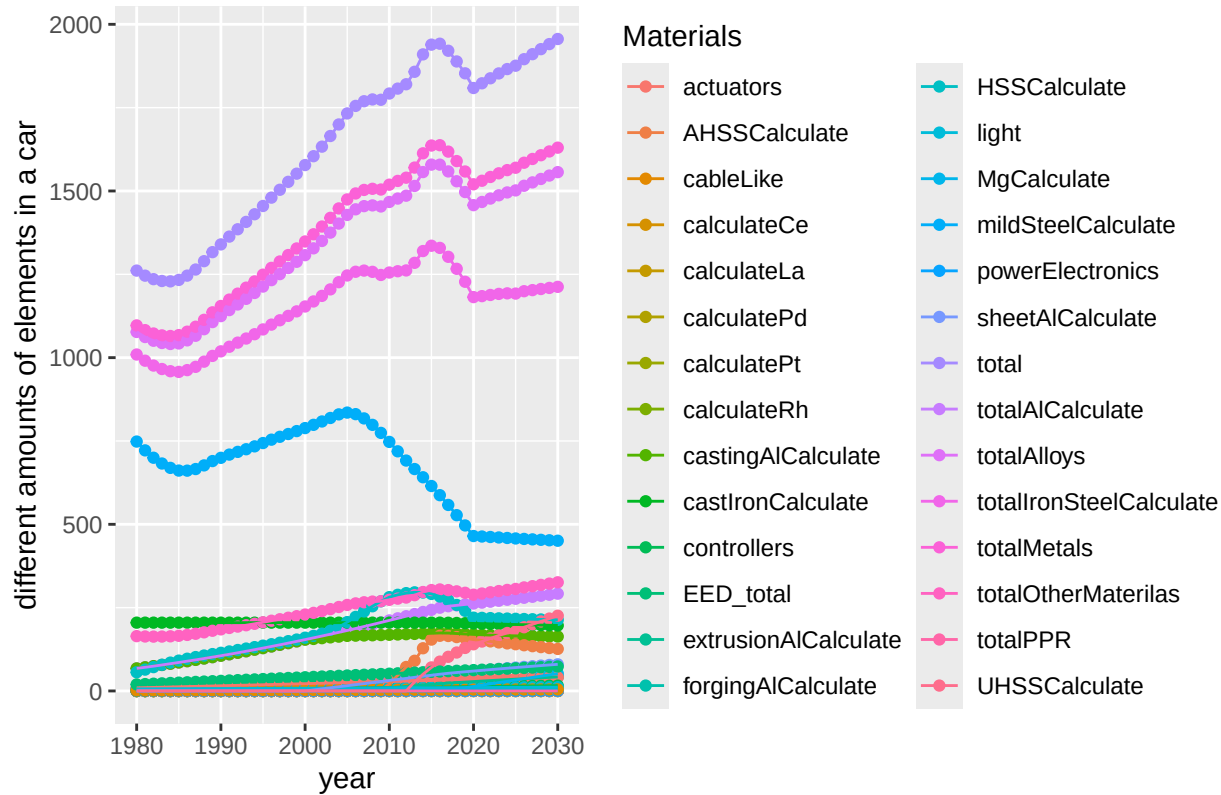
Total amount of metals in a car (V0101010103)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010104" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010104)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

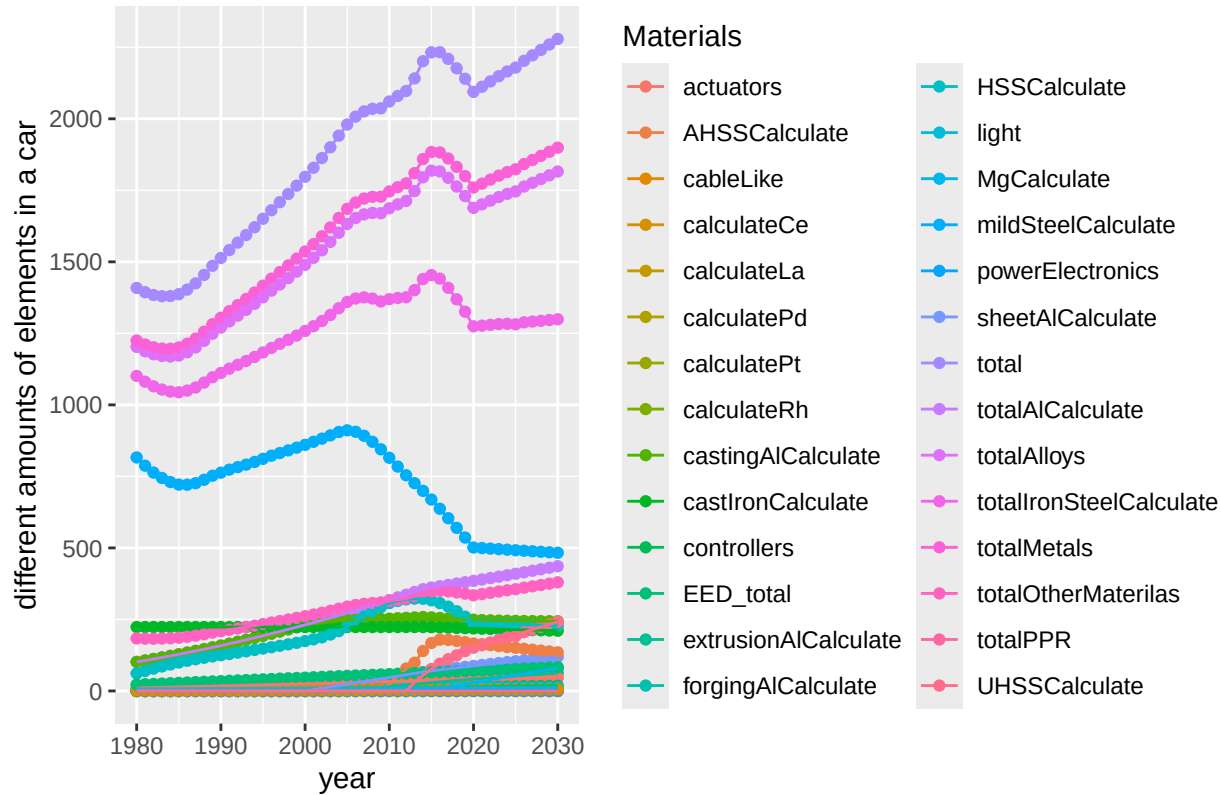
Total amount of metals in a car (V0101010104)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010105" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010105)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

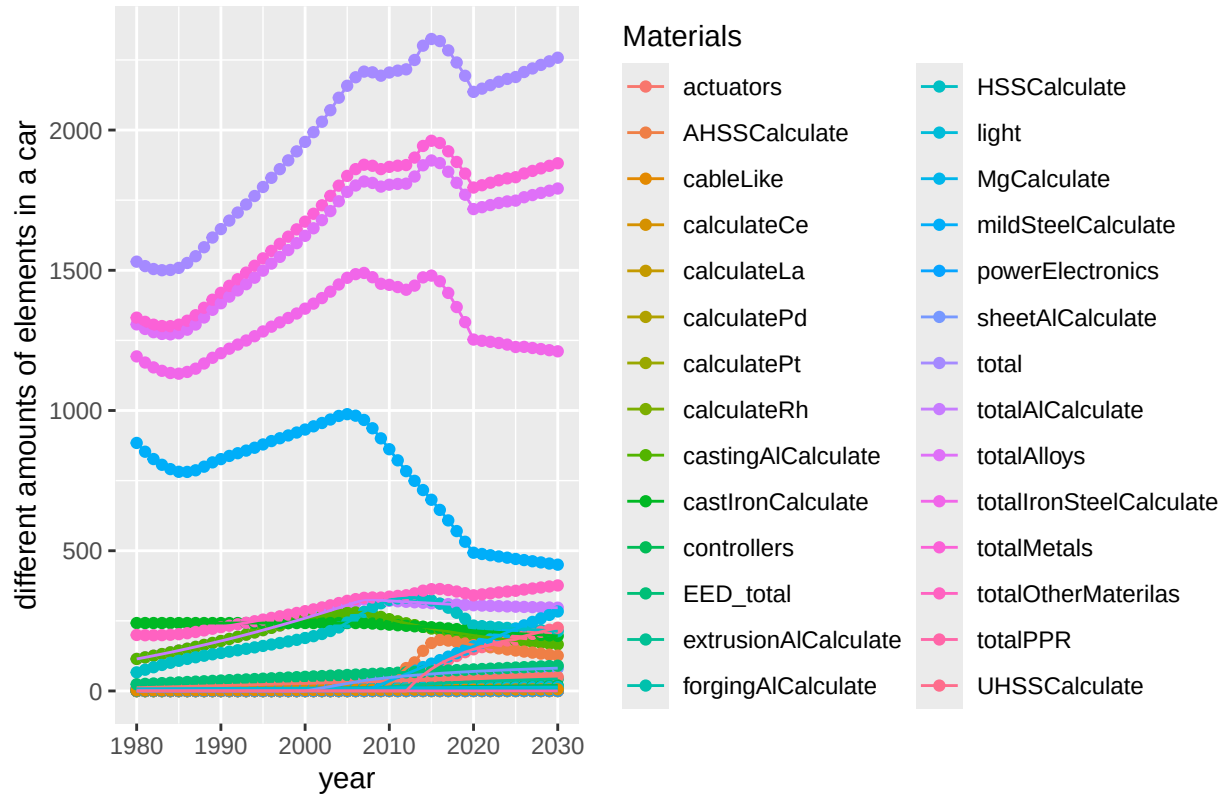
Total amount of metals in a car (V0101010105)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010106" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010106)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

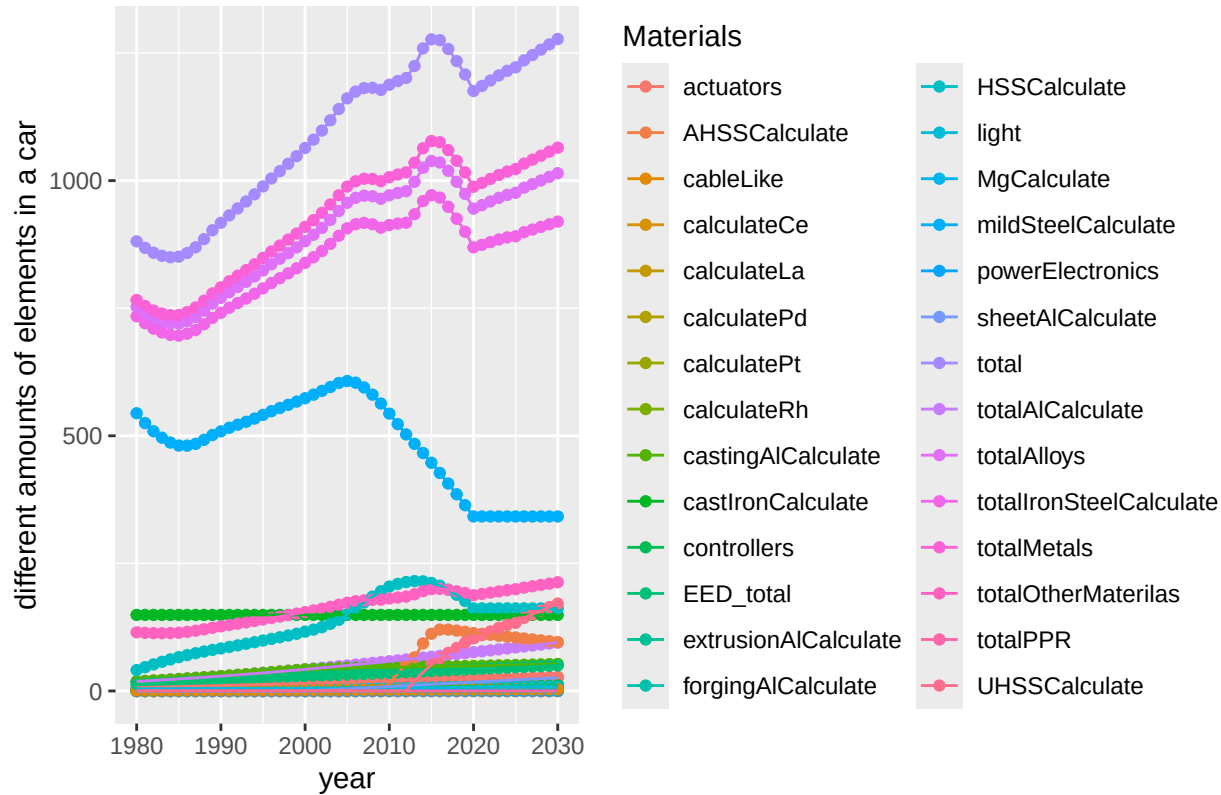
Total amount of metals in a car (V0101010106)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010201" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010201)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

Total amount of metals in a car (V0101010201)

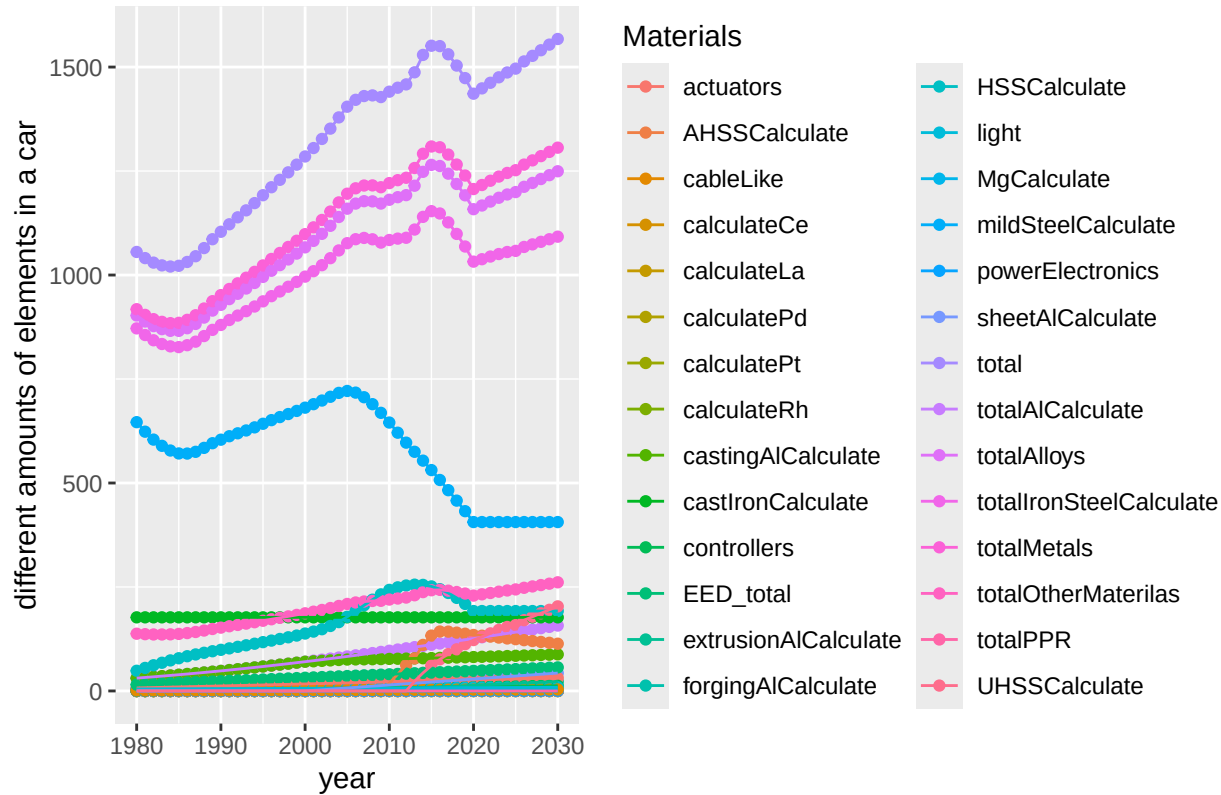


```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010202" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010202)") +
  xlab("year") +
  ylab("different amounts of elements in a car")

plot(g)
```

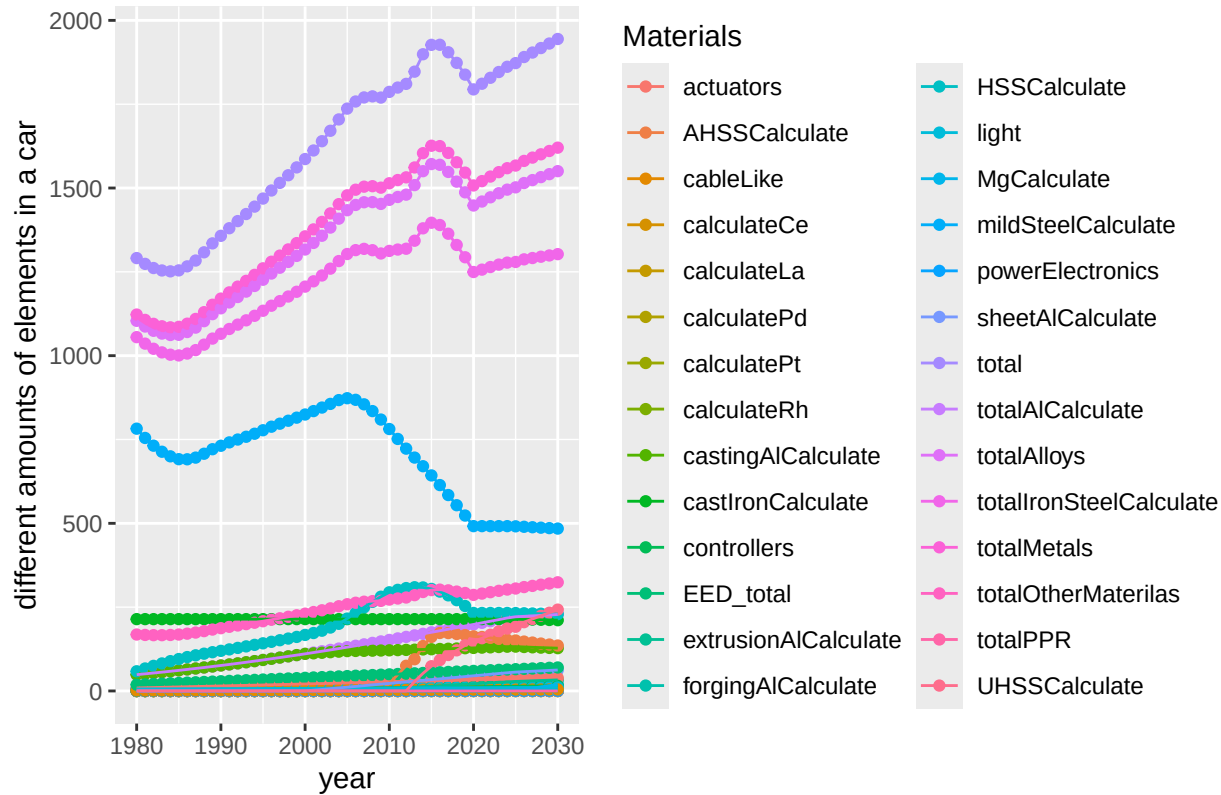
Total amount of metals in a car (V0101010202)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010203" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010203)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

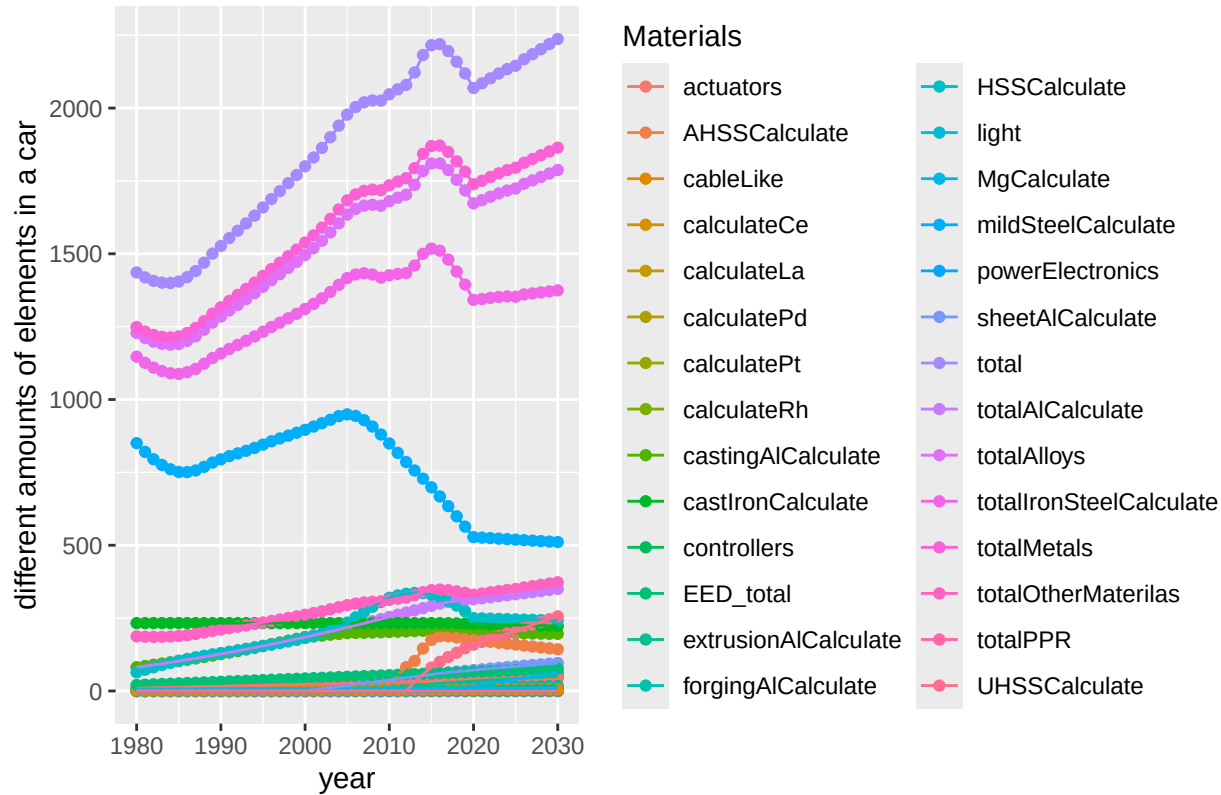
Total amount of metals in a car (V0101010203)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010204" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010204)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

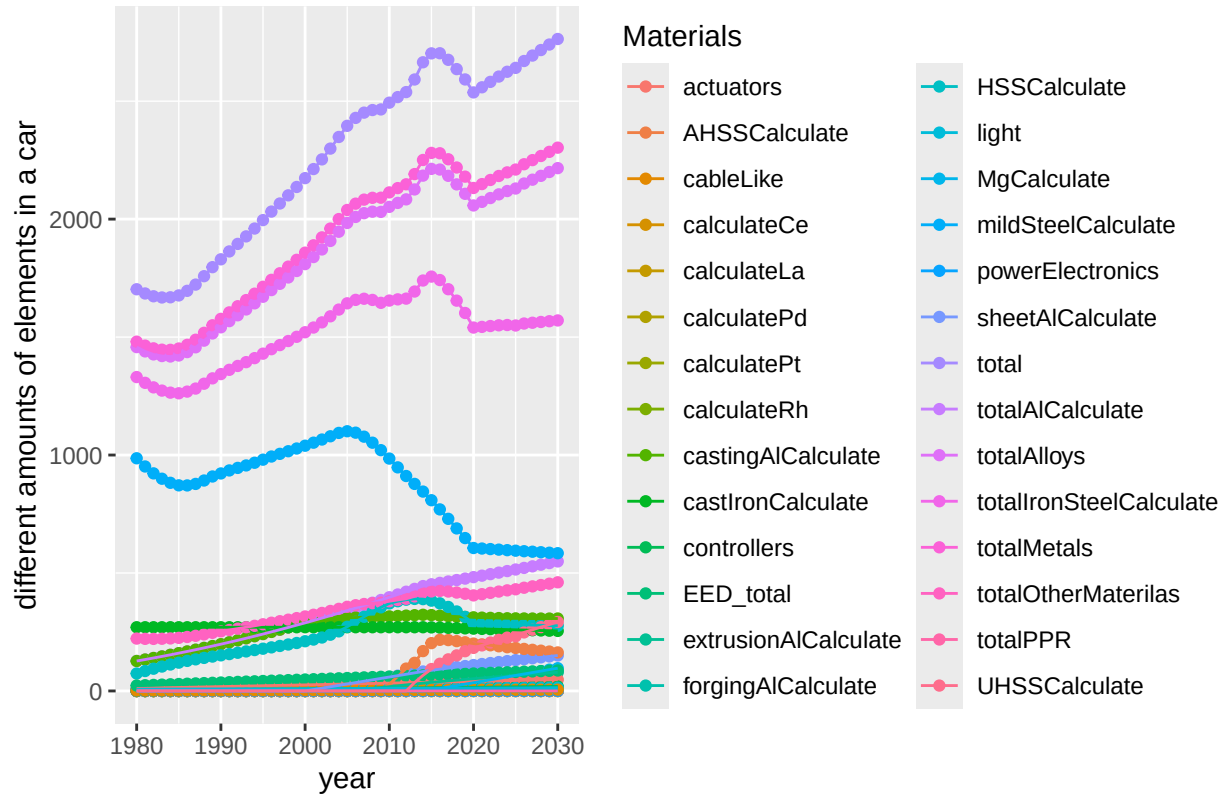
Total amount of metals in a car (V0101010204)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010205" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010205)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

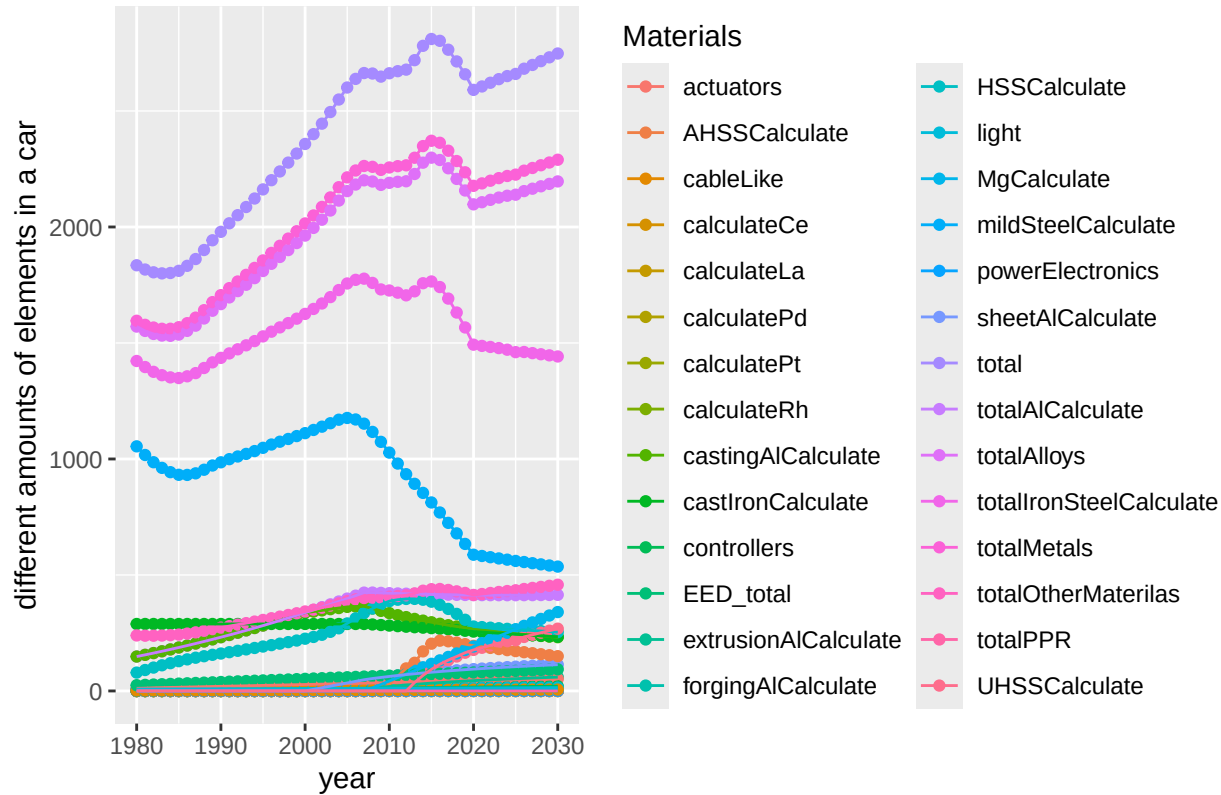
Total amount of metals in a car (V0101010205)



```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010206" & productKeyLevel1 == "elvPetrol")

g <- ggplot(data = temp, mapping = aes(x = year, y = values, color = Materials)) +
  geom_point() +
  geom_line() +
  ggtitle("Total amount of metals in a car (V0101010206)") +
  xlab("year") +
  ylab("different amounts of elements in a car")
plot(g)
```

Total amount of metals in a car (V0101010206)



Do a comparison with the real weight of the different segment cars

```
temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010000" & Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

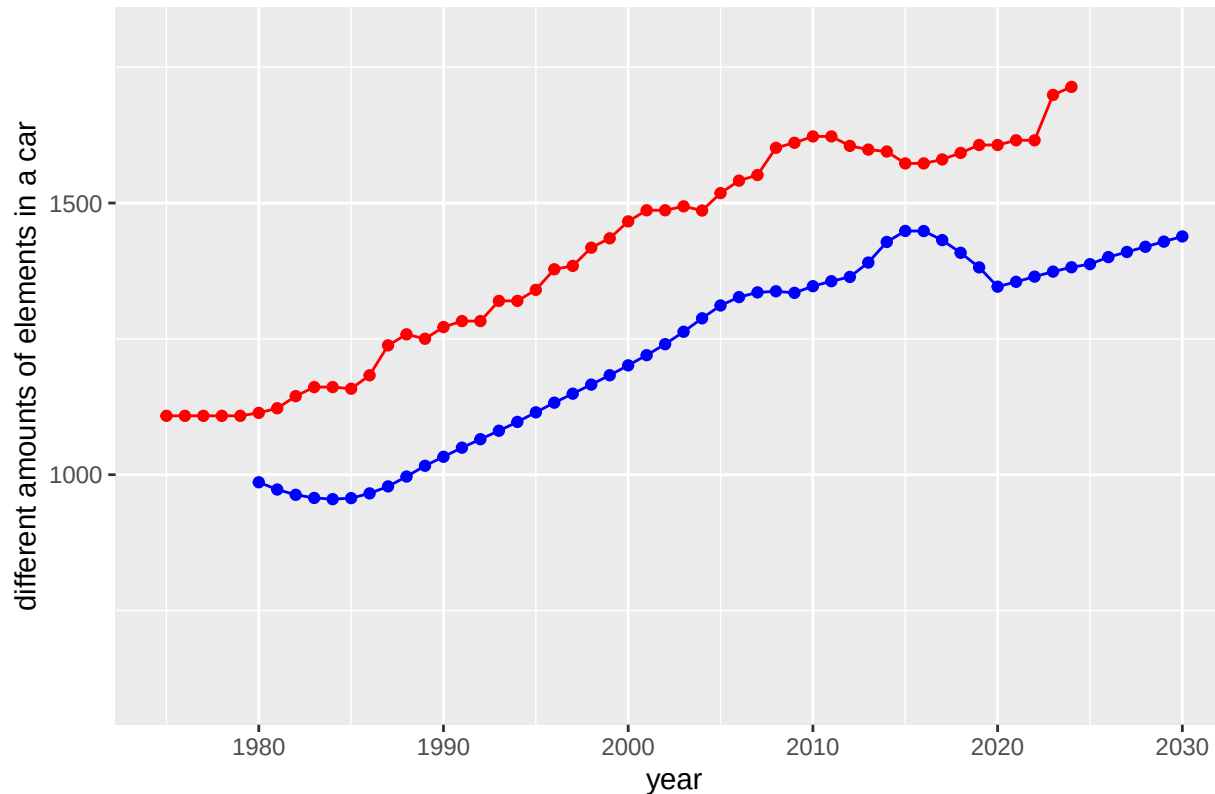
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_line(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_point(data = carWeight, mapping = aes(x = Year, y = StandardMass), color = "red") +
  geom_line(data = carWeight, mapping = aes(x = Year, y = StandardMass), color = "red") +
  ggtitle("Total amount of metals in a car (V0101010000)") +
  scale_y_continuous(limits = c(600, 1800)) +
```

```

xlab("year") +
ylab("different amounts of elements in a car")
plot(g)

```

Total amount of metals in a car (V010101000)



```

temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010102" & Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

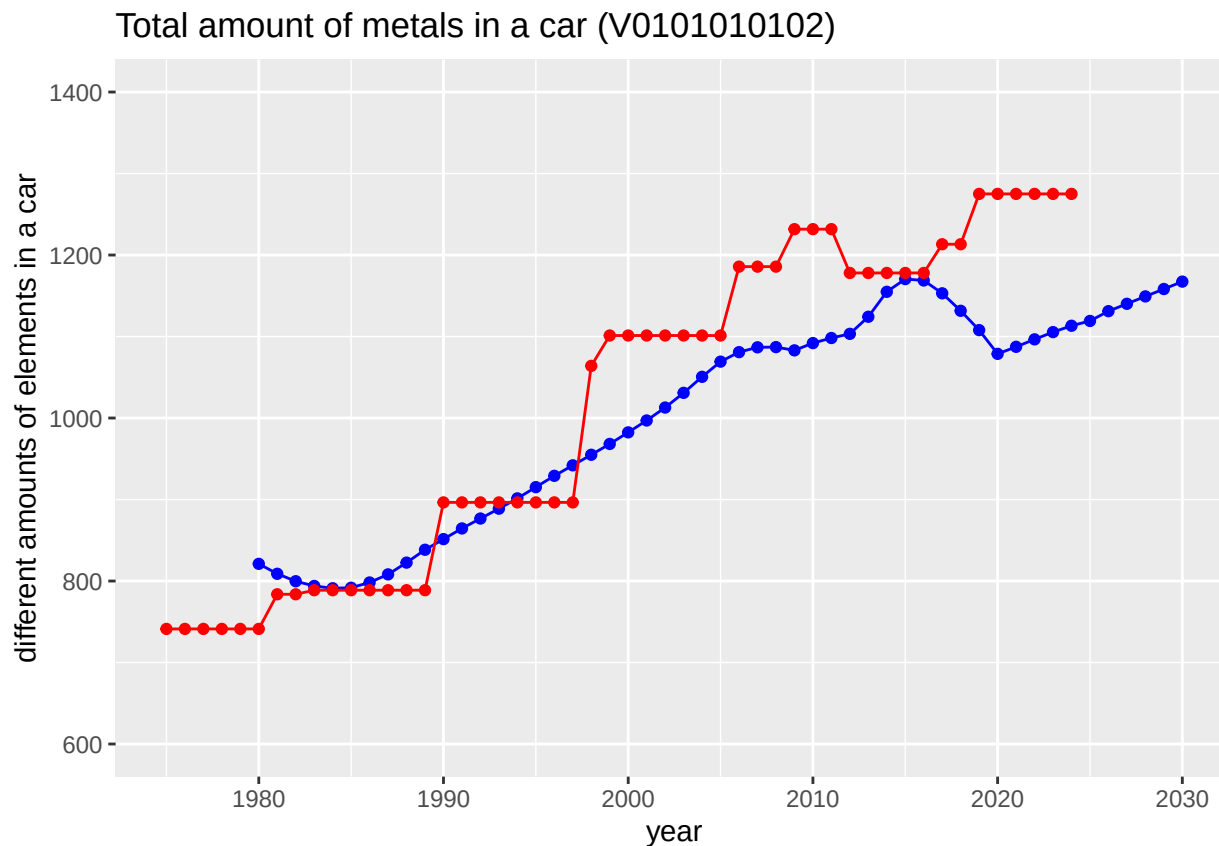
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_line(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_point(data = carWeight, mapping = aes(x = Year, y = SegmentBMass), color = "red") +
  geom_line(data = carWeight, mapping = aes(x = Year, y = SegmentBMass), color = "red") +
  ggtitle("Total amount of metals in a car (V0101010102)") +
  scale_y_continuous(limits = c(600, 1400)) +

```

```

xlab("year") +
ylab("different amounts of elements in a car")
plot(g)

```



```

temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010103" & Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

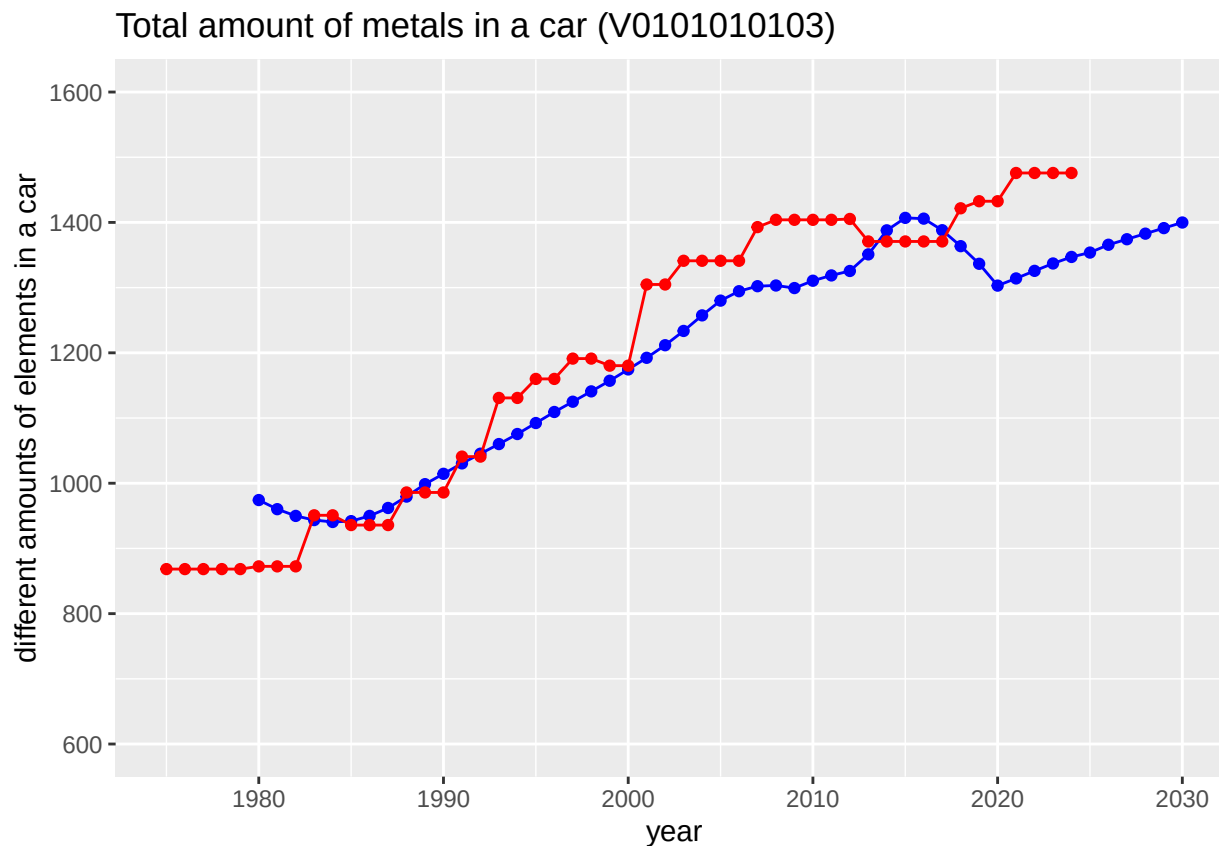
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_line(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_point(data = carWeight, mapping = aes(x = Year, y = SegmentCMass), color = "red") +
  geom_line(data = carWeight, mapping = aes(x = Year, y = SegmentCMass), color = "red") +
  ggtitle("Total amount of metals in a car (V0101010103)") +
  scale_y_continuous(limits = c(600, 1600)) +

```

```

xlab("year") +
ylab("different amounts of elements in a car")
plot(g)

```



```

temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010104" & Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

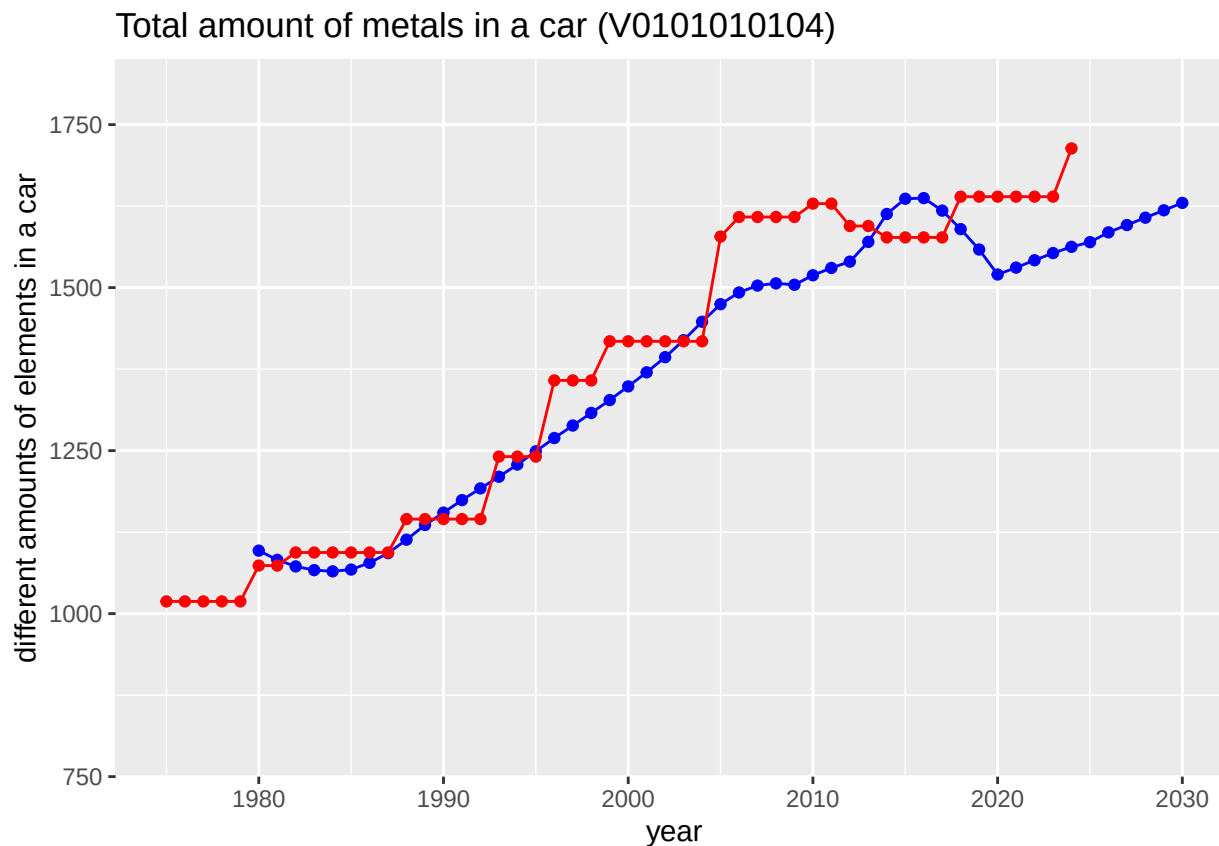
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_line(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_point(data = carWeight, mapping = aes(x = Year, y = SegmentDMass), color = "red") +
  geom_line(data = carWeight, mapping = aes(x = Year, y = SegmentDMass), color = "red") +
  ggtitle("Total amount of metals in a car (V0101010104)") +
  scale_y_continuous(limits = c(800, 1800)) +

```

```

xlab("year") +
ylab("different amounts of elements in a car")
plot(g)

```



```

temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010105" & Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

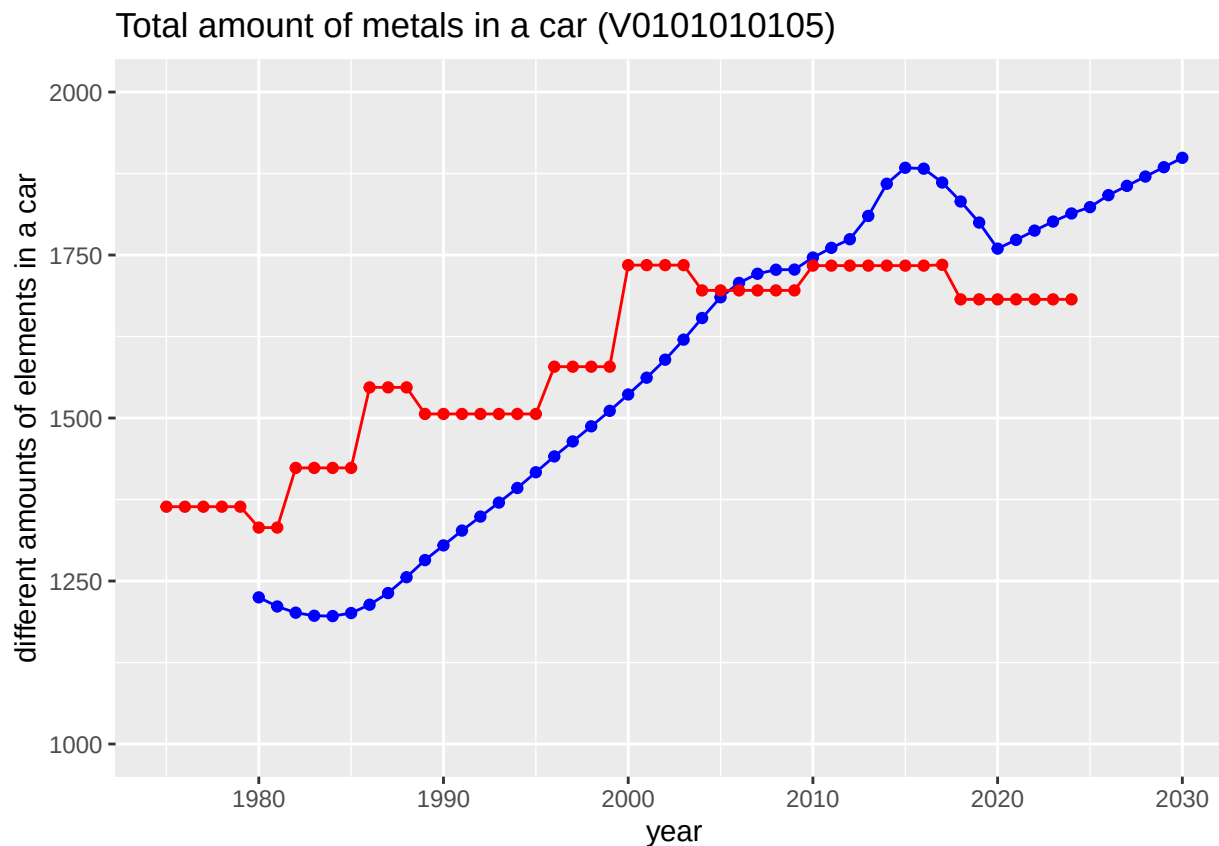
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_line(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_point(data = carWeight, mapping = aes(x = Year, y = SegmentEMass), color = "red") +
  geom_line(data = carWeight, mapping = aes(x = Year, y = SegmentEMass), color = "red") +
  ggtitle("Total amount of metals in a car (V0101010105)") +
  scale_y_continuous(limits = c(1000, 2000)) +

```

```

xlab("year") +
ylab("different amounts of elements in a car")
plot(g)

```



```

temp <- Calculate_long %>% filter(productKeyLevel2 == "V0101010106" & Materials == "totalMetals" & productKeyLevel1 == "elvPetrol")

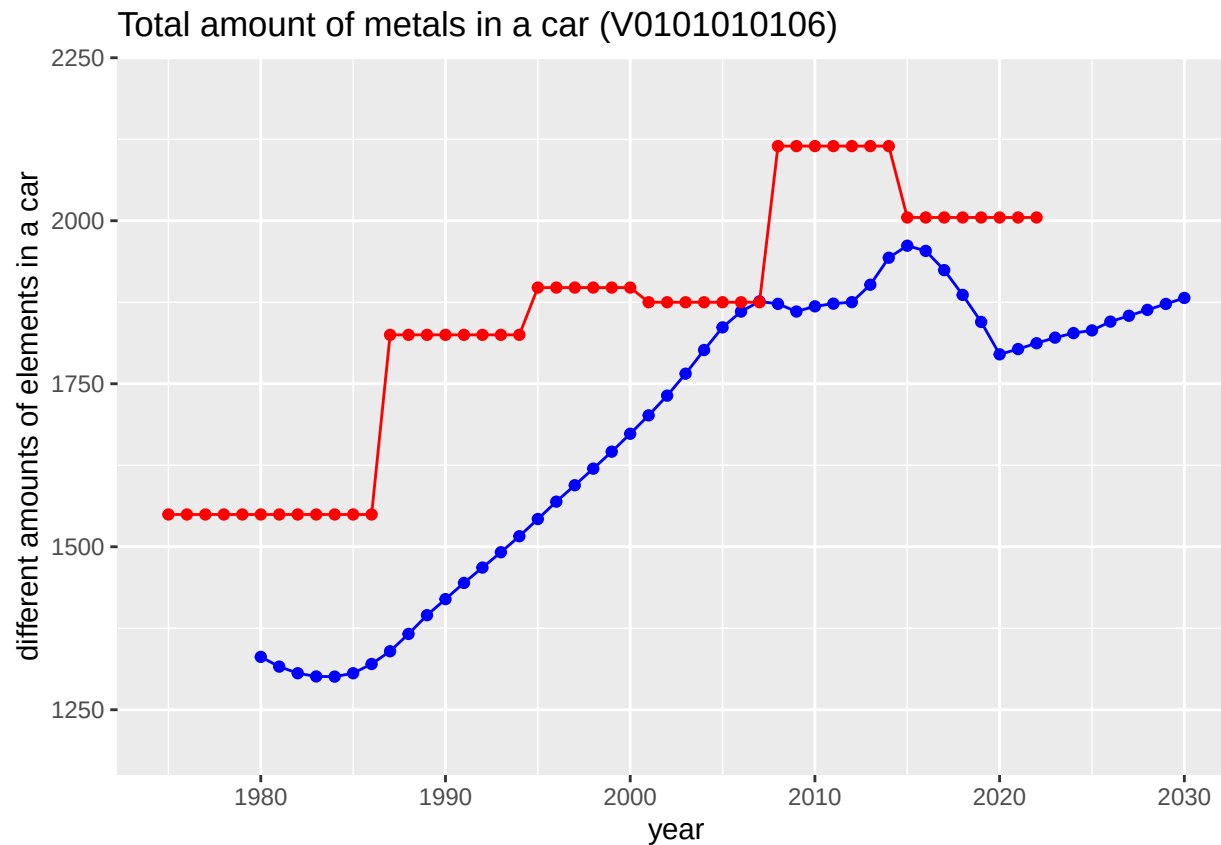
g <- ggplot() +
  geom_point(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_line(data = temp, mapping = aes(x = year, y = values), color = "blue") +
  geom_point(data = carWeight, mapping = aes(x = Year, y = SegmentFMass), color = "red") +
  geom_line(data = carWeight, mapping = aes(x = Year, y = SegmentFMass), color = "red") +
  ggtitle("Total amount of metals in a car (V0101010106)") +
  scale_y_continuous(limits = c(1200, 2200)) +

```

```

xlab("year") +
ylab("different amounts of elements in a car")
plot(g)

```



Add data from 2031 to 2050, which are identical with the 2030 data

```

data_2030 <- Calculate %>% filter(year == 2030)
#data_2030
years <- 2031:2050

```

```
for (i in 1:nrow(data_2030)) {  
  new_data <- data_2030 %>% slice(rep(i, length(years))) %>% mutate(year = years)  
  Calculate <- Calculate %>% bind_rows(new_data)  
}  
  
#Calculate
```

```
## write to excel sheet using perl  
  
# Create a new workbook and add the data  
wb <- createWorkbook()  
addWorksheet(wb, "FutuRaM-ELV")  
writeData(wb, "FutuRaM-ELV", Calculate)  
  
# Save the workbook to an Excel file  
saveWorkbook(wb, filename21_output, overwrite = TRUE)
```